

FUNDAMENTALS OF PROBLEM SOLVING

79 algorithms for OI

© Mingeon Joung[†]
Donghyun Gu[†]

[†] SSHS, Seoul, South Korea

Fundamentals of Problem Solving

Mingeon Jeong

Donghyun Gu

15th November 2025

Contents

I	서론	1
1	시간 복잡도와 공간 복잡도	3
1.1	개요	3
1.2	점근 표기법	4
1.3	제한 시간과 시간 복잡도	4
1.4	연습문제	5
II	기본 알고리즘	7
2	정렬	9
2.1	비교 기반 정렬 알고리즘	9
2.1.1	버블 정렬	9
2.1.2	삽입 정렬	11
2.1.3	선택 정렬	11
2.1.4	퀵 정렬	12
2.1.5	병합 정렬	14
2.2	정수 정렬 알고리즘	16
2.2.1	계수 정렬	16
2.2.2	기수 정렬	17
2.3	내장 정렬 함수 사용하기	18
2.4	연산자 오버로딩	20
2.5	순열 얻기	20
2.6	정렬의 활용	21
2.7	연습문제 A	22
2.8	연습문제 B	23

2.9 연습문제 C	24
2.10 힌트	27
3 구간 합	33
3.1 부분 합을 구하는 문제	33
3.2 이차원 부분 합	34
3.3 시간복잡도 줄이기	36
3.4 누적 합의 역연산	36
3.5 연습문제 A	37
3.6 연습문제 B	37
3.7 연습문제 C	38
3.8 힌트	40
4 이분 탐색	43
4.1 빠르게 탐색하기	43
4.2 STL 내장 함수	45
4.3 연습문제 A	47
4.4 연습문제 B	48
4.5 연습문제 C	49
5 매개변수 탐색	51
5.1 답을 탐색한다는 아이디어	51
5.2 결정 문제와 최적해 문제	53
5.3 연습문제 A	53
5.4 연습문제 B	54
5.5 연습문제 C	56
6 병렬 이분 탐색	59
6.1 퀴리와 이분 탐색	59
6.2 연습문제 A	61
6.3 연습문제 B	62
6.4 연습문제 C	62
7 투 포인터	65
7.1 가능한 구간을 찾는 문제	65
7.2 투 포인터가 사용될 조건	67
7.3 연습문제 A	67
7.4 연습문제 B	67
7.5 연습문제 C	70

8 비트마스크	73
8.1 배열 대신 숫자 하나로	73
8.2 <code>std::bitset</code>	74
8.3 연습문제 A	76
8.4 연습문제 B	76
8.5 연습문제 C	78
9 좌표 압축	85
9.1 값의 범위를 줄이기	85
9.2 연습문제 A	87
9.3 연습문제 B	88
9.4 연습문제 C	89
10 스위핑	91
10.1 선분 길이 구하기	91
10.2 연습문제 A	93
10.3 연습문제 B	93
10.4 연습문제 C	94
11 백트래킹	97
11.1 이론	97
11.2 <i>N</i> -Queen 문제	98
11.3 순열 구하기	100
11.4 격자 이동	101
11.5 최적화	102
11.6 연습문제 A	102
11.7 연습문제 B	103
11.8 연습문제 C	104
11.9 힌트	106
12 분할 정복	109
12.1 다시 보는 병합 정렬	109
12.2 가장 가까운 두 점 찾기	110
12.3 마스터 정리	112
12.4 연습문제 A	113
12.5 연습문제 B	113
12.6 연습문제 C	114

III 자료구조 (Easy)	117
13 연결 리스트	119
13.1 자료구조를 왜 만드는가?	119
13.2 연결리스트	120
13.3 연결리스트의 구현	121
13.4 STL의 연결리스트	123
13.5 연습문제 A	124
13.6 연습문제 B	124
13.7 연습문제 C	125
14 스택, 큐, 덱	127
14.1 스택	127
14.2 큐	129
14.3 덱	131
14.4 연습문제 A	134
14.5 연습문제 B	134
14.6 연습문제 C	135
15 힙과 이진 탐색 트리	137
15.1 힙	137
15.2 이진 탐색 트리	140
15.3 연습문제 A	144
15.4 연습문제 B	144
15.5 연습문제 C	145
16 해시	147
16.1 해시	147
16.2 연습문제 A	150
16.3 연습문제 B	150
16.4 연습문제 C	150
17 분리 집합	151
17.1 자료구조를 왜 만드는가?	151
17.2 연결리스트	152
17.3 연결리스트의 구현	153
17.4 STL 라이브러리의 연결리스트	155
17.5 연습문제 A	156
17.6 연습문제 B	156

17.7 연습문제 C	157
18 회소 배열	159
18.1 자료구조를 왜 만드는가?	159
18.2 연결리스트	160
18.3 연결리스트의 구현	161
18.4 STL 라이브러리의 연결리스트	163
18.5 연습문제 A	164
18.6 연습문제 B	164
18.7 연습문제 C	165
 IV 동적계획법 (Easy)	 167
19 1차원 동적계획법	169
19.1 동적계획법 개론	169
19.2 동적계획법의 구현	170
19.2.1 top-down	170
19.2.2 bottom-up	171
19.3 일차원 동적계획법 문제의 풀이	172
19.4 연습문제 A	173
19.5 연습문제 B	173
19.6 연습문제 C	173
 20 배낭 문제	 175
 21 다차원 동적계획법	 177
 V 그리디	 179
 22 그리디	 181
 VI 그래프 이론	 183
23 그래프 기초	185
23.1 그래프의 정의	185
23.2 그래프의 표현	186
23.3 몇 가지 용어들	187
23.4 연습문제 A	188

23.5 연습문제 B	188
23.5.1 기초문제	188
23.5.2 응용문제	188
23.5.3 도전문제	189
24 그래프의 종류	191
24.1 그래프의 분류	191
24.1.1 방향성	191
24.1.2 가중치	191
24.2 특수한 그래프	192
24.3 연습문제 A	194
24.4 연습문제 B	194
24.4.1 기초문제	194
24.4.2 응용문제	195
24.4.3 도전문제	195
25 그래프 탐색	197
25.1 깊이 우선 탐색	197
25.2 너비 우선 탐색	199
25.3 0-1 BFS	200
25.4 연습문제 A	201
25.5 연습문제 B	201
25.5.1 기초문제	201
25.5.2 응용문제	202
25.5.3 도전문제	203
26 최소 스패닝 트리	205
26.1 최소 스패닝 트리	205
26.2 Kruskal 알고리즘	205
26.3 Prim 알고리즘	207
26.4 연습문제 A	207
26.5 연습문제 B	208
26.5.1 기초문제	208
26.5.2 응용문제	208
26.5.3 도전문제	209
27 이중 연결 요소	211
27.1 단절점	211
27.2 단절선	212

27.3 이중 연결 요소	214
27.4 블록-컷 트리	215
27.5 연습문제 A	216
27.6 연습문제 B	216
27.7 연습문제 C	216
28 위상 정렬	217
28.1 위상 정렬	217
28.2 큐 활용 위상정렬 알고리즘	218
28.3 DFS 활용 위상정렬 알고리즘	219
28.4 연습문제 A	220
28.5 연습문제 B	220
28.5.1 기초문제	220
28.5.2 응용문제	220
28.5.3 도전문제	220
29 강한 연결 요소	221
29.1 강한 연결 요소	221
29.2 Kosaraju 알고리즘	222
29.3 Tarjan 알고리즘	224
29.4 2-SAT 문제	224
29.5 연습문제 A	224
29.6 연습문제 B	224
29.6.1 기초문제	224
29.6.2 응용문제	224
29.6.3 도전문제	224
30 다익스트라 알고리즘	225
30.1 문제 정의	225
30.2 다익스트라 알고리즘	225
30.2.1 $O(N^2)$ 구현	226
30.2.2 $O(M \log M)$ 구현	227
30.3 연습문제 A	229
30.4 연습문제 B	229
30.4.1 기초문제	229
30.4.2 응용문제	229
30.4.3 도전문제	230
31 벨만-포드 알고리즘	233

31.1 문제 정의	233
31.2 벨만-포드 알고리즘	233
31.3 연습문제 A	235
31.4 연습문제 B	235
31.4.1 기초문제	235
31.4.2 응용문제	235
31.4.3 도전문제	236
32 플로이드-워셜 알고리즘	237
32.1 문제 정의	237
32.2 플로이드-워셜 알고리즘	237
32.3 연습문제 A	239
32.4 연습문제 B	239
32.4.1 기초문제	239
32.4.2 응용문제	240
32.4.3 도전문제	241
VII 트리 알고리즘	243
33 트리 기초	245
34 트리 탐색	247
35 최소 공통 조상	249
36 오일러 투어 테크닉	251
37 센트로이드	253
38 Heavy-Light 분할	255
39 트리에서의 동적계획법	257
40 작은 집합에서 큰 집합으로 합치는 테크닉	259
VIII 자료구조 (Hard)	261
41 제곱근 분할	263
42 mo's	265

43 세그먼트 트리	267
44 느리게 갱신되는 세그먼트 트리	269
45 바이너리 인덱스 트리	271
46 머지 소트 트리	273
47 다차원 세그먼트 트리	275
48 퍼시스턴트 세그먼트 트리	277
49 스플레이 트리	279
50 링크/컷 트리	281
51 오프라인 동적 연결성	283
 IX 동적계획법 (Hard)	 285
52 단조 큐를 이용한 동적계획법 최적화	287
53 텍을 이용한 구간 최댓값 트릭	289
54 비트필드를 이용한 동적계획법	291
55 블록 곱질을 이용한 동적계획법 최적화	293
56 분할 정복을 이용한 동적계획법 최적화	295
57 커넥션 프로파일을 이용한 동적계획법	297
58 Knuth 최적화	299
59 Aliens 트릭	301
60 Slope 트릭	303
61 Hirschburg 알고리즘	305
62 벨리캠프-매시	307
63 기타마사법	309

X 문자열	311
64 KMP	313
65 아호-코라식	315
66 트라이	317
67 접미사 배열과 LCP 배열	319
XI 기하	321
68 선분 교차 판정	323
69 블록 겹질	325
70 회전하는 캘리퍼스	327
XII 네트워크 플로우	329
71 이분 매칭	331
72 최대 유량	333
73 최소 비용 최대 유량	335
74 최대 유량 최소 컷 정리	337
XIII 수학	339
75 게임 이론	341
76 소수 판정 및 소인수분해	343
77 모듈러 곱셈 역원	345
78 중국인의 나머지 정리	347
79 확장 유클리드 호제법	349
80 고속 푸리에 변환	351

Part I

서론

Chapter 1

시간 복잡도와 공간 복잡도

1.1 개요

Problem Solving에 필요한 알고리즘 공부에 앞서, 알고리즘을 평가하는 방법을 알 필요가 있다. 보통 알고리즘의 평가는 **시간과 메모리**의 두 가지 측면에서 이루어진다. Problem Solving에서 시간 제한과 메모리 제한은 문제를 풀기 전에 꼭 확인해야 하는 요소 중 하나이며, 시간 제한에 따라 문제 풀이가 달라지기도 한다.

당연하게도 시간과 메모리는 제약 조건(Constraint)에 대한 함수로 표시될 것이다. 아주 간단히 1부터 n 까지 수의 합을 구하는 문제와 이에 대한 아래 두 가지 알고리즘을 보자:

- 알고리즘 1: 반복문을 이용해 1부터 n 까지 순서대로 구한다.
- 알고리즘 2: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ 공식을 이용해 구한다.

직관적으로 알고리즘 1보다 알고리즘 2가 더 빠른 시간에 작동할 것이라고 추측할 수 있고, 실제로 그렇다(궁금한 사람은 $n = 100,000,000$ 정도를 대입하여 직접 실행해보자). 그렇다면 정확히 얼마나 더 빠를까?

이에 대한 답을 명확히 줄 수 있는 수학적 기호가 바로 시간 복잡도와 공간 복잡도이다. 알고리즘 1의 시간 복잡도와 공간 복잡도를 보면 반복문에서 n 번의 연산을 하고, 답에 저장하는 변수 하나만 필요하므로 메모리는 1이 필요하다고 생각할 수 있다. 반면 알고리즘 2의 경우에는 반복문 없이 총 세 번의 연산으로 답을 구할 수 있다. 이 알고리즘의 경우에는 시간 복잡도는 3로 줄어들고 공간 복잡도는 여전히 1이다. 즉, 알고리즘 2가 알고리즘 1보다 n 배 빠르다고 말할 수 있다.

이런 의문이 들었을 수도 있다: 정확히는 $n/3$ 배 빠른 것 아닌가? 위에서 n 으로 표시한 이유는

시간 복잡도와 공간 복잡도에서 상수가 크게 중요하지 않기 때문이다. 시간 복잡도와 공간 복잡도는 정확히 얼마의 시간이 걸리는지 알기 위한 것이 아니라, 제약조건에 따라 어느 정도로 시간이 빠르게 증가하는지를 확인하고자 하는 것이다. 즉, 우리는 시간에 가장 큰 영향을 주는 항만 남겨 표기해야 한다. 그리고 이를 수학적으로 나타내기 위해 점근 표기법(asymptotic notation)을 사용한다.

1.2 점근 표기법

점근 표기법이란, 함수가 증가 또는 감소하는 경향성을 나타내기 위한 표기법이다. 대표적으로는 Big-O, small-o, Big-Theta, small-theta, Big-Omega, small-omega 표기법이 있으며, 이 중 이 책에서는 PS에서 자주 쓰이는 Big-O와 Big-Theta notation에 대해서만 서술할 것이다. 이들의 정의는 다음과 같다.

Definition 1.1 (Big-O notation). $\limsup_{x \rightarrow \infty} f(x)/g(x) < \infty$ 이면 $f(x) = O(g(x))$ 이다.

Definition 1.2 (Big-Theta notation). $\limsup_{x \rightarrow \infty} f(x)/g(x) < \infty$ 이고 $\liminf_{x \rightarrow \infty} f(x)/g(x) > 0$ 이면 $f(x) = \Theta(g(x))$ 이다.

위 정의에서 보드시피 Big-O notation은 $f(x)$ 의 점근적 상한, Big-Theta notation은 $f(x)$ 의 점근적 상한이자 하한을 나타내는 것으로, Big-Theta가 더 엄밀한 값을 나타낸다는 것을 알 수 있다. 하지만 PS에서는 이 둘을 혼용해서 모두 Definition 1.2의 정의로 사용하며, 이 책에서도 이후로 나오는 Big-O notation에 대한 정의는 예의 정의로 쓰일 것이다.

이제 시간 복잡도를 표기하는 방법을 알아보기 위해 앞의 예시를 다시 한 번 가져와보자.

- 알고리즘 1: 반복문을 이용해 1부터 n 까지 순서대로 구한다.
- 알고리즘 2: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ 공식을 이용해 구한다.

1.1절에서 분석한 내용과 같이 알고리즘 1에는 n 번의 연산이 쓰이며, 알고리즘 2에는 3번의 연산이 쓰인다. 이를 Big-O notation으로 표현한다면 알고리즘 1은 $O(n)$ 의 시간 복잡도를, 알고리즘 2는 $O(1)$ 의 시간 복잡도를 가진다고 표현할 수 있다(Definition 1.2에서는 계수가 포함되어도 상관이 없지만, 이제부터는 그 상수를 생략하고 표기할 것이다).

공간 복잡도에 대해서도 같은 표기법을 사용한다는 것을 알아두자.

1.3 제한 시간과 시간 복잡도

현재 컴퓨터는 1초에 약 10^8 번 정도, 많게는 10^9 번 정도의 연산을 수행할 수 있다고 알려져 있다. PS 문제를 푼다면 본인이 생각한 풀이의 시간 복잡도를 분석하여 그 식에 입력 제한을 대입하여 제한 시간 안에 풀이가 답을 찾는지 확인하는 것이 좋다. 다음은 PS에서 사용되는 대표적인 시간 복잡도이다.

- 상수 시간 복잡도, $O(1)$
- 로그 시간 복잡도, $O(\log N)$, $O(\log^2 N)$ 등
- 루트 시간 복잡도, $O(\sqrt{N})$
- 다항 시간 복잡도, $O(N)$, $O(N^2)$ 등
- 지수 시간 복잡도, $O(2^N)$, $O(3^N)$ 등
- 팩토리얼 시간 복잡도, $O(N!)$

보통 시간 복잡도는 위 예시들의 합 또는 곱으로 이루어진다.

1.4 연습문제

Problem 1.1. 크기 N 의 어떤 집합 S 의 모든 부분집합을 출력하려고 한다. 완전 탐색 방식의 시간복잡도는 무엇인가?

Problem 1.2. N 개의 노드로 이루어진 완전 이진 트리의 리프에서 루트까지 올라가는 경로를 출력하고자 한다. 시간복잡도는 무엇인가?

(추가될 예정)

Part II

기본 알고리즘

Chapter 2

정렬

정렬(sorting)은 어떤 배열의 값들을 작은 수부터 큰 수 또는 큰 수부터 작은 수의 순서로 나열하는 것을 말한다. 이 장에서는 총 7가지 정렬 알고리즘과 정렬 기준 설정에 대해 배울 것이다.

2.1 비교 기반 정렬 알고리즘

비교 기반 정렬 알고리즘은 어떤 두 수를 비교해 얻은 결과만을 이용해 정렬하는 알고리즘이다. 다른 말로, 배열 arr 에 속하는 어떤 두 수 x, y 를 비교해 더 작은 수가 앞에 와야 함을 이용해 정렬하는 것이다.

이러한 비교 기반 정렬 알고리즘의 특징은 시간복잡도의 상한이 $O(N \log N)$ 이라는 것이다. 그 이유는 N 개의 데이터에서 존재할 수 있는 모든 초기 데이터가 $N!$ 가지인데, 적절한 한 번의 비교를 통해 가능한 경우의 수를 절반으로 줄일 수 있기 때문에 최소 연산 횟수는 $O(\log N!)$ 로 정해져 있기 때문이다(스털링 근사에 의해 $O(N!) = O(N \log N)$ 이 된다).

이 절에서는 총 다섯 종류의 비교 기반 정렬 알고리즘을 다룬다. 앞 3개는 평균 시간복잡도가 $O(N^2)$ 인 알고리즘, 뒤 2개는 평균 시간복잡도가 $O(N \log N)$ 인 알고리즘이다. 비교 기반 정렬 알고리즘은 모두 중요하므로 주의 깊게 보도록 하자.

2.1.1 버블 정렬

버블 정렬은 인접한 두 수를 비교해 큰 수가 앞에 있으면 뒤집는 과정을 반복함으로써 정렬하는 방식이다. 정확히 말하면 $1 \leq i < N$ 인 모든 i 에 대해 i 번째 수와 $i + 1$ 번째 수의 크기 순서가

반대이면 i 번째 수와 $i + 1$ 번째 수를 뒤집는 과정을 정렬될 때까지 반복하는 것이다. Code 2.1에 구현 코드가 있다.

Code 2.1. 버블 정렬 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, arr[1010];
5
6  bool check() {
7      for (int i = 1; i < n; i++) {
8          if (arr[i] > arr[i + 1]) return true;
9      }
10     return false;
11 }
12
13 int main() {
14     cin >> n;
15     for (int i = 1; i <= n; i++) cin >> arr[i];
16
17     while (check()) {
18         for (int i = 1; i < n; i++) {
19             if (arr[i] > arr[i + 1]) swap(arr[i], arr[i + 1]);
20         }
21     }
22
23     for (int i = 1; i <= n; i++) cout << arr[i] << " ";
24     return 0;
25 }
```

버블 정렬의 시간복잡도는 어떻게 될까? 뒤집는 과정이 반복되는 횟수에 N 을 곱한 것이 시간복잡도가 될 것이다. 그렇다면 뒤집는 과정은 얼마나 많이 반복될까? 수학적으로 N 번 이하임을 증명할 수 있다. 즉, 버블 정렬의 시간복잡도는 $O(N^2)$ 이다. 증명은 Problem 2.2을 참고하라.

버블 정렬의 교환 횟수는 어떻게 될까? 한 번 교환할 때마다 $i < j$ 이고 $arr[i] > arr[j]$ 인 순서쌍 (i, j) 의 개수가 하나씩 줄어든다는 것은 자명한 사실이다. 즉, 어떤 배열 arr 에 대해 $i < j$ 이고 $arr[i] > arr[j]$ 인 순서쌍 (i, j) 의 개수로 정의되는 *Inversion Count*가 버블 정렬의 교환 횟수가 된다. *Inversion Count*를 구하는 문제를 Problem 2.3에서 풀어보라.

2.1.2 삽입 정렬

삽입 정렬은 앞에서부터 정렬해나가는 방식으로 x 번째까지 정렬되어 있을 때 $x + 1$ 번째를 자신보다 작은 수가 나올 때까지 앞으로 한 칸씩 옮기며 정렬하는 방식이다. 이 방법을 사용했을 때 정렬이 됨은 자명하게 확인할 수 있다. Code 2.2를 보라.

Code 2.2. 삽입 정렬 구현 코드

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, arr[1010];
5
6  int main() {
7      cin >> n;
8      for (int i = 1; i <= n; i++) cin >> arr[i];
9
10     for (int i = 1; i <= n; i++) {
11         for (int j = i; j >= 2; j--) {
12             if (arr[j] < arr[j - 1]) swap(arr[j], arr[j - 1]);
13             else break;
14         }
15     }
16
17     for (int i = 1; i <= n; i++) cout << arr[i] << " ";
18     return 0;
19 }
```

삽입 정렬의 재미있는 특징은 거의 정렬되어 있다면 빠른 시간에 완료된다는 것이다. 왜냐하면 x 번째 수 앞에 있는 수들 중 x 번째 수보다 큰 수의 개수와 교환 횟수가 동일하기 때문이다 ($N = 5$ 일 때 배열 1, 3, 5, 4, 2와 5, 4, 3, 2, 1의 교환 횟수를 비교해 확인해보라). 이러한 특징은 전체 배열이 거의 정렬되었을 때 의도적으로 삽입 정렬을 사용하는 하이브리드 정렬에 자주 사용된다. 하지만 여전히 시간복잡도는 $O(N^2)$ 이다. 완전히 역순인 최악의 경우에는 교환 횟수가 $N(N - 1)/2$ 이기 때문이다.

2.1.3 선택 정렬

선택 정렬은 배열에서 최솟값을 찾고 그 수를 제일 앞으로 보내는 과정을 반복함으로써 정렬하는 방식이다. 당연히 k 개의 수를 앞으로 보냈다면 $k + 1$ 번째 수부터 최솟값을 찾아야 한다. Code 2.3을 보라.

Code 2.3. 선택 정렬 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, arr[1010];
5
6  int main() {
7      cin >> n;
8      for (int i = 1; i <= n; i++) cin >> arr[i];
9
10     for (int i = 1; i <= n; i++) {
11         int idx, val = 1e9;
12         for (int j = i; j <= n; j++) {
13             if (val > arr[j]) {
14                 idx = j;
15                 val = arr[j];
16             }
17         }
18         swap(arr[i], arr[idx]);
19     }
20
21     for (int i = 1; i <= n; i++) cout << arr[i] << " ";
22     return 0;
23 }

```

시간복잡도는 $O(N^2)$ 이다.

2.1.4 퀵 정렬

퀵 정렬은 어떤 기준값인 **pivot**을 잡아 **pivot**보다 작은 수는 앞으로 보내고, 큰 수는 뒤로 보낸 후 각 부분을 다시 정렬하는 재귀적 정렬 방식이다.

그렇다면 어떻게 **pivot**을 기준으로 양쪽으로 수를 나눌까? 일단 **pivot**의 경우에는 어떤 수를 선택해도 상관없기 때문에, 보통 첫 번째 수를 잡는다. 이후, 두 개의 포인터(정확히는 인덱스 번호를 저장하는 정수형 변수이다) **p**, **q**를 사용하는데, **p**는 왼쪽 끝에서 오른쪽 끝으로, **q**는 오른쪽 끝에서 왼쪽 끝으로 이동할 것이다.

매 반복마다 **p**는 $arr[p]$ 가 **pivot**보다 커질 때까지, **q**는 $arr[q]$ 가 **pivot**보다 작아질 때까지 이동한다. 그러면 두 가지 경우가 가능하다:

- p와 q의 순서가 뒤집힌 경우에는 분리가 끝났다는 뜻이다. 따라서 기준값인 pivot가 중간에 오도록 q번째 수와 pivot를 바꾼다. q번째 수와 바꾸는 이유는 q번째 수가 pivot보다 작기 때문이다.
- p와 q의 순서가 뒤집히지 않은 경우에는 분리가 끝나지 않은 경우이다. 이 경우에는 단순히 p번째 수와 q번째 수를 바꾸어 분리가 되도록 한다.

이 과정을 구현한 것이 Code 2.4이다.

Code 2.4. 퀵 정렬 구현 코드

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, arr[1010];
5
6  void quicksort(int st, int ed) {
7      if (st >= ed) return;
8      int pivot = st;
9      int lt = st + 1, rt = ed;
10
11     while (true) {
12         while (lt <= ed && arr[lt] <= arr[pivot]) lt++;
13         while (rt > st && arr[rt] >= arr[pivot]) rt--;
14         if (lt > rt) {
15             swap(arr[pivot], arr[rt]);
16             break;
17         }
18         else swap(arr[lt], arr[rt]);
19     }
20
21     quicksort(st, rt - 1);
22     quicksort(rt + 1, ed);
23 }
24
25 int main() {
26     cin >> n;
27     for (int i = 1; i <= n; i++) cin >> arr[i];
28
29     quicksort(1, n);
30
31     for (int i = 1; i <= n; i++) cout << arr[i] << " ";
```

```

32     return 0;
33 }

```

이 알고리즘의 시간복잡도는 어떻게 될까? 결론만 말하면 재귀 함수가 재귀적으로 호출된 횟수 (재귀의 깊이라고 한다)와 `pivot`를 기준으로 나누기 위해 필요한 연산 횟수의 곱이 시간복잡도이다(Problem 2.6 참고). 평균적으로 재귀의 깊이는 $O(\log N)$ 일 것이고, 나누는 과정의 시간복잡도는 $O(N)$ 이므로 평균 시간복잡도는 $O(N \log N)$ 이다. 하지만, 최악의 경우 재귀의 깊이는 $O(N)$ 이므로 최악 시간복잡도는 $O(N^2)$ 가 된다.

2.1.5 병합 정렬

병합 정렬은 가운데 값을 기준으로 양쪽 배열을 정렬한 후 양쪽의 결과를 바탕으로 합쳐 정렬된 배열을 만드는 방식이다. 양쪽 배열이 정렬되어 있기 때문에 다음 관찰이 성립한다.

Observation. 합쳐진 배열의 첫 번째 수는 왼쪽 배열의 첫 번째 수와 오른쪽 배열의 첫 번째 수 중 작은 수이다.

이 관찰을 반복적으로 적용할 수 있는데, 왜냐하면 첫 번째 수를 빼고 나머지 두 배열을 합친 결과가 정렬된 배열이 되기 때문이다. 따라서 왼쪽 배열의 현재 수와 오른쪽 배열의 현재 수를 나타내는 포인터(인덱스) `p`와 `q`를 이용해 구현할 수 있다. `lt[p]`와 `rt[q]` 중 작은 수를 넣고 작은 쪽의 포인터를 오른쪽으로 한 칸 움직이면 된다.

Code 2.5. 병합 정렬 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, arr[1010];
5
6  void mergesort(int st, int ed) {
7      if (st >= ed) return;
8
9      int mid = (st + ed + 1) / 2;
10     mergesort(st, mid - 1);
11     mergesort(mid, ed);
12
13     vector<int> temp;
14     int p = st, q = mid;
15
16     while (true) {

```

```
17         if (p >= mid && q > ed) break;
18         if (p >= mid) {
19             temp.push_back(arr[q]);
20             q++;
21             continue;
22         }
23         if (q > ed) {
24             temp.push_back(arr[p]);
25             p++;
26             continue;
27         }
28         if (arr[p] <= arr[q]) {
29             temp.push_back(arr[p]);
30             p++;
31         }
32         else {
33             temp.push_back(arr[q]);
34             q++;
35         }
36     }
37
38     for (int i = st; i <= ed; i++) arr[i] = temp[i - st];
39 }
40
41 int main() {
42     cin >> n;
43     for (int i = 1; i <= n; i++) cin >> arr[i];
44
45     mergesort(1, n);
46
47     for (int i = 1; i <= n; i++) cout << arr[i] << " ";
48     return 0;
49 }
```

뒤에서 다시 배우겠지만, 이렇게 구간을 분할한 후 다시 합치는 과정을 통해 문제를 해결하는 방법을 분할 정복(divide and conquer)라고 한다. 병합 정렬은 대표적인 분할 정복 알고리즘의 예시이다.

병합 정렬은 퀵 정렬과 다르게 항상 배열의 길이가 절반이 됨이 보장된다. 따라서 재귀의 깊이는 최악의 경우에도 $O(\log N)$ 이므로 최악 시간복잡도도 $O(N \log N)$ 이다.

2.2 정수 정렬 알고리즘

정수 정렬 알고리즘은 모든 값이 정수일 때 비교가 아닌 다른 방식으로 정렬하는 방법으로, 정수의 특징을 이용하는 경우가 많다. 이 정수 정렬 알고리즘의 아이디어는 다양한 문제에서 사용될 수 있으므로 꼭 한 번 구현해보는 것을 추천한다.

2.2.1 계수 정렬

계수 정렬은 값의 개수를 저장한 후 작은 값부터 그 값의 개수만큼 새로운 배열에 추가함으로써 정렬하는 방식이다.

Code 2.6. 계수 정렬 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, arr[1010], cnt[1010];
5  const int x = 1000;
6
7  int main() {
8      cin >> n;
9      for (int i = 1; i <= n; i++) cin >> arr[i];
10
11     for (int i = 1; i <= n; i++) cnt[arr[i]]++;
12
13     vector<int> res;
14     for (int i = 1; i <= x; i++) {
15         for (int j = 1; j <= cnt[i]; j++) res.push_back(i);
16     }
17
18     for (int i = 0; i < n; i++) cout << res[i] << " ";
19     return 0;
20 }
```

이전까지의 정렬과 다르게 `cnt[]` 배열을 선언해 개수를 저장해야 한다. 따라서 `cnt[]` 배열의 인덱스 개수는 최댓값 X 이상이어야 하고, 공간복잡도가 값의 크기에 영향을 받게 된다. 또한, 반복문이 1부터 X 까지 진행되며 `cnt[i]`개 만큼 i 를 추가하는 과정이 포함되므로 시간복잡도도 값의 크기에 영향을 받게 된다.

이제 시간복잡도를 계산해보자. `cnt[i]`의 최댓값은 N 이고, i 를 총 X 개 확인해야 하므로 $O(NX)$

일까? 하지만, 실제로 `res`에 값이 추가되는 횟수는 `cnt[i]`의 총 합이 N 이므로 N 번이다. 즉, i 에서 `cnt[i]`가 N 일 수 없다. 따라서 다음과 같이 계산해야 정확히 시간복잡도를 구할 수 있다.

$$O\left(\sum_{i=1}^X (1 + \text{cnt}[i])\right) = O(N + X)$$

결론적으로 시간복잡도 $O(N + X)$, 공간복잡도 $O(N + X)$ 가 된다.

2.2.2 기수 정렬

기수 정렬은 정수의 중요한 성질인 진법을 활용한다. 각 정수를 진법을 통해 자리수로 나누어 저장하고, 낮은 자리수부터 계수 정렬의 원리를 활용하여 정렬하는 방식이다.

먼저 필요한 자리수 d 를 계산한다. 이후 각 자리수에 대해 다음을 반복한다.

- i 번째 자리수를 계산한다. 15번째 줄을 참고하라.
- j 번째 수의 i 번째 자리수가 t 이면 `cnt[t]`에 j 를 추가한다. 개수를 저장하지 않고 인덱스를 저장하는 이유는 t 는 실제 배열 `arr`에 포함된 수가 아니기 때문이다.
- i 번째 자리수가 작은 것부터 `cnt[t]` 배열에 저장된 인덱스를 이용해 새로운 배열 `res`에 추가한다. 이후 `res`의 값을 `arr`에 대입한다.

다음 Code 2.7은 위 알고리즘을 구현한 것이다.

Code 2.7. 기수 정렬 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, arr[1010], cnt[1010];
5  const int x = 1000, r = 10;
6
7  int main() {
8      cin >> n;
9      for (int i = 1; i <= n; i++) cin >> arr[i];
10
11     double d = log(x)/log(r);
12     for (int i = 0; i <= d; i++) {
13         vector<int> cnt[r], res;
14         for (int j = 1; j <= n; j++) {
15             int t = (arr[j] % (int)pow(r, i+1)) / (int)pow(r, i);
16             cnt[t].push_back(j);

```

```

17     }
18     for (int k = 0; k < r; k++) {
19         for (int j = 0; j < (int)cnt[k].size(); j++)
20             res.push_back(arr[cnt[k][j]]);
21     }
22     for (int j = 1; j <= n; j++) arr[j] = res[j - 1];
23 }
24
25 for (int i = 1; i <= n; i++) cout << arr[i] << " ";
26 return 0;
27 }

```

주의해야 할 점은 이미 이전 자리수를 기준으로 정렬된 순서가 깨지면 안된다는 점이다. 같은 t 를 갖더라도 그 앞 자리 수가 작은 수가 더 앞에 와야 하기 때문이다. 이러한 정렬을 안정 정렬 (stable sorting)이라고 하며, 기수 정렬에서 사용되는 계수 정렬은 무조건 안정 정렬이어야 한다.

기수 정렬의 시간복잡도와 공간복잡도는 계수 정렬과 비슷하게 구할 수 있다. Problem 2.10을 참고하라.

2.3 내장 정렬 함수 사용하기

C++의 표준 템플릿 라이브러리(STL)에는 내장 정렬 함수 `std::sort`가 있다. `std::sort(start, end, compare)`로 정의되며, `start`는 시작 포인터, `end`는 끝 포인터, `compare`은 비교 기준을 나타내는 함수 포인터이다. 즉, `[start, end)` 구간을 `compare`을 기준으로 정렬해준다. <https://en.cppreference.com/w/cpp/algorithm/sort>에서 제공하는 예시 코드를 이용해 구체적으로 설명하겠다.

Code 2.8. `std::sort` 예시 코드

```

1  #include <algorithm>
2  #include <array>
3  #include <functional>
4  #include <iostream>
5  #include <string_view>
6
7  int main()
8  {
9      std::array<int, 10> s {5, 7, 4, 2, 8, 6, 1, 9, 0, 3};

```

```

10
11     auto print = [&s](std::string_view const rem)
12     {
13         for (auto a : s)
14             std::cout << a << ' ';
15         std::cout << ": " << rem << '\n';
16     };
17
18     std::sort(s.begin(), s.end());
19     print("sorted with the default operator<");
20
21     std::sort(s.begin(), s.end(), std::greater<int>());
22     print("sorted with the standard library compare function object");
23
24     struct
25     {
26         bool operator()(int a, int b) const { return a < b; }
27     }
28     customLess;
29
30     std::sort(s.begin(), s.end(), customLess);
31     print("sorted with a custom function object");
32
33     std::sort(s.begin(), s.end(), [](int a, int b)
34             {
35                 return a > b;
36             });
37     print("sorted with a lambda expression");
38 }

```

- compare 함수를 빈칸으로 놓으면 기본 < 연산자에 의해 정렬된다.
- compare 함수로 STD에서 지원하는 비교 함수 객체를 사용할 수 있다. `std::less<T>`로 놓으면 오름차순으로, `std::greater<T>`로 놓으면 내림차순으로 정렬된다.
- 직접 함수 객체를 정의해 사용할 수 있다. 함수 객체 선언 방식은 24에서 28번 줄을 참고하라.
- 람다 표현법으로 직접 compare에 함수를 추가할 수 있다. 람다 표현법은 33에서 36번 줄을 참고하라.

추가로, 비교 시 값이 같다면 앞에 있는 수가 정렬된 배열에서도 앞에 오도록 정렬할 수 있다. 이러한 정렬을 안정 정렬(stable sorting)이라고 하며, STL에서는 `std::stable_sort`를 사용하면 된다. 함수 사용 방법은 똑같다.

2.4 연산자 오버로딩

내장 정렬 함수에서 `compare`에 비교 기준을 넣어 원하는 기준에 따라 정렬할 수도 있지만, 연산자 오버로딩(operator overloading)을 사용할 수도 있다. 원하는 비교 기준을 연산자 <에 오버로딩하여 `compare` 함수를 따로 쓰지 않고 정렬하는 것이다. 다음 Code 2.9는 구조체에서 연산자 오버로딩을 통해 정렬하는 예시 코드이다. 구조체에서 연산자 오버로딩을 해야 하는 경우가 생기면 Code 2.9를 참고해 `operator` 함수를 정의하라.

Code 2.9. 연산자 오버로딩 예시 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n;
5  struct Data{
6      int teamA, teamB;
7      bool operator<(const Data &other) const {
8          return teamA - teamB < other.teamA - other.teamB;
9      }
10 } arr[1010];
11
12 int main() {
13     cin >> n;
14     for (int i = 1; i <= n; i++) cin >> arr[i].teamA >> arr[i].teamB;
15
16     sort(arr + 1, arr + n + 1);
17
18     for (int i = 1; i <= n; i++)
19         cout << arr[i].teamA << " " << arr[i].teamB << "\n";
20     return 0;
21 }

```

2.5 순열 얻기

때로는 정렬된 수열의 어떤 수가, 원래 수열에서 몇 번째에 있었는지 알아야 할 필요가 있다. 다음 예시 문제가 그러하다.

Example 2.1. (1015번) 길이 N 인 수열 A_i 와 순열 P_i 를 생각하자. 그러면 $A_i = B_{P_i}$ 가 되도록 길이 N 인 수열 B_i 를 유일하게 결정할 수 있다. A_i 가 주어질 때 B_i 가 단조증가하도록 만드는

사전 순으로 가장 앞선 P_i 를 출력하라. $N \leq 50$, $A_i \leq 1000$ 이다.

중요한 관찰은 i 번째 수가 정렬된 수열 B 에서 몇 번째인지 나타내는 수가 P_i 라는 것이다. 그리고, P_i 로 가능한 값은 똑같은 수가 존재할 수 있기 때문에 구간으로 나오게 되는데, P_i 가 사전 순으로 가장 앞서게 하려면 작은 i 부터 작은 값을 할당해야 한다. 정렬된 배열에서 a 번째부터 b 번째까지 값이 모두 t 로 같다면, $A_i = t$ 인 모든 i 에 대해 $P_i \in [a, b]$ 여야 하고, 이러한 i 는 정확히 $b - a + 1$ 개이므로 작은 i 부터 $a, a + 1, \dots$ 의 순으로 P_i 를 정하면 된다.

이것을 하려면 정렬된 수열의 어떤 수가 원래 수열에서 몇 번째인지 알아야 한다. `std::pair` 객체를 활용하면 쉽게 할 수 있다. 아래 코드를 보자.

```
1 pair<int, int> arr[55];
2 // ...
3     for (int i = 1; i <= n; i++) {
4         cin >> arr[i].first;
5         arr[i].second = i;
6     }
7     sort(arr + 1, arr + n + 1);
8 // ...
```

`std::pair` 객체를 사용해서 정렬하고자 하는 배열을 첫 번째로, 그리고 인덱스를 두 번째로 넣으면, 첫 번째 값을 기준으로 정렬되므로 제대로 정렬이 된다. 그리고 두 번째 값을 읽으면 그 값이 원래 배열에서 어디에 있었는지 알 수 있게 된다.

위 문제에서는 P_i 가 최소가 되게 해야 하므로, 안정 정렬을 해야 원하는 위치를 나타내는 순열(permutation)을 얻게 된다. 이를 이용하면 문제를 해결할 수 있다.

2.6 정렬의 활용

다음 문제는 정렬이 필요하다.

Example 2.2. (1059번) 크기 N 인 집합 S 에 대해 n 을 포함하지만 S 의 그 어떤 원소도 포함하지 않는 서로 다른 닫힌 구간 $[a, b]$ ($1 \leq a < b$)의 개수를 구하여라. $N \leq 50$ 이고 S 의 모든 원소는 1000 이하의 서로 다른 수이며, n 은 S 에서 가장 큰 수보다 작거나 같은 자연수이다.

$S = \{5, 10\}$ 일 때, S 의 그 어떤 원소도 포함하지 않는 서로 다른 닫힌 구간은 $[1, 4]$ 에 속하거나, $[6, 9]$ 에 속하거나, $[10, \infty)$ 에 속해야 한다. 이 구간들은 겹치지 않기 때문에 결국 n 을 포함하면서 S 의 그 어떤 원소도 포함하지 않는 서로 다른 닫힌 구간은 딱 하나의 구간에 속해야 한다.

이 구간을 찾을 때 정렬이 필요하다. n 좌우의 가장 가까운 S 에 속하는 원소를 찾으면 그것이

구간이 되는데, 정렬하면 n 이 사이에 들어가는 두 수를 찾으면 각각이 가장 가까운 원소임이 보장되기 때문이다. 이후에는 수학적으로 n 을 포함해야 함을 고려해 가능한 왼쪽 끝점과 오른쪽 끝점의 경우의 수를 각각 구해 곱하면 답을 얻을 수 있다.

또 다른 문제를 하나 보자.

Example 2.3. 어떤 다항식을 $\sum c_i x^{k_i}$ ($c_i \neq 0$)로 표현하는 것은 다항식을 최소 개수의 항으로 표현하는 방식이다. 최소 개수의 항으로 표현된 다항식 두 개가 입력으로 주어질 때, 두 다항식의 곱을 최소 개수의 항으로 표현하라.

정렬은 곱한 결과를 최소 개수의 항으로 표현하기 위해 필요하다. 곱셈을 이중 반복문으로 구현하고 만들어지는 항을 차례대로 추가하고, 지수가 같은 두 개 이상의 항을 찾아 하나로 합쳐 주어야 한다. 이를 해결하는 가장 쉬운 방법은 각 항마다 전체를 순회하면서 지수가 같은 다른 항을 모두 찾아 합치는 것이다. 그런데, 지수를 기준으로 정렬하면, **지수가 같은 항은 항상 인접하게 될** 것이므로 훨씬 더 작은 시간복잡도로 구현할 수 있게 된다.

2.7 연습문제 A

Problem 2.1. 비교 기반 정렬 알고리즘 다섯 가지를 직접 구현하라. 길이 $N \leq 10000$ 의 같은 랜덤 데이터에 대한 시간을 측정해 분석하라.

Problem 2.2. 버블 정렬의 정당성을 증명하라. 증명을 바탕으로 버블 정렬의 시간복잡도가 $O(N^2)$ 임을 증명하라.

Problem 2.3. Inversion Count를 $O(N^2)$ 의 시간복잡도에 구현하라.

Problem 2.4. Code 2.2의 13번째 줄의 `break` 문을 없애면 어떻게 되는가?

Problem 2.5. Code 2.3의 11번째 줄에서 `val`을 `1e9`로 설정하였다. `1e9`가 갖는 의미가 무엇인가?

Problem 2.6. 퀵 정렬이나 병합 정렬과 같은 재귀함수 기반의 알고리즘의 시간복잡도는 어떻게 구할까? 퀵 정렬과 병합 정렬을 예시로 하여 고민해보라.

Problem 2.7. 퀵 정렬의 시간복잡도는 최악의 경우 $O(N^2)$ 이다. 이 문제를 해결할 수 있는 개선된 퀵 정렬이 있을까? 직접 생각해봐도 좋고, 검색하여 찾아보아도 좋다.

Problem 2.8. Code 2.5의 9번째 줄을 설명하라. `mid = (st + ed) / 2`로 정의하면 어떻게 될까?

Problem 2.9. 정수 정렬 알고리즘 두 가지를 직접 구현하라.

Problem 2.10. 값의 개수를 N , 값의 범위를 X 이하의 정수, 사용하는 진법을 r 이라고 할 때, 기수 정렬 알고리즘의 시간복잡도와 공간복잡도를 구하라. 시간복잡도가 r 의 변화에 따라 어떻게 변하는가?

Problem 2.11. 위의 7개 정렬 방식 중 안정 정렬인 것은 무엇인가?

Problem 2.12. Code 2.8의 실행 결과를 예측하고, 실제로 실행해 확인해보라.

Problem 2.13. Code 2.9의 arr의 마지막 값은 무엇인가?

2.8 연습문제 B

Problem 2.14. (2107번) 수직선에 N 개의 닫힌 구간 $[a_i, b_i]$ 가 있다. 또, a_i 와 b_i 는 모두 서로 다르다. 즉, 어떤 한 정수는 많아야 한 구간의 끝점이다. 어떤 경우에는 한 구간이 다른 구간에 포함될 수도 있다. 한 구간이 포함하는 다른 구간의 개수의 최댓값을 찾는 프로그램을 작성하라. $N \leq 25000$ 이고 $1 \leq A_i < B_i \leq 2 \cdot 10^9$ 이다.

Problem 2.15. (4701번) N 명의 알파벳 이름이 주어질 때, 절반은 S 보다 사전순으로 뒤이고 절반은 S 보다 사전순으로 앞이거나 같은 S 를 찾고 싶다. 이러한 S 들 중 사전순으로 가장 앞선 S 를 출력하라. $N \leq 1000$ 인 짝수이고 알파벳 이름의 길이는 30 이하이다.

Problem 2.16. (9078번) 길이 N 의 주어진 수열이 이웃하는 숫자 두 개를 묶어서 원하는 곳에 넣는 작업만으로 정렬할 수 있는지 판단하는 프로그램을 작성하라. $N \leq 100$ 이다.

Problem 2.17. (17553번*) N 개의 좌표 (x, y) 가 주어질 때, (a, b) 와 (c, d) 와의 거리가 같은 점은 많아야 1개이며 절반은 (a, b) 와 더 가깝고 나머지 절반은 (c, d) 와 더 가까운 (a, b) 와 (c, d) 를 하나 정하라. 단, a, b, c, d 는 모두 절댓값이 10^{18} 이하인 정수여야 한다. $N \leq 10^5$ 이고 (x, y) 의 x 와 y 는 모두 절댓값이 10^9 이하인 정수이다.

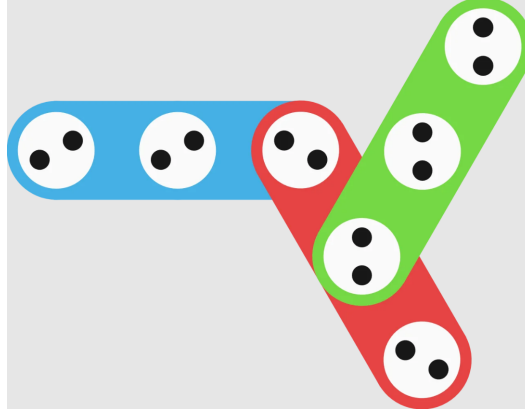
Problem 2.18. (21091번*) 길이가 N 인 두 순열 A 와 B 가 주어질 때, 순열 A 의 부분 구간 $[l, r]$ 을 오름차순 정렬 또는 내림차순 정렬하는 작업으로 순열 B 를 만드는 방법 한 가지를 제시하라. 작업 사용 횟수는 N 번 이하이지만 하면 되고, 최소화할 필요는 없다. $N \leq 500$ 이다.

Problem 2.19. (21112번) 길이가 N 인 수열을 차이가 K 이하인 수를 바꾸는 것만으로 정렬할 수 있을 K 의 최솟값을 구하라. $N \leq 2 \cdot 10^5$ 이고 수열의 원소는 10^{18} 이하의 자연수이다.

Problem 2.20. (25323번*) 길이가 N 인 수열에서 곱이 제곱수인 두 수만을 바꾸어 정렬할 수 있는지 판별하라. $N \leq 5 \cdot 10^5$ 이고 수열의 원소는 10^{18} 이하의 자연수이다.

Problem 2.21. (28076번) 멀티탭의 방향과 쏘는 구멍이 30도 차이나는 멀티탭 N 개를 생각하자. 다른 말로 하나의 멀티탭 위에 다른 멀티탭을 쏘으면 두 멀티탭 사이의 각도는 60

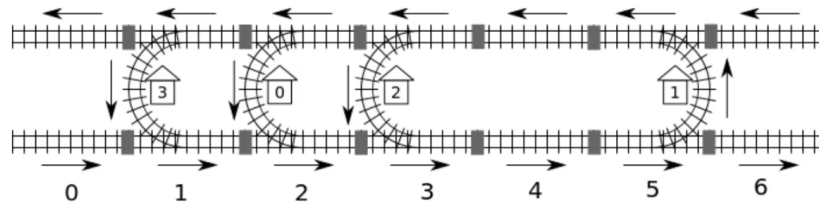
도이다. 멀티탭을 적절히 꽂아 위에서 보았을 때 보이는 멀티탭 칸의 수가 최소가 되도록 할 때, 최소값을 출력하라. $N \leq 2 \cdot 10^5$ 이고 멀티탭에 있는 칸 수는 2 이상 10^8 이하의 정수이다.



2.9 연습문제 C

Problem 2.22. (8883번**) 주어진 다각형을 가로 x_s , 세로 y_s 의 직사각형 종이 여러 개로 덮으려고 한다. 이때 종이의 각 변은 x 축 또는 y 축과 평행해야 하고 주어진 다각형의 회전은 불가능하다. 오차를 피하기 위해 지도가 종이의 바깥으로 10^{-6} 까지 나가는 것을 허용한다. 필요한 종이의 수의 최소값을 구하여라. 다각형의 꼭짓점의 수 N 은 3 이상 50 이하의 정수이고 지도의 꼭짓점의 좌표 (x, y) 는 $0 \leq x \leq 10x_s$ 와 $0 \leq y \leq 10y_s$ 를 만족한다.

Problem 2.23. (10069번*, IOI14) 0부터 $M - 1$ 까지 번호가 부여된 M 개의 블록과 북쪽에는 서쪽 방향의 철도, 남쪽에는 동쪽 방향의 철도가 있는 길을 생각하자. 철도 사이의 블록 중 N 개에는 0번부터 $N - 1$ 번까지의 기차역이 하나씩 존재하며, 기차역이 있는 블록에는 북쪽 철로에서 남쪽 철로로 이어지는 세로 방향 철로 또는 남쪽 철로에서 북쪽 철로로 이어지는 세로 방향 철로가 있다. 또한 각 블록을 잇는 수평 방향 철로 사이에는 커넥터가 있다(회색 직사각형).



두 역 사이의 최단거리는 두 역 사이를 기차로 이동할 때 지나는 커넥터의 최소 개수이며, 모든 두 역 쌍에 대해 도달할 수 있음이 보장된다. 우리는 0번 기차역이 C 번 블록 위에 있고 0번 기차역에 있는 철로의 방향이 북에서 남이라는 사실과 Q 번의 두 역 사이의 최단거리에 대한

질의만으로 각 역이 있는 블록의 번호와 각 역에 있는 철로의 방향이 북에서 남인지, 남에서 북인지 알아내어야 한다.

- (서브테스크 1) $N \leq 100$ 이고 Q 에 제한이 없으며, 0번 역을 제외하면 나머지 역의 철로 방향은 남에서 북이다.
- (서브테스크 2) $N \leq 100$ 이고 Q 에 제한이 없으며, 0번 역의 오른쪽에 있는 철로 방향은 남에서 북이고, 왼쪽에 있는 철로 방향은 북에서 남이다.
- (서브테스크 3) $N \leq 5000$ 이고 Q 는 $N(N-1)/2$ 이하여야 한다.
- (서브테스크 4) $N \leq 5000$ 이고 Q 는 $3(N-1)$ 이하여야 한다.

Problem 2.24. (10720번) 둘레의 길이가 L 인 원형 레일을 L 등분 한 후 각 지점에 0부터 $L-1$ 까지 번호를 붙였다. 그 위에 N 마리의 시계 방향으로 이동하는 개미와 M 마리의 반시계 방향으로 이동하는 개미가 있다. 모든 개미의 이동 속도는 초당 1이고, 개미는 충돌 이후 반대 방향으로 같은 속도로 이동하게 된다. 모든 개미가 처음과 같은 위치 및 이동 방향을 갖기 위해 필요한 시간 T 를 출력하라. 단, 모든 개미는 구별되므로, 처음 그 자리에 있었던 개미가 다시 그 자리로 오는 때를 찾아야 한다. $L \leq 600$, $1 \leq N+M \leq 600$, $N \geq 1$, $M \geq 1$ 이다.

Problem 2.25. (15767번**) 이차원 좌표평면에 N 개의 원이 있다. 다음 과정을 통해 원을 지워나간다.

- (1) 지워지지 않았고 반지름이 가장 큰 원들 중 인덱스가 가장 작은 원을 찾는다. 이 원을 *target*이라 하자.
- (2) *target*과 겹치는 원을 모두 지운다. 겹친다는 것은 두 원의 내부 및 경계의 교집합이 공집합이 아님을 말한다.

위 과정을 원이 모두 사라질 때까지 반복하면 각 원은 특정 반복에서 지워진다. 각 원이 지워진 반복에서 *target* 원의 번호를 출력하는 프로그램을 작성하라. $N \leq 3 \cdot 10^5$ 이고 원의 중심 좌표는 절댓값이 10^9 이하의 정수, 원의 반지름은 10^9 이하의 자연수이다.

Problem 2.26. (18480번**) 겹치지 않는($r_i < l_{i+1}$) N 개의 닫힌 구간 $[l_i, r_i]$ 의 합집합 A 에 대해 크기가 4인 A 의 부분집합들 중 합이 s 인 부분집합의 개수를 구하라. $N \leq 400$, $0 \leq s \leq 8 \times 10^8$, $0 \leq l_i \leq r_i \leq 2 \cdot 10^8$ 이다.

Problem 2.27. (18738번**) 1부터 $2N$ 까지의 숫자가 정확히 한 번씩 적힌 $2N$ 장의 카드를 두 플레이어가 N 장씩 나누어 갖는다. 그리고 다음과 같은 규칙으로 게임을 한다.

- 홀수번째 라운드에서는 선 플레이어가 먼저 카드를 내고, 짝수번째 라운드에서는 후 플레이어 먼저 카드를 낸다.
- 각 라운드에서 두 번째로 카드를 내는 플레이어는 상대방이 낸 카드보다 큰 숫자가 적힌 카드를 1개 이상 갖고 있다면 그러한 카드(들) 중 하나를 내야 한다.
- 각 라운드에서 더 큰 숫자를 낸 플레이어가 그 라운드를 승리한다.

- 각 라운드에서 제출한 카드는 제거된다.

선 플레이어와 후 플레이어가 받은 N 장의 카드가 각각 주어질 때, 선 플레이어가 이길 수 있는 라운드의 수의 최댓값을 구하여라. $N \leq 1000$ 이다.

2.10 힌트

hint 2.1.

hint 2.2. 정렬의 정의로 ‘모든 i 와 $n > 0$ 에 대해 $arr[i] < arr[i + n]$ ’를 사용하라.

hint 2.3. Inversion Count가 무엇과 같다고 했었는지 생각해보라.

hint 2.4.

hint 2.5.

hint 2.6.

hint 2.7.

hint 2.8.

hint 2.9.

hint 2.10.

hint 2.11.

hint 2.12.

hint 2.13.

hint 2.14. N^2 알고리즘을 짜기에 N 이 크다고 생각할 수 있지만, 단순 연산은 상수가 작아 시간 안에 돌기 충분하다.

hint 2.15. 기본적으로 케이스에 따라 분류해서 풀어야 하는데, 예외 케이스가 많아서 틀리기 쉽다. 그나마 쉬운 구현 방법은, 답이 무조건 31자리 이하 문자열임을 생각해서, 모든 31자리 이하 문자열들을 나열한 수열에서 $n/2+1$ 번째 문자열 바로 앞에 있는 문자열 T 를 찾는다. 이렇게 하면, $n/2$ 번째 문자열과 같거나 더 사전순으로 뒤이고, T 번째 문자열과 **같거나** 더 사전순으로 앞인 가장 짧은 문자열을 찾는 문제로 바뀐다. 그러면 생각할 예외 케이스가 크게 줄어든다.

hint 2.16. 두 개의 단계별 힌트가 있다.

[1]

Inversion Count와 관련된 보존량을 생각해보자. 두 개씩 묶어서 이동시킬 때, Inversion Count의 ?는 항상 일정하다.

[2]

보존량은 Inversion Count의 2로 나눈 나머지와 같다. 정렬된 배열에서 0이므로, 처음 배열의 Inversion Count가 홀수이면 불가능하다. 반면, 짝수일 때는 항상 가능하다. 짝수일 때 항상 가능하다는 것은 정렬할 수 있는 방법 하나만 제시하면 충분하다. 이는 직접 생각해보자.

hint 2.17. (a, b) 와 (c, d) 를 마음대로 정할 수 있음에 주목하자. 일단, (a, b) , (c, d) 와의 거리가 같은 점의 개수 조건을 무시하고 최대한 쉬운 직선을 그리자. 그리고 직선을 약간 회전시키면, 개수 조건을 만족시키게 할 수 있다!

hint 2.18. 두 개의 단계별 힌트가 있다.

[1]

i 번째에 있는 어떤 수 k 가 첫 번째에 오게 하려면 $[1, i]$ 에 모두 k 보다 크거나 작은 수가 있어야 한다. 이것은 제일 처음에 '1 N I' 연산을 써서 $1, 2, 3, \dots, N$ 으로 정렬시키면 만족시킬 수 있다. '1 k D' 연산을 쓰면 내림차순 정렬된다.

[2]

첫 번째 수는 결정되었으므로, 두 번째 수를 같은 방법으로 결정한다. 그런데, '2 N I' 연산을 써서 다시 정렬시킨다면 N 번 이하의 연산으로 수행할 수 없을 것이다. 중요한 것은, 처음 정렬을 했기 때문에 여전히 $[2, i]$ 에 모두 k 보다 크거나 작은 수가 있다는 조건이 유지된다는 것이다. 왜 그런지는 스스로 고민해보자.

hint 2.19. 세 개의 단계별 힌트가 있다.

[1]

K 가 정렬된 배열에서 i 번째 수와 $i+1$ 번째 수의 차이의 최댓값이면 항상 정렬 가능하다. 정렬된 배열에서 a 번째에 해당하는 수와 정렬된 배열에서 $b(> a)$ 번째에 해당하는 수를 바꾸려면, a 와 $a+1$ 번째를 위치를 바꾸고, $a+1$ 번째와 $a+2$ 번째를 바꾸고, ... 을 반복하면 되기 때문이다.

[2]

하지만, 배열 자체가 굳이 교환이 필요없는 고립된 몇 개의 구간으로 나뉘진다면 어떨까? $2, 1, 102, 101$ 순으로 나열된 배열에서 2와 101은 바꿀 필요가 없기 때문에 답은 99가 아닌 1이 된다. 여기서 $2, 1$ 과 $102, 101$ 은 두 구간 사이에서 교환이 필요없는 고립된 구간이다.

[3]

고립된 구간은 정렬된 배열의 각 수의 원래 위치를 알면 쉽게 찾을 수 있다. 이것은 $1, 2, \dots, N$ 이 섞인 순열(permutation)으로 나올텐데, 순열에서 원래 그 구간에 속한 값들끼리만 서로 섞이게 된다. 예를 들어 $\dots, 6, 7, 8, 5, \dots$ 와 같이 있다면, $6, 7, 8, 5(5, 6, 7, 8)$ 는 고립된 구간이다.

hint 2.20. 두 개의 단계별 힌트가 있다.

[1]

동치 관계(equivalence relation)를 생각하자. 정의와 성질을 모르는 사람들은 아래의 설명을

참고하라.

이항 관계는 어떤 집합 S 에 대해 $\{(x, y) \mid x \in S, y \in S\}$ 의 부분집합인 R 을 말한다. 이항 관계 중에서 다음 세 조건을 만족하는 관계를 특별히 동치 관계라고 한다.

- (1) $(i, i) \in R$
- (2) $(i, j) \in R$ 이면 $(j, i) \in R$
- (3) $(i, j) \in R$ 이고 $(j, k) \in R$ 이면 $(i, k) \in R$

동치 관계가 R 에 대해서, 집합 S 를 동치 관계가 성립하는 것끼리 분할할 수 있다. 즉, $S_1 \cup S_2 \cup \dots \cup S_p = S$ 이면서 $S_i \cap S_j = \emptyset$ ($i \neq j$)이게 S 를 분할하면서, S_i 에 속하는 임의의 두 원소 사이에 동치 관계가 성립하고, S_i 에 속하는 임의의 원소와 S_j ($i \neq j$)에 속하는 임의의 원소 사이에 동치 관계가 성립하지 않게 할 수 있다.

[2]

i 번째 수와 j 번째 수를 바꿀 수 있을 때 (i, j) 를 R 에 포함시키면, R 은 $S = \{1, 2, \dots, n\}$ 에 대한 이항 관계 중에서 동치 관계가 된다. 즉, A_i 와 A_j 를 바꿀 수 있다는 것과 $(i, j) \in R$ 은 동치이다. 그리고, 결국 정렬된 배열 B 에 대해 A_i 위치에 B_i 가 와야 한다.

hint 2.21. 멀티탭이 꽂히는 순서를 3으로 나눈 나머지가 다른 두 멀티탭은 하나가 다른 하나에 완전히 포함될 수 없다. 이에 착안하면 정확히 세 개의 멀티탭만 제일 위에 보이게 할 수 있고, 이 경우가 최적이다.

hint 2.22. <https://icpcarchive.github.io/World%20Finals/ICPC%20World%20Finals/2013%20ICPC%20World%20Finals/solution.pdf>

hint 2.23. <https://blog.myungwoo.kr/46>

hint 2.24. 처음 개미에 임의로 번호를 부여한다. 그러면 두 개미간의 충돌은, 스쳐 지나가면서 서로의 번호를 교환하는 것과 같다. 답이 $L \cdot (N + M)$ 이하임이 보장되고 충돌은 L 초를 주기로 돌아오기 때문에, 충돌하는 시각을 모두 기록하고 단순 시뮬레이션하면 $O(L \cdot (N + M)^2)$ 에 구현할 수 있다.

hint 2.25. <https://jjang36524.tistory.com/26>

hint 2.26. 네 개의 단계별 힌트가 있다.

[1] 자주 쓰이는 트릭으로 다항식을 활용하는 방법이 있다. 어떤 수 a 를 뽑는 것을, 다항식에서 x^a 을 선택하는 것으로 바꾸면, 합이 s 가 되게 하는 경우의 수는 다항식들을 곱한 다항식에서 x^s 의 계수가 된다.

[2] 집합을 고르는 것이므로 같은 수의 선택은 불가하고 고르는 순서는 중요하지 않다. 따라서, $A(x) = \sum_{1 \leq i \leq n} (x^{l_i} + x^{l_i+1} + \dots + x^{r_i})$ 라 두고 포함과 배제의 정리를 사용하자.

[3] 전체 경우를 (a, b, c, d) 와 같이 전부 다른 경우, (a, a, b, c) 와 같이 정확히 두 개만 같은 경우, (a, a, b, b) 로 두 개씩 같은 경우, (a, a, a, b) 로 세 개만 같은 경우, (a, a, a, a) 로 네 개 다 같은 경우로 나누자. 즉,

$$A(x)^4 = (a, b, c, d) + (a, a, b, c) + (a, a, b, b) + (a, a, a, b) + (a, a, a, a)$$

가 된다. 이제 각각을 구하자.

- (a, a, a, a) : 같은 수를 정확히 네 개 선택하는 것은 $A(x^4)$ 으로 쓸 수 있다. 즉, $A(x^4)$ 과 같다.
- (a, a, a, b) : $a = b$ 인 경우를 무시하면 같은 수를 정확히 세 개 선택하고, 어떤 수를 한 번 선택하는 것은 $A(x^3)A(x)$ 가 된다. 그런데, $a = b$ 인 항은 $A(x^4)$ 으로 쓸 수 있고 $a \neq b$ 인 항은 지수가 달라 한 번씩 나타나므로, $A(x^3)A(x) - A(x^4)$ 가 된다. 순서를 바꾸는 방법의 수 $\frac{4!}{3!} = 4$ 를 곱하면 (a, a, a, b) 가 나온다.
- (a, a, b, b) : $a = b$ 인 경우를 무시하면, 같은 수를 정확히 두 개 선택하는 것을 두 번 반복하는 것은 $A(x^2)A(x^2)$ 가 된다. 그런데, $a = b$ 인 항은 $A(x^4)$ 이고, $a \neq b$ 인 항은 지수가 같아 두 번씩 나타나므로, $\frac{1}{2} [A(x^2)^2 - A(x^4)]$ 이 된다. 순서를 바꾸는 방법의 수 $\frac{4!}{2!2!} = 6$ 을 곱하면 (a, a, b, b) 가 나온다.
- (a, a, b, c) : a, b, c 가 모두 달라야 한다는 조건을 무시하면 $A(x^2)A(x)A(x)$ 가 된다. 여기서 $b \neq c$ 를 적용하면 위와 같은 방식으로 $\frac{1}{2}A(x^2) [A(x)^2 - A(x^2)]$ 가 된다. 이제 $a \neq b$ 와 $a \neq c$ 를 생각하면, (a, a, a, b) 에서 구했던 항인 $A(x^3)A(x) - A(x^4)$ 을 추가로 빼면 된다. 이렇게 구한 항에 순서를 바꾸는 방법의 수 $\frac{4!}{2!} = 12$ 를 곱하면 (a, a, b, c) 가 나온다.

이렇게 구한 (a, b, c, d) 를 $4! = 24$ 로 나누면 원하는 답이 된다. 즉,

$$\frac{1}{24} [A(x)^4 - 6A(x^2)A(x)^2 + 3A(x^2)^2 + 8A(x^3)A(x) - 6A(x^4)]$$

이다.

[4] [3]의 식에서 뒤 네 개의 항은 $A(x) = \sum_{1 \leq i \leq n} \left(\frac{x^{r_i+1} - x^{l_i}}{x-1} \right)$ 로 두면 항의 개수가 $2n$ 개인 다항식 세 개의 곱셈이므로 $O(n^3)$ 에 답을 구할 수 있다. 나눗셈은 $x^k - 1$ 꼴의 다항식으로 나누는 것만 처리하면 되고, 나누어 떨어짐이 보장되므로, 곱셈 공식을 이용해 해결할 수 있다.

제일 앞 항은 meet in the middle로 해결한다. 먼저, x^k 을 $(x-1)^4$ 으로 나눈 몫은 $\binom{3}{3} \cdot x^{k-4} + \binom{4}{3} \cdot x^{k-5} + \dots$ 로 표현되기 때문에, $A(x)^4$ 에 포함된 $c \cdot x^k$ 에 대해 $c \cdot \binom{k-1-s}{3}$ 을 다 더하면 된다. 이것을 meet in the middle로 해결하려면 $A(x)^2$ 에 포함된 $c \cdot x^k$ 와 $A(x)^2$ 의 곱인 다항식에서 답을 구하고, k 를 증가시키며 빠른 시간이 이 값을 업데이트해야 한다. 이것은

$$\binom{a+b}{k} = \sum_{i=0}^k \binom{a}{i} \binom{b}{k-i}$$

을 이용하여 해결할 수 있다. $A(x)^2 = \sum d \cdot x^l$ 로 쓰면, 빠르게 업데이트해야 하는 값은 $T_3 =$

$\sum d \cdot \binom{k+l-1-s}{3}$ 가 되는데, 위 식을 사용하면 $T_2 = \sum d \cdot \binom{k+l-1-s}{2}$, $T_1 = \sum d \cdot \binom{k+l-1-s}{1}$, $T_0 = \sum d \cdot \binom{k+l-1-s}{0}$ 와 k 의 증가량을 Δk 에 대한 식으로 $O(1)$ 의 시간에 T_3, T_2, T_1, T_0 모두 업데이트 할 수 있게 된다.

hint 2.27. <https://qoj.ac/contest/446/problem/1441/statistics>

Chapter 3

구간 합

구간 합은 어떤 배열의 s 번째 인덱스 값부터 e 번째 인덱스의 값을 빠르게 구하는 방법을 말한다. 만약 반복문을 이용해 전부 다 더한다면 $e - s$ 번의 연산이 필요하겠지만, 구간 합을 이용하면 1번의 연산으로 해결할 수 있다. 구간 합은 단순하지만 많은 PS 문제, 특히 동적계획법 문제에서 자주 사용되므로 꼭 숙지하도록 하자.

3.1 부분 합을 구하는 문제

다음 문제를 생각해보자.

Example 3.1. (11659번) 길이 N 인 어떤 배열 A 에서 다음 쿼리 Q 개를 해결하는 프로그램을 작성하라.

- 쿼리: $A_s + A_{s+1} + \dots + A_e$ 를 구하여라.

만약 반복문을 통해 구한다면 $O(NQ)$ 의 시간복잡도를 가질 것이다. 그런데 잘 생각해보면 낭비되는 계산이 있다. 만약 $s = 3, e = 10$ 인 쿼리와 $s = 4, e = 11$ 인 쿼리를 처리해야 한다고 생각해보자. 그러면 $s = 4$ 에서 $e = 10$ 인 부분의 덧셈은 두 번 진행되고 이는 매우 불필요하다!

이 중복 문제를 해결하기 위해 누적 합 S_i 를 생각해보자. 누적 합 S_i 는 $S_i - S_{i-1} = A_i, S_0 = 0$ 으로 정의되는 수열이다. S_i 의 의미는 1번째부터 i 번째까지 수의 합이다. 이를 이용하면 s 번째 수부터 e 번째 수까지의 합을 $S_e - S_{s-1}$ 로 구할 수 있다. 구현은 다음 코드 Code 3.1을 보라.

Code 3.1. 일차원 부분 합

```
1 #include <bits/stdc++.h>
```

```

2  using namespace std;
3
4  int n, arr[101010], sum[101010], q;
5
6  int main() {
7      cin >> n;
8      for (int i = 1; i <= n; i++) cin >> arr[i];
9      for (int i = 1; i <= n; i++) sum[i] = sum[i - 1] + arr[i];
10
11     cin >> q;
12     while (q--) {
13         int s, e;
14         cin >> s >> e;
15         cout << sum[e] - sum[s - 1] << "\n";
16     }
17
18     return 0;
19 }

```

3.2 이차원 부분 합

Example 3.1을 확장한 다음 문제를 생각해보자.

Example 3.2. (11660번) 크기 $N \times M$ 인 어떤 격자 A 에서 다음 쿼리 Q 개를 해결하는 프로그램을 작성하라.

- 쿼리: $\sum_{i=s}^e \sum_{j=l}^r A_{i,j}$ 을 구하여라.

이 문제에도 똑같이 부분 합의 원리를 적용할 수 있다. $S_{i,j}$ 를 $(1,1)$ 부터 (i,j) 까지의 합이라고 정의하자. 그러면 (s,l) 부터 (e,r) 까지의 합은 다음 식으로 구할 수 있다.

$$S_{e,r} - S_{s-1,r} - S_{e,l-1} + S_{s-1,l-1}$$

위 식이 성립한다는 것은 그림을 그리면 쉽게 알 수 있을 것이다. 첫 번째 $S_{e,r}$ 에 해당하는 영역을 칠하고 $S_{s-1,r}$ 영역에 해당하는 영역과 $S_{e,l-1}$ 에 해당하는 영역을 지워보라. 그러면 $S_{s-1,l-1}$ 에 해당하는 영역이 두 번 지워지므로 다시 더해주자. 그러면 원하는 영역이 된다.

마지막 남은 문제는 $S_{i,j}$ 를 구하는 것이다. 가장 쉬운 방법은 각 가로줄마다 부분 합을 구하고, 만들어진 새로운 배열에서 각 세로줄에 대해 부분 합을 구하는 것이다. 이 과정을 통해

만들어진 배열의 (i, j) 번 칸은 $(1, 1)$ 부터 (i, j) 까지의 합이 된다. 구현은 다음 코드 Code 3.2를 보라.

Code 3.2. 이차원 부분 합

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, arr[1010][1010], sum[1010][1010], q;
5
6  int main() {
7      cin >> n;
8      for (int i = 1; i <= n; i++) {
9          for (int j = 1; j <= m; j++) {
10             cin >> arr[i][j];
11         }
12     }
13     for (int i = 1; i <= n; i++) {
14         for (int j = 1; j <= m; j++) {
15             sum[i][j] = sum[i][j - 1] + arr[i][j];
16         }
17     }
18     for (int i = 1; i <= n; i++) {
19         for (int j = 1; j <= m; j++) {
20             sum[i][j] = sum[i - 1][j] + sum[i][j];
21         }
22     }
23
24     cin >> q;
25     while (q--) {
26         int s, e, l, r;
27         cin >> s >> l >> e >> r;
28         cout << sum[e][r] - sum[s - 1][r] - sum[e][l - 1] + sum[s - 1][l
29             - 1] << "\n";
30     }
31     return 0;
32 }
```

3.3 시간복잡도 줄이기

부분 합이 중요한 이유는 시간복잡도를 획기적으로 줄일 수 있기 때문이다. 다음 문제를 보자.

Example 3.3. (13586번) ‘Go-’ 게임은 $N \times N$ 의 바둑판 위에 흑과 백 모두 P 개의 돌을 놓은 후, 바둑판에 있는 모든 부분 **정사각형**들 중 자기 색의 돌을 1개 이상 포함하며 상대 색의 돌을 포함하지 않는 정사각형의 개수가 더 많은 사람이 이기는 간단한 바둑 변형 게임이다. 바둑판의 정보가 주어질 때 흑과 백에 대해 정사각형의 개수를 계산해 출력하라. $N \leq 500$, $P \leq \min(\frac{N}{2}, 500)$ 이다.

부분 합을 사용하지 않으면, 고려해야 하는 정사각형의 수는 $O(N^3)$ 이고 각 돌의 개수를 세는데 $O(N^2)$ 의 시간이 필요하므로 $O(N^5)$ 이 된다. 하지만, 부분합을 사용하면 $O(1)$ 의 각 돌의 개수를 셀 수 있어 $O(N^3)$ 에 답을 구할 수 있다.

3.4 누적 합의 역연산

다음 문제를 보자.

Example 3.4. (10713번) i 번 철도역과 $i+1$ 번 철도역 사이를 잇는 i 번 노선은 IC카드 구매 없이 A_i 원에 이용하거나 IC카드 구매 후 A_i 원보다 싼 가격인 B_i 원에 이용할 수 있다. IC카드의 구매 비용은 C_i 이며 IC카드는 노선마다 고유하다. 여러분은 여행 시작 전 일부 노선의 IC카드를 구매해 여행에 드는 교통비를 최소화하고 싶다. M 일 동안 여행할 도시의 목록 P_j 가 주어질 때, 여행 시작 전 IC카드를 사는 비용을 포함하여 필요한 교통비의 최솟값을 출력하라. $2 \leq N \leq 10^5$, $2 \leq M \leq 10^5$, $1 \leq B_i < A_i \leq 10^5$, $1 \leq C_i \leq 10^5$ 이다.

때로는 누적 합을 통해 원하는 배열을 얻어야 하는 경우가 있다. 다음 관찰을 생각하자.

Observation. $A_0 = 0$ 이라 하고, $B_i = A_i - A_{i-1}$ 로 정의한 배열에 대해, $\sum_{i=1}^k B_i = A_k$ ($k \geq 1$)가 성립한다.

결국 얻어야 하는 배열 A_i 에 대해, i 번째에 $A_i - A_{i-1}$ 을 저장하면 누적 합을 시행했을 때 A_i 가 남게 된다. 이것을 활용하면 시간복잡도를 줄일 수 있다.

일단, IC카드를 구매하는 것이 유리할 조건은 i 번 노선을 타는 횟수 x 에 대해 $A_i \cdot x > B_i \cdot x + C_i$ 와 동치이다. 즉, i 번 노선을 타는 횟수를 빠르게 구하면 문제를 해결할 수 있다. 이제 i 번 노선을 타는 횟수를 생각해 보면 P_{j-1} 부터 P_j 까지는 순차적으로 이동해야 하기 때문에, $[P_{j-1}, P_j)$ 또는 $[P_j, P_{j-1})$ 에 모두 1을 더해야 한다.

만약 직접 모든 칸에 1씩 더한다면 $O(NM)$ 의 시간이 필요하다. 하지만 연속된 구간에 전부 같은 수를 더하기 때문에, $B_i = A_i - A_{i-1}$ 에 해당하는 차이가 바뀌는 지점은 양 끝 두 지점이 전부이다. 따라서 B_i 는 $O(M)$ 의 시간에 계산할 수 있다. 즉, A_i 를 $O(N + M)$ 의 시간에 구할

수 있다.

3.5 연습문제 A

Problem 3.1. Code 3.1의 시간복잡도는 무엇인가?

Problem 3.2. Example 3.1에서 A_i 를 x 로 변경하는 새로운 종류의 쿼리가 추가된다면 문제를 어떻게 해결해야 할까? (이는 Part VIII에 나오는 내용으로, 답을 찾지 못해도 상관없다. 궁금한 사람은 Part VIII을 먼저 보라.)

Problem 3.3. Code 3.2의 시간복잡도는 무엇인가?

Problem 3.4. 삼차원 부분 합은 식이 어떻게 될까? d 차원에서는?

3.6 연습문제 B

Problem 3.5. (8694번*) N 개의 물간에 w_i 만큼의 물이 들어 있다. **인접한** 물컵에 물을 옮기는 것만으로 모든 물컵에 든 물의 양을 똑같이 하려고 할 때 필요한 최소 물 이동 횟수를 출력하라. 물컵의 최대 크기는 무한하다고 가정할 수 있고 w_i 의 합은 N 의 배수이다. $N \leq 10^6$ 이고 w_i 는 10^6 인 정수이다.

Problem 3.6. (11525번) 분모가 N 이하의 자연수이고 크기가 0 이상 1 이하인 기약분수를 크기 순으로 나열한 수열을 N 번째 페리 수열이라고 한다. N 번째 페리 수열의 길이를 구하는 Q 개의 쿼리를 처리하는 프로그램을 작성하라. $N \leq 10000$, $Q \leq 10000$ 이다. (Hint. 오일러 토션트 함수)

Problem 3.7. (18628번*) 각 칸에 알파벳 소문자가 하나 씩 적혀 있는 $N \times M$ 격자의 좌상단에서 우하단까지 오른쪽 또는 아래로만 이동하여 도달하는 서로 다른 경로 세 개가 경로상에 있는 알파벳을 나열한 문자열이 같도록 잘 고를 수 있는지 판별하라. $N \leq 3000$, $M \leq 3000$ 이다.

Problem 3.8. (22046번, IOI21) A, T, C로만 이루어진 두 DNA 염기서열 A 와 B 사이의 돌연변이 거리는 A 의 염기서열의 두 문자를 바꾸는 작업만으로 염기서열 B 를 만들기 위해 필요한 최소 작업 횟수로 정의된다. 여러분은 길이 N 의 염기서열 A 와 B 가 주어질 때 Q 번 (x, y) 를 입력받아 부분 염기서열 $A[x..y]$ 와 $B[x..y]$ 사이의 돌연변이 거리를 구해야 한다.

- (서브테스크 1) $N \leq 10^5$, $Q \leq 10^5$, $y - x \leq 2$
- (서브테스크 2) $N \leq 10^5$, $Q \leq 500$, $y - x \leq 1000$, A 와 B 를 이루는 문자는 A, T 뿐이다.
- (서브테스크 3) $N \leq 10^5$, $Q \leq 10^5$, A 와 B 를 이루는 문자는 A, T 뿐이다.
- (서브테스크 4) $N \leq 10^5$, $Q \leq 500$, $y - x \leq 1000$

- (서브테스크 5) $N \leq 10^5$, $Q \leq 10^5$

Problem 3.9. (28167번*) $N + 1 \times M + 1$ 크기의 격자가 있다. $1 \leq i \leq N$ 과 $1 \leq j \leq M$ 에서 각 i 번 행에 대해 왼쪽 j 칸을 칠할 확률 p_{ij} 가 주어지고, 각 j 번 열에 대해 위쪽 i 칸을 칠할 확률 q_{ij} 가 주어진다. 그리고 $\sum_j p_{ij}$ 와 $\sum_i q_{ij}$ 는 1이다. 색으로 구분된 구역의 총 개수의 기댓값을 구하라. 같은 구역은 같은 색의 칸이 상하좌우로 인접하게 연결되어 있는 것에 한한다. $N \leq 1000$, $M \leq 1000$ 이고 모든 확률은 유리수에 대한 mod 998 244 353 표현으로 주어지며, 이 형식으로 출력해야 한다.

3.7 연습문제 C

Problem 3.10. (1792번) 다음 쿼리 N 개를 처리하라. $N \leq 50000$ 이다.

- 세 정수 a, b, d 가 주어질 때, $1 \leq x \leq a$, $1 \leq y \leq b$, $\gcd(x, y) = d$ 인 순서쌍 (x, y) 의 개수를 구하여라.
- $1 \leq d \leq a \leq 50000$, $1 \leq d \leq b \leq 50000$

Problem 3.11. N 개의 방이 있는 호텔에서 매 손님이 올 때마다 랜덤으로 비어 있는 인접한 두 개의 방을 할당한다. 이때 가능한 각 인접한 방 쌍이 할당될 확률은 모두 같다. 더 이상 빈 인접한 두 방이 존재하지 않을 때 k 번 방이 비어있을 확률을 mod $10^9 + 7$ 에 대한 표현으로 출력하라. 총 T 개의 케이스에 대해 (N, k) 쌍을 입력받아 답을 출력해야 한다.

- (14340번) $T \leq 100$, $N \leq 10^4$
- (14341번) $T \leq 100$, $N \leq 10^7$

Problem 3.12. (16001번) 길이 N 인 문자열이 주어질 때, 몇 개의 문자를 바꾸어 역순 문자열이 정순 문자열과 같은 팰린드롬을 만드려고 한다. 이때 비용은 다음과 같이 계산된다.

- 현재 포인터 p 가 i 번째 문자를 가리킨다면 i 번째 문자를 A_i 의 비용으로 원하는 알파벳으로 교체할 수 있다.
- 현재 포인터 p 가 i 번째 문자를 가리킬 때, j 번째 문자를 가리키게 하고 싶다면 $c \times |i - j|$ 의 비용이 필요하다.

처음 포인터 p 가 가리키는 문자가 k 번째 문자일 때 팰린드롬을 만들기 위해 필요한 최소 비용을 모든 $1 \leq k \leq N$ 에 대해 계산하라. $N \leq 10^6$, A_i 와 c 는 10^9 이하의 자연수이다.

Problem 3.13. (19509번**) T 번 N 이 주어질 때마다 다음을 만족하는 길이 N 인 수열을 하나 출력하라. $T \leq 200$, $N \leq 2000$ 이다.

- 수열의 모든 값은 $3(N + 6)$ 이하의 양의 정수여야 한다.
- $1 \leq l \leq r \leq N$ 을 만족하는 모든 $N(N + 1)/2$ 개의 정수 순서쌍 (l, r) 에 대해 l 번째 수부터 r

번째 수까지의 합을 구하면 모든 순서쌍에서 전부 다르다.

Problem 3.14. (20581번) 길이 N 의 바이너리 문자열($0, 1$ 로만 이루어진 문자열)은 총 2^N 개 있다. 길이 N 의 세 바이너리 문자열 S_1, S_2, S_3 와 음 아닌 정수 r_1, r_2, r_3 가 입력으로 들어올 때 다음 조건을 만족하는 길이 N 의 바이너리 문자열의 개수를 출력하는 프로그램을 작성하라. $N \leq 10000$ 이다.

- 두 바이너리 문자열 S, T 의 *바이너리 거리*를 $\sum_i |S_i - T_i|$ 로 정의할 때, S_i 과의 바이너리 거리가 r_i 이하인 i 가 존재해야 한다.

3.8 힌트

hint 3.1.

hint 3.2.

hint 3.3.

hint 3.4.

hint 3.5. 왼쪽부터 바로 오른쪽으로 물을 넘기되, 음수량을 넘겨도 상관이 없다. 순서를 조금 바꿔서 오른쪽에 물이 충분히 차면 그때 반대로 넘기면 되기 때문이다. 즉, 답의 상한은 $N - 1$ 이다.

hint 3.6. 1 이상 n 이하의 자연수 중에서 n 과 서로소인 수의 개수를 나타내는 함수를 오일러 토션트 함수라 하고, $\phi(n)$ 이라 쓴다. 이를 이용하면 분모가 N 이하인 페리 수열의 길이를 나타낼 수 있다. 참고로,

$$\text{factorization: } n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k} \implies \phi(n) = n \prod_{i=1}^k \left(1 - \frac{1}{p_i}\right)$$

가 성립한다.

hint 3.7. 두 개의 단계별 힌트가 있다.

[1]

같은 알파벳이 나열된 다른 경로가 나오려면, 어떤 칸을 기준으로 대각선 왼쪽 아래에 같은 알파벳이 있어야 한다. 따라서, 어떤 칸에 대해 바로 위 칸에 적힌 알파벳과 바로 왼쪽 칸에 적힌 알파벳이 같은 칸을 먼저 확인하자.

[2]

[1]에서 확인한 칸을 좋은 칸이라 하자. 그림을 그려보면 적어도 좋은 칸이 두 개 이상 존재해야 하고, 각각의 위치를 (r_1, c_1) 과 (r_2, c_2) 라 할 때, $r_1 < r_2, c_1 < c_2$ 또는 $(r_1, c_1) = (r_2, c_2 - 1)$ 또는 $(r_1, c_1) = (r_2 - 1, c_2)$ 이어야 한다.

hint 3.8. 주어진 구간에서 A, C, T의 개수가 다르면 불가능하다. 따라서 이 경우에는 -1을 리턴하면 된다. 이제 답이 있는 경우만을 생각하자.

어떤 두 문자에 대해 상호 위치가 변경된 경우에는 한 번의 시행으로 해결할 수 있다. 이외의 경우는 A, C, T가 순환하는 경우이므로 한 쌍당 두 번의 시행으로 해결할 수 있다. 이것이 최적이므로 누적합을 이용해 상호 위치 변경 횟수, 순환하는 개수를 세면 쿼리당 $O(1)$ 에 해결할 수 있다.

hint 3.9. 총 두 개의 단계별 힌트가 있다.

[1]

색칠된 같은 구역은 언제나 하나이다. 각 가로줄에서 왼쪽에서부터 적어도 하나는 칠하고, 각 세로줄에서 위쪽에서부터 적어도 하나는 칠하기 때문이다.

[2]

어떤 칸이 색칠되어 있다면, 적어도 위쪽 칸이나 왼쪽 칸 중 하나는 색칠되어 있음이 보장된다. 따라서, 색칠되지 않은 같은 구역과, 자신은 색칠되지 않았으면서 아래 칸과 오른쪽 칸이 모두 색칠된 칸은, 일대일 대응된다. 이러한 칸의 수는 누적 합을 사용하면 $O(NM)$ 에 빠르게 계산할 수 있다.

hint 3.10. 총 세 개의 단계별 힌트가 있다.

[1]

뫼비우스 반전 공식을 사용해야 한다. 뫼비우스 반전 정리에 의하면

$$g(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d)g\left(\frac{n}{d}\right)$$

이 성립하는데, $\epsilon(n) = \begin{cases} 1 & (n=1) \\ 0 & (n>1) \end{cases}$ 과 $1(n) = 1$ 에 대해 $1(n) = \sum_{d|n} \epsilon(d)$ 이므로 $\epsilon(n) = \sum_{d|n} \mu(d)1\left(\frac{n}{d}\right)$ 이다.

[2]

구하는 값은

$$\sum_{i=1}^{a/d} \sum_{j=1}^{b/d} \epsilon(\gcd(i, j))$$

이고, 뫼비우스 반전 공식에 의해

$$\sum_{i=1}^{a/d} \sum_{j=1}^{b/d} \sum_{e|\gcd(i, j)} \mu(e)$$

와 같고, e 를 기준으로 정리하면 i 와 j 가 모두 e 의 배수인 경우 $\mu(e)$ 가 더해질 것이므로

$$\sum_{e \geq 1} \left\lfloor \frac{a}{de} \right\rfloor \left\lfloor \frac{b}{de} \right\rfloor \mu(e)$$

과 같다.

[3]

[2]의 식을 그냥 계산한다면 $O(a/d)$ 의 시간이 소요되므로, $O(N \times a/d)$ 의 시간복잡도가 되어 TLE를 받는다. 여기서, $\lfloor \frac{a}{de} \rfloor$ 의 값의 종류의 수는 적음을 이용할 수 있다. 예를 들어 $\lfloor \frac{10}{e} \rfloor$ 을 e 에 따라 적으면 10, 5, 3, 2, 2, 1, 1, 1, 1, 1로 다섯 종류의 수만 나온다. 수의 종류의 수는 $O(\sqrt{a/d})$ 이므로(harmonic lemma) 같은 수를 묶어서 계산하면 $O(\sqrt{a/d})$ 의 시간에 [2]의 식을 계산할 수 있다.

hint 3.11. <https://github.com/kamyu104/GoogleCodeJam-2016/blob/master/World%20Finals/family-hotel.py>

hint 3.12. <https://tech.kakao.com/posts/354>

hint 3.13. 인용: https://m.blog.naver.com/PostView.naver?blogId=vermeil_&logNo=223146073298&navType=by

2년 간의 싸움 끝에 승리했다.

$x = (N \text{보다 큰 소수})$ 로 정하자. 그리고 누적 합 배열을 만드는데,

$$pf[i] = i * p * 2 + (i * (i + 1) / 2) * x$$

로 정의하자. 이는 우리가 만든 수열의 1 i번째 값의 합이다. 그리고 놀랍게도 저렇게 만들면 합이 안 겹친다 ㅋㅋ;

$pf[x] - pf[y] = pf[z] - pf[w]$ 를 풀면, 해는 $(x = z)$ and $(y = w)$ 가 된다.

이제 구현만 열심히 하면 끝

<https://github.com/jbkarvens/baekjoon/tree/main/%EB%B0%B1%EC%A4%80>

hint 3.14. <https://qoj.ac/contest/501/problem/1245/statistics>

Chapter 4

이분 탐색

탐색(search) 알고리즘은 어떤 배열에서 원하는 값이 있는지 판별하고, 있다면 몇 번째에 있는지 찾는 알고리즘을 말한다. 가장 단순한 탐색 알고리즘은 모든 배열의 값과 비교하여 $O(N)$ 에 탐색하는 방법이다. 이번 장에서는 $O(N)$ 보다 빠르게 탐색하는 방법을 배운다.

4.1 빠르게 탐색하기

다음과 같은 문제를 생각하자.

Example 4.1. 길이 N 의 어떤 수열 A 가 주어질 때, 다음과 같은 쿼리(query) Q 개를 처리하는 프로그램을 작성하라.

- 쿼리: 수 x_j 가 수열 A 에 존재하면 몇 번째에 존재하는지 출력하고, 존재하지 않으면 -1 을 출력하라.

모든 수와 같은지 비교하는 완전 탐색(complete search)으로 문제를 해결할 경우 $O(NQ)$ 의 시간이 걸릴 것이다. 하지만 만약 배열이 정렬되어 있다면 어떨까? 우리는 다음 관찰이 성립함을 확인할 수 있다.

Observation. k 번째 값보다 원하는 값 x_j 가 크면 $i < k$ 번째 값은 확인할 필요가 없고, 작으면 $i > k$ 번째 값은 확인할 필요가 없다.

이분 탐색(binary search)의 아이디어는 이 관찰로부터 비롯된다. 만약 k 를 x_j 가 있을 가능성이 있는 범위의 중간 지점으로 잡는다면, 한 번의 비교로 구간의 길이를 절반으로 줄일 수 있고, 적어도 $O(\log N)$ 번 안에 탐색을 완료할 수 있다.

이를 쉽게 이해하기 위해 up-down 게임을 생각해보라. up-down 게임은 술래가 100 이하의 숫자 x 를 생각하면 다른 플레이어가 숫자 y 를 말해 $x > y$ 이면 down, $x < y$ 이면 up이라고 말해주는 게임이다. 이 게임에서 최적 전략은 무엇일까? 당연히 처음에 50을 물어보고 답을 들을 것이다. 왜냐하면 up이라는 답을 들어도, down이라는 답을 들어도 남는 숫자의 개수가 가장 작은 숫자가 중간값인 50이기 때문이다. 여러분은 이미 이분 탐색을 사용하고 있다!

(누군가는 10을 물었을 때 down이라는 답을 들으면 더 빨리 수를 찾는 것 아니냐고 말하기도 한다. 하지만 반대로 생각해보라. 만약 up이라는 답을 들었다면 아직도 90개의 숫자가 가능한 후보이다. Problem Solving에서는 항상 최악 시간복잡도를 생각해야 한다는 사실을 잊지 말자.)

이제 이분 탐색을 구현을 해보자. 우리는 가능한 구간의 시작과 끝을 나타내는 변수 `st`, `ed`를 선언해야 한다. 그리고 그 중간 위치 `mid`와 현재 값을 비교해 구간을 줄이면 된다. 구현은 Code 4.1을 참고하라.

Code 4.1. 이분 탐색 알고리즘

```
1 void binary_search(int x) {
2     int x, ans = -1;
3     cin >> x;
4     int st = 1, ed = n;
5     while (st < ed) {
6         int mid = (st + ed) / 2;
7         if (x == arr[mid]) {
8             ans = mid;
9             break;
10        }
11        if (x < arr[mid]) ed = mid - 1;
12        if (x > arr[mid]) st = mid + 1;
13    }
14    cout << ans << "\n";
15 }
```

`while`문 하나로도 해결할 수 있을 정도로 알고리즘은 매우 단순하다. 하지만 이분 탐색은 수많은 문제에서 쓰이는 아주 중요한 알고리즘이므로 꼭 숙지하도록 하자.

4.2 STL 내장 함수

이분 탐색은 사용되는 경우가 매우 많기 때문에 STL에서 내장 함수로 지원한다. 두 종류의 함수 `std::lower_bound(start, end, value)`와 `std::upper_bound(start, end, value)`가 있다. 각각의 함수는 다음을 반환한다.

- `std::lower_bound(start, end, value)`: `[start,end)`에서 `value` 이상인 가장 앞의 수를 가리키는 포인터를 리턴한다. 없으면 `end`를 리턴한다.
- `std::upper_bound(start, end, value)`: `[start,end)`에서 `value` 초과인 가장 앞의 수를 가리키는 포인터를 리턴한다. 없으면 `end`를 리턴한다.

`std::sort`와 다르게 `compare` 인자가 없다. 즉, 비교 기준을 임의로 설정할 수 없다. 만약 다른 비교 기준이 필요하다면 이상과 초과는 < 연산자를 바탕으로 판단하므로 연산자 오버로딩을 사용하라.

다음 Code 4.2와 Code 4.3은 cppreference.com에서 제공하는 예시 코드이다. 예시 코드를 보고 사용방법을 숙지하자.

Code 4.2. `std::lower_bound` 예시 코드: https://en.cppreference.com/w/cpp/algorithm/lower_bound

```
1  #include <algorithm>
2  #include <iostream>
3  #include <vector>
4
5  struct PriceInfo { double price; };
6
7  int main()
8  {
9      const std::vector<int> data{1, 2, 4, 5, 5, 6};
10
11     for (int i = 0; i < 8; ++i)
12     {
13
14         auto lower = std::lower_bound(data.begin(), data.end(), i);
15
16         std::cout << i << " <= ";
17         lower != data.end()
18             ? std::cout << *lower << " at index " <<
19               std::distance(data.begin(), lower)
20             : std::cout << "not found";
```

```

20     std::cout << '\n';
21 }
22
23     std::vector<PriceInfo> prices{{100.0}, {101.5}, {102.5}, {102.5},
24                                     {107.3}};
25
26     for (double to_find : {102.5, 110.2})
27     {
28         auto prc_info = std::lower_bound(prices.begin(), prices.end(),
29                                           to_find,
30                                           [](const PriceInfo& info, double value)
31                                           {
32                                               return info.price < value;
33                                           });
34         prc_info != prices.end()
35             ? std::cout << prc_info->price << " at index " << prc_info -
36               prices.begin()
37             : std::cout << to_find << " not found";
38         std::cout << '\n';
39     }
40 }

```

Code 4.3. `std::upper_bound` 예시 코드: https://en.cppreference.com/w/cpp/algorithm/upper_bound

```

1  #include <algorithm>
2  #include <iostream>
3  #include <vector>
4
5  struct PriceInfo { double price; };
6
7  int main()
8  {
9      const std::vector<int> data{1, 2, 4, 5, 5, 6};
10
11     for (int i = 0; i < 7; ++i)
12     {
13         // Search first element that is greater than i
14         auto upper = std::upper_bound(data.begin(), data.end(), i);

```

```

15
16     std::cout << i << " < ";
17     upper != data.end()
18         ? std::cout << *upper << " at index " <<
19           std::distance(data.begin(), upper)
20           : std::cout << "not found";
21     std::cout << '\n';
22 }
23
24 std::vector<PriceInfo> prices{{100.0}, {101.5}, {102.5}, {102.5},
25                               {107.3}};
26
27 for (double to_find : {102.5, 110.2})
28 {
29     auto prc_info = std::upper_bound(prices.begin(), prices.end(),
30                                     to_find,
31                                     [](double value, const PriceInfo& info)
32                                     {
33                                         return value < info.price;
34                                     });
35     prc_info != prices.end()
36         ? std::cout << prc_info->price << " at index " << prc_info -
37           prices.begin()
38           : std::cout << to_find << " not found";
39     std::cout << '\n';
40 }

```

4.3 연습문제 A

Problem 4.1. 완전 탐색으로 Example 4.1을 해결하는 코드를 작성하라.

Problem 4.2. 이분 탐색으로 Example 4.1을 해결하는 코드를 작성하라. 시간복잡도는 무엇인가?

Problem 4.3. STL 내장 함수를 이용해 Example 4.1을 해결하는 코드를 작성하라.

4.4 연습문제 B

Problem 4.4. (17124번) 길이 N 의 배열 A 와 길이 M 의 배열 B 로 길이 N 의 배열 C 를 만드려고 한다. C_i 는 B 에 있는 수들 중 A_i 와의 차이가 가장 작은 수이다. 만약 이러한 수가 여러 개라면 크기가 가장 작은 수를 고른다. $\sum_{i=1}^N C_i$ 를 구하여라. $N \leq 10^6$, $M \leq 10^6$ 이다.

Problem 4.5. (19554번) $X \in [1, N]$ 인 X 를 찾아라. 단, 이 문제는 인터랙티브 문제로 입출력을 통해 채점 프로그램과 소통해야 한다.

- P 를 출력하여 질의한다. 그러면
- 채점 프로그램은 $P < X$ 이면 -1 을 출력한다.
- 채점 프로그램은 $P = X$ 이면 0 을 출력하고 프로그램을 종료한다.
- 채점 프로그램은 $P > X$ 이면 1 을 출력한다.

$N \leq 10^9$ 이며 질의 횟수는 50회 이하여야 한다. (hint. 첫 번째 인터랙티브 문제이다. 직접 제출하여 채점해보라. `cout` « `flush`로 버퍼를 비우는 것을 잊지 말자.)

Problem 4.6. (1056번*) 다음 세 연산으로 N 을 만들기 위해 필요한 최소 연산 횟수를 출력하라. $N \leq 10^{18}$ 이다.

- $N \rightarrow N + 1$
- $N \rightarrow N - 1$, $N > 1$
- $N \rightarrow N^x$, x 는 자연수

Problem 4.7. (1208번*) N 개의 정수로 이루어진 수열에서 몇 개를 골라 더한 값이 S 가 되는 경우의 수를 구하여라. $N \leq 40$ 이고 수열의 값은 절댓값이 10^5 이하인 정수, S 는 절댓값이 10^6 이하인 정수이다.

Problem 4.8. (5631번) N 개의 점의 집합 S 와 두 점 (a_x, a_y) 와 (b_x, b_y) 가 주어질 때 다음 쿼리 Q 개를 처리하는 프로그램을 작성하라. $N \leq 2 \cdot 10^5$, $Q \leq 20000$, 모든 점의 좌표는 20000 이하의 음 아닌 정수이다.

- 입력으로 주어지는 R_1 과 R_2 에 대하여 (a_x, a_y) 가 중심이고 반지름이 R_1 인 원에 속하는 S 의 원소의 개수와 (b_x, b_y) 가 중심이고 반지름이 R_2 인 원에 속하는 S 의 원소의 개수의 합을 출력하라. R_1 과 R_2 는 모두 13000 이하의 자연수이다.

Problem 4.9. (10425번) 주어진 21000자리 이하의 피보나치 수가 몇 번째 피보나치 수인지

출력하라. 피보나치 수의 점화식은 아래를 참고하라. 답이 10^5 이하임이 보장된다.

$$F_n = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F_{n-1} + F_{n-2} & n > 1 \end{cases}$$

Problem 4.10. (15425번*) 순서대로 1번부터 N 번까지 번호가 붙어 있는 N 개의 고층 빌딩이 있다. 각 고층 빌딩의 높이는 H_i 이고 고층 빌딩에서 얻을 수 있는 금화의 개수는 G_i 이다. 여러분은 다음 조건을 만족하도록 몇 개의 빌딩을 순서 없이 고를 수 있다.

- 고른 빌딩을 번호가 증가하도록 나열하면 높이가 단조증가해야 한다.
- 고른 빌딩에 있는 금화의 개수의 합은 K 이상이어야 한다.

위 조건을 만족하도록 고층 빌딩을 고르는 경우의 수를 출력하라. $N \leq 40$, $H_i \leq 10^9$, $G_i \leq 10^9$ 이다.

Problem 4.11. (22882번*) 정수로만 이루어진 $N \times N$ 의 이차원 배열에서 1행 1열에서 N 행 N 열까지 최단거리로 이동하는 경로를 하나 골라 길이 $2N - 1$ 의 수열 a_k 를 만드려고 한다. 이러한 과정으로 만들어지는 수열 중

$$\max_{1 \leq i \leq j \leq 2N-1} \left(\sum_{k=i}^j a_k \right)$$

의 값이 K 인 수열의 개수를 출력하라. $N \leq 20$ 이고 이차원 배열에 적힌 수는 절댓값이 10^9 이하인 정수이다.

Problem 4.12. (24047번*) 길이 N 의 배열을 버블 정렬로 오름차순 정렬하는 아래 의사 코드를 참고하여 K 번 교환 후 배열을 출력하라. K 번 교환 전에 정렬이 완료된다면 -1 을 출력하라. $N \leq 5 \cdot 10^5$, $K \leq 10^8$ 이고 배열의 각 원소는 10^9 이하의 자연수이다.

```
bubble_sort(A[1..N]) { # A[1..N]을 오름차순 정렬한다.
  for last <- N downto 2
    for i <- 1 to last - 1
      if (A[i] > A[i + 1]) then A[i] <-> A[i + 1] # 원소 교환
}
```

4.5 연습문제 C

Problem 4.13. (20084번) 숨겨진 길이 N 의 바이너리 문자열 T 가 있다. 여러분은 S 가 T 의 연속 부분문자열인지 판단하는 질의를 t 번 이내로 사용하여 T 를 알아내어야 한다.

- (서브테스크 1) $N \leq 5, t = 31$
- (서브테스크 2) $N \leq 100, t = 256$
- (서브테스크 3) $N \leq 1000, t = 1024$

Chapter 5

매개변수 탐색

매개변수 탐색(parametric search)은 최적해 문제(optimal problem)을 풀기 위한 강력한 도구로 이분 탐색을 이용해 최적해를 찾는 방법이다. 기본적으로 답을 탐색한다는 아이디어를 기반으로 하며, $O(\log X)$ 의 시간복잡도를 곱해 최적해 문제를 결정 문제(decision problem)으로 바꿀 수 있는 방법이다. 이번 장에서는 앞 장에서 배운 이분 탐색을 바탕으로 매개변수 탐색을 하는 방법을 배운다.

5.1 답을 탐색한다는 아이디어

이분 탐색이 어떤 값 x 가 정렬된 배열에 있는지 빠르게 탐색하는 방법이었다. 이 과정에서 x 와 $arr[mid]$ 를 비교하였다. 이를 일반적으로 확장한다면 어떤 조건 $f(x)$ 에 대해

- $f(mid)$ 가 참일 때 답이 mid 이상이고,
- $f(mid)$ 가 거짓일 때 답이 mid 미만이면

항상 이분 탐색의 원리를 적용할 수 있다. 이러한 관찰에 착안하여 답을 탐색하겠다는 아이디어가 매개변수 탐색이다. 구체적인 내용은 아래 Observation을 참고하자.

Observation. 조건 $f(x)$ 를 '답이 x 인 경우가 존재하는가?'로 잡자. 그러면 다음 조건을 만족할 때 답에 대한 이분탐색을 통해 최적해를 찾아낼 수 있다. 구체적인 부등식 조건은 가능한 값의 범위만 배수로 쪼아짐이 보장되면 어떤 것이든 상관없다.

- 답이 mid 인 경우가 존재할 때 최적해가 mid 이상이고,
- 답이 mid 인 경우가 존재하지 않을 때 최적해가 mid 미만이면

이러한 방식을 매개변수 탐색이라고 한다. 매개변수 탐색을 시행하면 최적해를 찾는 문제를 답이 x 일 수 있는지 판단하는 문제로 바뀐다. 이와 관련된 자세한 내용은 6.2절을 참고하라.

이제 예시 문제를 통해 구현 방법을 알아보자.

Example 5.1. 자연수로만 이루어진 어떤 배열 A 가 주어질 때, A_i 중 x 보다 큰 수의 개수가 k 개 이하가 되는 x 의 최솟값을 구하여라.

물론 Example 5.1은 일반적인 이분 탐색을 통해서도 풀 수 있을 것이다. 하지만 연습을 위해 매개변수 탐색을 통해 풀어보자. 먼저 매개변수 탐색의 조건을 만족하는지 확인해야 한다. $x = mid$ 일 때 조건을 만족하면 답은 mid 이하이다. 또한 $x = mid$ 일 때 조건을 만족하지 않는다면 답은 mid 초과이다. 따라서 조건을 만족함을 확인할 수 있다. 그리고 이 부등식 조건을 그대로 코드에 구현하면 된다. Code 5.1은 이를 바탕으로 구현한 코드이다. Code 5.1에서 `isAnswer(x)`는 x 가 답이 될 수 있는지 확인하는 함수이다.

Code 5.1. Example 5.1의 정답 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, arr[101010], k;
5  const int X = 1e9;
6
7  bool isAnswer(int x) {
8      int cnt = 0;
9      for (int i = 1; i <= n; i++) if (arr[i] > x) cnt++;
10     return cnt <= k;
11 }
12
13 int main() {
14     cin >> n;
15     for (int i = 1; i <= n; i++) cin >> arr[i];
16
17     cin >> k;
18     int st = 0, ed = X;
19     while (st < ed) {
20         int mid = (st + ed) / 2;
21         if (isAnswer(mid)) ed = mid;
22         else st = mid + 1;
23     }
24

```

```

25     cout << st << "\n";
26
27     return 0;
28 }

```

X 를 A_i 의 범위라 하면 가능한 답은 1과 X 사이이므로 `isAnswer` 함수가 호출되는 횟수는 $O(\log X)$ 번이고 전체 시간복잡도는 $O(N \log X)$ 가 된다.

5.2 결정 문제와 최적해 문제

PS의 문제는 크게 **결정 문제(decision problem)**와 **최적해 문제(optimal problem)**로 나눌 수 있다. 결정 문제는 어떤 조건을 만족하는 해가 존재하는지 판별하고 그 해를 출력하는 문제를 말하고, 최적해 문제는 어떤 조건을 만족하는 해들 중 가장 원하는 값이 큰 해를 찾아 출력하는 문제이다. 자명하게도 최적해 문제는 결정 문제보다 어렵다.

그렇다면 최적해 문제를 결정 문제로 환원할 수 있을까? 만약 어떤 최적해 문제의 답이 k 이하일 때 항상 가능성이 보장된다면, 1부터 차례대로 가능한지 확인해보면 되므로 결정 문제로 바꿀 수 있다. 즉, k 번 결정 문제를 해결하여 최적해 문제의 답을 구할 수 있다.

위 방식을 개선시키기 위해 명제 p_i 를 "답이 i 일 때 가능하다"라고 정의해보자. 그러면 $i \leq k$ 이면 p_i 는 참이고, $i > k$ 이면 p_i 는 거짓이다. 즉, 우리가 p_i 를 나열한 배열을 생각한다면 앞쪽은 전부 참이고, 뒤쪽은 전부 거짓이므로 **정렬된 배열**로 볼 수 있다(참인 경우 1을 적고, 거짓인 경우 0을 적는다고 생각하면 편하다).

따라서 이 배열 p_i 에 이분 탐색을 적용할 수 있고, 이것이 매개변수 탐색이다. 즉, 배열의 크기를 X 라고 하면 $O(\log X)$ 번의 결정 문제를 해결하여 최적화 문제의 답을 구할 수 있다. 또한 X 는 가능한 답의 범위가 될 것이다.

매개변수 탐색은 $O(\log X)$ 의 시간을 곱해 최적해 문제를 결정 문제로 만들어주는 강력한 도구이다. 물론 p_i 배열이 정렬된 상태, 즉 어떤 값을 기준으로 명제의 참/거짓이 뒤집혀야 한다.

5.3 연습문제 A

Problem 5.1. Example 5.1을 이분 탐색으로 풀어라.

Problem 5.2. N 개의 정수 A_i 가 주어진다. 다음 값이 K 이상이 되는 가장 큰 자연수 x 를 출력하라. $N \leq 10^6$, $1 \leq A_i \leq 10^9$ 이다.

$$\sum_{i=1}^N \max(A_i - x, 0)$$

Problem 5.3. N 개의 정수 A_i 가 주어진다. 다음 값이 K 이상이 되는 가장 작은 자연수 t 를 출력하라. $N \leq 10^6$, $1 \leq A_i \leq 10^9$ 이다.

$$\sum_{i=1}^N \left\lfloor \frac{t}{A_i} \right\rfloor$$

5.4 연습문제 B

Problem 5.4. (1166번) 한 변의 길이가 A 인 정육면체 N 개를 각 변이 평행하도록 $L \times W \times H$ 크기의 직육면체 박스에 모두 넣으려고 한다. 가능한 N, L, W, H 가 주어질 때 가능한 A 의 최댓값을 구하라. 절대 및 상대 오차는 10^{-9} 까지 허용한다. N, L, W, H 는 모두 10^9 이하의 자연수이다.

Problem 5.5. (3090번) 정수 N 개로 이루어진 수열 A_i 에서 수열의 한 수를 골라 1만큼 감소시키는 연산을 t 번 이내로 수행하여 인접한 수의 차이의 최댓값의 최솟값을 출력하라. 다른 말로,

$$\max_{1 \leq i < N} |A_{i+1} - A_i|$$

의 최솟값을 출력하라. $N \leq 10^5$, $t \leq 10^9$, $A_i \leq 10^9$ 이다.

Problem 5.6. (10227번, IOI10) 1부터 RC 까지의 자연수가 정확히 한 번씩 적힌 $R \times C$ 크기의 격자에서 홀수 H 와 W 가 주어질 때 $H \times W$ 크기의 부분 격자에 속한 원소의 중앙값의 최솟값을 출력하라. $R \leq 3000$, $C \leq 3000$ 이다.

Problem 5.7. (11233번) 세 수 a, b, c 가 주어질 때 정삼각형 내부의 점 중 세 꼭짓점으로부터의 거리가 a, b, c 인 점이 존재할 수 있는 정삼각형의 넓이를 출력하라. 만약 그러한 정삼각형이 없다면 -1 을 출력하면 된다. 고정 소숫점 방식으로 소숫점 셋째 자리까지 출력해야 한다. a, b, c 는 모두 100 이하의 양의 실수이다.

Problem 5.8. (15470번*) 중심이 같고 반지름이 다른 두 원이 있고, 반지름이 더 작은 원 위의 점 N 개가 있다. 이 점들을 변이 교차하지 않도록 이은 다각형의 각 변의 길이 l_i 와 반지름이 더 큰 원의 원주 길이 c 가 주어질 때 바깥 원과 다각형 사이의 면적을 출력하라. $N \leq 10$, $10 \leq c \leq 1000$ 이다.

Problem 5.9. (18543번*) 두 명의 플레이어가 다음 규칙으로 게임을 한다.

- 선 플레이어가 처음 선택권을 갖는다.
- 선택권을 가진 플레이어가 1부터 m 사이의 정수 중 하나를 뽑는다. 각 정수가 나올 확률은 모두 같다.
- 수를 뽑은 플레이어는 이 수를 상대에게 넘기고 자신의 선택권을 유지할지, 아니면 자신이

이 수를 갖고 상대방에게 선택권을 넘길지 선택한다.

두 플레이어가 모두 최적으로 플레이할 때, n 번의 숫자를 가진 이후 선 플레이어와 후 플레이어의 숫자 합 차이의 기댓값을 d_n 이라 하자. 그러면 $m \geq 3$ 에서 $\lim_{n \rightarrow \infty} d_n$ 의 값이 유리수로 존재함이 보장된다. 여러분은 m 이 주어질 때마다 $\lim_{n \rightarrow \infty} d_n$ 의 값을 $\text{mod } 10^9 + 7$ 에 대한 표현으로 출력해야 한다. $3 \leq m \leq 10^9$ 이고 m 이 주어지는 횟수 t 는 5×10^5 회 이하이다.

Problem 5.10. (20106번*, IOI13) 0번부터 $N-1$ 번까지의 번호가 붙여져 있는 일렬로 나열된 N 개의 문과 N 개의 스위치가 있다. 각 스위치는 어느 문에 연결되어 있는 스위치인지 모른다. 또 스위치를 켜를 때 문이 열려야 정상이지만, 일부 스위치의 경우에는 켜를 때 문이 닫히는 오류가 있다. 여러분은 최대 70000번 스위치를 조정한 후 처음으로 닫힌 문이 몇 번 문인지 확인할 수 있다. 이 결과를 토대로 i 번째 스위치가 문이 열리기 위해 켜져야 하는지(1) 꺼져야 하는지(0) 출력하고 i 번 스위치와 연결된 문의 번호도 출력해야 한다.

- (서브테스크 1) $N \leq 5000$, i 번 스위치는 i 번 문과 연결되어 있다.
- (서브테스크 2) $N \leq 5000$, 모든 스위치는 꺼졌을 때 문이 열린다.
- (서브테스크 3) $N \leq 100$
- (서브테스크 4) $N \leq 2000$
- (서브테스크 5) $N \leq 5000$

Problem 5.11. (23791번) 길이 N 의 두 배열 A 와 B 가 주어질 때, 다음 쿼리 Q 개를 처리하는 프로그램을 작성하라. $N \leq 10^5$, $Q \leq 10^5$ 이고 $1 \leq A_i \leq 2^{31} - 1$, $1 \leq B_i \leq 2^{31} - 1$ 이다.

- i, j, k 를 입력받아 A_1 부터 A_i , 그리고 B_1 부터 B_j 까지의 숫자들 중 k 번째로 작은 수를 출력하라. 다른 말로 A_1 부터 A_i , 그리고 B_1 부터 B_j 까지의 숫자들 중에서 x 이하인 수의 개수가 k 인 가장 작은 x 를 출력하라.

Problem 5.12. (23107번*) 주어진 길이 N 의 수열 a_i, b_i, c_i 에 대해 다음 조건을 만족하는 순서쌍 $S = (s_0, s_1, \dots, s_r)$ 을 모두 생각하자.

- r 은 자연수
- $s_0 = 0, s_r = N$
- $s_i < s_{i+1}$ ($0 \leq i < r$)

$f(S)$ 를 다음과 같이 정의할 때 가능한 모든 S 에 대해 $f(S)$ 의 최댓값을 출력하라. $N \leq 2 \cdot 10^5$ 이고 a_i, b_i, c_i 는 절댓값이 10^9 이하인 정수이다.

$$f(S) = \min_{0 \leq i < r} \left\{ b_{s_i} + c_{s_{i+1}-1} + \sum_{s_i \leq j < s_{i+1}} a_j \right\}$$

Problem 5.13. (27811번*) 양쪽 더미에 L 개와 R 개의 돌이 있다. i 번째 라운드에 더 많은

개수의 돌이 있는 더미에서 i 개의 돌을 가져간다. 만약 돌의 개수가 같으면 왼쪽 더미에서 i 개의 돌을 가져간다. 만약 i 번째 라운드에 양쪽 더미 모두에 있는 돌의 개수가 i 개 미만이면 돌을 가져가지 못하므로 i 번째 라운드가 진행되지 못하고 게임이 종료된다. 진행된 라운드의 숫자와, 게임 종료 후 양쪽 더미에 남은 돌의 개수를 출력하라. T 번 L 과 R 이 주어졌을 때 출력해야 한다. $T \leq 1000$, $L \leq 10^{18}$, $R \leq 10^{18}$ 이다.

5.5 연습문제 C

Problem 5.14. (1557번) 1이 아닌 제곱수로 나누어지지 않는 자연수를 크기 순으로 나열할 때 K 번째로 나타나는 수를 출력하라. $K \leq 10^9$ 이다.

Problem 5.15. (2814번) 소수 p 에 대해 가장 작은 소인수가 p 인 자연수를 크기 순으로 나열할 때 N 번째로 나타나는 수를 출력하라. $N \leq 10^9$, $p \leq 10^9$ 이다.

Problem 5.16. (5563번) 주어진 N 개의 점을 두 그룹으로 분할하려고 한다. 각 그룹에는 한 개 이상의 점이 포함되어야 한다. 같은 그룹에 속하는 임의의 두 점 사이의 거리의 최댓값의 최솟값을 출력하라. 두 점 (x_1, y_1) 과 (x_2, y_2) 사이의 거리는 $|x_1 - x_2| + |y_1 - y_2|$ 로 계산된다. $3 \leq N \leq 10^5$ 이고 점의 x 좌표와 y 좌표는 절댓값이 10^5 이하인 정수이다.

Problem 5.17. (9014번) 주어진 N 개의 점을 두 개의 기준점 (a_1, b_1) 과 (a_2, b_2) 중 적어도 하나와 연결하려고 한다. 그런데 각각의 기준점은 적어도 K 개의 점과 연결되어야 한다. 여러 분은 N 개의 점과 기준점과의 거리의 최댓값을 최소화해야 한다. 가능한 최솟값을 소숫점 첫째 자리에서 반올림한 값을 출력하라. 두 점 (x_1, y_1) 과 (x_2, y_2) 사이의 거리는 $|x_1 - x_2| + |y_1 - y_2|$ 로 계산된다. $2 \leq N \leq 10^5$, $K \leq N/2$ 이고 점의 x 좌표와 y 좌표는 절댓값이 10^5 이하인 짝수이다.

Problem 5.18. (16655번) N 명의 사람이 S 달러를 나누어 가지려고 한다. 하지만 N 명의 사람 각각이 요구하는 금액 a_i 의 합이 S 보다 커 문제가 되고 있다. 다음과 같이 공정한 분할과 올바른 분할을 정의할 때 공정한 분할의 결과를 출력하라. $N \leq 5000$, $a_i \leq 5000$, $S \leq \sum_{i=1}^N a_i$ 이다.

- 공정한 분할은 합이 S 인 수열 c_i 들 중 모든 (i, j) 쌍에 대해 (a_i, a_j) 에 대한 합 $c_i + c_j$ 의 올바른 분할이 (c_i, c_j) 인 수열이다.
- (x, y) 에 대한 합 z 의 올바른 분할은 $x + y \geq z$ 인 경우에 아래와 같이 정의된다.

$$\begin{cases} (\frac{z}{2}, \frac{z}{2}) & x \geq z, y \geq z \\ (\frac{x}{2}, z - \frac{x}{2}) & x \leq z \leq y \\ (z - \frac{y}{2}, \frac{y}{2}) & y \leq z \leq x \\ (\frac{x+z-y}{2}, \frac{y+z-x}{2}) & x \leq z, y \leq z \end{cases}$$

Problem 5.19. (18592번**) 중심이 (x_i, y_i, z_i) 이고 반지름이 r_i 인 N 개의 구를 생각하자. $N(N-1)/2$ 개의 (i, j) 쌍에 대하여 $d(i, j)$ 를 두 구 사이의 최단거리로 정의하자(겹치면 0이다). 수식으로는 아래와 같이 나타낼 수 있다.

$$d(i, j) = \max \left(0, \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2} - r_i - r_j \right)$$

모든 (i, j) 쌍에 대한 $[d(i, j)]$ 를 크기 순으로 나열할 때 가장 작은 k 개를 증가하는 순서로 출력하라. $N \leq 10^5$, $k \leq 300$, $0 \leq x_i, y_i, z_i \leq 10^6$, $1 \leq r_i \leq 10^6$ 이다.

Problem 5.20. (26610번) N 개의 돌이 있는 더미에서 i 번째 턴에 $1 \leq k \leq T$ 인 자연수 k 에 대해 $(X+i) \times k$ 개의 돌을 가져가야 하는 게임을 생각하자. 만약 더 이상 조건을 만족하게 돌을 가져갈 수 없으면 게임이 종료된다. 여러분은 다음 쿼리 Q 개에 대해 답해야 한다. $Q \leq 3 \times 10^5$, $1 \leq X < N \leq 5 \times 10^8$, $2 \leq T \leq 10^8$ 이다.

- 이 게임이 종료되었을 때 가장 적은 개수의 돌을 남겨야 하고 만약 그러한 방법이 여러 개라면 가장 빠르게 게임을 끝내야 한다. 최적의 방식으로 플레이했을 때 돌을 가져가는 횟수를 출력하라.

Chapter 6

병렬 이분 탐색

병렬 이분 탐색(parallel binary search)은 매개변수 탐색 과정을 여러 번 처리해야 할 때 빠르게 처리하기 위한 방법이다. 기본적으로 매개변수 탐색을 여러 번 처리할 때 같은 값 x 에 대해 x 가 답이 될 수 있는지 판단하는 것이 중복적으로 나타나고, 이 시간을 줄이겠다는 아이디어에서 만들어진 알고리즘이다. 이번 장에서는 6장에서 배운 매개변수 탐색을 바탕으로 병렬 이분 탐색을 하는 방법을 배운다.

* 주의: 병렬 이분 탐색은 뒤쪽 챕터의 알고리즘과 함께 쓰이는 경우가 전부이므로, 뒤의 내용을 공부하다가 필요한 시점에 다시 보는 것을 추천한다.

6.1 쿼리와 이분 탐색

Example 5.1에 쿼리를 추가한 문제를 생각하자.

Example 6.1. 자연수로만 이루어진 어떤 배열 A 가 주어질 때, 다음 쿼리 Q 개를 처리하는 프로그램을 작성하라.

- 쿼리: A_i 중 x 보다 큰 수의 개수가 k 개 이하가 되는 x 의 최솟값을 구하여라.

만약 매개변수 탐색을 Q 번 진행한다면 시간복잡도가 $O(NQ \log X)$ 가 될 것이다. 하지만, 병렬 이분 탐색을 사용할 경우 $O(N \log Q \log X + Q \log X)$ 로 줄일 수 있다.

각 쿼리의 k 를 q_i 라고 놓자. 그리고 각 쿼리마다 현재 가능한 값의 범위 $[s_i, e_i]$ 를 생각하자. 초깃값은 모든 i 에서 $s_i = 1, e_i = X$ 일 것이다. 이제 $x = m = (s + e)/2$ 에서 x 보다 큰 수의 개수를 세자. 이 개수보다 q_i 가 큰 쿼리는 s_i 가 바뀌고, q_i 가 그 개수 이하인 쿼리는 e_i 가 바뀔 것이다.

이제 각 $[s_i, e_i]$ 가 절반으로 감소하였다. 이와 동시에 가능한 $x = m$ 의 값은 두 가지가 되었다. 만약 이 두 가지 $x = m$ 에 대해 각각 $O(N)$ 의 시간으로 x 보다 큰 수의 개수를 계산한다면 시간복잡도의 개선이 전혀 일어나지 않을 것이다(Problem 6.1). 병렬 이분 탐색을 통해 시간 복잡도를 개선하려면 **한 번의 탐색으로 다양한 x 값에 대해 원하는 값(여기서는 x 보다 큰 수)을 구할 수 있어야 한다.**

이를 위해 이분탐색을 사용할 수 있다. 가능한 각 쿼리에서 m 의 값을 $x_1, x_2, \dots, x_q$ 라고 놓고 이를 정렬하자. 그리고 배열 c_i 에 값이 x_{i-1} 초과 x_i 이하인 수의 개수를 저장하자(편의상 $x_0 = 0, x_{q+1} = X$ 로 놓는다). 이 과정은 A_i 초과인 가장 앞에 있는 수 x_j 를 배열 x 에서 이분탐색을 통해 찾으면, $(x_{j-1}, x_j]$ 에 A_i 가 존재하므로 c_j 를 1 증가시키면 된다. 따라서 $O(N \log Q)$ 의 시간에 c_i 를 계산할 수 있다. 또한 c_i 의 정의에 의해 $i + 1$ 부터 $q + 1$ 까지의 c_i 의 부분 합이 각 x_i 보다 큰 수의 개수이므로 각 x_q 에 대한 답을 $O(Q)$ 의 시간에 찾을 수 있다. 구현은 다음 Code 6.1을 참고하라.

Code 6.1. Example 6.1의 정답 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, arr[101010], q, k[101010];
5  int s[101010], e[101010], c[101010], sum[101010];
6  const int X = (1<<30);
7
8  int main() {
9      cin >> n;
10     for (int i = 1; i <= n; i++) cin >> arr[i];
11     cin >> q;
12     for (int i = 1; i <= q; i++) cin >> k[i];
13
14     for (int i = 1; i <= q; i++) {
15         s[i] = 1;
16         e[i] = X;
17     }
18     while (s[1] < e[1]) {
19         for (int i = 1; i <= q + 1; i++) c[i] = sum[i] = 0;
20         vector< pair<int, int> > m;
21         m.push_back({0, 0});
22         for (int i = 1; i <= q; i++) m.push_back({(s[i] + e[i]) / 2, i});
23         m.push_back({X, q + 1});
24         sort(m.begin(), m.end());
25     }

```

```

26     for (int i = 1; i <= n; i++) {
27         int j = lower_bound(m.begin(), m.end(), make_pair(arr[i] +
28             1, 0)) - m.begin();
29         c[j]++;
30     }
31     for (int i = 1; i <= q + 1; i++) sum[i] = sum[i - 1] + c[i];
32     for (int i = 1; i <= q; i++) {
33         if (sum[q + 1] - sum[i] <= k[m[i].second]) e[m[i].second] =
34             m[i].first;
35         else s[m[i].second] = m[i].first + 1;
36     }
37
38     for (int i = 1; i <= q; i++) cout << s[i] << " ";
39
40     return 0;
41 }

```

정리하면 병렬 이분 탐색은 최적해 문제 쿼리를 업데이트 쿼리와 결정 문제 쿼리로 환원하는 것이다:

- 업데이트 쿼리: 결정 문제의 고정된 기준값 m 이 증가할 때, 결정 문제를 빠르게 해결하기 위해 저장해야 하는 값들을 갱신
- 결정 문제 쿼리: 주어진 m 에 대해 결정 문제를 해결

이처럼 업데이트 쿼리가 추가되기 때문에 업데이트가 있는 상황에서 주어진 문제를 빠르게 해결하기 위해 사용되는 자료구조들과 함께 쓰이는 경우가 많다.

6.2 연습문제 A

Problem 6.1. Example 6.1에 대한 병렬 이분 탐색 과정에서 Q 개의 쿼리에서 나온 모든 m 에 대해 $O(N)$ 의 시간으로 탐색한다면 시간복잡도가 어떻게 되는가?

Problem 6.2. Code 6.1에서 X 를 1«30으로 놓은 이유가 무엇인가?

Problem 6.3. Code 6.1에서 부분 합을 구간 $[i + 1, q + 1]$ 로 처리한 이유가 무엇인가? 왜 $[i + 1, q]$ 가 아닌가?

Problem 6.4. 사실 Example 6.1은 병렬 이분 탐색 없이 해결할 수 있다. 병렬 이분 탐색 없이

해결하는 코드를 짜고, 그 코드의 시간복잡도를 구하라.

6.3 연습문제 B

Problem 6.5. x_i 가 $x_{i-1} \cdot a + b$ 를 $10^9 + 7$ 로 나눈 나머진인 점화관계를 가진 길이 N 의 수열 x_i 에 대해 x_i 에서 q 번째로 작은 원소를 찾는 쿼리 Q 개를 처리하는 프로그램을 작성하라. 정확히 말하면 y 보다 작은 x_i 의 원소의 개수가 q 인 가장 작은 y 를 찾아야 한다. 단, x_i 를 모두 저장할만큼 메모리가 충분하지 않다.

- (12926번) $N \leq 10^6$, $Q \leq 100$, x_0, a, b 는 $10^9 + 7$ 미만의 음 아닌 정수
- (12921번) $N \leq 2 \cdot 10^6$, $Q \leq 1000$, x_0, a, b 는 $10^9 + 7$ 미만의 음 아닌 정수

6.4 연습문제 C

* 주의: 본 챕터의 연습문제 C는 뒤에 나오는 자료구조를 알아야 풀 수 있는 문제로 구성되어 있다. 만약, 이 책을 처음부터 읽고 있으며 뒤의 자료구조에 관한 지식이 없다면 연습문제 C를 보며 아이디어만 생각하거나 나중에 다시 풀어보자.

Problem 6.6. (8217번) 원이 M 분할되어 각각 1번부터 M 번까지 번호가 붙어 있다. 특히, 1번과 M 번은 붙어 있다. 그리고 각각은 N 명의 사람 중 한 명이 소유하고 있으며 이 정보는 입력으로 주어진다. 또한 Q 일 동안 연속된 부채꼴 $[l_i, r_i]$ 번에서 각각 a_i 만큼의 석유가 나온다 ($1 \leq i \leq Q$). 참고로 $l_i > r_i$ 이면 $[l_i, r_i]$ 는 $[l_i, m] \cup [1, r_i]$ 를 뜻한다. 각 사람이 자신이 가진 땅의 석유를 매일 모을 때, 각 사람마다 원하는 목표치인 p_j ($1 \leq j \leq N$) 이상의 석유를 얻기 위해 기다려야 하는 날의 수를 출력하라. $N \leq 3 \times 10^5$, $M \leq 3 \times 10^5$, $Q \leq 3 \times 10^5$ 이고 $a_i \leq 10^9$, $p_i \leq 10^9$ 이다.

Problem 6.7. (9877번) 10^9 이하의 음 아닌 정수가 적힌 $N \times M$ 격자에서 정수 T 가 주어질 때 각 셀에 대해 각 칸에 대해 다음 쿼리를 처리하라. $N \leq 500$, $M \leq 500$ 이다.

- 상하좌우로 인접한 칸 중 적힌 수의 차이가 D 이하인 칸으로 이동하는 것만 허용할 때 시작 칸 (a, b) 에서 도달할 수 있는 칸의 개수가 T 개 이상이 되도록 D 를 정하자. 가능한 D 의 최솟값을 출력하라.

Problem 6.8. (13952번) 방해물이 있는 $N \times N$ 격자에서 다음 쿼리 Q 개를 처리하라. $N \leq 1000$, $Q \leq 3 \times 10^5$ 이다.

- (x_a, y_a) 가 중심인 홀수 길이의 정사각형이 방해물에 걸리지 않고 상하좌우로 평행하게 이동하여 중심이 (x_b, y_b) 인 위치까지 이동할 수 있는 정사각형 길이의 최댓값을 출력하라. 그 어떤 정사각형도 불가능하면 0을 출력하면 된다.

Problem 6.9. (16976번) 컴퓨터를 한 대 갖고 있는 N 명의 사람이 M 종류의 게임 중 하나의 게임을 구매했다. 그리고 순차적으로 Q 번 두 사람이 가진 컴퓨터 사이에 랜선을 연결할 것이다. i 번 게임을 구매한 모든 사람의 컴퓨터가 연결되었을 때 i 번 게임을 시작할 때, 각 게임이 몇 개의 랜선을 연결한 이후에 시작될 수 있는지 출력하라. $N \leq 10^5$, $M \leq 10^5$, $Q \leq 10^5$ 이다.

Problem 6.10. (20654번) N 개의 액체가 있고, 각 액체는 맛 d_i 와 단위용량당 가격 p_i 가 정해져 있다. 이 액체 중 몇 개를 원하는 양만큼 넣어 음료를 만들고 싶다. 단, i 번 액체는 l_i 이하만큼 넣어야 한다. 이때 다음 쿼리 M 개를 처리하는 프로그램을 작성하라. $N \leq 10^5$, $M \leq 10^5$ 이고 d_i, p_i, l_i 는 10^5 이하의 자연수, g_i, L_i 는 10^{18} 이하의 자연수이다.

- 음료를 만드는데 필요한 액체의 가격이 g_j 이하이며 음료의 용량이 L_j 이상인 음료 중에서 음료에 포함되는 액체의 맛 d_i 의 최솟값의 가능한 최댓값을 출력하라.

Problem 6.11. (21095번*) 길이 N 의 수열 A_i 가 주어질 때, 다음 쿼리 Q 개를 처리하는 프로그램을 작성하라.

- l, r, k 가 주어질 때 모든 $1 \leq i \leq k$ 에 대해 $C_i + C_{(i \bmod k)+1} \leq W$ 이면서 $A[l..r]$ 에서 몇 개의 수를 지워 k 개의 수를 남긴 부분 수열인 C_i 가 존재하는 W 의 최솟값을 출력하라.

Problem 6.12. (25458번*) <https://www.acmicpc.net/problem/25458>

Problem 6.13. (27199번*) 주어진 홀수 길이의 배열에서 연속된 세 숫자 중 중앙값만 남기는 연산을 배열의 길이가 1이 될 때까지 해 남길 수 있는 수의 최댓값을 배열의 중앙값 평균이라고 하자. 길이 N 의 배열 A_i 와 Q 개의 (l, r) 쌍이 주어질 때 $A[l..r]$ 의 중앙값 평균을 출력하는 프로그램을 작성하라. $N \leq 50000$, $Q \leq 10^5$, $0 \leq A_i \leq 10^9$ 이고 모든 쿼리에서 $r - l + 1$ 은 홀수이다.

Chapter 7

투 포인터

투 포인터(Two pointer)는 한 배열의 어떤 부분 구간에 대한 문제를 풀 때, 두 개의 포인터를 사용하여 전체 구간을 탐색하지 않고 빠르게 문제를 해결하는 방식이다. 보통 제일 앞에 두 개의 포인터 p 와 q 를 잡고 두 포인터를 오른쪽으로 이동시키며 문제를 해결한다. 이 장에서는 투 포인터로 해결할 수 있는 문제가 무엇이고 어떻게 해결하는지 다룬다.

7.1 가능한 구간을 찾는 문제

다음과 같은 문제를 생각하자.

Example 7.1. (15565번) 0 또는 1로만 이루어진 길이 N 의 이진 배열에서 1의 개수가 k 개 이상인 구간들 중 가장 짧은 구간의 길이를 출력하라.

만약 가능한 $O(N^2)$ 개의 모든 구간을 고려한다면 부분 합을 사용할 때 각 구간이 조건을 만족함을 $O(1)$ 에 확인할 수 있으므로 시간복잡도가 $O(N^2)$ 이 된다. 하지만 투 포인터 알고리즘을 사용하면 $O(N)$ 으로 줄일 수 있다.

이를 위해 해야 하는 중요한 관찰 두 가지가 있다.

Observation. 다음이 성립한다.

- 구간 $[s, e]$ 가 조건을 만족한다면 $[s, e]$ 를 포함하는 구간은 확인 필요가 없다. $[s, e]$ 보다 길이가 길기 때문이다.
- 구간 $[s, e]$ 가 조건을 만족하지 않는다면 $[s, e]$ 에 포함되는 구간은 확인할 필요가 없다. 자명히 1의 개수가 k 개보다 작기 때문이다.

이제 고정된 s 에 대해 살펴봐야 할 e 의 범위를 생각해보자. 기본적으로 e 는 증가한다고 가정한다. 만약 $[s, e]$ 가 조건을 만족한다면 $i > e$ 인 $[s, i]$ 는 살펴볼 필요가 없다. 따라서 s 를 $s + 1$ 로 증가시키자. 반대로 $[s, e]$ 가 조건을 만족하지 않는다면 $i < e$ 인 $[s, i]$ 는 살펴볼 필요가 없다. 따라서 e 를 $e + 1$ 로 증가시키자. 이렇게 되면 각 과정에서 s 가 1 증가하거나 e 가 1 증가하므로 최대 $2N$ 번 반복되며, $O(N)$ 의 시간복잡도에 문제를 해결할 수 있게 된다. 구현은 Code 7.1을 참고하라.

Code 7.1. Example 7.1의 정답 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, arr[101010], k;
5  int sum[101010], ans;
6
7  int main() {
8      cin >> n;
9      for (int i = 1; i <= n; i++) cin >> arr[i];
10     cin >> k;
11
12     for (int i = 1; i <= n; i++) sum[i] = sum[i - 1] + arr[i];
13
14     int s = 1, e = 1;
15     ans = n;
16     while (true) {
17         if (e > n) break;
18         if (sum[e] - sum[s - 1] >= k) {
19             ans = min(ans, e - s + 1);
20             s++;
21         }
22         else e++;
23     }
24
25     cout << ans << "\n";
26
27     return 0;
28 }
```

투 포인터라는 이름은 s 와 e 를 인덱스가 아니라 배열의 s 번째 값과 e 번째 값을 가리키는 포인터로 생각하여 나온 이름이다. s 와 e 를 포인터로 생각할 수 있는 이유는 s 와 e 는 증가만 하고,

이러한 특징이 포인터가 증가해 다음 값을 가리키는 것과 비슷하기 때문이다.

7.2 투 포인터가 사용될 조건

그렇다면 투 포인터가 사용될 수 있는 조건은 무엇일까? 다시 위의 관찰로 돌아가보면 다음이 성립해야 함을 알 수 있다.

Observation. 구간 $[s, e]$ 를 평가하는 함수 $f(x)$ 와(값이 클수록 좋다고 가정) 구간 $[s, e]$ 가 조건을 만족하는지 여부를 리턴하는 함수 $g(x)$ 에 대해

- 어떤 구간 $[s, e]$ 의 $f(x)$ 보다 $[s, e]$ 를 포함하는 구간의 f 값이 더 작고,
- 어떤 구간 $[s, e]$ 의 $g(x)$ 가 거짓이면 $[s, e]$ 에 포함되는 구간의 g 값이 거짓이어야 한다.

Example 7.1은 $f(x) = -(e - s + 1)$ (음의 부호는 작을수록 좋기 때문에 붙였음), $g(x) = (1 \text{의 개수} \geq k)$ 인 상황으로 볼 수 있다.

마지막으로 임의의 투 포인터 문제의 시간복잡도를 논의해보자. 매 과정에서 $g([s, e])$ 가 성립하는지 확인하고, 전체 과정은 $2N$ 번 이하이므로 $g([s, e])$ 를 계산하는 시간복잡도가 $T(n)$ 이면 전체 시간복잡도는 $N \cdot T(N)$ 이다.

7.3 연습문제 A

Problem 7.1. Example 7.1은 부분 합을 사용하지 않아도 문제를 해결할 수 있다. 부분 합을 사용하지 않고 Example 7.1을 해결하는 코드를 작성하라.

7.4 연습문제 B

Problem 7.2. (1806번) 길이가 N 인 수열의 연속한 부분수열들 중 수열의 원소들의 합이 S 이상인 가장 짧은 부분수열의 길이를 출력하라. $N \leq 10^5$, $S \leq 10^9$ 이고 각 수열의 원소는 10000 이하의 자연수이다.

Problem 7.3. (2470번, KOI10) N 개의 수가 주어질 때 두 개를 골라 합을 0에 가장 가깝게 만드려고 한다. 그때 고른 두 수를 **아무거나** 하나 출력한다. $N \leq 10^5$ 이고 각 수는 절댓값이 10^9 이하인 정수이며 모두 다르다.

Problem 7.4. (2473번, KOI10) N 개의 수가 주어질 때 세 개를 골라 합을 0에 가장 가깝게 만드려고 한다. 그때 고른 세 수를 **아무거나** 하나 출력한다. $N \leq 5000$ 이고 각 수는 절댓값이 10^9 이하인 정수이며 모두 다르다.

Problem 7.5. (7453번) N 개의 수가 주어질 때 네 개를 골라 합이 0이 되는 방법의 수를 출력하라. $N \leq 4000$ 이고 각 수는 절댓값이 2^{28} 이하인 정수이다.

Problem 7.6. (13364번*) N 개의 점을 순서대로 연결하여 그린 $N - 1$ 개의 선분의 집합과 M 개의 점을 순서대로 연결하여 그린 $M - 1$ 개의 선분의 집합을 생각하자. 각각을 경로로 하여 두 물체가 이동한다. 즉, 첫 번째 물체는 첫 번째 점에서 N 번째 점의 방향으로, 두 번째 물체는 첫 번째 점에서 M 번째 점의 방향으로 이동한다. 두 물체가 같은 속도로 이동할 때 두 물체 사이의 거리가 가장 가까울 때 그 값을 출력하라. 단, 한 물체가 먼저 마지막 점에 도달하면 물체는 소멸한다고 가정한다. $N, M \leq 10^5$ 이고 각 점의 x 좌표와 y 좌표는 10000 이하의 음 아닌 정수이다.

Problem 7.7. (16598번) 연속된 출석 일수가 길수록 좋은 보상을 주는 앱이 있다. 그런데 버그가 나서 추가로 P 일이 출석으로 인정되는 문제가 생겼다! 이 앱에 출석한 N 일의 날짜 d_i 가 주어질 때, 가능한 연속 출석 일수의 최댓값을 출력하라. 즉, 적절히 출석으로 인정될 P 일을 고를 때 가능한 최대 연속 출석 일수를 구하는 것이다. $N \leq 2 \cdot 10^5$, $P \leq 2 \cdot 10^5$, $0 \leq d_1 < d_2 < \dots < d_n \leq 10^6$ 이다.

Problem 7.8. (20085번*, IOI16) 자연수로 이루어진 길이 N 의 수열 a_i 와 범위 $[l, r]$ 이 주어진다. 이때 길이 N 의 수열에서 적절히 몇 개의 수를 골라 합이 $[l, r]$ 의 범위에 속하게 할 수 있는지 판단하고, 가능하다면 선택되는 수열의 수의 인덱스를 오름차순으로 출력하라. 단, $r - l \leq \max a_i - \min a_i$ 가 보장된다.

- (서브테스크 1) $n \leq 100$, $a_i \leq 100$, $1 \leq l \leq r \leq 1000$, a_i 는 모든 i 에서 같다.
- (서브테스크 2) $n \leq 100$, $a_i \leq 1000$, $1 \leq l \leq r \leq 1000$, $\max a_i - \min a_i \leq 1$
- (서브테스크 3) $n \leq 100$, $a_i \leq 1000$, $1 \leq l \leq r \leq 1000$
- (서브테스크 4) $n \leq 10000$, $a_i \leq 10000$, $1 \leq l \leq r \leq 10000$
- (서브테스크 5) $n \leq 10000$, $a_i \leq 5 \times 10^5$, $1 \leq l \leq r \leq 5 \times 10^5$
- (서브테스크 6) $n \leq 100$, $a_i < 2^{31}$, $1 \leq l \leq r < 2^{31}$

Problem 7.9. (20133번*) N 개의 점이 주어질 때 서로 다른 *모래시계 모양*의 개수를 출력하라. $N \leq 1500$ 이고 점의 x 좌표와 y 좌표는 절댓값이 10^9 이하인 정수이며 어떤 세 점도 한 직선을 이루지 않는다.

- *모래시계 모양*은 두 삼각형이 한 점을 공유하고 두 삼각형이 겹치지 않는 모양이다.
- 서로 다른 *모래시계 모양*이라는 것은 구성하는 점과 선분이 중 하나라도 다른 것이다.

Problem 7.10. (20922번) 길이 N 의 수열이 주어질 때, 구간에 포함된 임의의 정수가 구간에 k 개 이하로 포함되는 가장 긴 연속 부분 수열의 길이를 출력하라. $N \leq 2 \cdot 10^5$, $1 \leq k \leq 100$ 이고 수열의 각 정수는 100000 이하의 자연수이다.

Problem 7.11. (20985번*) 1부터 N 까지 번호가 매겨진 N 개의 눈덩이가 있다. i 번 눈덩이는

x_i 에 있고 처음 무게는 0이다. 이 눈덩이는 바람에 의해 평원을 구르며 크기가 커지게 된다. 정확히는 다음과 같은 방식이다.

- Q 일 동안 정수 w 로 표현되는 바람이 분다. 이 바람이 불면 하루가 끝나고 나서 눈덩이가 x_i 에서 $x_i + w$ 로 이동한다. 또한 모든 눈덩이는 동시에 이동한다.
- 구간 $[a, a + 1]$ 에 눈이 있다면 그 구간을 지나간 눈덩이의 무게가 1 증가한다. 그리고 눈이 사라진다. 만약 눈이 없다면 무게의 변화는 없다.

Q 일이 지난 이후 N 개의 눈덩이의 무게를 각각 출력하라.

Problem 7.12. (25405번*, KOI22) 레벨이 L_i 인 N 명의 게임 캐릭터에 대해 다음과 같은 규칙의 트레이닝을 M 번 진행하였을 때 모든 트레이닝이 끝난 이후 각 캐릭터들의 레벨을 오름차순으로 정렬하여 출력하라.

- 한 번의 트레이닝은 레벨이 낮은 순서대로 K 개의 캐릭터를 고르고 선택된 캐릭터들의 레벨을 1 올리는 것이다. 레벨이 같은 캐릭터가 여러 명인 경우에는 아무거나 고른다.
- (서브테스크 1) $N \leq 1000, M \leq 1000, K \leq N, L_i \leq 10^9$
- (서브테스크 2) $N \leq 10^5, M \leq 10^9, K = 1, L_i \leq 10^9$
- (서브테스크 3) $N \leq 10^5, M \leq 10^5, K \leq N, L_i \leq 10^9$
- (서브테스크 4) $N \leq 10^5, M \leq 10^9, K \leq N, L_i \leq 10^9$

Problem 7.13. (26829번*) n 개의 참-거짓 질문에 대한 답변이 모두 다른 m 명의 사람이 있다. 그들의 답변은 $[0, 2^n]$ 사이의 숫자로 표현되며, i 번째 비트가 1이라는 것은 $i + 1$ 번째 질문에 대해 참으로 답변했음을 의미한다. m 명을 인덱스 기준으로 $[1, p]$ 와 $[p + 1, m]$ 의 두 그룹으로 나눌 때 다음 조건을 만족하도록 나누는 경우의 수를 구하여라. $n \leq 30, m \leq \min(2^n, 2 \times 10^6)$ 이다.

- 모든 $1 \leq i \leq n$ 에 대해 i 번째 질문에 참이라 대답한 사람이 두 그룹 모두 존재하거나 두 그룹 모두 존재하지 않아야 한다.
- 모든 $1 \leq i \leq n$ 에 대해 i 번째 질문에 거짓이라 대답한 사람이 두 그룹 모두 존재하거나 두 그룹 모두 존재하지 않아야 한다.

Problem 7.14. (27916번*) N 개의 아파트가 수직선 상에 있고, 각 아파트의 위치는 A_i 로 표현된다. 각 아파트는 정확히 하나의 아파트 단지에 속해야 하고, 아파트 단지는 최소 M 개의 아파트를 포함하는 연속된 번호를 가진 아파트의 집합이어야 한다. 다른 말로 모든 아파트 단지는 $[L, R]$ 로 표현될 수 있고, 이렇게 표현된다면 아파트 단지의 크기는 $A_R - A_L$ 이다. 다음 쿼리 Q 개를 해결하는 프로그램을 작성하라.

- 아파트 단지의 크기를 X_i 이하로 할 수 있는 아파트 단지의 구성 방법이 존재한다면 1을, 존재하지 않는다면 0을 출력하라.

$1 \leq A_1 < A_2 < \cdots < A_N \leq 10^9$, $0 \leq X_i \leq 10^9$ 이고 서브테스크마다 추가 제약 조건이 있다.

- (서브테스크 1) $N \leq 3 \times 10^5$, $M = N$, $Q = 1$
- (서브테스크 2) $N \leq 1000$, $M \leq N$, $Q \leq 3 \times 10^5$
- (서브테스크 3) $N \leq 3 \times 10^5$, $M \leq N$, $Q = 1$
- (서브테스크 4) $N \leq 1000$, $M \leq N$, $Q \leq 3 \times 10^5$
- (서브테스크 5) $N \leq 3 \times 10^5$, $M \leq N$, $Q \leq 3 \times 10^5$

Problem 7.15. (28081번) 영역 $[0, W] \times [0, H]$ 를 y 축과 평행한 M 개의 선 $x = x_i$ 과 x 축과 평행한 N 개의 선 $y = y_i$ 로 나누면 각각의 직사각형 영역이 만들어진다. 이 직사각형 영역들 중 넓이가 K 이하인 영역의 개수를 출력하라. $2 \leq H, W \leq 10^9$, $N, M \leq 10^5$, $1 \leq x_1 < x_2 < \cdots < x_M < W$, $1 \leq y_1 < y_2 < \cdots < y_N < H$ 이다.

7.5 연습문제 C

Problem 7.16. (8191번) 길이 N 의 수열 A_i 가 주어질 때, M 번 k 가 주어질 때마다 다음 쿼리를 해결하는 프로그램을 작성하라. $N \leq 10^6$, $M \leq 50$, $A_i \leq 10^9$, $k \leq 10^9$ 이다.

- $A_i > k$ 인 한 i 를 골라 A_i 를 1 줄이고 $i > 1$ 일 때 A_{i-1} 을 1 늘리거나 $i < N$ 일 때 A_{i+1} 을 1 늘리는 연산을 사용할 수 있다. 이 연산을 통해 $A_i \geq k$ 인 연속된 구간의 길이를 최대화시키고자 할 때 가능한 최댓값을 출력하라.

Problem 7.17. (10010번) N 개의 도시가 있고 일부 도시 쌍이 양방향 도로로 연결되어 있다. 구체적으로 $1 \leq i < N$ 에 대해 0번 도시와 i 번 도시가 연결되어 있고, $1 \leq i < N-1$ 에 대해 i 번 도시와 $i+1$ 번 도시가 연결되어 있고, 마지막으로 $N-1$ 번 도시와 1번 도시가 연결되어 있다. 앞에서 설명한 순서대로 각 도로를 이동하는데 걸리는 시간이 주어질 때, 최단거리가 가장 먼 두 도시 사이의 거리를 출력하라. $N \leq 5 \times 10^5$ 이고 각 도로를 이동하는데 걸리는 시간은 양의 정수이며, 임의의 두 도시 사이의 최단거리는 10^9 이하이다.

Problem 7.18. (10132번) 원형 철길 상의 N 개의 기차역이 있다. $1 \leq i < N$ 에 대해 i 번째 지점과 $i+1$ 번째 사이의 거리는 l_i , N 번째 지점과 첫 번째 지점 사이의 거리는 l_N 이다. 또한, 이 곳에서는 총 s 대의 기차가 사용된다. 다음은 이 철도 체계에 대한 설명이다.

- i 번째 기차는 연료를 꼭 채우고 d_i 만큼 이동할 수 있다.
- 연료는 역에서만 채울 수 있으므로 각 기차는 다음 역에 도달할만큼 충분히 연료가 남아있지 않다면 연료를 채워야 한다. 물론 그 양이 충분하더라도 원하면 연료를 채울 수 있다.
- 각 s 대의 기차가 하루에 딱 한번 한 기차역에서 출발해 한 번 순환한 후 다시 출발한 기차 역으로 돌아온다. 이때 다음 날의 운행을 위해 연료를 꼭 채우고 퇴근해야 한다. 즉, 기차가 운행을 시작할 때에도 연료는 꼭 차 있는 상태이다.

각 기차에 대해 출발역을 적절히 고르면 도착 이후 마지막 급유를 포함한 기차의 급유 횟수를

최소화할 수 있을 것이다. 각 기차에 대해 가능한 급유 횟수의 최솟값을 출력하라. 만약 어떤 역을 출발역으로 선택하더라도 기차가 역 중간에 연료 부족으로 멈추는 일이 생긴다면 NIE를 출력하라. $N \leq 10^9$, $s \leq 100$, $\sum l_i \leq 10^9$, $1 \leq d_i \leq \sum l_i$ 이다.

* 참고: NIE는 폴란드어로 "NO"라는 뜻이다.

Problem 7.19. (14058번) 음 아닌 정수 K 와 양의 정수 P , 그리고 집합 $A = \{a_1, a_2, \dots, a_N\}$ 를 생각하자. 모든 $1 \leq i \leq N$ 에 대해 $0 \leq a_i < P$ 이고 a_i 는 모두 다르다. 이제 다음 규칙의 게임을 생각하자.

- 선 플레이어부터 번갈아가며 A 에서 수 하나를 지운다.
- 수를 K 번 지운 이후 남은 수들의 합이 P 의 배수이면 선 플레이어가 이기고 아니면 선 플레이어가 진다.

두 플레이어가 모두 자신이 이길 수 있도록 플레이한다고 가정하였을 때, 총 T 개의 테스트케이스에 대해 이기는 플레이어를 출력하라. 선 플레이어가 이기면 X를, 후 플레이어가 이기면 Y를 출력하면 된다. $1 \leq K \leq N \leq 5000$, $P \leq 10^{18}$ 이다.

Problem 7.20. (17030번) 길이 N 의 수열 p_i 가 주어진다. 여기서 p_i 는 확률로 $0 \leq p_i \leq 1$ 이다. 모든 $l \leq i \leq r$ 에 대해 각 수를 p_i 의 확률로 선택할 때 선택되는 수가 **정확히 하나**인 확률이 최대가 되는 구간 $[l, r]$ 을 찾아 그때 최대 확률을 출력하라. $N \leq 10^6$ 이고, 모든 확률 입력 및 출력은 10^6 배 된 값으로 이루어진다. 즉, 입력되는 수를 10^6 으로 나눈 것이 실제 p_i 이고, 출력도 실제 최대 확률에 10^6 을 곱한 값의 정수부를 출력해야 한다.

Problem 7.21. (18888번) N 개의 서로 다른 점 P_i 들의 집합 S 가 주어진다. 임의의 세 점은 한 직선 위에 있지 않다. 네 꼭짓점이 모두 S 의 원소이며 점 P_i 가 사각형의 경계를 제외한 내부에 포함되는 사각형의 개수를 $f(P_i)$ 라고 할 때, $\sum_{i=1}^N f(P_i)$ 를 출력하라. $5 \leq N \leq 2500$ 이고 P_i 의 x 좌표와 y 좌표는 모두 절댓값이 10^9 이하인 정수이다.

Problem 7.22. (20087번**, IOI16) 0번 역부터 $N - 1$ 번 역까지 잇는 철도가 하나 있다. 모든 $0 \leq i < N - 1$ 에 대해 i 번 역과 $i + 1$ 번 역 사이를 지나가는데 걸리는 시간은 l_i 이다. 또한 각 역에는 지선철도로 다른 한 역과 연결되어 있을 수 있다. 만약 연결되어 있다면 $d_i \neq 0$ 이고 i 번 지선철도의 길이는 d_i 이다. $d_i = 0$ 은 지선철도가 없음을 나타낸다. 여러분은 **지선철도로만 연결된 역이 아닌 번호가 붙어 있는 역** 중 두 개를 골라 c 의 시간에 두 역 사이를 이동할 수 있는 급행철도 노선을 신설하려고 한다. **지선철도로만 연결된 번호가 없는 역을 포함하여** 두 역을 고를 때 두 역 사이 거리의 최댓값을 최소화시킬 수 있는 곳에 급행철도 노선을 신설할 때, 가능한 두 역 사이의 거리의 최댓값의 최솟값을 출력하라.

- 모든 경우 $1 \leq l_i \leq 10^9$, $0 \leq d_i \leq 10^9$, $1 \leq c \leq 10^9$ 이다.
- (서브테스크 1) $2 \leq N \leq 10$
- (서브테스크 2) $2 \leq N \leq 100$

- (서브테스크 3) $2 \leq N \leq 250$
- (서브테스크 4) $2 \leq N \leq 500$
- (서브테스크 5) $2 \leq N \leq 3000$
- (서브테스크 6) $2 \leq N \leq 10^5$
- (서브테스크 7) $2 \leq N \leq 3 \times 10^5$
- (서브테스크 8) $2 \leq N \leq 10^6$

Problem 7.23. (24261번) 길이 N 의 수열 A_i 와 길이 M 의 수열 B_i 가 주어진다. 또한, 모든 $1 \leq i \leq N$ 에 대해 $A_i \in [1, M]$ 이고 모든 $1 \leq j \leq M$ 에 대해 $B_j \in [1, N]$ 가 성립한다. A_i 에서 한 개 이상의 원소를 골라 더한 합과 B_i 에서 한 개 이상의 원소를 골라 더한 합이 같아지는 경우 하나를 출력하라. 선택되는 원소의 개수와 인덱스를 오름차순으로 각각 출력하면 된다. $N, M \leq 10^6$ 이다.

Problem 7.24. (25442번, IOI22) 길이 N 의 배열 a_i 와 집합 A 가 주어질 때, 최대 빈도와 최소 빈도를 다음과 같이 정의하자.

- 정수 k 에 대해 k 의 빈도는 집합 A 의 각 원소 x 에 대해 $a_x = k$ 인 x 의 개수이다.
- 최대 빈도는 모든 $1 \leq k \leq 10^9$ 에 대해 k 의 빈도중 최댓값이다.
- 최소 빈도는 모든 $1 \leq k \leq 10^9$ 에 대해 k 의 빈도중 0을 제외했을 때 최솟값이다.

또한 처음 집합 $A = \emptyset$ 이고 다음 세 가지 연산을 원하는 순서로 여러 번 사용할 수 있다. 시행 결과는 누적된다.

- $1 \leq i \leq N$ 인 i 를 하나 골라 A 에서 i 가 있다면 i 를 제거한다.
- $1 \leq i \leq N$ 인 i 를 하나 골라 A 에서 i 가 없다면 i 를 추가한다.
- A 의 최대 빈도를 알아낸다.

세 연산을 적절히 사용하여 집합 $\{1, 2, \dots, N\}$ 의 최소 빈도를 알아내어라.

- 모든 경우에서 $1 \leq a_i \leq 10^9$ 이다.
- (서브테스크 1) $N \leq 200$, 세 연산 각각을 40000번 이하로 사용
- (서브테스크 2) $N \leq 1000$, 세 연산 각각을 40000번 이하로 사용
- (서브테스크 3) $N \leq 2000$, 세 연산 각각을 $3N$ 번 이하로 사용

Problem 7.25. (26104번) N 개의 서로 다른 점 P_i 들의 집합 S 가 주어진다. 임의의 세 점은 한 직선 위에 있지 않다. 네 꼭짓점이 모두 S 의 원소이며 그 어떤 점 P_i 도 사각형의 경계를 제외한 내부에 포함되지 않는 사각형의 개수를 출력하라. $1 \leq N \leq 300$ 이고 P_i 의 x 좌표와 y 좌표는 모두 절댓값이 10^9 이하인 정수이다.

Chapter 8

비트마스크

8.1 배열 대신 숫자 하나로

크기 N 의 `bool`형 배열을 처리해야 하는 상황을 생각해보자. 물론 배열 자체로도 임의 인덱스에 접근할 수 있기 때문에 충분히 효율적이지만 배열로 사용하는 것에는 한계가 있다. 함수에 배열을 넘기는 것은 추가 작업이 필요하고, 배열은 인덱스나 값 비교에 쉽게 사용될 수 없다. 이러한 문제를 해결할 수 있는 방법이 비트마스크이다. `bool`형 배열의 i 번째(0-based) 수를 2진법으로 나타내었을 때 2^i 자리수로 대응시킨 숫자를 생각하자. 그러면 이 숫자는 2^N 보다 작은 음 아닌 정수가 될 것이다. 코딩에서 배열보다는 숫자가 훨씬 다루기 편하다는 점에서 비트마스킹은 큰 이점을 갖는다.

이제 `bool`형 배열과 숫자 사이의 변환을 해주는 코드를 살펴보자. Code 8.1을 보라.

Code 8.1. 비트마스크 함수

```
1 int boolToNum(int n, bool *arr) {
2     int ret = 0;
3     for (int i = 0; i < n; i++) {
4         if (arr[i]) ret += (1<<i);
5     }
6     return ret;
7 }
8
9 bool numToBool(int x, int k) {
10     return (bool)(x & (1<<k));
}
```

```

11 }
12
13 int setToOne(int x, int k) {
14     return x | (1<<k);
15 }
16
17 int setToZero(int x, int k) {
18     return x & (1<<k);
19 }

```

Code 8.1의 네 함수는 각각 다음 역할을 한다:

- `boolToNum(n, arr)`: 길이 n 의 `bool`형 배열을 i 번째 값을 2^i 자리수로 가지는 숫자로 변경
- `numToBool(x, k)`: 비트를 나타내는 수 x 에서 k 번째 비트 값 반환
- `setToOne(x, k)`: 비트를 나타내는 수 x 에서 k 번째 비트 값을 1로 변경
- `setToZero(x, k)`: 비트를 나타내는 수 x 에서 k 번째 비트 값을 0으로 변경

이 과정에서 비트 쉬프트, 비트 AND, 비트 OR, 비트 XOR 등의 비트 연산을 사용하는 것이 유용하다. 특히, 2^k 의 경우에는 $1 \ll k$ 로 간단하게 표현할 수 있음을 상기하자. 또한 비트 연산은 매우 빠르기 때문에 비트 연산을 사용하는 것이 프로그램의 수행 시간을 줄이는데 도움이 된다.

비트마스크는 주로 N 개 각각을 선택하거나 선택하지 않은 상태를 쉽게 표현하기 위해 사용된다. 백트래킹으로 쉽게 해결할 수 있는 크기 N 의 집합의 모든 부분집합을 출력하는 문제를 생각해보자. 백트래킹으로 풀 때에는 레벨을 나타내는 변수 L 을 현재 보는 원소의 인덱스로 잡고 재귀함수를 구현해야 한다. 하지만 비트마스크를 쓰면 단순히 $i = 0$ 부터 $i = 2^N - 1$ 까지의 모든 수에 대한 반복으로 쉽게 모든 부분집합을 출력할 수 있다(Problem 8.3).

8.2 std::bitset

STD에서는 `bitset`이라고 불리는 비트를 저장하는 특별한 컨테이너(container)를 지원한다. `bitset`의 맴버 함수(member function)으로는 `all`, `any`, `none`, `flip`, `set`, `reset` 등이 있다. 예시 사용법은 Code 8.2를 참고하라.

Code 8.2. <https://en.cppreference.com/w/cpp/utility/bitset>

```

1 #include <bitset>
2 #include <cassert>
3 #include <cstdint>
4 #include <iostream>

```

```
5
6 int main()
7 {
8     typedef std::size_t length_t, position_t; // the hints
9
10    // constructors:
11    constexpr std::bitset<4> b1;
12    constexpr std::bitset<4> b2{0xA}; // == 0B1010
13    std::bitset<4> b3{"0011"}; // can also be constexpr since C++23
14    std::bitset<8> b4{"ABBA", length_t(4), /*0:*/'A', /*1:*/'B'}; // ==
        0B0000'0110
15
16    // bitsets can be printed out to a stream:
17    std::cout << "b1:" << b1 << "; b2:" << b2 << "; b3:" << b3 << ";
        b4:" << b4 << '\n';
18
19    // bitset supports bitwise operations:
20    b3 |= 0b0100; assert(b3 == 0b0111);
21    b3 &= 0b0011; assert(b3 == 0b0011);
22    b3 ^= std::bitset<4>{0b1100}; assert(b3 == 0b1111);
23
24    // operations on the whole set:
25    b3.reset(); assert(b3 == 0);
26    b3.set(); assert(b3 == 0b1111);
27    assert(b3.all() && b3.any() && !b3.none());
28    b3.flip(); assert(b3 == 0);
29
30    // operations on individual bits:
31    b3.set(position_t(1), true); assert(b3 == 0b0010);
32    b3.set(position_t(1), false); assert(b3 == 0);
33    b3.flip(position_t(2)); assert(b3 == 0b0100);
34    b3.reset(position_t(2)); assert(b3 == 0);
35
36    // subscript operator[] is supported:
37    b3[2] = true; assert(true == b3[2]);
38
39    // other operations:
40    assert(b3.count() == 1);
41    assert(b3.size() == 4);
42    assert(b3.to_ullong() == 0b0100ULL);
43    assert(b3.to_string() == "0100");
```

44 }

8.3 연습문제 A

Problem 8.1. Code 8.1의 BoolToNum(n , arr)은 나머지 세 종류의 함수로 구현할 수 있다. BoolToNum(n , arr)를 나머지 세 종류의 함수로 구현하는 프로그램을 작성하라.

Problem 8.2. 아래 함수 setToOne, setToZero에는 문제가 있다. 무엇이 문제인지 설명하라.

```

1 int setToOne(int x, int k) {
2     return x + (1<<k);
3 }
4 int setToZero(int x, int k) {
5     return x - (1<<k);
6 }
```

Problem 8.3. 비트마스크를 이용해 크기 N 의 집합의 모든 부분집합을 출력하는 코드를 작성하라.

Problem 8.4. (26512번) 2의 보수 표현법은 음수를 이진법 숫자로 표현하기 위한 방식이다. 2의 보수 표현법은 첫 번째 자리를 부호 비트로 하고 남은 자리를 이용해 숫자를 표시하되, 남은 자리가 클수록 더 큰 숫자가 되도록 하는 것이다. 예를 들어 8bit 자료형인 char 자료형을 생각해 보자. 첫 번째 비트는 부호 비트이고, 나머지 7개의 비트는 값을 나타낸다. 부호가 결정된 상태에서 7개의 비트가 모두 0이면 가장 작은 수이고, 모두 1이면 가장 큰 수이다. 즉, 1111111_2 는 음수 중 가장 큰 수인 -1 이고, 실제로 가장 작은 수인 -128 을 표현하려면 10000000_2 로 써야 한다. 이제 정수 X, Y 가 주어질 때, 8-bit 2의 보수 표현법으로 $X, Y, -X, -Y, X - Y$ 를 표현하는 코드를 작성하라. 다섯 개의 값 모두 -128 이상 127 이하임이 보장된다.

8.4 연습문제 B

Problem 8.5. (1062번) N 개의 단어가 주어질 때, 26개의 알파벳 소문자 중 K 개를 골라 그 문자들로만으로 이루어진 단어의 개수를 세자. 이 값의 최댓값을 출력하라. $N \leq 50$ 이고 문자열의 길이는 전부 15 이하이다.

Problem 8.6. (1887번) T 이하의 자연수 중 0개 이상의 수를 고른 집합이 주어진 N 개의 집합을 포함하지 않도록 고르는 경우의 수를 출력하라. $T \leq 20$, $N \leq 52$ 이고, 주어진 집합은

공집합이 아닌 $\{1, 2, \dots, T\}$ 의 부분집합임이 보장된다.

Problem 8.7. (6989번*) 다음과 같은 방식으로 점수를 산출하는 방법을 고려해보자.

- (1) i 번 문항을 틀린 경우 i 번 문항에 대한 점수는 0이다.
- (2) i 번 문항을 맞춘 경우 i 번 문항에 대한 점수는
 - $i > 1$ 이고 $i - 1$ 번 문항을 맞춘 경우, $i - 1$ 번 문항에 대한 점수에 i 번 문항에 지정된 특정 숫자 S_i 를 더한 값이다.
 - $i = 1$ 이거나 $i - 1$ 번 문항을 맞추지 못한 경우, S_i 이다.
- (3) 총점은 각 문항에 대한 점수의 합이다.

S_i 가 주어질 때 총점으로 나올 수 없는 값들 중 K 이상인 값들을 생각하자. 이 값들의 최솟값을 출력하라. $N \leq 150, 1 \leq S_i \leq 100, 1 \leq K \leq 2 \cdot 10^9$ 이다.

Problem 8.8. (8903번*) 다섯 개의 수로 이루어진 N 개의 순서쌍 중, K 개의 순서쌍을 고르자. 이 K 개의 순서쌍의 i 번째 각 성분의 최댓값을 S_i 라고 할 때, $S_1 + S_2 + S_3 + S_4 + S_5$ 의 최댓값을 출력하라. $K \leq N \leq 10^4$ 이고 주어지는 N 개의 순서쌍의 각 성분은 0 이상 10^4 이하의 정수이다.

Problem 8.9. (12833번) 세 수 A, B, C 가 주어질 때, A 에 B 를 C 번 bitwise XOR 연산한 결과를 출력하라. A, B, C 는 모두 0보다 크고 10^9 보다 작은 정수이다.

Problem 8.10. (15936번) 0 이상 2^N 미만인 수들을 생각하자. 다음과 같은 규칙으로 수를 변경하려고 한다.

- 현재 수가 0 이상 2^N 미만인 수 x 라면, $x < y < 2^N$ 이고 x 와 y 의 bitwise XOR 결과가 2의 거듭제곱꼴인 y 를 반환한다.

N, M, K 가 주어질 때,

- 한 번의 변경으로 M 을 만들 수 있는 가장 큰 정수
- M 에서 한 번의 변경을 만들 수 있는 가장 작은 정수
- 어떤 수 a 에서 정확히 K 번의 변경으로 b 를 만들 수 있는 순서쌍 (a, b) 의 개수를 100 003으로 나눈 나머지

을 출력하라. $2 \leq K \leq N \leq 10^5, 1 \leq M \leq \min(10^8, 2^N - 1)$ 이고 답이 존재함이 보장된다.

Problem 8.11. (25152번*) K 가 주어지면 0 또는 1로만 구성된 $R \times C$ 크기의 이차원 배열을 출력해야 한다. 이때, 출력된 배열은 왼쪽 상단에서 오른쪽 또는 아래 방향으로 0인 칸만을 통해 이동해 오른쪽 하단에 도달하는 서로 다른 경로의 수가 K 인 배열이어야 한다. R 과 C 의 선택은 자유롭지만, 두 값 모두 1000 이하여야 한다.

Problem 8.12. (9527번) $f(x)$ 를 x 를 이진수로 표현했을 때 1의 개수라고 하자. $f(A) + f(A +$

$1) + \dots + f(B)$ 를 출력하라. $1 \leq A \leq B \leq 10^{16}$ 이다.

Problem 8.13. (26180번*) 원주 위에 n 개의 점이 있고, 각 점에는 0 또는 1의 숫자가 적혀 있다. 하지만, 여러분은 n 이 얼마인지 모르고 다음 두 가지 동작을 $3n+500$ 회 이하로 사용하여 알아내어야 한다.

- 왼쪽 또는 오른쪽으로 인접한 점으로 이동한다.
- 현재 점에 적힌 숫자가 0이면 1로, 1이면 0으로 바꾼다.

매 작업이 끝난 이후 현재 위치한 점에 적힌 숫자를 알 수 있다. 올바른 n 의 값을 찾는 프로그램을 작성하라. $3 \leq n \leq 5000$ 이다. 이 문제는 인터랙티브 문제이므로, 입출력을 통한 인터렉션에 관한 세부사항은 마음대로 정해도 좋다.

8.5 연습문제 C

Problem 8.14. (2833번) $N \times N$ 크기의 체스판에서 다음 규칙으로 나이트를 T 번 움직이려고 한다.

- 나이트는 체스의 기존 규칙에 따라 수평으로 한 칸, 수직으로 두 칸 떨어진 칸이나 수직으로 두 칸, 수평으로 한 칸 떨어진 칸으로 이동해야 한다.
- 나이트는 체스판 밖으로 움직일 수 없다.
- 현재 이동이 i 번째 이동이라면, 나이트가 이동하려는 칸에 적힌 숫자는 i 의 배수여야 한다.

체스판에 적힌 숫자들과 이동 횟수 T 가 주어졌을 때 가능한 도착 위치의 개수를 출력하고, 도착 위치들을 행이 증가하는 순서로, 행이 같다면 열이 증가하는 순서로 출력하라. $3 \leq N \leq 30$, $1 \leq T \leq 10^6$ 이며, 체스판에 적힌 숫자는 10^9 이하의 자연수이다.

Problem 8.15. (4207번) 그리고 $1 \leq i, j \leq n$ 이고 $i \neq j$ 인 모든 (i, j) 에 대해 $i \rightarrow j$ 의 관계와 $j \rightarrow i$ 의 관계 중 **정확히 하나**가 존재한다. 여기서 관계는 다음을 의미한다:

- 만약 $a \rightarrow b$ 의 관계가 존재한다면, a 를 골랐을 때 자동으로 b 도 선택된다. 하지만, b 를 골랐을 때 a 가 선택되지 않는다.

1부터 n 까지의 자연수 중 최소 개수를 골라 모든 자연수가 선택되도록 할 때, 최소 개수를 찾고, 최소 개수로 고르는 실례를 **아무거나** 하나 찾아라. $1 \leq n \leq 75$ 이다.

Problem 8.16. (13169번) n 개의 수 $A_1, A_2, \dots, A_n$ 과 집합 $N = \{1, 2, \dots, n\}$ 에 대해, $\oplus$ 를 bitwise XOR 연산이라고 하면

$$S_I = \sum_{k \in I \subseteq N} A_k$$

$$X = \bigoplus_{I \subseteq N} S_I$$

이다. X 를 구하는 프로그램을 작성하라. $1 \leq n \leq 30, 0 \leq A_i < 2^{30}$ 이다.

Problem 8.17. (17458번**) 전체집합 $U = \{0, 1, \dots, 2^k - 1\}$ 와 공집합이 아닌 부분집합 $A \subseteq U$ 를 생각하자. 다음 조건들을 만족하는 집합을 좋은 집합이라고 하자:

- 모든 $x, y \in A$ 에 대해 x 와 y 의 bitwise AND 연산의 결과도 A 에 속한다.
- 모든 $x, y \in A$ 에 대해 x 와 y 의 bitwise OR 연산의 결과도 A 에 속한다.

$[0, 2^k - 1]$ 에 속하는 n 개의 서로 다른 정수가 주어질 때, 이 정수들을 모두 포함하는 좋은 집합 $A \subseteq U$ 의 개수를 출력하라. $1 \leq k \leq 7, 0 \leq n \leq 2^k$ 이다.

Problem 8.18. (17752번) 두 플레이어 A와 B는 협동하여 A가 들은 수 X 를 B에게 전달해야 한다. 이때, 4×4 크기의 게임판을 통해서만 전달할 수 있다. 이 과정은 다음과 같이 진행된다.

- 게임의 진행자 K는 게임판의 임의의 위치에 말을 하나 놓는다. 이어서, 길이 10,000의 A와 B로만 이루어진 문자열에 적힌 순서로 A와 B를 부른다.
 - 이 문자열에는 같은 문자가 연속하여 많아야 100번 등장한다.
- A는 자신이 불러졌을 때, 현재 게임판에 놓인 말의 위치를 보고 상하좌우로 인접한 칸으로 옮긴다. 당연히 게임판 밖으로는 움직일 수 없다.
- B는 자신이 불러졌을 때, 현재 게임판에 놓인 말의 위치를 보고 예상되는 X 의 값을 외치거나 상하좌우로 인접한 칸으로 옮긴다.
 - 만약 X 의 값을 외친 경우, 실제 X 의 값을 맞추었다면 두 플레이어의 승리로 종료되고, 맞추지 못하였다면 두 플레이어의 패배로 종료된다.
 - 말을 옮긴 경우에는 계속 진행된다. 당연히 게임판 밖으로는 움직일 수 없다.

A와 B의 전략을 나타내는 코드를 작성하라. 추가로, A는 서브테스크 번호 T 와 숫자 X 를 듣고 전략을 구상할 수 있으며, B도 서브테스크 번호 T 를 듣고 전략을 구상할 수 있다. 이 문제는 두 스텝 문제이므로, 함수에 관한 세부사항은 본문을 참고하라.

번호	배점	추가 제약 조건
1	20%	$T = 1$ 이고 주어지는 문자열은 ABAB...과 같이 AB가 반복되는 형태이다.
2	20%	$T = 2$ 이고 처음 말의 위치는 (1,1)임이 보장된다.
3	60%	$T = 3$ 이고 추가 제약 조건은 없다.

Problem 8.19. A부터 Z까지 이름이 정해진 26개의 레지스터가 있고, 각 레지스터는 64-bit의 부호 없는 정수를 저장할 수 있다. 또 다음 일곱 개의 레지스터 연산이 있다.

- `mov R X`: X 에 저장된 값을 R 에 저장한다.
- `and R X`: X 에 저장된 값과 R 에 저장된 값을 bitwise AND 후, R 에 저장한다.
- `or R X`: X 에 저장된 값과 R 에 저장된 값을 bitwise OR 후, R 에 저장한다.
- `xor R X`: X 에 저장된 값과 R 에 저장된 값을 bitwise XOR 후, R 에 저장한다.

- **not** R : R 에 저장된 값을 bitwise NOT 후, R 에 저장한다.
- **shl** R X : R 에 저장된 값을 왼쪽으로 X bit shift한 후, R 에 저장한다. 남은 칸은 0으로 채워진다.
- **shr** R X : R 에 저장된 값을 오른쪽으로 X bit shift한 후, R 에 저장한다. 남은 칸은 0으로 채워진다.

(a) (18716번) [문제 1]를 해결하는 레지스터 연산을 나열하라. 레지스터 연산의 수는 64회 이하여야 한다.

[문제 1] 레지스터 A에 임의의 입력 숫자 x 가 주어졌고 다른 칸들에는 0이 저장되어 있다. A에 저장된 값을 8만큼 증가시켜라(modulo 2^{64} 으로). 즉, 레지스터 A에 $x + 8$ 을 저장하라. 나머지 레지스터에 저장된 값은 무엇이던 상관없다.

(b) (18717번) [문제 2]를 해결하는 레지스터 연산을 나열하라. 레지스터 연산의 수는 300회 이하여야 한다.

[문제 2] 레지스터 A에 임의의 입력 숫자 x 가 주어졌고 다른 칸들에는 0이 저장되어 있다. 이진법 표현에서 x 와 같은 개수의 1을 가지고 있는 x 보다 큰 가장 작은 수를 y 라고 하자. 예를 들어 $x = 23$ 이면 $y = 27$ 이다. y 를 레지스터 A에 저장하라. y 는 2^{64} 미만의 값임이 보장되고, 나머지 레지스터에 저장된 값은 무엇이던 상관없다.

Problem 8.20. (18913번) 임의의 두 (i, j) ($1 \leq i \neq j \leq n$)에 대해 $i \rightarrow j$ 의 간선과 $j \rightarrow i$ 의 간선 중 정확히 하나가 존재하는 유형 그래프가 주어진다. $n \leq 3,000$ 에서 그래프의 간선을 14개 이하의 색으로 컬러링하여 모든 $i \rightarrow j \rightarrow k$ ($i \neq k$) 경로의 두 간선의 색이 다르도록 할 수 있음이 보장되어 있다. 가능한 컬러링을 아무거나 출력하라.

Problem 8.21. (19674번) 1부터 N 까지의 수가 섞인 길이 N 의 순열 A 가 있으나, 여러분은 A 의 각 원소를 알 수 없다. 여러분은 그레이더와 다음 기능을 하는 함수를 통해 상호작용하여 A 의 각 원소를 알아내야 한다.

- 1부터 N 까지의 수를 K 개씩 총 B 개의 버킷으로 분류해 함수의 인자로 호출한다. $N = B \times K$ 임이 보장된다.
- 그러면 함수에서는 각 버킷에 포함된 수의 인덱스에 있는 수 K 개를 반환한다. 단, K 개의 수로 묶인 i 번째 묶임이 i 번째 버킷에 있었던 수들임을 보장하지 않는다. 즉, 함수의 인자에서의 버킷 순서와 반환된 배열에서의 버킷 순서는 다를 수 있다. 또, 한 묶임에서도 j 번째 수가 그 버킷에서 j 번째에 있었던 수임을 보장하지 않는다. 즉, 함수의 인자에서 각 버킷의 원소 순서와 반환된 배열에서 각 버킷의 원소 순서는 다를 수 있다.
 - 예를 들어 $[1, 2, 3]$, $[4, 5, 6]$ 의 두 개의 버킷을 만들었고, $A = [2, 4, 5, 3, 1, 6]$ 이면 $[[3, 6, 1], [5, 4, 2]]$ 은 가능한 하나의 반환 배열이다.

- 반환 배열의 첫 번째 묶임은 입력에서 두 번째 버킷이었던 수들의 인덱스에 존재하는 수 $[3, 1, 6]$ 이 임의의 순서로 섞여 있다.
- 반환 배열의 두 번째 묶임은 입력에서 첫 번째 버킷이었던 수들의 인덱스에 존재하는 수 $[2, 4, 5]$ 가 임의의 순서로 섞여 있다.
- 용어를 *버킷*과 *묶임*으로 다르게 쓰고 있음을 상기하라.

이 함수를 Q 번 이하로 호출하여 순열 A 를 알아내어라. $B \geq 2, K \geq 2, 6 \leq N \leq 1,000$, 그리고 $N = B \times K$ 이며, Q 는 하단의 서브테스크를 참고하라.

번호	배점	추가 제약 조건											
1	2%	$N = 6, B = 2, K = 3, Q = 100, ST = 1$											
2	3%	$N = 6, B = 3, K = 2, Q = 100, ST = 2$											
3	12%	$Q = 12, ST = 3$, 그레이더 함수 <code>randomResult</code> 는 버킷을 섞지 않는다. 여전히 각 버킷의 숫자는 섞인다.											
4	16%	$K = 2, Q = 4, ST = 4$											
5	15%	$B = 2, Q = 12, ST = 5$											
6	52%	$B > 2, K > 2, Q = 2000, ST = 6$, 각 테스트케이스에서 Q 의 최댓값 $Q_{\max}$ 에 따라 점수가 결정된다.											
		$Q_{\max}$ 의 조건	획득 점수	$Q_{\max} > 2000$	0%	$500 < Q_{\max} \leq 2000$	8%	$50 < Q_{\max} \leq 500$	17%	$9 < Q_{\max} \leq 50$	$22 + 30 \times \left(\frac{50 - q}{41}\right)^2 \%$	$Q_{\max} \leq 9$	52%
		$Q_{\max}$ 의 조건	획득 점수										
		$Q_{\max} > 2000$	0%										
		$500 < Q_{\max} \leq 2000$	8%										
		$50 < Q_{\max} \leq 500$	17%										
$9 < Q_{\max} \leq 50$	$22 + 30 \times \left(\frac{50 - q}{41}\right)^2 \%$												
$Q_{\max} \leq 9$	52%												

Problem 8.22. (21768번) 길이 2^N 의 음이 아닌 정수로 이루어진 수열 $A_0, A_1, \dots, A_{2^N-1}$ 을 생각하자. $0 \leq i, j \leq 2^N - 1$ 이며 $A_i + A_j < A_{i \wedge j} + A_{i \vee j}$ 인 (i, j) 가 존재한다면 아무거나 하나 찾고, 존재하지 않는다면 -1을 출력하라. $\wedge$ 는 bitwise AND, $\vee$ 은 bitwise OR 연산자를 의미하고, $0 \leq N \leq 20, 0 \leq A_i \leq 10^7$ 이다.

번호	배점	추가 제약 조건
1	14%	$N \leq 14$
2	17%	$N \leq 17$
3	69%	추가 제약 조건은 없음

Problem 8.23. (21860번) 수열 $A_1, A_2, \dots, A_N$ 이 주어질 때, bitwise XOR 연산 $\oplus$ 에 대해

$$\bigoplus_{1 \leq i < j \leq N} A_i + A_j$$

의 값을 구하라. $1 \leq N \leq 10^6$, $1 \leq A_i \leq 5 \times 10^8$ 이다.

Problem 8.24. (21915번) 25개의 칸이 9개의 빨간 칸, 8개의 파란 칸, 7개의 흰색 칸, 1개의 특수 칸으로 이루어져 있다. 그리고 각 칸에는 좌상단부터 a부터 y까지 알파벳 라벨이 있다.



이제 단어 w 와 자연수 g ($1 \leq g \leq 9$)를 고르는 것을 반복할 것이다. 고를 때마다 다음 일이 발생한다.

- w 를 이루는 각 알파벳을 앞에서부터 보며 그 알파벳에 대응하는 칸이 선택된 적이 없다면 남기고, 그렇지 않다면 지운다. z 의 경우에는 지운다.
- 남은 알파벳 중 g 번째 알파벳에 해당하는 칸을 선택한다. g 번째 알파벳이 없거나 g 번째 알파벳에 해당하는 칸이 빨간 칸이 아니라면 틀린다.

우리의 목표는 **계속 틀리지 않고** 모든 빨간 칸을 선택하는 것이다. n 개의 길이 30 이하의 단어 목록이 주어질 때, q 개의 25개의 칸의 배치와, 각 칸이 선택되었는지의 여부가 주어질 때마다 목표를 달성하기 위한 w 와 g ($1 \leq g \leq 9$)의 쌍을 순서대로 출력하라. 만약 없다면 IMPOSSIBLE을 출력하면 된다. $1 \leq n \leq 10^5$, $1 \leq q \leq 10^5$ 이다.

Problem 8.25. (24068번) $N \times M$ 크기의 격자에 다음 규칙을 만족하도록 음이 아닌 자연수를 채워넣자.

- 상하좌우로 인접한 두 칸에 적힌 수를 2진법 표현했을 때 값이 서로 다른 비트가 하나 이하여야 한다.
- 모든 칸에 적힌 수는 달라야 한다.

이렇게 채우는 방법 중에서 최댓값이 최소가 되는 배치를 하나 출력하라. $1 \leq N, M \leq 2,000$ 이다.

Problem 8.26. (27511번) N 명에서 다음 세 단계로 진행되는 게임을 하려고 한다.

- 1단계:
 - N 개의 자리에 있는 원탁에 0부터 $N - 1$ 까지의 수가 적힌 종이 N 장이 하나씩 놓여 있다.
 - 각 자리에 정해진 순서로 한 명씩 앉아 자신과 자신의 **오른쪽** 사람이 가진 종이를 보고 0 이상 10^9 미만의 새로운 숫자를 적는다.
 - 원래 있었던 종이는 각자 가져가 버린다. 그러면 **1단계와 관련된 기억이 지워진다.**

- 2단계:
 - 각 자리에 정해진 순서로 한 명씩 앉아 자신과 자신의 **왼쪽과 오른쪽** 사람이 가진 종이를 보고 0 이상 10^9 미만의 새로운 숫자를 적는다.
 - 1단계에서 놓았던 종이는 각자 가져가 버린다. 그러면 **2단계와 관련된 기억이 지워진다.**
- 3단계:
 - 각 자리에 정해진 순서로 한 명씩 앉아 자신과 자신의 **왼쪽과 오른쪽** 사람이 가진 종이를 보고 0 이상 10^9 미만의 새로운 숫자를 적는다.
 - 2단계에서 놓았던 종이는 각자 가져가 버린다. 그러면 **3단계와 관련된 기억이 지워진다.**

이렇게 하여 3단계에서 놓은 종이를 전부 펼쳤을 때 인접한 두 자리에 적힌 숫자가 다르면서 모든 숫자가 3보다 작도록 하고 싶다. 이를 위한 플레이어들의 전략을 작성하라. $10 \leq N \leq 100,000$ 이다.

Problem 8.27. (27906번) 다음과 같은 규칙의 모자 게임을 생각하자.

- N 명의 플레이어가 있고 한 명의 진행자가 있다.
- 각 플레이어는 일부 플레이어의 모자만 볼 수 있다. 특히, 자기 자신의 모자는 절대 볼 수 없다.
- 모든 플레이어는 각 플레이어가 볼 수 있는 모자들을 알고 있다.
- 각 플레이어가 눈을 감으면 게임의 진행자가 각 플레이어에게 흰색 모자와 검은색 모자 중 하나를 씌운다.
- 이후 "흰색 모자를 w_1 개 이상 w_2 개 이하로 씌웠습니다"라고 말한다.
- 진행자가 몇 명의 플레이어에게 "당신이 쓴 모자의 색을 알고 있나요?"라고 Q 번 질의한다. 그들은 동시에 대답해야 하며, 모두가 대답을 들을 수 있다.

모든 출제자가 완벽히 논리적으로 사고하며 근거 없는 추측을 하지 않는다고 가정할 때, 각 질의에서 각 출제자의 대답을 알아내어라. $1 \leq N \leq 20$, $1 \leq Q \leq 30$ 이다.

Chapter 9

좌표 압축

좌표 압축(coordinate compression) 또는 값 압축(value compression)이라고 불리는 테크닉은 전체 값들의 절대적인 값보다 상대적인 대소관계만이 영향을 주는 상황에서 값의 범위를 줄이는 방법이다. 입력의 개수에 비해 값의 범위가 매우 넓을 때 유용하게 사용할 수 있다. 특히 값 자체를 인덱스의 의미로 사용해야 하는 경우에 유용하다. 이번 장에서는 좌표 압축을 하는 방법과 사용 예시를 배운다.

9.1 값의 범위를 줄이기

다음 Example 9.1을 보자.

Example 9.1. 이차원 좌표평면에 있는 N 개의 점이 주어진다. 각 좌표 (x_i, y_i) 에 대해 x 좌표가 더 작은 점의 수를 출력하는 프로그램을 작성하라. 즉, 총 N 개의 수를 출력해야 한다.

가장 단순한 방법은 N 개를 전부 보면서 (x_i, y_i) 가 등장하면 x_i 의 개수에 1을 더한 이후 누적합을 사용하는 것이다. 하지만 x_i 의 범위가 매우 넓으면 문제가 된다. 만약 x_i 가 10^9 이하의 자연수라면 10^9 크기의 배열을 만들어야 할 것이다.

그런데 굳이 10^9 이하의 모든 값에 대한 정보를 저장하는 배열을 만들어야 할까? 최대 N 개의 값이 들어오기 때문에, 개수를 세는 배열에서 최대 N 개의 값만 0이 아니고, 0이 아닌 인덱스만 고려해도 충분하다. 이러한 관찰에서 시작된 알고리즘이 좌표 압축이다. 좌표 압축의 정의는 어떤 배열 A 에서 각 A_i 를 A_i 이하의 수의 종류의 수로 정하는 것이다. 간단하게 말해, A_i 가 배열 A 에서 얼마나 큰지를 상대적으로 나타내는 값이다. A_i 가 A 에서 k 번째로 작으면 좌표 압축한 배열에서 A_i 는 k 이다.

이렇게 좌표 압축을 하면 빠지는 점 없이 모든 점을 처리할 수 있으며, 각 j 에 대해 $A_i = j$ 인 i 가 적어도 하나는 존재하기 때문에 기존의 목표를 달성하였다고 볼 수 있다. 이제 마지막으로 남은 문제는 좌표 압축을 한 배열에서 답을 구해도 되는지이다. 우리는 다음 관찰 하나로 좌표 압축된 배열에서 답을 구해도 된다는 사실을 정당화할 수 있다.

Observation. 좌표 압축한 이후에 수의 대소관계는 바뀌지 않는다. 다른 말로 원래 배열에서 더 컸던 수는 좌표 압축한 배열에서도 여전히 더 크다.

문제에서 x_i 보다 x 좌표가 작은 점의 개수를 찾으라고 하였으므로 좌표 압축을 한 배열에서 문제를 풀어도 전혀 문제가 없다.

마지막으로 위 내용을 수학적으로 명확하게 정리해보자. A 를 좌표 압축한 배열을 B 라고 할 때,

- B 의 모든 원소는 A 의 길이 이하이다.
- $A_i < A_j$ 와 $B_i < B_j$ 는 필요충분관계, $A_i = A_j$ 와 $B_i = B_j$ 도 필요충분관계이다.

그렇다면 좌표 압축은 어떻게 할까? 다음 Example 9.1에 대한 정답 코드인 Code 9.1로 알아보자.

Code 9.1. Example 9.1의 정답 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, cnt[101010], sum[101010];
5  pair<int, int> arr[101010];
6  vector<int> val;
7
8  int main() {
9      cin >> n;
10     for (int i = 1; i <= n; i++) cin >> arr[i].first >> arr[i].second;
11
12     for (int i = 1; i <= n; i++) val.push_back(arr[i].first);
13     sort(val.begin(), val.end());
14     val.erase(unique(val.begin(), val.end()), val.end());
15     for (int i = 1; i <= n; i++)
16         arr[i].first = lower_bound(val.begin(), val.end(), arr[i].first)
17             - val.begin() + 1;
18
19     for (int i = 1; i <= n; i++) cnt[arr[i].first]++;
20     for (int i = 1; i <= n; i++) sum[i] = sum[i - 1] + cnt[i];

```

```

20     for (int i = 1; i <= n; i++) cout << sum[arr[i].first - 1] << " ";
21
22     return 0;
23 }

```

위 코드에서 좌표 압축에 해당하는 부분은 13, 14, 16번 줄이다. 세 줄에 적힌 코드는 각각 다음과 같은 역할을 한다.

- 13, 14번째 줄은 `val`를 정렬하고 중복된 값을 제거하는 코드이다.
- 16번째 줄은 `arr`에 포함된 서로 다른 값을 바탕으로 좌표 압축한 새로운 배열을 만드는 코드이다.

13번째 줄에 있는 `std::unique`에 대한 자세한 내용은 다음 C++ 레퍼런스 사이트에서 확인할 수 있다: <https://en.cppreference.com/w/cpp/algorithm/unique>

모든 좌표 압축은 코드가 동일하다. 그리고 모든 좌표 압축은 **코드 세 줄(13, 14, 16번 줄)**로 끝난다. 매우 자주 쓰이는 테크닉이니 꼭 이 코드 세 줄 만큼은 외워두도록 하자.

9.2 연습문제 A

Problem 9.1. Example 9.1을 좌표 압축 없이 해결하는 코드를 작성하라.

Problem 9.2. Code 9.1에서 다시 원래 배열의 값을 알고 싶다면 어떻게 하면 되겠는가?

Problem 9.3. (18870번) 주어진 길이 N 의 수열에 좌표압축을 한 결과를 출력하라. 시작은 0 번이며, $N \leq 10^6$, 수열의 값은 절댓값이 10^9 이하인 정수이다.

Problem 9.4. (18869번) 길이 N 인 M 개의 수열이 주어질 때, $\binom{M}{2}$ 개의 쌍들 중 다음 조건을 만족하는 조합의 개수를 출력하라.

- 수열 $A_1, A_2, \dots, A_N$ 과 $B_1, B_2, \dots, B_N$ 이 임의의 $1 \leq i, j \leq N$ 에 대하여

$$A_i < A_j \Rightarrow B_i < B_j$$

$$A_i = A_j \Rightarrow B_i = B_j$$

$$A_i > A_j \Rightarrow B_i > B_j$$

를 만족해야 한다.

$2 \leq M \leq 100$, $3 \leq N \leq 10,000$ 이며 수열의 값은 10^6 이하의 자연수이다.

9.3 연습문제 B

Problem 9.5. (2370번) n 개의 구간 $[s_i, e_i]$ 가 주어지면 작은 i 부터 i 번 구간 $[s_i, e_i]$ 을 i 번 색으로 칠하자. 이미 색이 칠해져 있다면 새로운 색으로 덮어서 칠하는 것이다. n 번 색까지 칠한 이후에 보이는 색의 종류의 수를 출력하라. $1 \leq n \leq 10^4$, $1 \leq s_i \leq e_i \leq 10^8$ 이다.

Problem 9.6. (5926번) 수직선 상의 n 개의 점이 있고, 각 점에는 숫자가 적혀 있다. 다음 조건을 만족하도록 만들 수 있는 구간 $[s, e]$ 를 생각하자.

- 임의의 숫자 k 에 대해 n 개의 점들 중 k 가 적힌 점이 존재한다면, $[s, e]$ 에 k 에 적힌 점이 존재해야 한다.

가능한 $e - s$ 의 최솟값을 출력하라. $1 \leq n \leq 5 \times 10^4$ 이고 점의 좌표와 점에 적힌 숫자는 모두 10^9 이하의 양의 정수이다.

Problem 9.7. (11540번) N 개의 문제 중 플레이어 A가 풀 수 있는 A 개의 문제 번호와 플레이어 B가 풀 수 있는 B 개의 문제 번호가 주어진다. 그들은 1번 문제부터 포면서 두 플레이어 중 한 명 이상이 풀 수 있다면 풀 것이다. 이때 문제를 풀 사람이 컴퓨터를 사용해야 한다. 이렇게 하여 최대한 많은 문제를 풀 때 컴퓨터를 사용하는 사람을 교체해야 하는 횟수를 출력하라. 처음 컴퓨터를 사용할 사람은 자유롭게 정할 수 있다. $1 \leq N \leq 10^9$, $1 \leq A, B \leq 5 \times 10^4$ 이다.

Problem 9.8. (11997번) 이차원 좌표평면에 n 개의 점이 있다. 이 좌표평면을 짝수 a, b 에 대해 $x = a$ 와 $y = b$ 로 네 부분으로 나누었을 때, 네 부분에 있는 점의 개수의 최댓값의 가능한 최솟값을 출력하라. $1 \leq n \leq 1000$ 이고 점의 x 좌표와 y 좌표는 모두 10^6 이하의 양의 홀수이다.

Problem 9.9. (16806번) 좌표평면에 n 개의 점이 있고, 각 점은 상하좌우 중 하나의 고유 방향을 가진다. 이제 좌표평면 상의 원하는 위치에 상하좌우 중 한 방향으로 공을 굴리자. 그러면, 공이 점을 지나는 순간 공의 방향은 지난 점의 고유 방향으로 바뀌고, 점은 사라진다. 이렇게 계속 운동하면 공이 유한 개의 점만 지남을 보장할 수 있고, 지난 점의 개수가 점수가 된다. 점수가 최대가 되도록 공의 위치와 이동 방향을 잘 정할 때, 가능한 최대 점수를 출력하라. $1 \leq n \leq 3000$ 이고 점의 x 좌표와 y 좌표는 모두 절댓값이 10^9 이하인 정수이다.

Problem 9.10. (18205번) 소숫점 6 자리로 주어지는 n 개의 수와 m 개의 쿼리가 있다. 쿼리의 내용은 다음과 같다:

- $1 \leq l \leq r \leq n$ 이 주어질 때, $x = A_i$ 인 $l \leq i \leq r$ 인 i 가 $\lfloor \frac{r-l+1}{2} \rfloor + 1$ 개 이상(과반)인 x 가 존재하는지 판별하라.

각 쿼리에 대해 쿼리가 참이면 `usable`을, 거짓이면 `unusable`을 출력하라. $1 \leq n \leq 10^4$, $1 \leq m \leq 10^4$ 이고 주어지는 수는 0.000000 이상 14.000000 이하이다.

Problem 9.11. (18707번) 각 칸마다 숫자가 적혀있는 $N \times M$ 크기의 격자는 K 개의 칸에는

1이 적혀있고 나머지 칸에는 전부 0이 적혀 있다. $[1, i] \times [1, j]$ 의 직사각형 영역(prefix sub-grid)에 적혀 있는 1의 개수가 홀수개인 (i, j) ($1 \leq i \leq N, 1 \leq j \leq M$)의 개수를 구하여라. **여러 개의 테스트케이스에 대해 해결해야 함에 유의하라.** $1 \leq N, M \leq 10^9, 0 \leq K \leq 10^3$ 이고, 입력으로는 1이 적혀 있는 칸의 좌표가 주어진다.

Problem 9.12. (20440번) n 개의 구간 $[s_i, e_i]$ 가 주어질 때, $x \in [s_i, e_i]$ 인 i 의 개수의 최댓값과, 최대가 될 때 x 의 값의 범위를 구간 $[s, e]$ 으로 표현하라. 단, 둘 이상이 가능하다면, **가장 시작 좌표가 작은 구간 하나만** 출력하라. $1 \leq n \leq 10^6, 0 \leq s_i \leq e_i \leq 2.1 \times 10^9$ 이다.

Problem 9.13. (24572번) $N \times N$ 크기의 격자에 T 개의 나무가 있다. 격자에 포함되면서 나무를 포함하지 않는 정사각형 격자의 한 변의 길이로 가능한 최댓값을 출력하라. $N \leq 5 \times 10^5, T \leq 100$ 이다.

9.4 연습문제 C

Problem 9.14. (2361번) 이차원 좌표평면에 n 개의 점이 있다. 이 좌표평면에서 $\lceil \frac{n}{2} \rceil$ 개 이상 (과반)의 점을 포함하는 최소 크기의 정사각형의 한 변의 길이를 찾아라. 정사각형의 경계에 있는 것도 정사각형에 포함되는 것으로 간주한다. $4 \leq n \leq 1000$ 이고 점의 x 좌표와 y 좌표는 모두 100,015 이하의 양의 실수로 소숫점 6자리까지 주어질 수 있다.

Problem 9.15. (3324번) $W \times H$ 크기의 격자에 N 개의 칸이 흰색 또는 검은색으로 칠해져 있고, 칠해진 칸에는 숫자가 적혀 있다.

- 두 격자 사이의 거리를 상하좌우로 인접한 칸으로 이동할 때 필요한 이동 횟수로 정의하자. 즉, (a, b) 위치와 (c, d) 위치 사이의 거리는 $|a - c| + |b - d|$ 이다.
- 흰색 또는 검은색으로 칠해진 칸과 그 칸에 적혀 있는 숫자 d 에 대하여 거리가 d 이하인 점은 도달할 수 있다.

각 칸에 도달할 수 있는 흰색 칸의 수가 검은색 칸의 수보다 많은 칸의 개수와, 검은색 칸의 수가 흰색 칸의 수보다 많은 칸의 개수를 순서대로 출력하라. $1 \leq W, H \leq 10^9, 1 \leq N \leq 3000$ 이고 각 칠해진 칸에 적혀 있는 수 d 는 $0 \leq d < 5 \times 10^8$ 이다.

Problem 9.16. 다음과 같은 방식으로 진행되는 투표가 있다:

- 후보는 두 명이어야만 한다. 선착순으로(?) 두 명만 받는다.
- 각 유권자는 최대 100,000개의 표를 두 후보에게 나누어 던질 수 있다. 이때 한 표도 행사하지 않는 것은 불가능하다. 즉, 1번 후보에게 a (≥ 0)개의 표를, 2번 후보에게 b (≥ 0)개의 표를 던졌다면 $1 \leq a + b \leq 100,000$ 이어야 한다.
- 두 후보가 받은 표의 합이 더 큰 후보가 당선된다. 만약 같다면 두 후보를 공동 당선(?) 처리한다.

여론조사에서는 2번 후보가 압도적 지지를 받았지만, 암흑의 단체에서 해킹을 통해 다음과 같은 방식의 로직을 추가하여 1번 후보만 당선되도록 조작하였다.

- 투표 결과의 전산 집계에서, 한 후보에게 x 표 이상을 투표했거나, 전체 y 표 이상을 투표한 경우 무시한다. 다른 말로, $a \geq x$, 또는 $b \geq x$, 또는 $a + b \geq y$ 인 표는 무시한다.

여러분은 공직선거법 위반 혐의로 암흑의 단체를 기소하기 위해 $1 \leq x, y \leq 10^5$ 를 만족하는 모든 (x, y) 쌍을 찾아 검토해야 한다. n 명의 유권자가 1번 후보와 2번 후보에게 던진 표의 수가 주어질 때, 가능한 (x, y) 쌍의 경우의 수를 구하는 프로그램을 작성하라. $1 \leq n \leq 1000$ 이다.

Problem 9.17. <https://www.acmicpc.net/problem/28461>

Chapter 10

스위핑

스위핑(sweeping)은 "쓸다"를 의미하는 단어 "sweep"에서 파생된 단어로, 한 방향으로 지나가면서 문제에 대한 해답을 구하는 알고리즘을 말한다. 다른 말로 특정 기준에 따라 정렬된 순서를 따라 값을 처리하면서 정렬되어 있다는 특성을 활용하여 문제를 해결하는 방법이다. 이러한 스위핑 알고리즘은 다양한 문제에서 다양한 방식으로 활용된다. 이번 장에서는 예제를 통해 스위핑 알고리즘의 사용 방법을 배운다.

10.1 선분 길이 구하기

다음 문제 Example 10.1(<https://www.acmicpc.net/problem/2170>)을 보자.

Example 10.1. N 개의 선분의 시작점과 끝점이 주어질 때, 선이 그려진 지점의 총 길이를 구하는 프로그램을 작성하라. 단, 선이 여러 번 그려진 곳도 하나로 센다.

이 문제는 대표적인 스위핑 문제이다. 먼저 다음과 같은 관찰을 하자.

Observation. 선이 그려진 지점은 몇 개의 선분으로 표현되고, 그 선분의 양 끝점은 원래 선분의 양 끝점들 중 하나이다.

두 선분이 겹쳐져 있으면 그 두 선분의 공통 범위는 가장 앞 시작점과 가장 뒤 끝점이 된다는 사실로부터 간단히 증명할 수 있다. 또한, 이 증명으로부터 현재까지 선이 그려진 지점을 저장해놓고 선분을 추가하면서 갱신하겠다는 아이디어를 낼 수 있다.

그렇다면 어떤 순서로 선분을 추가해야 할까? 정답은 선분의 시작점이 오름차순으로 증가하는 순서이다. 왜냐하면 이렇게 순서를 정해야 이전까지 그려진 선들 중 가장 오른쪽에 있는 선분

만 고려해도 되기 때문이다. 이를 증명하기 위해 이전까지 나온 선분들 중 가장 오른쪽 선분을 (l, r) 이라고 하자. 두 가지 경우가 가능하다.

- 현재 선분 (s, e) 가 $s > r$ 인 경우에는 가장 오른쪽 선분을 (s, e) 로 바꾸면 된다. 이 이후에 나오는 선분은 시작점이 s 오른쪽이기 때문에 절대로 (l, r) 과 겹칠 수 없기 때문이다.
- 현재 선분 (s, e) 가 $s \leq r$ 인 경우에는 가장 오른쪽 선분을 $(l, \max(r, e))$ 로 바꾸면 된다. 겹치기 때문에 합쳐진 선분도 연속한 하나의 선분으로 표현될 수 있기 때문이다.

이 과정을 N 개의 선분에 대해 시행하게 되면 $O(N)$ 의 시간에 겹치는 구간을 모두 구할 수 있다. 구현은 다음 코드 Code 10.1을 참고하라.

Code 10.1. Example 10.1의 정답 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, ans = 0, l, r;
5  pair<int, int> arr[1010101];
6
7  int main() {
8      ios_base::sync_with_stdio(false);
9      cin.tie(0);
10     cin >> n;
11     for (int i = 1; i <= n; i++) cin >> arr[i].first >> arr[i].second;
12     sort(arr + 1, arr + n + 1);
13
14     l = arr[1].first; r = arr[1].second;
15     for (int i = 2; i <= n; i++) {
16         if (r <= arr[i].first) {
17             ans += r - l;
18             l = arr[i].first; r = arr[i].second;
19         }
20         else r = max(r, arr[i].second);
21     }
22     ans += r - l;
23     cout << ans << "\n";
24     return 0;
25 }
```

10.2 연습문제 A

Problem 10.1. Code 10.1의 8, 9번째 줄에 적힌 코드가 무엇인지 조사해보라. 그리고 8, 9번째 코드를 빼고 2170번 문제에 제출해보자. 어떻게 되는가?

Problem 10.2. Example 10.1을 풀 때, 끝 점 기준으로 정렬해도 스위핑이 가능한가? 가능한 방법이 있다면 찾고 코드로 구현하라.

10.3 연습문제 B

Problem 10.3. <https://www.acmicpc.net/problem/2594>

Problem 10.4. <https://www.acmicpc.net/problem/3845>

Problem 10.5. <https://www.acmicpc.net/problem/5874>

Problem 10.6. <https://www.acmicpc.net/problem/6326>

Problem 10.7. <https://www.acmicpc.net/problem/11182>

Problem 10.8. <https://www.acmicpc.net/problem/15889>

Problem 10.9. <https://www.acmicpc.net/problem/21041>

Problem 10.10. <https://www.acmicpc.net/problem/22643>

Problem 10.11. <https://www.acmicpc.net/problem/1118>

Problem 10.12. <https://www.acmicpc.net/problem/1319>

Problem 10.13. <https://www.acmicpc.net/problem/1617>

Problem 10.14. <https://www.acmicpc.net/problem/2130>

Problem 10.15. <https://www.acmicpc.net/problem/2397>

Problem 10.16. <https://www.acmicpc.net/problem/2836>

Problem 10.17. <https://www.acmicpc.net/problem/4438>

Problem 10.18. <https://www.acmicpc.net/problem/5541>

Problem 10.19. <https://www.acmicpc.net/problem/6529>

- Problem 10.20. <https://www.acmicpc.net/problem/6818>
- Problem 10.21. <https://www.acmicpc.net/problem/10107>
- Problem 10.22. <https://www.acmicpc.net/problem/15922>
- Problem 10.23. <https://www.acmicpc.net/problem/17166>
- Problem 10.24. <https://www.acmicpc.net/problem/19584>
- Problem 10.25. <https://www.acmicpc.net/problem/23740>
- Problem 10.26. <https://www.acmicpc.net/problem/25174>

10.4 연습문제 C

- Problem 10.27. <https://www.acmicpc.net/problem/1218>
- Problem 10.28. <https://www.acmicpc.net/problem/1266>
- Problem 10.29. <https://www.acmicpc.net/problem/2017>
- Problem 10.30. <https://www.acmicpc.net/problem/6117>
- Problem 10.31. <https://www.acmicpc.net/problem/12771>
- Problem 10.32. <https://www.acmicpc.net/problem/16809>
- Problem 10.33. <https://www.acmicpc.net/problem/17159>
- Problem 10.34. <https://www.acmicpc.net/problem/17196>
- Problem 10.35. <https://www.acmicpc.net/problem/17625>
- Problem 10.36. <https://www.acmicpc.net/problem/17692>
- Problem 10.37. <https://www.acmicpc.net/problem/17739>
- Problem 10.38. <https://www.acmicpc.net/problem/17973>
- Problem 10.39. <https://www.acmicpc.net/problem/18279>
- Problem 10.40. <https://www.acmicpc.net/problem/18579>

Problem 10.41. <https://www.acmicpc.net/problem/20306>

Problem 10.42. <https://www.acmicpc.net/problem/22020>

Problem 10.43. <https://www.acmicpc.net/problem/23197>

Problem 10.44. <https://www.acmicpc.net/problem/25415>

Problem 10.45. <https://www.acmicpc.net/problem/27235>

Problem 10.46. <https://www.acmicpc.net/problem/27290>

Chapter 11

백트래킹

백트래킹(backtracking)은 해를 찾는 도중 해가 아니어서 막히면, 되돌아가서 다시 해를 찾아가는 알고리즘으로 최적화 문제와 결정 문제를 푸는데 쓰이며, 구현 시 재귀함수를 사용한다. 대표적인 문제들로 재귀함수의 사용법과 백트래킹을 익혀보자.

11.1 이론

스도쿠와 같은 퍼즐을 컴퓨터로 어떻게 풀 수 있을까? 가장 쉬운 방법은 모든 방법을 시도하는 것이다. 이를 위해 경우의 수를 셀 때 썼던 수형도(나뭇가지 그림)과 같은 아이디어를 쓸 수 있다. 첫 번째 칸에서 1~9 중 하나를 시도하고, 그 다음 칸에서 또 1~9 중 하나를 시도하고, 이를 반복하는 완전 탐색을 하는 것이다. 빈 칸이 t 개인 스도쿠를 푼다면, 총 9^t 개의 경우를 시도하게 될 것이다.

하지만 꼭 모든 경우를 탐색해야 할까? 스도쿠에서 처음 두 칸에 모두 1을 채우는 것은, 뒤에 칸이 어떻게 채워지든지 불가능하다. 즉, 탐색을 하다가 중간에 불가능하다는 결론이 나오면 그 이후는 시도할 필요가 없다! 이 경우에는 다시 되돌아가서 다른 경우를 시도할 수 있다. 이처럼, 완전 탐색을 하되, 자명하게 불가능한 경우를 제외하여 시간의 효율을 조금 더 향상시킨 것이 백트래킹이다.

즉, 문제가 상수 개 중 하나를 선택하는 것을 여러 번 반복한다면, 백트래킹을 고려할 수 있다. 이때, 어느 순서로 채우는지에 따라 소요되는 시간이 크게 영향받을 수 있으므로, 좋은 방식을 찾는 것이 중요하다. 이제, 몇 가지의 예시 문제들로 백트래킹을 익히자.

11.2 N-Queen 문제

다음과 같은 문제를 생각해보자.

Example 11.1 (*N-Queen problem*). $N \times N$ 체스판 위에 N 개의 퀸을 놓고자 한다. 가능한 경우가 존재하면 가능한 경우 한 가지를 출력하고, 그렇지 않으면 -1 을 출력하라. 단, $N \leq 25$ 이다.

효율적 구현을 위해서는 어떤 순서로 선택을 할 지 결정해야 한다. 다음 관찰이 도움을 준다.

Observation. 한 가로줄에는 최대 하나의 퀸만 위치할 수 있다. 따라서 N 개의 퀸을 놓으려면 정확히 하나의 가로줄에 하나의 퀸이 위치해야 한다.

즉, 각 가로줄마다 하나의 퀸이 어디 위치할 지 N 개 중에 하나를 선택하는 문제로 바꿀 수 있다. 이렇게 제일 위 가로줄부터 채우는 방식을 쓰기 위해 재귀함수 `solve`의 파라미터로 현재까지 퀸의 위치를 정한 가로줄의 개수 L 을 사용할 것이다. 당연히 L 은 현재까지 놓은 퀸의 개수와 같다. 그러면 `solve(L)`은 $L+1$ 번째 가로줄에 있는 N 개의 칸에 퀸이 놓일 수 있는지 시험해 가능한 경우에 대해 모두 `solve(L+1)`을 호출하는 것으로 문제를 해결할 수 있다. 이 과정을 반복해 $L = N$ 에 도달하면 N 개의 퀸을 놓았다는 뜻이므로, 현재 퀸의 위치를 출력하고 코드를 종료하면 된다. 만약 모든 경우를 시험했음에도 불구하고 코드가 종료되지 않았다면, 불가능하다는 뜻이므로 -1 을 출력하고 코드를 종료하면 된다.

그렇다면 어떤 칸에 퀸이 놓일 수 있는지 어떻게 판단할 수 있을까? 그 퀸이 있는 세로줄, 가로줄, 그리고 두 방향에 대각선들에 있는 칸에 대해 다른 퀸이 있는지 확인하면 된다. 한 가로줄에 퀸 하나임은 보장되므로 가로줄은 확인할 필요가 없고, 아래쪽에 퀸이 없음이 보장되므로 위쪽에 있는 가로줄만 확인하면 된다. 이 과정은 정답 코드의 `isvaild(r, c)` 함수에서 처리된다.

Code 11.1. Example 11.1의 정답 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, board[25][25];
5
6  bool isvaild(int r, int c) {
7      for (int i = 0; i < r; i++) {
8          if (board[i][c] == 1) return false;
9      }
10
11     int k = 1;
12     while (true) {

```

```
13         if (r + k >= n || c - k < 0) break;
14         if (board[r + k][c - k] == 1) return false;
15         k++;
16     }
17     k = 1;
18     while (true) {
19         if (r - k < 0 || c - k < 0) break;
20         if (board[r - k][c - k] == 1) return false;
21         k++;
22     }
23
24     return true;
25 }
26
27 void solve(int L) {
28     if (L == n) {
29         for (int i = 0; i < n; i++) {
30             for (int j = 0; j < n; j++) cout << board[i][j] << " ";
31             cout << "\n";
32         }
33         exit(0);
34     }
35     for (int i = 0; i < n; i++) {
36         if (isvaild(L, i)) {
37             board[L][i] = 1;
38             solve(L + 1);
39             board[L][i] = 0;
40         }
41     }
42 }
43
44 int main() {
45     cin >> n;
46     solve(0);
47     cout << "-1\n";
48     return 0;
49 }
```

11.3 순열 구하기

다음으로 백트래킹으로 주어진 조건을 만족하는 수열을 구하자. 고등학교 확률과 통계에서 배우는 것과 비슷한 느낌의 문제다.

Example 11.2. $1 \leq M \leq N \leq 8$ 을 만족하는 두 정수 N 과 M 이 주어질 때, 각 조건을 만족하는 길이가 M 인 서로 다른 수열을 사전 순으로 모두 출력하는 프로그램을 작성하라.

- (15649번) 1부터 N 까지 자연수 중에서 중복 없이 M 개를 고른 수열
- (15650번) 1부터 N 까지 자연수 중에서 중복 없이 M 개를 고른 단조증가 수열
- (15651번) 1부터 N 까지 자연수 중에서 중복이 가능하게 M 개를 고른 수열
- (15652번) 1부터 N 까지 자연수 중에서 중복이 가능하게 M 개를 고른 단조증가 수열

이 문제에서도 Code 11.1에서 L 에 대응하는 것을 잡아야 한다. 첫 번째 수부터 차례로 고르는 것이 제일 편하다. 따라서 현재까지 고른 수의 개수를 L 로 두고 구현한다. 또한, 이 문제에서는 수열을 출력해야 하므로 지금까지 선택한 것을 저장해야 한다. 동적 자료구조인 `std::vector`을 아래와 같이 활용하면 편하다.

```

1 #include <vector>
2 using namespace std;
3
4 vector<int> path;
5 void solve(int L) {
6     // ...
7     path.push_back(i);
8     solve(L+1);
9     path.pop_back();
10    // ...
11 }
```

책에서는 이러한 방식을 재귀함수의 경로 기록이라고 할 것이다. 책에서 `path`라는 벡터 변수가 사용된다면, 함수 호출에 따른 경로를 기록한 경우가 많을 것이다.

이제 직접 네 문제를 구현해보라. 조건이 약간씩 다르기 때문에 세부적인 코드는 조금씩 변경해야 할 것이다. 이후의 과정은 독자에게 맡긴다.

이제, 비슷하지만 조금 더 어려운 문제를 보자.

Example 11.3. $1 \leq M \leq N \leq 8$ 을 만족하는 두 정수 N 과 M , 그리고 N 개의 수로 이루어진 수열 A_i 가 주어질 때, 각 조건을 만족하는 길이가 M 인 서로 다른 수열을 사전 순으로 모두 출력하는 프로그램을 작성하라. 단, N 개의 주어지는 수들 중에는 같은 수가 있을 수 있다.

- (15663번) 수열 A_i 에서 중복 없이 M 개를 고른 수열
- (15664번) 수열 A_i 에서 중복 없이 M 개를 고른 **단조증가** 수열
- (15665번) 수열 A_i 에서 **중복이 가능하게** M 개를 고른 수열
- (15666번) 수열 A_i 에서 **중복이 가능하게** M 개를 고른 **단조증가** 수열

이 문제의 차이점은 N 개의 수 중에 같은 수가 있을 수 있다는 것이다. 같은 수가 있으면, 이전 문제와 같은 방식으로 구현한 코드에서 출력하는 수열이 모두 다르다는 보장을 할 수 없다. 즉, 어떤 경우에 같아지는지 생각하고, 그 경우를 배제해야 한다. 같아지는 경우는 배열에 똑같은 수가 a 번째와 b 번째에 있을 때, 선택된 위치 중에서 a 만을 b 로 바꾼 경우이다. 즉, 아무것도 선택되지 않거나, a 만 선택되거나, a 와 b 가 모두 선택되는 경우만 남기면 된다. 이것은 똑같은 수 중에서 더 앞에 있는 수가 선택되지 않았으면, 그 수를 선택하지 않는 방법으로 해결할 수 있다.

판단이 어렵다고 생각할 수 있지만, 배열을 처음에 정렬하고 시작함을 상기하자. 정렬된 배열에서 똑같은 수는 항상 연속된 위치에 있다. 이를 이용하면 구현을 쉽게 할 수 있다. 이후의 과정은 독자에게 맡긴다.

11.4 격자 이동

백트래킹을 사용하면 길을 찾을 수도 있다. 격자에서 상하좌우로만 이동할 수 있다면, 이것 자체도 4개 중에 하나를 선택하는 것을 반복하는 것이기 때문이다. 이와 관련된 문제를 하나 보자.

Example 11.4. (1502번) $n \times m$ 격자에 두 개의 1이 적혀 있다. 두 개의 1 사이를 상하좌우로 인접한 칸으로 선을 그어 이으려고 한다. 선이 모든 칸을 다 지나야 할 때 가능한 이동 경로를 지나는 칸의 좌표로 출력하는 프로그램을 작성하라. 만약 가능한 경우가 없다면 -1을 출력한다. m 과 n 은 2 이상 8 이하의 짝수이다.

이 문제에서도 L 에 대응하는 것을 생각하자. 지금까지 지난 칸의 수를 L 로 잡으면 편하게 구현할 수 있을 것이다. 물론, 현재 칸에 따라 다음 칸이 달라지기 때문에 현재 칸의 위치를 나타내는 좌표도 파라미터로 넣어야 한다.

이 문제에 유용한 테크닉이 하나 있다. 격자에서 상하좌우로 이동하는 것은 현재 위치에서 $(-1, 0)$, $(1, 0)$, $(0, -1)$, $(0, 1)$ 중 하나를 더하는 것과 같은데, 이것을 직접 작성해서 처리해도 되지만, 아래 코드와 같이 배열을 활용하면 조금 더 간단하게 할 수 있다.

```
1 int dx[4] = {-1, 0, 1, 0}, dy[4] = {0, 1, 0, -1};
2
3 void solve(int L, int x, int y) {
4     // ...
```

```

5   for (int k = 0; k < 4; k++) {
6       int X = x + dx[k], Y = y + dy[k];
7   }
8   // ...
9   }

```

이 방식은 상하좌우가 아니라 대각선까지 포함해 8칸이 이동 가능한 경우에도 비슷하게 적용할 수 있다. 이 방법을 사용하여 구현해보자. 만약, 구현 과정에서 시간 초과가 난다면 절대 불가능한 경우를 제외해 경우의 수를 줄여야 한다. 예를 들어 모든 칸을 지나지 않고 두 번째 1에 도착하는 경우는 절대 불가능한 경우이다.

11.5 최적화

이번에는 백트래킹에서 어떻게 불가능한 경우를 잘 추려내서 경우의 수를 줄일 수 있는지 예시를 통해 감을 익히자. 2048이라는 유명한 게임과 관련된 문제이다.

Example 11.5. $N \times N$ 격자에서 2048 게임(<https://play2048.co/>)을 시뮬레이션하고자 한다. 실제 게임과 다르게 **블록의 생성이 없을 때**, 초기 상태에서 X 번 이동하여 만들 수 있는 가장 큰 숫자를 구하는 프로그램을 작성하라.

- (12100번) $1 \leq N \leq 20$, $X = 5$
- (12094번*) $1 \leq N \leq 20$, $X = 10$

$X = 5$ 인 경우는 가능한 경우가 4^5 으로 작기 때문에 완전 탐색을 해도 충분하다. 하지만, $X = 10$ 인 경우, $N^2 \times 4^{10}$ 이 생각보다 커서 최적화가 필요하다. 일단 보드가 바뀌지 않는 경우 그 방향으로의 이동은 할 필요가 없다. 이는 자명한 최적화다.

정말 중요한 최적화는 문제에서 최댓값을 요구한다는 것이다. 즉, 현재 상황에서 어떻게 이동하더라도 최댓값이 갱신될 수 없으면 더 탐사할 필요가 없다. 이는 백트래킹 문제에서 자주 사용되는 중요한 테크닉이다. 더 구체적으로 말하면, 항상 최댓값은 많아야 2배 증가하므로, 현재 값에 남은 횟수 만큼 2를 곱한 값이 최댓값 이하라면 더 탐사할 필요가 없다.

상술한 두 가지 최적화를 추가하면 $X = 10$ 인 경우에도 제한 시간 내에 문제를 해결할 수 있다.

11.6 연습문제 A

Problem 11.1. Code 11.1의 시간 복잡도는 무엇인가?

Problem 11.2. Code 11.1에서 `isvaild(r, c)` 함수의 시간복잡도를 $T(n)$ 이라고 할 때 전체 시간복잡도를 $T(n)$ 으로 나타내라. 이 사실로부터 알 수 있는 것은 무엇인가?

Problem 11.3. $N = 25$ 일 때, CPU의 연산 속도를 1초당 10^9 회라고 가정하고 Code 11.1의 수행 시간을 구해보라. 예상되는 실행 시간은 얼마인가? 실제 실행 시간과 얼마나 차이나는가? 이 차이는 왜 발생하는가?

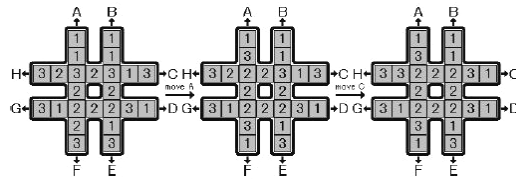
Problem 11.4. (9021번**) Example 11.4는 백트래킹을 사용하지 않고 수학적 해법을 찾을 수 있다. 수학적 해법을 찾아, m 과 n 이 2 이상 100 이하의 짝수인 경우를 해결해보라. 이렇게 각 문제마다 고유의 규칙을 찾아 답을 구성하는 것을 구성적(constructive)이라고 한다.

11.7 연습문제 B

Problem 11.5. (1128번) N 개의 물건이 있고, 각 물건에는 무게와 가격이 있다. 또, 가방이 있는데, 가방에 담을 수 있는 무게의 최댓값은 C 이다. 가방에 물건을 적절히 담아, 가방에 담긴 물건의 무게 합이 C 이하이면서 가격의 합이 최대가 되도록 하려고 한다. 가능한 최댓값을 구하라. 단, $N \leq 50$, $C \leq 10^{16}$ 이고 물건의 무게와 가격은 모두 10^{16} 이하이다. 또한, 물건의 무게는 모두 피보나치 수 중 하나이다(1, 2, 3, 5, 8, 13, ...).

Problem 11.6. (1799번) $N \times N$ 체스판에서 비숍을 놓을 수 있는 칸과 놓을 수 없는 칸이 주어질 때, 어떤 두 비숍도 공격당하지 않도록 놓을 수 있는 최대 개수를 출력하라. $N \leq 10$ 이다.

Problem 11.7. (5070번) 아래와 같은 모양의 퍼즐을 생각하자.



다음은 이 퍼즐에 대한 설명이다.

- 각 블록에는 1부터 3까지 수 중 하나가 적혀 있으며 각각은 정확히 8개씩 있다.
- 목표는 가운데 정사각형의 여덟 블록이 모두 같은 숫자가 되도록 하는 것이다.
- 사용할 수 있는 동작은 7개의 블록으로 이루어진 줄에서 끝 블록을 반대쪽 끝으로 이동시키고 나머지 여섯 개의 블록을 한 칸씩 미는 8가지의 동작이다. 각각의 동작은 A H까지 라벨링되어 있다.

최소 횟수로 동작을 사용하여 퍼즐을 해결하려고 한다. 최소 횟수로 해결하는 방법들 중 사전 순으로 가장 앞선 동작의 나열을 출력하고, 그 나열을 쓸 때 가운데 정사각형의 여덟 블록이 어떤 숫자가 되는지 출력하라.

Problem 11.8. (25795번) 화이트 초콜릿 조각과 다크 초콜릿 조각을 각각 N 개를 붙여 초콜

릿을 만들었다. 이 나열 중 특별한 조건을 만족하는 초콜릿을 예쁜 초콜릿이라고 한다.

- 화이트 + 다크 는 예쁜 초콜릿이다.
- 화이트 + 예쁜 초콜릿 + 다크 는 예쁜 초콜릿이다.
- 예쁜 초콜릿 + 예쁜 초콜릿 은 예쁜 초콜릿이다.
- 위 세 가지 규칙으로 만들어지지 않는 나머지 초콜릿은 예쁜 초콜릿이 아니다.

초콜릿의 점수는 정수 a 에서 시작해 왼쪽부터 화이트 초콜릿 조각이 있으면 b 를 더하고 다크 초콜릿 조각이 있으면 c 를 곱하여 계산한 값을 10^5 으로 나눈 나머지도다. N, a, b, c 가 주어질 때 예쁜 초콜릿들 초콜릿의 점수가 가장 높을 때 최댓값을 구하여라. $N \leq 15$ 이고 a, b, c 는 모두 10^5 이하이다.

11.8 연습문제 C

Problem 11.9. (2071번) $N \times N$ 의 검은 돌만 놓여 있는 바둑판의 집의 개수를 세려고 한다. 기존의 바둑과 다르게 모든 방향에서 블록으로 가려져 있는 곳만을 집으로 인정한다. 즉, 테두리는 둘러싸일 수 없으므로 집이 아니다. 하지만 바둑판을 이미 치워 바둑판의 모양은 알 수 없고 각 N 개의 가로줄과 세로줄에 있는 바둑돌의 수, 각 $2N - 1$ 개의 우상향과 우하향 방향의 대각선에 있는 바둑돌의 수만을 알고 있다. 이 정보를 입력으로 받아 바둑판에 있는 집의 크기를 계산하는 프로그램을 작성하라. 단, $N \leq 15$ 이고 주어진 정보로 바둑판의 모양을 유일하게 결정할 수 있음이 보장된다.

Problem 11.10. (2396번) 길이가 L 인 막대 M 개를 잘라 토막 N 개를 얻었다. 다시 이 토막으로 길이가 같은 막대 여러 개를 만드려고 한다. 가능한 방법들 중에서 토막의 길이가 가장 짧을 때 그 길이를 출력하라. $N \leq 100$ 이고 토막의 길이도 100 이하이다.

Problem 11.11. (18461번**) 길이가 N 인 순열(permutation)에서 최장 증가 부분 수열(LIS)과 같은 길이를 가지며 교집합이 공집합인 증가하는 부분 수열 두 개가 존재하는 순열의 개수를 구하여라. $N \leq 75$ 이다.

- 길이가 N 인 순열은 1부터 N 까지의 수가 정확히 한 번씩 나타나는 수열이다.
- 증가 부분 수열은 길이 N 의 주어진 수열에서 몇 개의 수를 제거하여 만든 증가 수열을 말하고, 최대 증가 부분 수열은 어떤 수열의 증가 부분 수열들 중 가장 긴 길이의 증가 부분 수열이다.

Problem 11.12. (24258번*) n 이 주어질 때 $a_1 a_2 a_3 \cdots a_n = a_1 + a_2 + \cdots + a_n$ 이며 $a_1 \geq a_2 \geq \cdots \geq a_n$ 인 순서쌍 $(a_1, a_2, \cdots, a_n)$ 의 개수를 출력하라. $2 \leq n \leq 10^{11}$ 이고 이러한 순서쌍의 개수는 주어진 조건에서 10^{18} 개 미만임이 보장된다.

Problem 11.13. (26272번) N 가지의 참-거짓 질문에 대한 대답 순서쌍이 모두 다른 2^N 명

의 사람들이 있다. 연구원은 매일 피실험자들을 대상으로 $2N$ 가지 유형의 실험을 진행하는데, $2i - 1$ 번 유형의 실험에서는 i 번 질문에 참이라 대답한 사람만 부르고, $2i$ 번 유형의 실험에서는 i 번 질문에 거짓이라 대답한 사람만 부른다. 피실험자의 편의를 위해 연구원들은 이전 실험에 참가한 피실험자들 중 절반 이상이 다음 실험의 참여 대상이 되도록 순서를 정하려고 한다. 하지만 매번 실험 순서가 같다면 실험이 편향될 수 있기 때문에 연구자들은 x 번 유형의 실험을 y 번째로 시행할 수 있는 실험 시행 순서 중 가장 사전순으로 앞선 것을 택할 것이다. 여러분은 Q 번 x 와 y 가 주어질 때마다 결정되는 실험 시행 순서 a_k 를 구해

$$\sum_{k=1}^{2N} ka_k$$

를 $10^9 + 7$ 로 나눈 나머지를 출력해야 한다. $2 \leq N \leq 10^9$ 이고, $1 \leq Q \leq 2 \cdot 10^5$ 이다.

11.9 힌트

hint 11.1.

hint 11.2.

hint 11.3.

hint 11.4. thregium의 practice_baekjoon 레포지토리

hint 11.5. 두 가지 최적화가 필요하다.

- 똑같은 무게를 가지는 여러 물건들 중에서 ?한 것을 먼저 골라야 한다.
- C 이하인 가장 큰 피보나치 수부터 $t_1, t_2, \dots$ 라 하면, 적당히 작은 상수 k 에 대해 무게가 $t_1, t_2, \dots, t_k$ 인 물건을 모두 고르지 않으면 [...]을 보장할 수 있다.

hint 11.6. N-Queen 문제와 비슷하게, 한 대각선에는 하나의 비숍만 놓일 수 있음을 활용해 보자.

hint 11.7.

hint 11.8. 모든 화이트 초콜릿을 자신보다 뒤에 있는 다크 초콜릿과 대응시키되, 모두 다른 다크 초콜릿과 대응시킬 수 있어야 예쁜 초콜릿이다. 이 조건은 현재까지 사용한 화이트 초콜릿의 개수와 다크 초콜릿의 개수로 쉽게 판별할 수 있다.

hint 11.9. utilForever의 BOJ 레포지토리

hint 11.10. hummingbird-12의 boj-solutions 레포지토리

hint 11.11. <https://qoj.ac/contest/443>

hint 11.12. 총 다섯 개의 단계별 힌트가 있다.

[1]

$i \geq j$ 인 i 와 j 에 대해 (a_i, a_j) 에서 $(a_i + 1, a_j - 1)$ 로 바꾸면, 합은 일정하지만 곱은 감소한다. 즉, 합이 일정한 상황에서 곱의 최소는 $(a_1, 1, 1, \dots, 1)$ 에서 갖는다. 그런데, $(a_1, 1, 1, \dots, 1)$ 에서는 항상 합이 곱보다 크기 때문에, 실질적으로 $(a_1, 2, 1, \dots, 1)$ 을 고려해야 한다. 이 상태에서 합보다 곱이 크다면 불가능하다. 이를 수식으로 풀면,

$$2a_1 \leq n + a_1 \Rightarrow a_1 \leq n, a_1 \times a_2 \times \dots \times a_n = a_1 + a_2 + \dots + a_n \leq 2a_1 \leq 2n$$

가 된다.

[2]

이제, a_2 부터 하나씩 결정하되, 곱으로 경우의 수를 줄일 것이기 때문에 2 이상의 값으로 결정하자. 그러면 2 이상의 값의 개수가 2개일 때부터(1개는 불가능하다) n 개일 때까지 모두 확인해야 하므로, 매번 아직 결정되지 않은 값을 모두 1로 두고 가능한 a_1 이 존재하는지 판단하자. 재귀함수의 파라미터로는 a_k 를 결정할 때의 k , a_k 의 상한, 현재까지 합, 현재까지 곱을 넣으면 충분하다.

[3]

곱에서 합을 뺀 값을 점점 작아지게 만들어보자. 아래 부등식에서 각 순서쌍은, 곱에서 합을 뺀 값을 나타낸다.

$$(a_1, a_2, \dots, a_n) \geq (a_1 + S, a_2, \dots, a_k, 1, 1, \dots, 1) \geq (a_1, a_2, \dots, a_k, 1, 1, \dots, 1)$$

$$\text{where } S = (a_{k+1} - 1) + (a_{k+2} - 1) + \dots + (a_n - 1)$$

즉, a_k 까지 수를 정하고 그 이후는 1이라 가정한 뒤 가능한 a_1 을 구했을 때, a_2 보다 작으면 그 이후는 무조건 불가능하다. $(a_1 = a_2, a_2, \dots, a_k, 1, 1, \dots, 1)$ 에서 곱에서 합을 뺀 값이 이미 양수이기 때문에, $(a_1 \geq a_2, a_2, \dots, a_k, 1, 1, \dots, 1)$ 에서도 양수이고, $(a_1, a_2, \dots, a_k, a_{k+1}, \dots, a_n)$ 에서도 양수가 되기 때문이다.

[4]

[3]에 의해 a_{k+1} 부터 전부 1일 때 가능한 유리수 a_1 의 값이 a_1 의 상한이 된다. 그리고 [1]에 의해 그 때 $a_1 + a_2 + \dots + a_n$ 이 최대이다. 이렇게 구한 합의 상한과 하한 안에, $a_2 \times a_3 \times \dots \times a_k$ 의 배수가 있어야 한다. 이를 활용하면 경우의 수를 더 줄일 수 있다.

[5]

[2]를 확장해, a_3 부터 하나씩 결정할 수 있다. 이 풀이의 장점은 현재까지의 곱에 따라 두 가지 방법을 사용할 수 있다는 것이다. $a_3 a_4 \dots a_k$ 가 작을 때는 a_2 의 상한이 $O\left(\sqrt{\frac{n}{a_3 a_4 \dots a_k}}\right)$ 이므로, 가능한 a_2 에 따라 a_1 을 구한다. 반면, $a_3 a_4 \dots a_k$ 가 클 때는 [4]와 같이 $a_1 + a_2 + \dots + a_n$ 의 값의 상한과 하한을 구하고, 그 범위에 속하는 $a_3 a_4 \dots a_k$ 의 배수마다 이차방정식의 근의 공식으로 a_1 과 a_2 를 한번에 구하는 것이다. 이 방법은 $O(n \times (a_3 a_4 \dots a_k)^{-2})$ 의 시간을 요구한다.

https://old.infosbg.com/Shumen_2021.html

hint 11.13. 이 문제는 직접 규칙을 찾으며 주어진 입력에 따라 케이스를 나누어 수학적으로 해결할 수도 있다. 하지만, 여기서는 백트래킹 풀이에 대한 힌트를 수록한다.

[1]

직전 점호 인원 중 절반이 넘게 소집해제되지 않는다는 것과, a_k 와 a_{k+1} 을 2로 나눈 몫이 다름은 동치이다.

[2]

만약, x 번 유형의 점호를 y 번째로 실시해야 한다는 조건이 없다면, 1, 3, 2, 4, ...와 같은 순서로 두는 것이 사전 순으로 가장 앞선다.

[3]

[2]의 순서에서 x 번 유형의 점호를 y 번째로 실시하기 위해 원래 x 번 유형이 있던 자리 주변과 y 번째 주변의 순서를 바꾸어야 할 것이다. 각각에서 사전순 최소를 백트래킹으로 탐색하자.

Chapter 12

분할 정복

분할 정복(divide and conquer)은 큰 문제를 작은 문제로 분할한 후 다시 합쳐 문제를 해결하는 방식이다. 분할 정복은 작은 부분에서 구한 결과를 바탕으로 다시 합치는 과정이 간단할 때 유용하다. 이러한 분할 정복은 최적해 문제에 자주 사용되는 특징이 있다. 이번 장에서는 앞에서 배웠던 병합 정렬과, 다른 예시를 통해 분할 정복의 활용성에 대해 알아본다.

12.1 다시 보는 병합 정렬

병합 정렬 과정(2.1.5절)을 다시 보자. 병합 정렬 과정의 핵심을 요약하면 전체 구간을 절반으로 분할한 후 분할된 각 구간이 정렬되어 있음을 이용하여 전체 구간을 $O(N)$ 의 시간에 정렬하였다. 이러한 방식을 분할 정복 알고리즘이라고 한다. 이러한 알고리즘의 특징 때문에 대부분의 분할 정복 알고리즘은 재귀 함수를 통해 구현된다. Code 2.5를 보면 `mergesort(st, ed)`가 자기 자신을 다시 호출하는 재귀함수임을 알 수 있다.

분할 정복은 병합 정렬과 같은 경우 외에도 최적해 문제에서 많이 사용된다. 만약 어떤 구간을 절반으로 분할하여 양쪽에서의 최적값을 찾았다면, 병합 과정에서는 중심을 무조건 포함하는 경우에 대해서만 최적값을 찾으면 되고, 이는 빠른 시간복잡도에 처리할 수 있다. 나중에 배우겠지만, 이러한 분할은 배열 뿐만 아니라 트리에서도 할 수 있어서() 분할 정복 자체는 굉장히 많은 문제와 같이 나올 수 있다고 볼 수 있다.

다음 부분에서 예시 문제인 가장 가까운 두 점을 찾는 문제를 통해 분할 정복으로 최적해 문제를 푸는 방법을 알아보자.

12.2 가장 가까운 두 점 찾기

다음 문제를 보자. <https://www.acmicpc.net/problem/2261>과 같다.

Example 12.1. 좌표평면에 주어진 N 개의 점 중 가장 가까운 두 점 사이의 거리의 제곱을 출력하는 프로그램을 작성하라.

모든 두 점 쌍에 대해 거리를 구하면 거리를 구하는 과정은 상수 시간에 할 수 있으므로 시간복잡도가 $O(N^2)$ 이 될 것이다. 이제 분할 정복을 이용하여 *like-linear*한 시간에 문제를 해결해보자. 먼저 점을 x 좌표 순으로 점을 정렬한 후 x 좌표를 기준으로 좌우로 분할한다. 그리고 왼쪽과 오른쪽에 대해 각각 최단거리를 구한 후 그 중간 부분에 있는 점들의 최단거리를 구할 것이다. 구체적으로 다음과 같이 진행한다.

- 점의 개수가 하나이면 리턴한다.
- 점의 개수가 두 개 이상이면
 - x 좌표 기준으로 점을 절반으로 나누어 왼쪽 점들 사이에서의 최단거리와 오른쪽 점들 사이에서의 최단거리를 구한다. (함수 호출)
 - 현재까지의 최단거리 d 에 대해 경계선 $x = x_c$ 에서 d 이내의 영역, 즉 $[x_c - d, x_c + d]$ 에 속하는 점들을 찾고 y 좌표 기준으로 정렬한다.
 - 각 점에 대해 y 좌표 차이가 d 이상이 되기 전까지 비교한다. 이후에는 무조건 거리 차이가 d 보다 크기 때문에 비교할 필요가 없다.

아래는 구현 코드이다.

Code 12.1. Example 12.1의 정답 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4  #define x first
5  #define y second
6  int n;
7  pair<ll, ll> point[1010101], ylist[1010101], temp[1010101];
8
9  bool compare(pair<ll, ll> p1, pair<ll, ll> p2) {
10     return p1.y < p2.y;
11 }
12
13 ll dis(pair<ll, ll> P, pair<ll, ll> Q) {
14     return (P.x - Q.x) * (P.x - Q.x) + (P.y - Q.y) * (P.y - Q.y);
15 }
```

```
16
17 ll solve(int st, int ed) {
18     if (st == ed) {
19         ylist[st] = point[st];
20         return 9e18;
21     }
22
23     int mid = (st + ed) / 2;
24     ll stdx = point[mid].x, d = 9e18;
25     d = min(d, solve(st, mid));
26     d = min(d, solve(mid + 1, ed));
27
28     sort(ylist + st, ylist + ed + 1, compare);
29
30     int r = -1;
31     for (int i = st; i <= ed; i++) {
32         if ((stdx - ylist[i].x) * (stdx - ylist[i].x) < d) temp[++r] =
            ylist[i];
33     }
34
35     for (int i = 0; i <= r; i++) {
36         for (int j = i + 1; j <= r; j++) {
37             if ((temp[j].y - temp[i].y) * (temp[j].y - temp[i].y) >= d)
                break;
38             d = min(d, dis(temp[i], temp[j]));
39         }
40     }
41     return d;
42 }
43
44 int main() {
45     ios_base::sync_with_stdio(false);
46     cin.tie(0);
47
48     cin >> n;
49     for (int i = 1; i <= n; i++) cin >> point[i].x >> point[i].y;
50
51     sort(point + 1, point + n + 1);
52     cout << solve(1, n) << "\n";
53
54     return 0;
```

12.3 마스터 정리

이제 위 알고리즘의 시간 복잡도를 알아내야 한다. 이를 위해 크기 위 알고리즘의 시간복잡도를 $T(N)$ 이라고 놓자. 그러면 병합 과정의 시간 복잡도는 정렬이 지배하므로(Problem 12.1) 다음 식이 성립한다.

$$T(N) = \begin{cases} O(1) & \text{if } N = 1 \\ 2T(N/2) + O(N \log N) & \text{otherwise} \end{cases}$$

이러한 형태의 식에서 $T(N)$ 을 구하는 방법은 이미 알려져 있다. 바로 Master Theorem이다.

Theorem 12.1 (Master Theorem). 다음과 같은 관계식

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ aT(n/b) + f(n) & \text{otherwise} \end{cases}$$

에서 $a \geq 1$, $b > 1$ 이고 $f(n)$ 이 점근적으로 양의 함수값을 가지는 함수일 때 어떤 양수 ε 대해

- $f(n) \in O(n^{\log_b a - \varepsilon}) \implies T(n) \in \Theta(n^{\log_b a})$
- $f(n) \in \Theta(n^{\log_b a}) \implies T(n) \in \Theta(n^{\log_b a} \log n)$
- $f(n) \in \Omega(n^{\log_b a + \varepsilon})$, $c < 1$ 과 충분히 큰 n 에 대해 $af(n/b) \leq cf(n) \implies T(n) \in \Theta(f(n))$

가 성립한다.

Remark. 마스터 정리의 두 번째 경우를 확장하면 다음을 얻는다:

$$f(n) \in \Theta(n^{\log_b a} \log^n) \implies T(n) \in \Theta(n^{\log_b a} \log^{k+1} n)$$

쉽게 설명하면 $O(n^{\log_b a})$ 보다 $f(n)$ 의 시간 복잡도가 더 작으면 $T(n)$ 의 시간복잡도가 $O(n^{\log_b a})$ 가 되고, 같으면 $O(n^{\log_b a} \log n)$, 크면 $O(f(n))$ 이 된다. 물론 정확히는 다르지만 재귀적 함수의 시간복잡도를 예측하는데 있어 이 정도의 이해로도 충분할 것이다.

이제 위 알고리즘의 시간복잡도를 마스터 정리로 구해보자. $a = 2$, $b = 2$, $f(N) = O(N \log N)$ 이므로 $T(N) = O(N \log^2 N)$ 이다.

12.4 연습문제 A

Problem 12.1. Code 12.1의 병합 과정에서 왜 정렬이 시간 복잡도를 지배하려면 병합의 세 번째 과정에서 각 점마다 확인해야 하는 점의 개수가 제한되어야 한다. 실제로 한 점마다 확인해야 하는 점의 개수는 $O(1)$ 이다. 이를 설명하라.

Problem 12.2. 사실 Example 12.1는 $O(N \log N)$ 에 해결할 수 있다. 해결하는 코드를 작성하고, 실제 백준 문제에 제출해 시간을 비교해보라. (*hint.* 병합 정렬의 아이디어를 활용하라.)

12.5 연습문제 B

Problem 12.3. <https://www.acmicpc.net/problem/1074>

Problem 12.4. <https://www.acmicpc.net/problem/1780>

Problem 12.5. <https://www.acmicpc.net/problem/2630>

Problem 12.6. <https://www.acmicpc.net/problem/6182>

Problem 12.7. <https://www.acmicpc.net/problem/6785>

Problem 12.8. <https://www.acmicpc.net/problem/14600>

Problem 12.9. <https://www.acmicpc.net/problem/24110>

Problem 12.10. <https://www.acmicpc.net/problem/24460>

Problem 12.11. <https://www.acmicpc.net/problem/1030>

Problem 12.12. <https://www.acmicpc.net/problem/1517>

Problem 12.13. <https://www.acmicpc.net/problem/1588>

Problem 12.14. <https://www.acmicpc.net/problem/1891>

Problem 12.15. <https://www.acmicpc.net/problem/2261>

Problem 12.16. <https://www.acmicpc.net/problem/2339>

Problem 12.17. <https://www.acmicpc.net/problem/5904>

Problem 12.18. <https://www.acmicpc.net/problem/8204>

Problem 12.19. <https://www.acmicpc.net/problem/11219>

Problem 12.20. <https://www.acmicpc.net/problem/11600>

Problem 12.21. <https://www.acmicpc.net/problem/14956>

Problem 12.22. <https://www.acmicpc.net/problem/15278>

Problem 12.23. <https://www.acmicpc.net/problem/16885>

Problem 12.24. <https://www.acmicpc.net/problem/18077>

Problem 12.25. <https://www.acmicpc.net/problem/19808>

Problem 12.26. <https://www.acmicpc.net/problem/22346>

Problem 12.27. <https://www.acmicpc.net/problem/24069>

12.6 연습문제 C

Problem 12.28. <https://www.acmicpc.net/problem/1187>

Problem 12.29. <https://www.acmicpc.net/problem/3406>

Problem 12.30. <https://www.acmicpc.net/problem/5482>

Problem 12.31. <https://www.acmicpc.net/problem/10842>

Problem 12.32. <https://www.acmicpc.net/problem/15604>

Problem 12.33. <https://www.acmicpc.net/problem/15842>

Problem 12.34. <https://www.acmicpc.net/problem/15863>

Problem 12.35. <https://www.acmicpc.net/problem/16128>

Problem 12.36. <https://www.acmicpc.net/problem/16349>

Problem 12.37. <https://www.acmicpc.net/problem/16904>

Problem 12.38. <https://www.acmicpc.net/problem/17017>

Problem 12.39. <https://www.acmicpc.net/problem/17679>

Problem 12.40. <https://www.acmicpc.net/problem/18314>

Problem 12.41. <https://www.acmicpc.net/problem/18614>

Problem 12.42. <https://www.acmicpc.net/problem/18621>

Problem 12.43. <https://www.acmicpc.net/problem/18732>

Problem 12.44. <https://www.acmicpc.net/problem/18827>

Problem 12.45. <https://www.acmicpc.net/problem/19635>

Problem 12.46. <https://www.acmicpc.net/problem/20464>

Problem 12.47. <https://www.acmicpc.net/problem/20799>

Problem 12.48. <https://www.acmicpc.net/problem/21543>

Problem 12.49. <https://www.acmicpc.net/problem/22922>

Problem 12.50. <https://www.acmicpc.net/problem/27657>

Part III

자료구조 (Easy)

Chapter 13

연결 리스트

연결리스트는 배열이 아닌 다른 자료구조의 이해를 위한 기본적인 자료구조이다. 이 장에서는 연결리스트에 대한 학습과 더불어 자료구조를 공부할 때 어떠한 점을 생각해야 하는지 같이 살펴보겠다.

13.1 자료구조를 왜 만드는가?

자료구조는 ‘자료’와 ‘구조’가 합쳐진 말로, 자료와 구조가 모두 있어야 자료구조라 할 수 있다. 여기서 자료는 말 그대로 저장하고 싶은 데이터를 말하는 것이고, 구조는 어떤 ‘연산’을 얼마나 빠르게 처리할 수 있는지와 대응된다. 여기서 연산은 삽입, 삭제, 탐색과 같이 데이터와 상호작용하는 것이다.

배열은 가장 기본적인 자료구조로 볼 수 있다. 먼저, 배열은 저장하고자 하는 데이터 타입 T 가 나열된 것이므로 당연히 ‘자료’가 있다.

0	1	2	3	4	5
---	---	---	---	---	---

‘구조’에 해당하는 연산도 있을까? 사실, 처음 배열을 배웠을 때부터 연산을 하고 있었다. `cin` » `arr[i]`를 반복문 안에서 반복하는 것은, 마지막 값이 $i-1$ 번 인덱스에 저장되어 있으므로, 자료구조의 끝에 삽입하는 연산과 같다. 실제로, 배열은 삽입, 삭제, 탐색 모두 가능하다.

모든 자료구조는 **특별한 장점**이 있어서 쓴다. 배열의 특별한 장점은 ‘임의 접근(random access)’을 빠르게 할 수 있다는 것이다. 정확히는, $O(1)$ 의 시간복잡도에 가능하다. 그리고, 모든 연산을 빠르게 처리할 수 있는 자료구조는 없다. 배열은 ‘임의 접근이 빠르다’는 장점을 얻은 대신에

‘삽입과 삭제가 느리다’는 단점을 얻었다. 삽입과 삭제는 $O(N)$ 의 시간이 필요하다. 삽입을 위해서는 그보다 오른쪽에 있는 것들을 전부 오른쪽으로 한 칸 옮겨야 하고, 삭제를 위해서는 그보다 오른쪽에 있는 것들을 전부 왼쪽으로 한 칸 옮겨야 하기 때문이다. 연결리스트는 반대이다. ‘임의 접근이 느리다’는 단점이 있지만, ‘삽입과 삭제가 빠르다’는 장점을 얻었다.

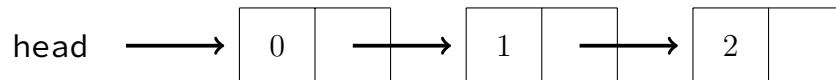
자료구조마다 장단점이 다르다는 것 자체가 자료구조를 만들어야 하는 이유가 된다. 어떤 문제가 삽입과 삭제의 횟수가 매우 많고 랜덤 접근은 단 한 번도 하지 않는다면 연결리스트를 활용하는 것이 배열을 활용하는 것보다 더 나은 시간복잡도로 문제를 해결할 수 있을 것이다. 이 관점에서 보면 자료구조가 먼저 만들어진 것이 아니라, 문제가 먼저 제시되었고, 그 문제를 위한 자료구조가 개발된 것에 가깝다. 비유를 하자면, 자료구조는 수학에서 정리(Theorem)의 느낌이다. 정리를 쓰면 정리를 증명하는 과정을 건너뛸 수 있는 것처럼, 자료구조를 쓰면 문제에서 필요한 최적화를 건너뛸 수 있다.

정리하면, 문제를 보고 그 문제에서 ‘여러 번 시행하기’를 요구하는 연산을 빠르게 수행할 수 있는 자료구조를 사용하는 것이 ‘자료구조를 사용한 PS 문제의 해결’이라고 말할 수 있다. 따라서, 자료구조는 ‘빠르게 할 수 있는 연산’과 똑같은 개념으로 받아들여야 한다.

지금부터 우리가 배울 자료구조는 문제에서 ‘여러 번 시행하기’를 자주 요구해서 잘 알려진 것들이다. 특히, 이 파트의 자료구조는 특히 자주 요구하므로, 꼭 각 자료구조가 어떤 연산과 같은지 이해하고 다음으로 넘어가길 바란다.

13.2 연결리스트

연결리스트는 이름에서도 나타나듯이 연결을 통해 자료를 저장하는 방식이다.

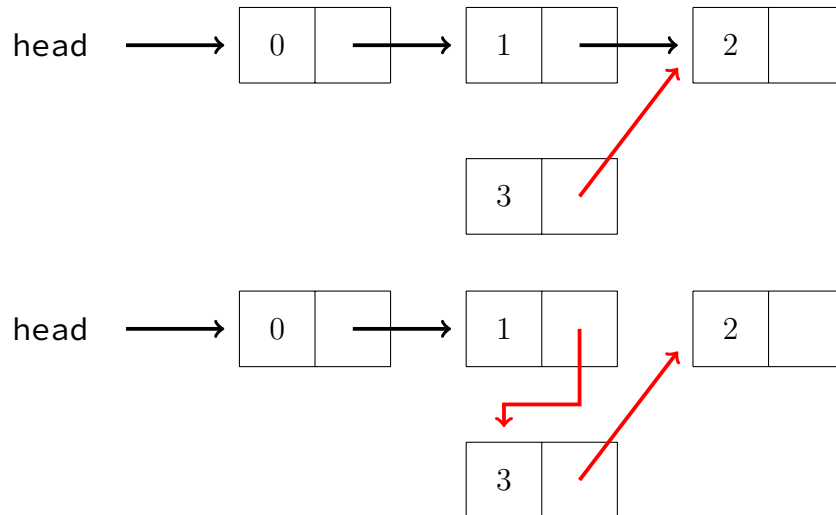


하나의 데이터에는 데이터 뿐만 아니라 포인터가 붙어 있다. 이 하나의 쌍을 ‘노드(node)’라고 부를 것이다. 각 노드의 포인터는 다음 노드를 가리킴으로써 다음 노드와 ‘연결’하고, 특별히 헤드 포인터(그림에서 head)가 시작 노드를 가리킨다. 이 헤드 포인터를 참조하면 연결리스트에 접근할 수 있는 것이다.

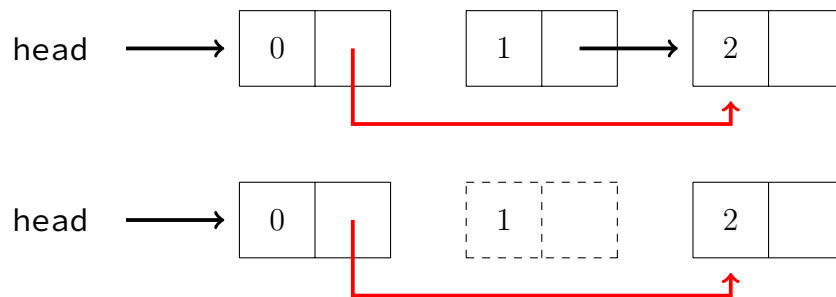
앞에서 연결리스트는 임의 접근을 포기하고 삽입과 삭제를 얻었다고 했다. 실제로 연결리스트에서 임의 접근, 삽입, 삭제가 어떻게 동작하는지 시뮬레이션해보자.

먼저 임의 접근이다. 임의로 k 번째(0-based) 위치에 접근하고 싶다면, 헤드 포인터가 가리키는 노드에서 k 번 더 포인터를 통해 이동한 위치의 값을 가져오면 된다. 연결리스트에서는 발판들을 건너 넘어가는 것처럼, 순서대로 직접 가야 한다. 따라서 k 번의 연산이 필요하고, 이를 점근 표기법으로 나타내면 $O(N)$ 이 되는 것이다.

두 번째로 삽입이다. ‘현재 노드 뒤’(Problem 18.1)에 삽입하는 연산을 생각해보자. 그러면 새로 삽입되는 노드의 포인터를 현재 노드의 포인터로 바꾸고, 현재 노드의 포인터를 새로 삽입되는 노드로 바꾸면 된다. 아래 그림을 참고하자.



세 번째로 삭제이다. ‘현재 노드 뒤’(Problem 18.2)에 있는 노드를 삭제하는 연산을 생각해보자. 그러면 현재 노드의 포인터를 현재 노드에서 바로 뒤에 있는 노드의 포인터로 바꾸고, 현재 노드 뒤에 있는 노드를 제거하면 된다. 아래 그림을 참고하자.



13.3 연결리스트의 구현

연결리스트를 구현하려면 노드 구조체에 데이터 타입 T와 노드를 가리키는 포인터를 담아야 한다. 이렇게 되면 자기참조로써 컴파일러에서 무한루프에 빠지는거 아닐까? 하지만 포인터 자체는 unsigned long long int와 같고, 별(*) 연산자와 붙었을 때 구조체의 정보를 불러오기 때문에 이러한 일은 발생하지 않는다.

C는 클래스와 관련된 것이 하나도 존재하지 않아서 생성자와 소멸자를 사용할 수 없지만, C++에서는 구조체로 선언하였다 하더라도 구조체 자체가 클래스로 구현되어 있어서 생성자와 소

멸자를 사용할 수 있다. 따라서 간단하게 `new`로 노드를 만들고, `del`로 노드를 제거할 수 있다. 메모리 누수 방지를 위해 노드 제거 작업은 꼭 필요하다.

이제 다음 코드 Code 18.1로 연결리스트의 구현을 보자. 포인터를 사용하는 자료구조의 구현은 본질적으로 이와 같고, 각 연산들의 로직만 다르므로 아래 구현 방식에는 익숙해졌으면 한다. 실제 시험장에서 연결리스트를 구현할 일은 없겠지만, 포인터를 사용하는 자료구조 구현은 나중에 때때로 쓰이기 때문이다.

Code 13.1. 연결리스트 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int n;
4
5  struct Node {
6      int a;
7      Node *nxt;
8      Node() {
9          a = 0;
10         nxt = NULL;
11     }
12 };
13 Node *head = new Node();
14
15 void add(Node *node, int val) {
16     Node *added = new Node();
17     added->a = val;
18     added->nxt = node->nxt;
19     node->nxt = added;
20 }
21
22 void rem(Node *node) {
23     Node *tar = node->nxt;
24     if (tar == NULL) return;
25     node->nxt = tar->nxt;
26     delete tar;
27 }

```

구조체 `Node` 안에 선언된 `Node()` 함수는 생성자로, `new`를 사용하여 구조체 변수를 만들 때 자동으로 실행된다. 함수 이름을 구조체 이름과 똑같이 설정하면 된다. 만약, 소멸자가 필

요하다면 `Node()`와 같이 물결을 붙여서 함수 이름을 정하면 된다. 이 소멸자는 `delete`를 사용하여 구조체 변수를 제거할 때 자동으로 실행된다.

13.4 STL의 연결리스트

STL에는 연결리스트가 있다. <https://en.cppreference.com/w/cpp/container/list>에 가면 어떤 함수를 지원하고 어떻게 사용하는지, 시간복잡도까지 모두 확인할 수 있다. 그 중에서 중요한 함수만을 소개하겠다. 그 전에 몇 가지 용어를 정의하자. STL container에서는 C언어의 포인터와 같은 기능을 하는 것을 **iterator**라고 표현한다. 그리고 iterator로 표현된 구간은 기본적으로 반열린구간 $[s, e)$ 로 처리된다. 또, 저장하는 데이터의 자료형은 `T`로 표기한다. 즉, 함수 인자에 `T`가 있다면 데이터를 함수 인자로 넣으라는 뜻이다.

- `front()`를 통해 제일 앞의 노드를 참조할 수 있다.
- `back()`를 통해 제일 뒤의 노드를 참조할 수 있다.
- `begin()`를 통해 제일 앞의 노드를 가리키는 포인터를 참조할 수 있다.
- `end()`를 통해 제일 뒤의 노드의 **다음 노드**를 가리키는 포인터를 참조할 수 있다.
- `push_front(T)`를 통해 제일 앞에 새로운 데이터를 추가할 수 있다.
- `push_back(T)`를 통해 제일 뒤에 새로운 데이터를 추가할 수 있다.
- `pop_front()`를 통해 제일 앞의 노드를 제거할 수 있다.
- `pop_back()`를 통해 제일 뒤의 노드를 제거할 수 있다.
- `erase(iterator)`를 통해 포인터(iterator)가 가리키는 노드를 제거할 수 있다.
- `insert(iterator, T)`를 통해 포인터(iterator)의 위치에 새로운 데이터를 추가할 수 있다.
- `empty()`를 통해 비어있는지 여부를 확인할 수 있다.
- `size()`를 통해 저장된 데이터의 개수를 확인할 수 있다.
- `clear()`를 통해 모든 데이터를 지울 수 있다.

상술된 함수들은 대부분의 STL container에서 공통적으로 존재하는 함수들이다. 따라서 같은 이름의 함수를 앞으로 계속 만나게 된다. 그리고, 각 함수들이 매번 비슷한 기능을 하므로 다양한 STL container를 만나면서 점차 친숙해지면 된다.

코드에서 사용하는 방법은 다음 예시 Code 13.2를 참고하라.

Code 13.2. `std::list`의 예시 코드

```
1 #include <algorithm>
2 #include <iostream>
3 #include <list>
4
5 int main()
```

```

6 {
7     // Create a list containing integers
8     std::list<int> l = {7, 5, 16, 8};
9
10    // Add an integer to the front of the list
11    l.push_front(25);
12    // Add an integer to the back of the list
13    l.push_back(13);
14
15    // Insert an integer before 16 by searching
16    auto it = std::find(l.begin(), l.end(), 16);
17    if (it != l.end())
18        l.insert(it, 42);
19
20    // Print out the list
21    std::cout << "l = { ";
22    for (int n : l)
23        std::cout << n << ", ";
24    std::cout << "};\n";
25 }

```

13.5 연습문제 A

Problem 13.1. 연결리스트에서 헤드 포인터와 삽입하고 싶은 위치의 인덱스 k (0-based)가 주어질 때 연산의 시간복잡도는 무엇인가? 삽입이 아닌 삭제에서는 어떠한가?

Problem 13.2. 연결리스트에서 **현재 포인터**를 $O(1)$ 에 삭제하려면 어떻게 수정하면 될까? 방법을 고안하고 직접 구현해보라.

13.6 연습문제 B

Problem 13.3. (5680번) <https://www.acmicpc.net/problem/5680>

Problem 13.4. (3294번) <https://www.acmicpc.net/problem/3294>

Problem 13.5. (4621번) <https://www.acmicpc.net/problem/4621>

Problem 13.6. (6542번) <https://www.acmicpc.net/problem/6542>

Problem 13.7. (22645번) <https://www.acmicpc.net/problem/22645>

Problem 13.8. (23309번) <https://www.acmicpc.net/problem/23309>

Problem 13.9. (25252번) <https://www.acmicpc.net/problem/25252>

Problem 13.10. (30885번) <https://www.acmicpc.net/problem/30885>

Problem 13.11. (31857번) <https://www.acmicpc.net/problem/31857>

Problem 13.12. (3217번) <https://www.acmicpc.net/problem/3217>

Problem 13.13. (21201번*) <https://www.acmicpc.net/problem/21201>

Problem 13.14. (17820번*) <https://www.acmicpc.net/problem/17820>

Problem 13.15. (23774번*) <https://www.acmicpc.net/problem/23774>

Problem 13.16. (28974번*) <https://www.acmicpc.net/problem/28974>

13.7 연습문제 C

Problem 13.17. (26032번) <https://www.acmicpc.net/problem/26032>

Problem 13.18. (24971번*) <https://www.acmicpc.net/problem/24971>

Problem 13.19. (17692번**) <https://www.acmicpc.net/problem/17692>

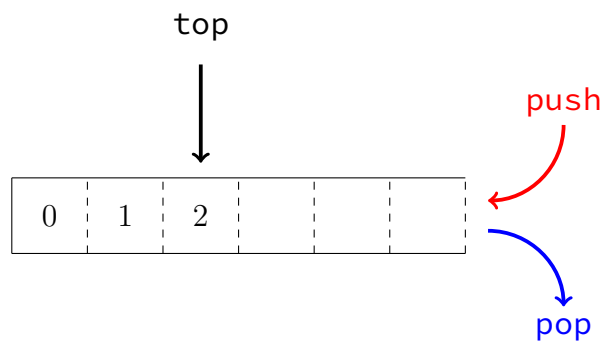
Chapter 14

스택, 큐, 덱

스택, 큐, 덱은 데이터의 양 끝에서 연산이 이루어지는 선형 자료구조라는 공통된 특징이 있다. 이 자료구조들은 양 끝에서의 연산만이 요구되는 상황에서 장점을 갖는다. 이번 장에서는 스택, 큐, 덱이 지원하는 연산들을 알아보고, 각각이 어떤 문제에 적합한지 알아볼 것이다.

14.1 스택

스택(stack)은 이름에서도 드러나지만 데이터를 쌓아가는 구조를 가진다. 데이터를 쌓아가는 구조를 만드려면 어떤 연산을 지원해야 할까? 제일 위의 데이터 참조, 데이터를 위에 추가, 제일 위의 데이터 제거의 세 가지 연산만 제공하면 충분할 것이다.



즉, 스택에서는 제일 위의 데이터가 굉장히 중요하다. 그래서 제일 위의 데이터를 **top**이라고 한다. **top**이라는 용어를 이용해서 다시 표현하면, 스택은 **top**의 값 가져오기, **top** 위에 새로운 데이터를 쌓기, **top** 제거하기의 세 가지 연산이 가능하다고 말할 수 있겠다. 그리고 이

모든 연산은 $O(1)$ 의 시간에 가능하다.

이처럼 제일 나중에 들어온 데이터가 제일 먼저 나가는 구조를 LIFO(last-in-first-out) 구조라고 부른다. 이러한 장점은 **나중에 것을 먼저 처리**해야만 하는 모든 문제에 적용될 수 있다. 나중에 호출된 함수의 작업이 끝나야 그 전에 호출된 함수를 실행하는 재귀함수나, 나중에 작성한 텍스트부터 지워야 하는 되돌리기(undo) 기능 등을 예시로 들 수 있다.

STL에는 스택이 있다. <https://en.cppreference.com/w/cpp/container/stack>에 가면 어떤 함수를 지원하고 어떻게 사용하는지, 시간복잡도까지 모두 확인할 수 있다. 다음은 대표적으로 사용되는 함수들이다.

- `top()`를 통해 제일 위(앞)의 값을 참조할 수 있다.
- `push(T)`를 통해 제일 위(앞)에 값을 추가할 수 있다.
- `pop()`를 통해 제일 위(앞)의 값을 제거할 수 있다.
- `empty()`를 통해 비어있는지 여부를 확인할 수 있다.
- `size()`를 통해 저장된 데이터의 개수를 확인할 수 있다.

실제로 STL의 스택을 사용하여 다음의 대표적인 스택 활용 문제를 해결해보자.

Example 14.1. (9012번) ‘(’와 ‘)’만으로 이루어진 문자열을 **괄호 문자열**이라 하자. **올바른 괄호 문자열**은 다음 세 조건 중 하나 이상을 만족해야 한다.

- 문자열 ‘()’이거나,
- 올바른 괄호 문자열 S 에 대해 (S) 이거나,
- 올바른 괄호 문자열 S 와 T 에 대해 ST 이다.

주어진 문자열이 올바른 괄호 문자열인지 판별하라.

이 문제는 왜 스택으로 해결할 수 있을까? 다음의 관찰을 생각해보자.

Observation. 괄호 문자열은 바깥 괄호쌍 안에 안쪽 괄호쌍이 규칙에 맞게 있어야 한다. 즉, **나중에** 나온 열린 괄호부터 닫힌 괄호와 매칭해줘야 한다. 이는 스택의 장점인 **나중에** 것을 먼저 처리하는 것으로 해결할 수 있다. 따라서 스택은 이 문제에 적절한 자료구조이다.

위의 사고 과정을 다시 되짚어보자. 우리는 문제를 보고 문제의 해결을 위한 알고리즘인 ‘나중에 ... 매칭해줘야 한다’를 고안했다. 그리고, 알고리즘의 요구사항인 ‘나중에’에 장점이 있는 자료구조인 스택을 떠올리고, 스택으로 문제를 해결할 수 있음을 알게 된 것이다.

다시 한번 말하지만, 자료구조는 **빠르게 할 수 있는 연산**과 같다. 따라서, 자료구조를 쓸 수 있는지 확인하려면, 자료구조의 장점 – 어떤 연산을 빠르게 할 수 있는가 – 과 알고리즘의 요구사항이 일치하는지 확인하면 된다.

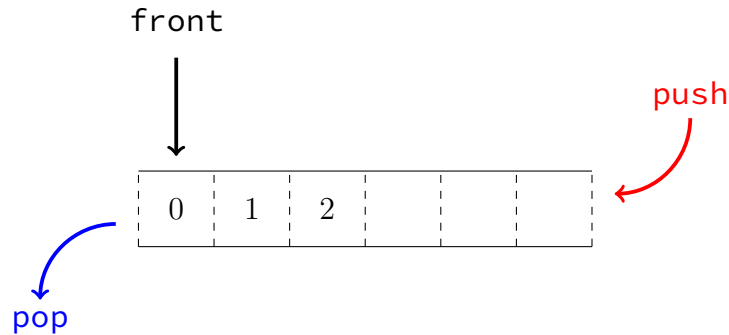
이제 관찰을 바탕으로 실제로 구현을 해보자. Code 14.1을 보자.

Code 14.1. Example 14.1을 해결하는 코드

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void solve(string s) {
5      stack<int> stk;
6      for (int i = 0; i < s.length(); i++) {
7          if (s[i] == '(') stk.push('(');
8          else {
9              if (stk.size() == 0) {
10                 cout << "NO\n";
11                 return;
12             }
13             else stk.pop();
14         }
15     }
16     if (stk.size() == 0) cout << "YES\n";
17     else cout << "NO\n";
18 }
19
20 int main() {
21     ios_base::sync_with_stdio(false);
22     cin.tie(0);
23
24     int t;
25     cin >> t;
26     while (t--) {
27         string s;
28         cin >> s;
29         solve(s);
30     }
31 }
```

14.2 큐

큐(queue)은 이름에서도 드러나지만 대기열과 비슷한 구조를 가진다. 대기열과 비슷한 구조를 만드려면 어떤 연산을 지원해야 할까? 제일 앞의 데이터 참조, 데이터를 뒤에 추가, 제일 앞의 데이터 제거의 세 가지 연산만 제공하면 충분할 것이다.



즉, 큐에서는 제일 앞의 데이터가 굉장히 중요하다. 그래서 제일 앞의 데이터를 **front**라고 한다. 또, 제일 뒤의 데이터를 **back**이라고 한다. **front**와 **back**이라는 용어를 이용해서 다시 표현하면, 스택은 **front**의 값 가져오기, **back** 뒤에 새로운 데이터를 추가하기, **front** 제거하기의 세 가지 연산이 가능하다고 말할 수 있겠다. 그리고 이 모든 연산은 $O(1)$ 의 시간에 가능하다.

이처럼 제일 먼저 들어온 데이터가 제일 먼저 나가는 구조를 FIFO(first-in-first-out) 구조라고 부른다. 이러한 장점은 **이른 것을 먼저 처리**해야만 하는 모든 문제에 적용될 수 있다. 선착순으로 입장하는 대기열(하드웨어에서는 프로세스 스케줄링)이나 BFS(링크) 등을 예시로 들 수 있다.

STL은 큐도 지원한다. <https://en.cppreference.com/w/cpp/container/queue>에 가면 어떤 함수를 지원하고 어떻게 사용하는지, 시간복잡도까지 모두 확인할 수 있다. 다음은 대표적으로 사용되는 함수들이다.

- `front()`를 통해 제일 앞의 노드를 참조할 수 있다.
- `back()`를 통해 제일 뒤의 노드를 참조할 수 있다.
- `push(T)`를 통해 제일 뒤에 값을 추가할 수 있다.
- `pop()`를 통해 제일 앞의 값을 제거할 수 있다.
- `empty()`를 통해 비어있는지 여부를 확인할 수 있다.
- `size()`를 통해 저장된 데이터의 개수를 확인할 수 있다.

실제로 STL의 큐를 사용하여 다음의 대표적인 큐 활용 문제를 해결해보자.

Example 14.2. (1158번) 1번부터 N 번까지의 사람이 시계방향으로 둘러앉아 있다. 주어진 양의 정수 K 에 대해 모두가 제거될 때까지 시계방향으로 이동하며 K 번째 사람을 제거한다. 제거되는 사람의 번호를 순서대로 출력하라.

이 문제는 왜 큐로 해결할 수 있을까? 다음의 관찰을 생각해보자.

Observation. 내가 가만히 있고, N 명의 사람들이 움직인다고 생각해보자. 사람들은 한 줄로

서 있고, K 번째 사람을 만나기 전까지 사람을 줄의 제일 뒤로 보내고, K 번째 사람을 만나면 그 사람을 죽일 것이다. 즉, 대기열과 비슷한 알고리즘을 해결할 수 있으므로, 큐는 이 문제에 적합한 자료구조이다.

이제 관찰을 바탕으로 실제로 구현을 해보자. Code 14.2을 보자.

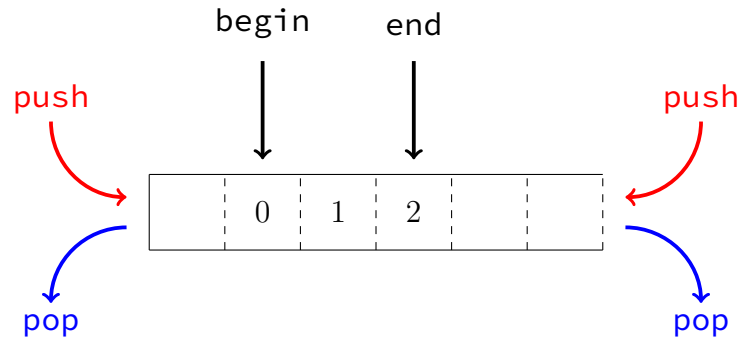
Code 14.2. Example 14.2을 해결하는 코드

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios_base::sync_with_stdio(false);
6     cin.tie(0);
7
8     int n, k;
9     cin >> n >> k;
10    queue<int> q;
11
12    for (int i = 1; i <= n; i++) q.push(i);
13
14    while (!q.empty()) {
15        for (int i = 1; i < k; i++) {
16            q.push(q.front());
17            q.pop();
18        }
19        cout << q.front();
20        q.pop();
21    }
22
23    return 0;
24 }
```

14.3 덱

덱(deque)은 double-ended queue의 약자로, 큐와 비슷하지만 양쪽 모두에서 push와 pop이 가능한 자료구조이다.

덱은 vector와 유사하게 제일 앞의 데이터를 front라고 한다. 또, 제일 뒤의 데이터를 back이라고 한다. front와 back이라는 용어를 이용해서 다시 표현하면, 스택은 front의 값 가



저오기, back 뒤에 새로운 데이터를 추가하기, front 제거하기의 세 가지 연산이 가능하다고 말할 수 있겠다. 그리고 이 모든 연산은 $O(1)$ 의 시간에 가능하다.

즉, 덱은 스택과 큐의 기능을 모두 지원하는 큰 범주의 자료구조라고 할 수 있겠다. 하지만, 그렇다고 항상 덱을 쓰는 것은 좋은 습관이 아니다. 스택이나 큐만의 연산만을 사용해도 충분한 경우, 스택이나 큐를 사용하는 것이 더 빠를 뿐만 아니라 코드의 가독성이나 알고리즘의 명확성 측면에서 도움이 된다.

STL은 덱도 지원한다. <https://en.cppreference.com/w/cpp/container/deque>에 가면 어떤 함수를 지원하고 어떻게 사용하는지, 시간복잡도까지 모두 확인할 수 있다. 다음은 대표적으로 사용되는 함수들이다.

- `front()`를 통해 제일 앞의 노드를 참조할 수 있다.
- `back()`를 통해 제일 뒤의 노드를 참조할 수 있다.
- `begin()`를 통해 제일 앞의 노드를 가리키는 포인터를 참조할 수 있다.
- `end()`를 통해 제일 뒤의 노드의 다음 노드를 가리키는 포인터를 참조할 수 있다.
- `push_front(T)`를 통해 제일 앞에 새로운 데이터를 추가할 수 있다.
- `push_back(T)`를 통해 제일 뒤에 새로운 데이터를 추가할 수 있다.
- `pop_front()`를 통해 제일 앞의 노드를 제거할 수 있다.
- `pop_back()`를 통해 제일 뒤의 노드를 제거할 수 있다.
- `erase(iterator)`를 통해 포인터(iterator)가 가리키는 노드를 제거할 수 있다.
- `insert(iterator, T)`를 통해 포인터(iterator)의 위치에 새로운 데이터를 추가할 수 있다.
- `empty()`를 통해 비어있는지 여부를 확인할 수 있다.
- `size()`를 통해 저장된 데이터의 개수를 확인할 수 있다.
- `clear()`를 통해 모든 데이터를 지울 수 있다.
- `[]`를 통해 랜덤 액세스가 가능하다.

실제로 STL의 덱을 사용하여 다음의 덱 활용 문제를 해결해보자.

Example 14.3. (18115번) 다음은 잘 알려진 세 가지 카드 기술이다.

- (1) 제일 위의 카드 1장을 바닥에 내려놓는다.
- (2) 위에서 두 번째 카드를 바닥에 내려놓는다. 카드가 2장 이상일 때만 쓸 수 있다.
- (3) 제일 밑에 있는 카드를 바닥에 내려놓는다. 카드가 2장 이상일 때만 쓸 수 있다.

처음에 섞인 카드 더미를 세 가지 카드 기술을 주어진 순서대로 총 N 번 사용했더니, 바닥에 놓인 카드가 위에서부터 $1, 2, \dots, N$ 의 순서로 놓이게 되었다. 처음 카드가 섞여 있던 상태를 위에서부터 순서대로 출력하라.

이 문제는 왜 덱으로 해결할 수 있을까? 다음의 관찰을 생각해보자.

Observation. 세 가지 카드 기술은 덱에서 지원하는 연산으로 해결할 수 있다. 중요하게, 덱은 일련의 연산을 역으로 다시 시행하여 원래 상태로 복구하는 것이 가능하다(persistent하다고 한다). 따라서 이 문제는 덱으로 해결할 수 있다.

이제 관찰을 바탕으로 실제로 구현을 해보자. Code 14.3을 보자.

Code 14.3. Example 14.3을 해결하는 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, arr[1010101];
5  deque<int> dq;
6
7  int main() {
8      ios_base::sync_with_stdio(false);
9      cin.tie(0);
10
11     cin >> n;
12     for (int i = 1; i <= n; i++) cin >> arr[i];
13
14     for (int i = n; i >= 1; i--) {
15         if (arr[i] == 1) dq.push_front(n-i+1);
16         if (arr[i] == 2) {
17             int t = dq.front();
18             dq.pop_front();
19             dq.push_front(n-i+1);
20             dq.push_front(t);
21         }
22         if (arr[i] == 3) dq.push_back(n-i+1);

```

```
23     }
24
25     for (auto iter = dq.begin(); iter != dq.end(); iter++) {
26         cout << *iter << " ";
27     }
28
29     return 0;
30 }
```

14.4 연습문제 A

Problem 14.1. 스택과 큐의 연산은 배열을 통해서 $O(1)$ 에 구현할 수 있다. 그렇다면 왜 배열을 쓰지 않고 굳이 별도의 자료구조를 만들어 사용할까?

Problem 14.2. 스택과 큐의 연산은 연결리스트를 통해서 $O(1)$ 에 구현할 수 있다. 구현 방법을 고안하라.

Problem 14.3. 스택과 큐는 persistent한가? 이유를 설명하라.

Problem 14.4.덱의 연산은 랜덤 액세스를 제외하면 연결리스트를 통해서 $O(1)$ 에 구현할 수 있다. 하지만 랜덤 액세스를 구현하려면 배열을 사용해야 한다. 메모리 사용량을 현재 담긴 데이터의 개수 N 에 대해 $O(N)$ 으로 제한하면서 배열 기반으로 덱을 구현하는 방법을 고안하라.

14.5 연습문제 B

Problem 14.5. (10828번) <https://www.acmicpc.net/problem/10828>

Problem 14.6. (10485번) <https://www.acmicpc.net/problem/10485>

Problem 14.7. (1784번) <https://www.acmicpc.net/problem/1784>

Problem 14.8. (10866번) <https://www.acmicpc.net/problem/10866>

Problem 14.9. (5430번) <https://www.acmicpc.net/problem/5430>

Problem 14.10. (1021번) <https://www.acmicpc.net/problem/1021>

Problem 14.11. (2493번, KOI09) <https://www.acmicpc.net/problem/2493>

Problem 14.12. (17298번) <https://www.acmicpc.net/problem/17298>

- Problem 14.13. (1918번) <https://www.acmicpc.net/problem/1918>
- Problem 14.14. (2504번, KOI08) <https://www.acmicpc.net/problem/2504>
- Problem 14.15. (11003번*) <https://www.acmicpc.net/problem/11003>
- Problem 14.16. (6549번*) <https://www.acmicpc.net/problem/6549>
- Problem 14.17. (3015번*) <https://www.acmicpc.net/problem/3015>
- Problem 14.18. (2104번*) <https://www.acmicpc.net/problem/2104>
- Problem 14.19. (14865번*, KOI17) <https://www.acmicpc.net/problem/14865>
- Problem 14.20. (10000번*) <https://www.acmicpc.net/problem/10000>
- Problem 14.21. (3025번*) <https://www.acmicpc.net/problem/3025>
- Problem 14.22. (3988번*) <https://www.acmicpc.net/problem/3988>
- Problem 14.23. (8021번*) <https://www.acmicpc.net/problem/8021>
- Problem 14.24. (19679번*) <https://www.acmicpc.net/problem/19679>
- Problem 14.25. (17421번*) <https://www.acmicpc.net/problem/17421>
- Problem 14.26. (11873번*) <https://www.acmicpc.net/problem/11873>
- Problem 14.27. (16909번*) <https://www.acmicpc.net/problem/16909>
- Problem 14.28. (2905번*) <https://www.acmicpc.net/problem/2905>

14.6 연습문제 C

- Problem 14.29. (16601번) <https://www.acmicpc.net/problem/16601>
- Problem 14.30. (8885번) <https://www.acmicpc.net/problem/8885>
- Problem 14.31. (8146번) <https://www.acmicpc.net/problem/8146>
- Problem 14.32. (19916번) <https://www.acmicpc.net/problem/19916>
- Problem 14.33. (15411번) <https://www.acmicpc.net/problem/15411>

- Problem 14.34. (15870번) <https://www.acmicpc.net/problem/15870>
- Problem 14.35. (1156번) <https://www.acmicpc.net/problem/1156>
- Problem 14.36. (22269번) <https://www.acmicpc.net/problem/22269>
- Problem 14.37. (3357번) <https://www.acmicpc.net/problem/3357>
- Problem 14.38. (28443번) <https://www.acmicpc.net/problem/28443>
- Problem 14.39. (27236번) <https://www.acmicpc.net/problem/27236>
- Problem 14.40. (31515번) <https://www.acmicpc.net/problem/31515>
- Problem 14.41. (7333번) <https://www.acmicpc.net/problem/7333>
- Problem 14.42. (9630번*) <https://www.acmicpc.net/problem/9630>
- Problem 14.43. (3303번**) <https://www.acmicpc.net/problem/3303>
- Problem 14.44. (16364번**) <https://www.acmicpc.net/problem/16364>
- Problem 14.45. (8646번**) <https://www.acmicpc.net/problem/8646>
- Problem 14.46. (23574번**) <https://www.acmicpc.net/problem/23574>
- Problem 14.47. (25010번**) <https://www.acmicpc.net/problem/25010>

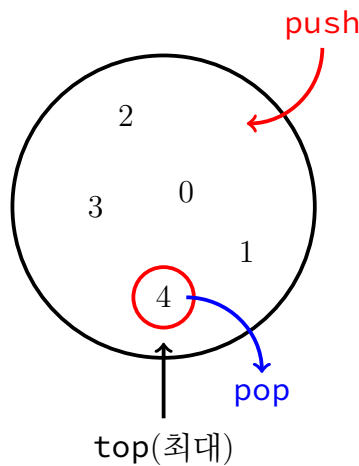
Chapter 15

힙과 이진 탐색 트리

힙과 이진 탐색 트리는 이진 트리 기반의 자료 구조로 데이터의 값과 관련된 특별한 작업을 로그 시간에 시행하는 연산을 지원하는 자료구조이다. 힙은 최대 또는 최소와 관련하여, 이진 탐색 트리는 정렬된 상태와 관련하여 연산을 지원한다. 이번 장에서는 힙과 이진 탐색 트리의 장점과 특징을 알아보고, 각각이 어떤 문제에 적합한지 알아볼 것이다.

15.1 힙

힙(heap)은 이진 트리 기반의 자료구조로, 대기열에서 **최댓값**부터 뽑는 자료구조로 고안된 것이다. 즉, 큐와 비슷하게 대기열의 추가가 가능하지만 **최댓값 참조** 및 **최댓값 제거**를 지원한다.



즉, 힙에서는 최대인 데이터가 굉장히 중요하다. 그래서 최대인 데이터를 **top**이라고 한다. **top**이라는 용어를 이용해서 다시 표현하면, 힙은 **top**의 값 가져오기, 새로운 데이터 추가하고 **top**(최댓값) 다시 찾기, **top** 제거하기의 세 가지 연산이 가능하다고 말할 수 있겠다. 그리고 이 모든 연산은 $O(\log N)$ 의 시간에 가능하다.

힙의 이러한 특징은 **최대인 것을 먼저 처리**해야만 하는 모든 문제에 적용될 수 있다. 중요도가 높은 것부터 먼저 처리하는 우선순위 기반 대기열이나 다익스트라 알고리즘(링크) 등에서 사용된다.

중요한 점은 여기서의 ‘최대’는 값의 최대에 한정되지 않는다. **비교 연산자가 잘 정의되어 있다면 어떤 데이터에서나 사용할 수 있다.** 구현에서 값을 직접 참조하지 않고 임의의 두 값 사이의 비교 결과만이 사용되기 때문이다. 예를 들어, 문자열의 사전순 비교에서 사전순으로 늦은 문자열이 ‘더 큰 것으로 정의’하여 힙을 만들면, 가장 사전순으로 늦은 문자열이 **top**에 위치하게 된다. 이 경우에는 문자열의 비교가 $O(1)$ 이 아니므로, 문자열의 비교에 걸리는 시간에 $O(\log N)$ 이 곱해진 것이 연산의 시간복잡도가 될 것임에 유의하라.

STL은 ‘우선순위 큐’라는 이름으로 힙 자료구조를 지원한다. https://en.cppreference.com/w/cpp/container/priority_queue에 가면 어떤 함수를 지원하고 어떻게 사용하는지, 시간복잡도까지 모두 확인할 수 있다. 다음은 대표적으로 사용되는 함수들이다.

- `top()`를 통해 **최댓값**을 참조할 수 있다.
- `push(T)`를 통해 값을 추가할 수 있다. 값 추가 이후 **최댓값**은 바로 계산된다.
- `pop()`를 통해 **최댓값**을 제거할 수 있다.
- `empty()`를 통해 비어있는지 여부를 확인할 수 있다.
- `size()`를 통해 저장된 데이터의 개수를 확인할 수 있다.

또한, 정의 방식에 대해서도 알 필요가 있다. `priority_queue<T, Container, Compare>`로 정의하는데, 각각은 다음을 나타낸다:

- **T**: 데이터의 자료형이다.
- **Container**: 데이터를 저장할 선형 자료구조이다. 내부의 알고리즘이 $O(\log N)$ 에 작동하도록 보장하는 것일 뿐 메모리 구조 자체가 다른 것이 아니기 때문에 필요하다. 모든 경우에 기본값인 `vector`를 사용하면 된다.
- **Compare**: 비교 기준으로, 다양한 방법이 있으나, 연산만을 담은 구조체를 선언하여 사용하는 방법이 가장 간단하다.

특별히, 가장 작은 수를 ‘더 큰 것으로 정의’하여 **최솟값**이 **top**에 위치하는 힙을 만드는 것은 STL에서 지원하는 `Compare` 객체로 해결할 수 있다:

```
1 // 1) using STL
2 std::priority_queue<int, std::vector<int>, std::greater<int>>
```

```

    min_priority_queue;
3
4 // 2) using compare struct
5 struct customLess {
6     bool operator()(const int l, const int r) const { return l > r; }
7 };
8 std::priority_queue<int, std::vector<int>, customLess>
    custom_priority_queue;

```

실제로 STL의 우선순위 큐를 사용하여 다음의 대표적인 힙 활용 문제를 해결해보자.

Example 15.1. (1715번) N 개의 수에서 수가 하나 남을 때까지 다음 과정을 반복하자.

- (1) 원하는 두 개의 수를 뽑는다.
- (2) 두 수의 합을 기록한다.
- (3) 뽑힌 두 개의 수를 제거하고, 두 수를 더한 값을 추가한다.

기록된 모든 수의 합으로 가능한 최솟값을 출력하라.

이 문제는 왜 힙으로 해결할 수 있을까? 다음의 관찰을 생각해보자.

Observation. 필요한 연산의 총 횟수는 같고, 기록된 수는 추가되는 수와 같다. 즉, 추가되는 수의 합이 최대한 작을수록 유리하다. 따라서 남아있는 수 중 가장 작은 두 수를 뽑는 것이 최적이다(Problem 15.1). 크기 순으로 최소 또는 최대인 수를 뽑고, 그 합을 삽입하는 연산을 빠르게 수행해야 하므로, 힙은 이 문제에 적합한 자료구조이다.

이제 관찰을 바탕으로 실제로 구현을 해보자. Code 15.1을 보자.

Code 15.1. Example 15.1을 해결하는 코드

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, sum = 0;
5 priority_queue< int, vector<int>, greater<int> > pq;
6
7 int main() {
8     ios_base::sync_with_stdio(false);
9     cin.tie(0);
10
11     cin >> n;
12     for (int i = 1; i <= n; i++) {

```

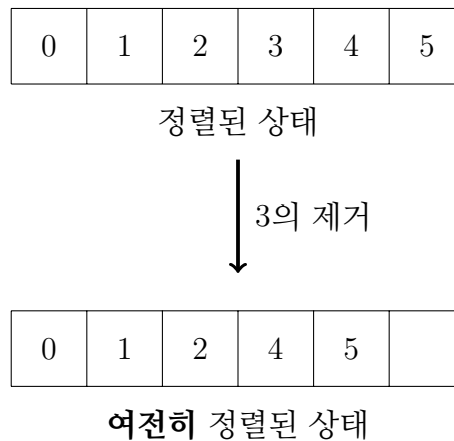
```

13     int a;
14     cin >> a;
15     pq.push(a);
16 }
17
18 while (pq.size() > 1) {
19     int a = pq.top(); pq.pop();
20     int b = pq.top(); pq.pop();
21     sum += a+b;
22     pq.push(a+b);
23 }
24
25 cout << sum << "\n";
26 }

```

15.2 이진 탐색 트리

이진 탐색 트리(binary search tree)은 이름에서도 드러나지만 이진 트리이지만 탐색을 위해 설계된 이진 트리이다. 여기서의 탐색은 크기 순으로 정렬된 것을 나타내는데, 실제로 데이터의 추가 및 삭제가 계속 일어나는 상황에서도 데이터를 **정렬된 상태로 유지**하는 중요한 성질이 있다.



이진 탐색 트리의 이러한 특징은 변화하는 배열에서 **정렬된 상태를 유지**해야만 하는 모든 문제에 적용될 수 있다. 또한, 레드-블랙 트리, AVL 트리 등의 방식으로 이진 탐색 트리의 각 연산은 $O(\log N)$ 의 시간에 작동한다. 누군가는 힙의 모든 연산을 이진 탐색 트리가 지원하니까 힙의 필요성이 없다고 생각할 수 있다. 하지만 스택 및 큐와 텍의 관계와 비슷하게, 이진 탐색 트리의

연산 시간복잡도는 $O(\log N)$ 이지만 **상수가 꽤 크다**는 것이 문제가 된다. 다른 말로 $O(N \log N)$ 의 알고리즘이라도 이진 탐색 트리를 사용한다면 $N \log N < 10^8$ 의 제약조건에서도 몇 초의 시간이 걸릴 수 있다. 즉, 힙의 기능이 문제 해결에 충분한 경우에는 이진 탐색 트리 대신 힙을 사용하는 것이 훨씬 좋다.

STL은 이진 탐색 트리를 두 가지 형태로 지원한다. 첫번째는 정렬된 형태의 집합으로, `set`이라고 한다. 집합이므로 **같은 수의 추가는 무시된다**. <https://en.cppreference.com/w/cpp/container/set>에 가면 어떤 함수를 지원하고 어떻게 사용하는지, 시간복잡도까지 모두 확인할 수 있다. 다음은 대표적으로 사용되는 함수들이다.

- `begin()`를 통해 제일 앞의 노드를 가리키는 포인터를 참조할 수 있다.
- `end()`를 통해 제일 뒤의 노드의 **다음 노드**를 가리키는 포인터를 참조할 수 있다.
- `insert(T)`를 통해 데이터를 추가할 수 있다. 즉시 정렬된다.
- `erase(T)`를 통해 주어진 값에 해당하는 노드를 제거할 수 있다.
- `erase(iterator)`를 통해 포인터(iterator)가 가리키는 노드를 제거할 수 있다.
- `count(T)`를 통해 주어진 데이터의 개수를 셀 수 있다. `set`에서는 당연히 0 또는 1이다.
- `find(T)`를 통해 주어진 데이터의 존재 여부를 판단할 수 있다. 반환형은 `bool`형 변수이다.
- `lower_bound(T)`를 통해 주어진 데이터 **이상인** 최솟값을 가리키는 포인터를 얻을 수 있다. 이 연산도 $O(\log N)$ 에 실행된다.
- `upper_bound(T)`를 통해 주어진 데이터 **초과인** 최솟값을 가리키는 포인터를 얻을 수 있다. 이 연산도 $O(\log N)$ 에 실행된다.
- `empty()`를 통해 비어있는지 여부를 확인할 수 있다.
- `size()`를 통해 저장된 데이터의 개수를 확인할 수 있다.
- `clear()`를 통해 모든 데이터를 지울 수 있다.

참고로, `[]`를 통해 **랜덤 액세스가 불가능**하다. 또, 같은 수의 추가를 고려하고 싶다면, `set` 대신에 `multiset`을 사용하면 된다. `erase(T)`에서 주어진 값에 해당하는 노드가 **모두 제거**된다는 사실을 제외하면 주요 함수의 사용법은 동일하다.

우선순위 큐와 마찬가지로 정의 방식에 대해서도 알 필요가 있다. `set<T, Compare, Allocator>`로 정의하는데, 각각은 다음을 나타낸다:

- `T`: 데이터의 자료형이다.
- `Compare`: 비교 기준으로, 우선순위 큐와 같다.
- `Allocator`: 몰라도 된다.

정의 방식은 `multiset`에서도 같다.

두 번째는, `map`이라고 불리는 자료구조로 딕셔너리와 똑같다. key-value 쌍으로 값이 저장되고, key가 정렬된 순서로 유지된다. 특이하게도, `map`은 key를 활용한 접근을 목적으로 고안되었기 때문에, **둘 이상의 key를 허용하지 않는다**. 또, `map`의 iterator는 key와 value를

`std::pair`로 묶은 변수를 가리킨다. 즉, `first`와 `second`로 접근해야 한다. <https://en.cppreference.com/w/cpp/container/map>에 가면 어떤 함수를 지원하고 어떻게 사용하는지, 시간복잡도까지 모두 확인할 수 있다. 다음은 대표적으로 사용되는 함수들이다.

- `begin()`를 통해 제일 앞의 노드를 가리키는 포인터를 참조할 수 있다.
- `end()`를 통해 제일 뒤의 노드의 **다음 노드**를 가리키는 포인터를 참조할 수 있다.
- `insert({key, T})`를 통해 (키, 데이터) 쌍을 추가할 수 있다. 즉시 정렬된다.
- `erase(key)`를 통해 주어진 키에 해당하는 노드를 제거할 수 있다.
- `erase(iterator)`를 통해 포인터(iterator)가 가리키는 노드를 제거할 수 있다.
- `count(T)`를 통해 주어진 키에 해당하는 데이터의 개수를 셀 수 있다. `map`에서는 당연히 0 또는 1이다.
- `find(T)`를 통해 주어진 키에 해당하는 데이터의 존재 여부를 판단할 수 있다. 반환형은 `bool`형 변수이다.
- `lower_bound(T)`를 통해 주어진 키 **이상인** 최소의 키를 가리키는 포인터를 얻을 수 있다. 이 연산도 $O(\log N)$ 에 실행된다.
- `upper_bound(T)`를 통해 주어진 키 **초과인** 최소의 키를 가리키는 포인터를 얻을 수 있다. 이 연산도 $O(\log N)$ 에 실행된다.
- `empty()`를 통해 비어있는지 여부를 확인할 수 있다.
- `size()`를 통해 저장된 데이터의 개수를 확인할 수 있다.
- `clear()`를 통해 모든 데이터를 지울 수 있다.
- `[]`에서 대괄호 안에 `key`를 넣으면 접근이 가능하다. 즉, `[T]`는 `*lower_bound(T).second`와 거의 같은 역할을 수행한다. 따라서 이 연산은 $O(\log N)$ 에 실행된다.

참고로, 같은 수의 추가를 고려하고 싶다면, `map` 대신에 `multimap`을 사용하면 된다. 단, `multimap`에서는 `[]`를 활용한 접근이 불가능하다. 나머지 주요 함수의 사용법은 동일하다.

맵도 정의 방식에 대해서 알 필요가 있다. `set<key, T, Compare, Allocator>`로 정의하는데, 각각은 다음을 나타낸다:

- `key`: 키의 자료형이다.
- `T`: 데이터의 자료형이다.
- `Compare`: 비교 기준으로, 우선순위 큐와 같다.
- `Allocator`: 몰라도 된다.

정의 방식은 `multimap`에서도 같다.

이제, 실제로 STL의 셋이나 맵을 사용하여 다음의 대표적인 이진 탐색 트리 활용 문제를 해결해보자.

Example 15.2. (2015번) 길이 N 의 수열과 수 K 가 주어질 때, $A_i + A_{i+1} + \dots + A_j = K$ 인 (i, j) ($1 \leq i \leq j \leq N$) 쌍의 개수를 출력하라.

이 문제는 왜 셋이나 맵으로 해결할 수 있을까? 다음의 관찰을 생각해보자.

Observation. 누적 합을 S_i 라 하였을 때, 결국 $S_j - S_{i-1} = K$ 인 (i, j) 의 쌍을 세는 문제와 같다. 따라서, 각 i 에 대해 $S_0 = 0, S_1, \dots, S_{i-1}$ 중에서 $S_i - K$ 의 개수를 세는 문제와 같다. 이는 삽입이 $O(N)$ 번 일어나는 집합에서 주어진 수의 개수를 세는 것이므로 맵은 이 문제에 적합한 자료구조이다. (그리고 (멀티)셋은 적합하지 않다)

이제 관찰을 바탕으로 실제로 구현을 해보자. Code 15.2을 보자.

Code 15.2. Example 15.2을 해결하는 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, k, arr[202020], sum[202020];
5  long long ans = 0;
6  map<int, int> m;
7
8  int main() {
9      ios_base::sync_with_stdio(false);
10     cin.tie(0);
11
12     int n, k;
13     cin >> n >> k;
14     for (int i = 1; i <= n; i++) {
15         cin >> arr[i];
16         sum[i] = sum[i-1] + arr[i];
17     }
18
19     for (int i = 1; i <= n; i++) {
20         m[sum[i-1]]++;
21         ans += m[sum[i]-k];
22     }
23
24     cout << ans << "\n";
25
26     return 0;
27 }
```

Code 15.2에서 보면, map의 [] 연산자는 존재하지 않는 key에 접근했을 때, 대입 연산을 하면

새로운 노드를 만들고 값을 참조하면 0을 반환하도록 깔끔하게 구현되어 있다. 이러한 특징은 코드에서의 예외 처리를 줄여 간결하게 만들어준다.

15.3 연습문제 A

Problem 15.1. Example 15.1에서 가장 작은 수를 뽑는 것이 최적임을 증명하라. (이와 같이 순간의 최적이 전체의 최적과 동일한 경우 그리디 알고리즘(링크)을 쓸 수 있다고 한다)

Problem 15.2. Example 15.2은 멀티셋으로 해결할 수 없다. 왜 그럴까? (*hint.* (멀티)셋에서는 랜덤 액세스가 불가능함을 생각해보라.)

15.4 연습문제 B

Problem 15.3. (11286번) <https://www.acmicpc.net/problem/11286>

Problem 15.4. (14425번) <https://www.acmicpc.net/problem/14425>

Problem 15.5. (1655번) <https://www.acmicpc.net/problem/1655>

Problem 15.6. (1202번) <https://www.acmicpc.net/problem/1202>

Problem 15.7. (2014번) <https://www.acmicpc.net/problem/2014>

Problem 15.8. (13334번) <https://www.acmicpc.net/problem/13334>

Problem 15.9. (17612번) <https://www.acmicpc.net/problem/17612>

Problem 15.10. (21939번) <https://www.acmicpc.net/problem/21939>

Problem 15.11. (19638번) <https://www.acmicpc.net/problem/19638>

Problem 15.12. (2957번*) <https://www.acmicpc.net/problem/2957>

Problem 15.13. (1539번*) <https://www.acmicpc.net/problem/1539>

Problem 15.14. (13303번*) <https://www.acmicpc.net/problem/13303>

Problem 15.15. (3988번*) <https://www.acmicpc.net/problem/3988>

Problem 15.16. (15461번*) <https://www.acmicpc.net/problem/15461>

Problem 15.17. (21279번*) <https://www.acmicpc.net/problem/21279>

- Problem 15.18. (9571번*) <https://www.acmicpc.net/problem/9571>
- Problem 15.19. (18879번*) <https://www.acmicpc.net/problem/18879>
- Problem 15.20. (28327번*) <https://www.acmicpc.net/problem/28327>
- Problem 15.21. (15901번*) <https://www.acmicpc.net/problem/15901>
- Problem 15.22. (17163번*) <https://www.acmicpc.net/problem/17163>
- Problem 15.23. (25734번*) <https://www.acmicpc.net/problem/25734>
- Problem 15.24. (2471번*) <https://www.acmicpc.net/problem/2471>
- Problem 15.25. (10760번*) <https://www.acmicpc.net/problem/10760>
- Problem 15.26. (26970번*) <https://www.acmicpc.net/problem/26970>
- Problem 15.27. (16764번*) <https://www.acmicpc.net/problem/16764>
- Problem 15.28. (11920번*) <https://www.acmicpc.net/problem/11920>

15.5 연습문제 C

- Problem 15.29. (9244번) <https://www.acmicpc.net/problem/9244>
- Problem 15.30. (20150번) <https://www.acmicpc.net/problem/20150>
- Problem 15.31. (20151번) <https://www.acmicpc.net/problem/20151>
- Problem 15.32. (32033번) <https://www.acmicpc.net/problem/32033>
- Problem 15.33. (17763번) <https://www.acmicpc.net/problem/17763>
- Problem 15.34. (9281번) <https://www.acmicpc.net/problem/9281>
- Problem 15.35. (28124번) <https://www.acmicpc.net/problem/28124>
- Problem 15.36. (6620번) <https://www.acmicpc.net/problem/6620>
- Problem 15.37. (31745번) <https://www.acmicpc.net/problem/31745>
- Problem 15.38. (3647번) <https://www.acmicpc.net/problem/3647>

Problem 15.39. (28198번) <https://www.acmicpc.net/problem/28198>

Problem 15.40. (19557번) <https://www.acmicpc.net/problem/19557>

Problem 15.41. (13864번) <https://www.acmicpc.net/problem/13864>

Problem 15.42. (32040번) <https://www.acmicpc.net/problem/32040>

Problem 15.43. (31972번) <https://www.acmicpc.net/problem/31972>

Problem 15.44. (28697번) <https://www.acmicpc.net/problem/28697>

Problem 15.45. (18998번) <https://www.acmicpc.net/problem/18998>

Problem 15.46. (13624번) <https://www.acmicpc.net/problem/13624>

Problem 15.47. (9482번) 삼차원 좌표계 상에 N 개의 점이 주어질 때 두 점 사이의 거리가 K 이하인 두 점 쌍이 10^5 가지를 넘지 않는다. 이러한 두 점 쌍의 개수를 출력하라. $N \leq 10^5$, $k \leq 10^9$ 이고 N 개의 점의 x, y, z 좌표는 모두 절댓값이 10^9 이하인 정수이다.

Problem 15.48. (13138번) <https://www.acmicpc.net/problem/13138>

Problem 15.49. (14435번*) <https://www.acmicpc.net/problem/14435>

Problem 15.50. (1250번*) <https://www.acmicpc.net/problem/1250>

Problem 15.51. (17728번*) <https://www.acmicpc.net/problem/17728>

Problem 15.52. (9245번*) <https://www.acmicpc.net/problem/9245>

Problem 15.53. (23670번*) <https://www.acmicpc.net/problem/23670>

Problem 15.54. (12009번*) <https://www.acmicpc.net/problem/12009>

Problem 15.55. (8253번*) <https://www.acmicpc.net/problem/8253>

Problem 15.56. (18801번**) <https://www.acmicpc.net/problem/18801>

Problem 15.57. (4000번**) <https://www.acmicpc.net/problem/18801>

Chapter 16

해시

해시는 임의의 길이를 갖는 데이터를 고정된 길이의 데이터로 매핑하는 함수를 활용하여 데이터를 저장하는 방식이다. 해시는 충돌이 발생할 수 있지만, 상수 시간에 주어진 데이터와 일치하는 데이터를 찾을 수 있다는 특징이 있다. 이번 장에서는 해시의 장점과 특징을 알아보고, 어떤 문제에 적합한지 알아볼 것이다.

16.1 해시

해시의 기본적인 아이디어는, 입력되는 값이 너무 크거나 문자열과 같아 인덱스로 사용할 수 없을 때, 작은 수로 바꾸어 그 값을 인덱스로 사용하는 것이다. 물론, 이것은 결국 원래의 값과 같은 값을 찾기 위함이므로 값이 다르면 바뀔 수도 최대한 다르게 만들어야 한다. 이때 작은 수로 바꾸는 함수를 해시 함수, 바뀐 값을 해시값이라고 부르고, 바뀐 값이 같아 발생하는 문제를 해시 충돌이라고 한다.

PS에서 해시를 사용할 경우는 극히 드물다. 대부분의 경우 `std::map`을 사용해도 충분하기 때문이다. 디렉터리와 같은 기능을 할 수 있으면서, 키가 정렬되어 있어 안정적으로 $O(\log N)$ 의 시간에 연산을 처리할 수 있다는 장점을 사용하는 것이다. 그런데, 때때로 $O(\log N)$ 이 더 곱해지는 것이 시간초과의 원인인 경우가 있다. N 이 10만 가까이 될 경우, 상수가 큰 $O(N \log^2 N)$ 은 꽤나 뻣뻣하고, $O(N^{1.5} \log N)$ 은 시간 안에 실행되기 어렵다. PS에서 해시는 이러한 문제를 해결하기 위한 최후의 수단에 가깝다. 키를 바탕으로 한 이분탐색과 같은 정렬 상태가 필요하지 않고, 단순 디렉터리와 같은 대응 기능만이 필요한 경우, 해시를 사용하여 $O(1)$ 의 시간에 값을 가져올 수 있다. 물론, 해시는 안정적이지 않다. 실제로 시간복잡도를 amortized $O(1)$ 과 같이 표기하는데, 어떤 한 번의 연산에서는 $O(1)$ 보다 훨씬 큰 $O(N)$ 가량의 시간이 요구될 수 있으나, N 번 정도의 충분한 시행이 있으면 평균 시간복잡도가 $O(1)$ 이라는 뜻이다.

이제 STL에서 지원하는 해시 자료구조를 보겠다. 이진 탐색 트리와 마찬가지로, set과 map의 두 가지 형태로 지원한다.

첫번째는 정렬되지 않은 집합으로, unordered_set이라고 한다(진정한 집합이다). 집합이므로 같은 수의 추가는 무시된다. https://en.cppreference.com/w/cpp/container/unordered_set에 가면 어떤 함수를 지원하고 어떻게 사용하는지, 시간복잡도까지 모두 확인할 수 있다. 다음은 대표적으로 사용되는 함수들이다.

- begin()를 통해 제일 앞의 노드를 가리키는 포인터를 참조할 수 있다.
- end()를 통해 제일 뒤의 노드의 다음 노드를 가리키는 포인터를 참조할 수 있다.
- insert(T)를 통해 데이터를 추가할 수 있다.
- erase(T)를 통해 주어진 값에 해당하는 노드를 제거할 수 있다.
- erase(iterator)를 통해 포인터(iterator)가 가리키는 노드를 제거할 수 있다.
- count(T)를 통해 주어진 데이터의 개수를 셀 수 있다. unordered_set에서는 당연히 0 또는 1이다.
- find(T)를 통해 주어진 데이터의 존재 여부를 판단할 수 있다. 반환형은 bool형 변수이다.
- empty()를 통해 비어있는지 여부를 확인할 수 있다.
- size()를 통해 저장된 데이터의 개수를 확인할 수 있다.
- clear()를 통해 모든 데이터를 지울 수 있다.

참고로, 대부분의 함수는 amortized $O(1)$ 에 실행되며, []를 통해 랜덤 액세스가 불가능하다. 또, 같은 수의 추가를 고려하고 싶다면, unordered_set 대신에 unordered_multiset을 사용하면 된다. erase(T)에서 주어진 값에 해당하는 노드가 모두 제거된다는 사실을 제외하면 주요 함수의 사용법은 동일하다.

우선순위 큐와 마찬가지로 정의 방식에 대해서도 알 필요가 있다. unordered_set<T, Hash, ...>로 정의하는데, 각각은 다음을 나타낸다:

- T: 데이터의 자료형이다.
- Hash: 데이터에 대한 해시 함수이다. STL에서 지원하는 해시 함수가 존재하는 경우 자동으로 지정된다. 직접 해시 함수를 만들어야 할 경우, <https://en.cppreference.com/w/cpp/utility/hash>의 예시를 참고하라.
- ...: 뒤의 추가 인자는 몰라도 된다.

정의 방식은 unordered_multiset에서도 같다.

두 번째는, unordered_map이다. key-value 쌍으로 값이 저장되고, key가 정렬된 순서로 유지된다. 동일하게, key를 활용한 접근을 목적으로 고안되었기 때문에, **둘 이상의 key를 허용하지 않는다**. 또, unordered_map의 iterator는 key와 value를 std::pair로 묶은 변수를 가리킨다. 즉, first와 second로 접근해야 한다. https://en.cppreference.com/w/cpp/container/unordered_map에 가면 어떤 함수를 지원하고 어떻게 사용하는지, 시간복잡

도까지 모두 확인할 수 있다. 다음은 대표적으로 사용되는 함수들이다.

- `begin()`를 통해 제일 앞의 노드를 가리키는 포인터를 참조할 수 있다.
- `end()`를 통해 제일 뒤의 노드의 **다음 노드**를 가리키는 포인터를 참조할 수 있다.
- `insert({key, T})`를 통해 (키, 데이터) 쌍을 추가할 수 있다.
- `erase(key)`를 통해 주어진 키에 해당하는 노드를 제거할 수 있다.
- `erase(iterator)`를 통해 포인터(iterator)가 가리키는 노드를 제거할 수 있다.
- `count(T)`를 통해 주어진 키에 해당하는 데이터의 개수를 셀 수 있다. `unordered_map`에서는 당연히 0 또는 1이다.
- `find(T)`를 통해 주어진 키에 해당하는 데이터의 존재 여부를 판단할 수 있다. 반환형은 `bool`형 변수이다.
- `empty()`를 통해 비어있는지 여부를 확인할 수 있다.
- `size()`를 통해 저장된 데이터의 개수를 확인할 수 있다.
- `clear()`를 통해 모든 데이터를 지울 수 있다.
- `[]`에서 대괄호 안에 `key`를 넣으면 접근이 가능하다. 즉, `[T]`는 `*lower_bound(T).second`와 거의 같은 역할을 수행한다.

참고로, 같은 수의 추가를 고려하고 싶다면, `unordered_map` 대신에 `unordered_multimap`을 사용하면 된다. 단, `unordered_multimap`에서는 `[]`를 활용한 접근이 불가능하다. 나머지 주요 함수의 사용법은 동일하다.

맵도 정의 방식에 대해서 알 필요가 있다. `unordered_map<key, T, Hash, ...>`로 정의하는데, 각각은 다음을 나타낸다:

- `key`: 키의 자료형이다.
- `T`: 데이터의 자료형이다.
- `Hash`: 키에 대한 해시 함수이다. STL에서 지원하는 해시 함수가 존재하는 경우 자동으로 지정된다. 직접 해시 함수를 만들어야 할 경우, <https://en.cppreference.com/w/cpp/utility/hash>의 예시를 참고하라.
- `...`: 뒤의 추가 인자는 몰라도 된다.

정의 방식은 `unordered_multimap`에서도 같다.

이제 해시를 사용해서 이전 장에서 이진 탐색 트리로 해결했었던 Example 15.2를 해결해보자. 관찰을 다시 적어보면 다음과 같다.

Observation. 누적 합을 S_i 라 하였을 때, 결국 $S_j - S_{i-1} = K$ 인 (i, j) 의 쌍을 세는 문제와 같다. 따라서, 각 i 에 대해 $S_0 = 0, S_1, \dots, S_{i-1}$ 중에서 $S_i - K$ 의 개수를 세는 문제와 같다.

맵 대신에, 해시맵(`unordered_map`)을 사용하여, 해시맵에 키 S_k 에 해당하는 값을 1씩 추가하고, 키 $S_i - K$ 에 해당하는 값을 읽어 다 더하면 똑같이 문제를 해결할 수 있다. 구현은

Code 15.2에서 한 줄만 바꾸면 되므로 생략한다.

16.2 연습문제 A

Problem 16.1. 해시 충돌의 해결 방안을 조사해보고, C++에서 어떤 방식을 사용하고 있는지 알아보라.

16.3 연습문제 B

Problem 16.2. (27160번) <https://www.acmicpc.net/problem/27160>

Problem 16.3. (9375번) <https://www.acmicpc.net/problem/9375>

Problem 16.4. (5568번) <https://www.acmicpc.net/problem/5568>

Problem 16.5. (9015번) <https://www.acmicpc.net/problem/9015>

Problem 16.6. (1184번) <https://www.acmicpc.net/problem/1184>

Problem 16.7. (3136번*) <https://www.acmicpc.net/problem/3136>

Problem 16.8. (24618번*) <https://www.acmicpc.net/problem/24618>

Problem 16.9. (27944번*) <https://www.acmicpc.net/problem/27944>

Problem 16.10. (11510번*) <https://www.acmicpc.net/problem/11510>

16.4 연습문제 C

Problem 16.11. (8241번) <https://www.acmicpc.net/problem/8241>

Problem 16.12. (14516번) <https://www.acmicpc.net/problem/14516>

Problem 16.13. (28785번) <https://www.acmicpc.net/problem/28785>

Problem 16.14. (7995번*) <https://www.acmicpc.net/problem/7995>

Chapter 17

분리 집합

연결리스트는 배열이 아닌 다른 자료구조의 이해를 위한 기본적인 자료구조이다. 이 장에서는 연결리스트에 대한 학습과 더불어 자료구조를 공부할 때 어떠한 점을 생각해야 하는지 같이 살펴보겠다.

17.1 자료구조를 왜 만드는가?

자료구조는 ‘자료’와 ‘구조’가 합쳐진 말로, 자료와 구조가 모두 있어야 자료구조라 할 수 있다. 여기서 자료는 말 그대로 저장하고 싶은 데이터를 말하는 것이고, 구조는 어떤 ‘연산’을 얼마나 빠르게 처리할 수 있는지와 대응된다. 여기서 연산은 삽입, 삭제, 탐색과 같이 데이터와 상호작용하는 것이다.

배열은 가장 기본적인 자료구조로 볼 수 있다. 먼저, 배열은 저장하고자 하는 데이터 타입 T 가 나열된 것이므로 당연히 ‘자료’가 있다.

0	1	2	3	4	5
---	---	---	---	---	---

‘구조’에 해당하는 연산도 있을까? 사실, 처음 배열을 배웠을 때부터 연산을 하고 있었다. `cin` » `arr[i]`를 반복문 안에서 반복하는 것은, 마지막 값이 $i-1$ 번 인덱스에 저장되어 있으므로, 자료구조의 끝에 삽입하는 연산과 같다. 실제로, 배열은 삽입, 삭제, 탐색 모두 가능하다.

모든 자료구조는 **특별한 장점**이 있어서 쓴다. 배열의 특별한 장점은 ‘임의 접근(random access)’을 빠르게 할 수 있다는 것이다. 정확히는, $O(1)$ 의 시간복잡도에 가능하다. 그리고, 모든 연산을 빠르게 처리할 수 있는 자료구조는 없다. 배열은 ‘임의 접근이 빠르다’는 장점을 얻은 대신에

‘삽입과 삭제가 느리다’는 단점을 얻었다. 삽입과 삭제는 $O(N)$ 의 시간이 필요하다. 삽입을 위해서는 그보다 오른쪽에 있는 것들을 전부 오른쪽으로 한 칸 옮겨야 하고, 삭제를 위해서는 그보다 오른쪽에 있는 것들을 전부 왼쪽으로 한 칸 옮겨야 하기 때문이다. 연결리스트는 반대이다. ‘임의 접근이 느리다’는 단점이 있지만, ‘삽입과 삭제가 빠르다’는 장점을 얻었다.

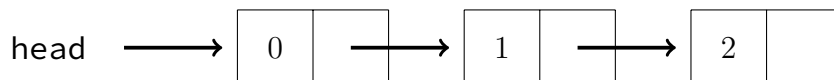
자료구조마다 장단점이 다르다는 것 자체가 자료구조를 만들어야 하는 이유가 된다. 어떤 문제가 삽입과 삭제의 횟수가 매우 많고 랜덤 접근은 단 한 번도 하지 않는다면 연결리스트를 활용하는 것이 배열을 활용하는 것보다 더 나은 시간복잡도로 문제를 해결할 수 있을 것이다. 이 관점에서 보면 자료구조가 먼저 만들어진 것이 아니라, 문제가 먼저 제시되었고, 그 문제를 위한 자료구조가 개발된 것에 가깝다. 비유를 하자면, 자료구조는 수학에서 정리(Theorem)의 느낌이다. 정리를 쓰면 정리를 증명하는 과정을 건너뛸 수 있는 것처럼, 자료구조를 쓰면 문제에서 필요한 최적화를 건너뛸 수 있다.

정리하면, 문제를 보고 그 문제에서 ‘여러 번 시행하기’를 요구하는 연산을 빠르게 수행할 수 있는 자료구조를 사용하는 것이 ‘자료구조를 사용한 PS 문제의 해결’이라고 말할 수 있다. 따라서, 자료구조는 ‘빠르게 할 수 있는 연산’과 똑같은 개념으로 받아들여야 한다.

지금부터 우리가 배울 자료구조는 문제에서 ‘여러 번 시행하기’를 자주 요구해서 잘 알려진 것들이다. 특히, 이 파트의 자료구조는 특히 자주 요구하므로, 꼭 각 자료구조가 어떤 연산과 같은지 이해하고 다음으로 넘어가길 바란다.

17.2 연결리스트

연결리스트는 이름에서도 나타나듯이 연결을 통해 자료를 저장하는 방식이다.

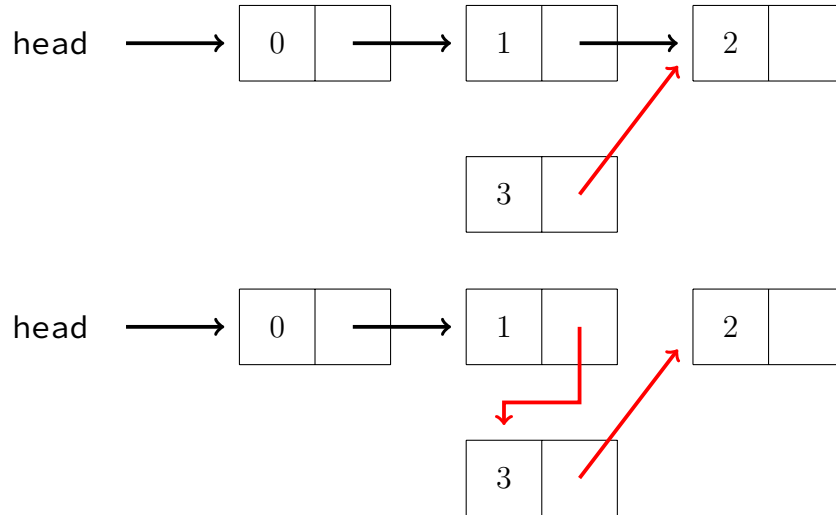


하나의 데이터에는 데이터 뿐만 아니라 포인터가 붙어 있다. 이 하나의 쌍을 ‘노드(node)’라고 부를 것이다. 각 노드의 포인터는 다음 노드를 가리킴으로써 다음 노드와 ‘연결’하고, 특별히 헤드 포인터(그림에서 head)가 시작 노드를 가리킨다. 이 헤드 포인터를 참조하면 연결리스트에 접근할 수 있는 것이다.

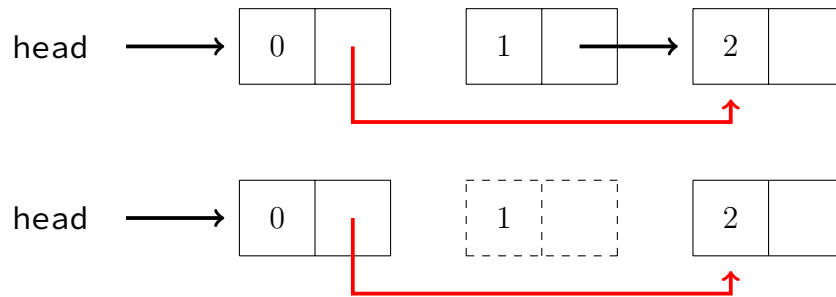
앞에서 연결리스트는 임의 접근을 포기하고 삽입과 삭제를 얻었다고 했다. 실제로 연결리스트에서 임의 접근, 삽입, 삭제가 어떻게 동작하는지 시뮬레이션해보자.

먼저 임의 접근이다. 임의로 k 번째(0-based) 위치에 접근하고 싶다면, 헤드 포인터가 가리키는 노드에서 k 번 더 포인터를 통해 이동한 위치의 값을 가져오면 된다. 연결리스트에서는 발판들을 건너 넘어가는 것처럼, 순서대로 직접 가야 한다. 따라서 k 번의 연산이 필요하고, 이를 점근 표기법으로 나타내면 $O(N)$ 이 되는 것이다.

두 번째로 삽입이다. ‘현재 노드 뒤’(Problem 18.1)에 삽입하는 연산을 생각해보자. 그러면 새로 삽입되는 노드의 포인터를 현재 노드의 포인터로 바꾸고, 현재 노드의 포인터를 새로 삽입되는 노드로 바꾸면 된다. 아래 그림을 참고하자.



세 번째로 삭제이다. ‘현재 노드 뒤’(Problem 18.2)에 있는 노드를 삭제하는 연산을 생각해보자. 그러면 현재 노드의 포인터를 현재 노드에서 바로 뒤에 있는 노드의 포인터로 바꾸고, 현재 노드 뒤에 있는 노드를 제거하면 된다. 아래 그림을 참고하자.



17.3 연결리스트의 구현

연결리스트를 구현하려면 노드 구조체에 데이터 타입 T와 노드를 가리키는 포인터를 담아야 한다. 이렇게 되면 자기참조로써 컴파일러에서 무한루프에 빠지는거 아닐까? 하지만 포인터 자체는 unsigned long long int와 같고, 별(*) 연산자와 붙었을 때 구조체의 정보를 불러오기 때문에 이러한 일은 발생하지 않는다.

C는 클래스와 관련된 것이 하나도 존재하지 않아서 생성자와 소멸자를 사용할 수 없지만, C++에서는 구조체로 선언하였다 하더라도 구조체 자체가 클래스로 구현되어 있어서 생성자와 소

멸자를 사용할 수 있다. 따라서 간단하게 `new`로 노드를 만들고, `del`로 노드를 제거할 수 있다. 메모리 누수 방지를 위해 노드 제거 작업은 꼭 필요하다.

이제 다음 코드 Code 18.1로 연결리스트의 구현을 보자. 포인터를 사용하는 자료구조의 구현은 본질적으로 이와 같고, 각 연산들의 로직만 다르므로 아래 구현 방식에는 익숙해졌으면 한다. 실제 시험장에서 연결리스트를 구현할 일은 없겠지만, 포인터를 사용하는 자료구조 구현은 나중에 때때로 쓰이기 때문이다.

Code 17.1. 연결리스트 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int n;
4
5  struct Node {
6      int a;
7      Node *nxt;
8      Node() {
9          a = 0;
10         nxt = NULL;
11     }
12 };
13 Node *head = new Node();
14
15 void add(Node *node, int val) {
16     Node *added = new Node();
17     added->a = val;
18     added->nxt = node->nxt;
19     node->nxt = added;
20 }
21
22 void rem(Node *node) {
23     Node *tar = node->nxt;
24     if (tar == NULL) return;
25     node->nxt = tar->nxt;
26     delete tar;
27 }

```

구조체 `Node` 안에 선언된 `Node()` 함수는 생성자로, `new`를 사용하여 구조체 변수를 만들 때 자동으로 실행된다. 함수 이름을 구조체 이름과 똑같이 설정하면 된다. 만약, 소멸자가 필

요하다면 `Node()`와 같이 물결을 붙여서 함수 이름을 정하면 된다. 이 소멸자는 `delete`를 사용하여 구조체 변수를 제거할 때 자동으로 실행된다.

17.4 STL 라이브러리의 연결리스트

STL 라이브러리는 연결리스트를 지원한다. <https://en.cppreference.com/w/cpp/container/list>에 가면 어떤 함수를 지원하고 어떻게 사용하는지, 시간복잡도까지 모두 확인할 수 있다. 그 중에서 중요한 함수만을 소개하겠다. 그 전에 몇 가지 용어를 정의하자. STL container에서는 C언어의 포인터와 같은 기능을 하는 것을 **iterator**라고 표현한다. 그리고 iterator로 표현된 구간은 기본적으로 반열린구간 $[s, e)$ 로 처리된다. 또, 저장하는 데이터의 자료형은 T로 표기한다. 즉, 함수 인자에 T가 있다면 데이터를 함수 인자로 넣으라는 뜻이다.

- `front()`를 통해 제일 앞의 노드를 참조할 수 있다.
- `back()`를 통해 제일 뒤의 노드를 참조할 수 있다.
- `begin()`를 통해 제일 앞의 노드를 가리키는 포인터를 참조할 수 있다.
- `end()`를 통해 제일 뒤의 노드의 다음 노드를 가리키는 포인터를 참조할 수 있다.
- `push_front(T)`를 통해 제일 앞에 새로운 데이터를 추가할 수 있다.
- `push_back(T)`를 통해 제일 뒤에 새로운 데이터를 추가할 수 있다.
- `pop_front()`를 통해 제일 앞의 노드를 제거할 수 있다.
- `pop_back()`를 통해 제일 뒤의 노드를 제거할 수 있다.
- `erase(iterator)`를 통해 포인터(iterator)가 가리키는 노드를 제거할 수 있다.
- `insert(iterator, T)`를 통해 포인터(iterator)의 위치에 새로운 데이터를 추가할 수 있다.
- `empty()`를 통해 비어있는지 여부를 확인할 수 있다.
- `size()`를 통해 저장된 데이터의 개수를 확인할 수 있다.
- `clear()`를 통해 모든 데이터를 지울 수 있다.

상술된 함수들은 대부분의 STL container에서 공통적으로 존재하는 함수들이다. 따라서 같은 이름의 함수를 앞으로 계속 만나게 된다. 그리고, 각 함수들이 매번 비슷한 기능을 하므로 다양한 STL container를 만나면서 점차 친숙해지면 된다.

코드에서 사용하는 방법은 다음 예시 Code 18.2를 참고하라.

Code 17.2. `std::list`의 예시 코드

```
1 #include <algorithm>
2 #include <iostream>
3 #include <list>
4
5 int main()
```

```

6 {
7     // Create a list containing integers
8     std::list<int> l = {7, 5, 16, 8};
9
10    // Add an integer to the front of the list
11    l.push_front(25);
12    // Add an integer to the back of the list
13    l.push_back(13);
14
15    // Insert an integer before 16 by searching
16    auto it = std::find(l.begin(), l.end(), 16);
17    if (it != l.end())
18        l.insert(it, 42);
19
20    // Print out the list
21    std::cout << "l = { ";
22    for (int n : l)
23        std::cout << n << ", ";
24    std::cout << "};\n";
25 }

```

17.5 연습문제 A

Problem 17.1. 연결리스트에서 헤드 포인터와 삽입하고 싶은 위치의 인덱스 k (0-based)가 주어질 때 연산의 시간복잡도는 무엇인가? 삽입이 아닌 삭제에서는 어떠한가?

Problem 17.2. 연결리스트에서 **현재 포인터**를 $O(1)$ 에 삭제하려면 어떻게 수정하면 될까? 방법을 고안하고 직접 구현해보라.

17.6 연습문제 B

Problem 17.3. (5680번) <https://www.acmicpc.net/problem/5680>

Problem 17.4. (3294번) <https://www.acmicpc.net/problem/3294>

Problem 17.5. (4621번) <https://www.acmicpc.net/problem/4621>

Problem 17.6. (6542번) <https://www.acmicpc.net/problem/6542>

Problem 17.7. (22645번) <https://www.acmicpc.net/problem/22645>

Problem 17.8. (23309번) <https://www.acmicpc.net/problem/23309>

Problem 17.9. (25252번) <https://www.acmicpc.net/problem/25252>

Problem 17.10. (30885번) <https://www.acmicpc.net/problem/30885>

Problem 17.11. (31857번) <https://www.acmicpc.net/problem/31857>

Problem 17.12. (3217번) <https://www.acmicpc.net/problem/3217>

Problem 17.13. (21201번*) <https://www.acmicpc.net/problem/21201>

Problem 17.14. (17820번*) <https://www.acmicpc.net/problem/17820>

Problem 17.15. (23774번*) <https://www.acmicpc.net/problem/23774>

Problem 17.16. (28974번*) <https://www.acmicpc.net/problem/28974>

17.7 연습문제 C

Problem 17.17. (26032번) <https://www.acmicpc.net/problem/26032>

Problem 17.18. (24971번*) <https://www.acmicpc.net/problem/24971>

Problem 17.19. (17692번**) <https://www.acmicpc.net/problem/17692>

Chapter 18

희소 배열

연결리스트는 배열이 아닌 다른 자료구조의 이해를 위한 기본적인 자료구조이다. 이 장에서는 연결리스트에 대한 학습과 더불어 자료구조를 공부할 때 어떠한 점을 생각해야 하는지 같이 살펴보겠다.

18.1 자료구조를 왜 만드는가?

자료구조는 ‘자료’와 ‘구조’가 합쳐진 말로, 자료와 구조가 모두 있어야 자료구조라 할 수 있다. 여기서 자료는 말 그대로 저장하고 싶은 데이터를 말하는 것이고, 구조는 어떤 ‘연산’을 얼마나 빠르게 처리할 수 있는지와 대응된다. 여기서 연산은 삽입, 삭제, 탐색과 같이 데이터와 상호작용하는 것이다.

배열은 가장 기본적인 자료구조로 볼 수 있다. 먼저, 배열은 저장하고자 하는 데이터 타입 T 가 나열된 것이므로 당연히 ‘자료’가 있다.

0	1	2	3	4	5
---	---	---	---	---	---

‘구조’에 해당하는 연산도 있을까? 사실, 처음 배열을 배웠을 때부터 연산을 하고 있었다. `cin` » `arr[i]`를 반복문 안에서 반복하는 것은, 마지막 값이 $i-1$ 번 인덱스에 저장되어 있으므로, 자료구조의 끝에 삽입하는 연산과 같다. 실제로, 배열은 삽입, 삭제, 탐색 모두 가능하다.

모든 자료구조는 **특별한 장점**이 있어서 쓴다. 배열의 특별한 장점은 ‘임의 접근(random access)’을 빠르게 할 수 있다는 것이다. 정확히는, $O(1)$ 의 시간복잡도에 가능하다. 그리고, 모든 연산을 빠르게 처리할 수 있는 자료구조는 없다. 배열은 ‘임의 접근이 빠르다’는 장점을 얻은 대신에

‘삽입과 삭제가 느리다’는 단점을 얻었다. 삽입과 삭제는 $O(N)$ 의 시간이 필요하다. 삽입을 위해서는 그보다 오른쪽에 있는 것들을 전부 오른쪽으로 한 칸 옮겨야 하고, 삭제를 위해서는 그보다 오른쪽에 있는 것들을 전부 왼쪽으로 한 칸 옮겨야 하기 때문이다. 연결리스트는 반대이다. ‘임의 접근이 느리다’는 단점이 있지만, ‘삽입과 삭제가 빠르다’는 장점을 얻었다.

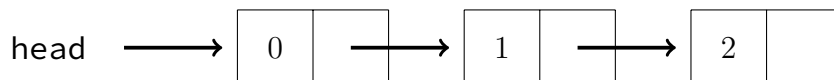
자료구조마다 장단점이 다르다는 것 자체가 자료구조를 만들어야 하는 이유가 된다. 어떤 문제가 삽입과 삭제의 횟수가 매우 많고 랜덤 접근은 단 한 번도 하지 않는다면 연결리스트를 활용하는 것이 배열을 활용하는 것보다 더 나은 시간복잡도로 문제를 해결할 수 있을 것이다. 이 관점에서 보면 자료구조가 먼저 만들어진 것이 아니라, 문제가 먼저 제시되었고, 그 문제를 위한 자료구조가 개발된 것에 가깝다. 비유를 하자면, 자료구조는 수학에서 정리(Theorem)의 느낌이다. 정리를 쓰면 정리를 증명하는 과정을 건너뛸 수 있는 것처럼, 자료구조를 쓰면 문제에서 필요한 최적화를 건너뛸 수 있다.

정리하면, 문제를 보고 그 문제에서 ‘여러 번 시행하기’를 요구하는 연산을 빠르게 수행할 수 있는 자료구조를 사용하는 것이 ‘자료구조를 사용한 PS 문제의 해결’이라고 말할 수 있다. 따라서, 자료구조는 ‘빠르게 할 수 있는 연산’과 똑같은 개념으로 받아들여야 한다.

지금부터 우리가 배울 자료구조는 문제에서 ‘여러 번 시행하기’를 자주 요구해서 잘 알려진 것들이다. 특히, 이 파트의 자료구조는 특히 자주 요구하므로, 꼭 각 자료구조가 어떤 연산과 같은지 이해하고 다음으로 넘어가길 바란다.

18.2 연결리스트

연결리스트는 이름에서도 나타나듯이 연결을 통해 자료를 저장하는 방식이다.

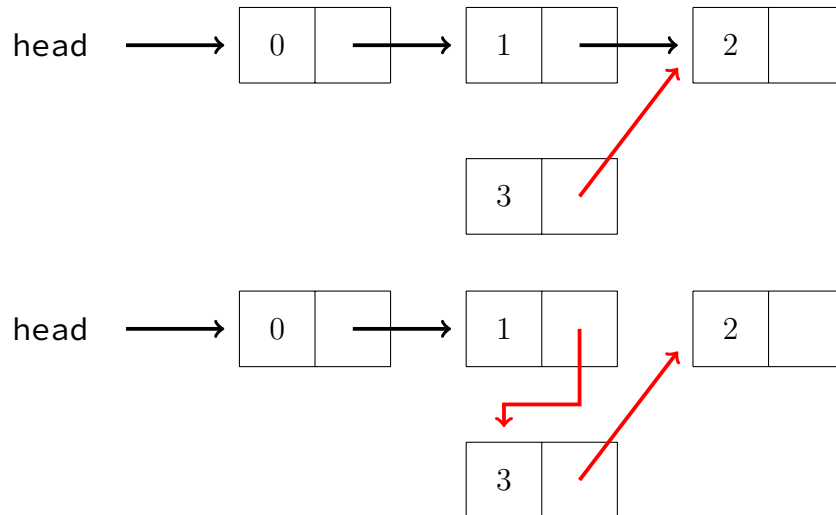


하나의 데이터에는 데이터 뿐만 아니라 포인터가 붙어 있다. 이 하나의 쌍을 ‘노드(node)’라고 부를 것이다. 각 노드의 포인터는 다음 노드를 가리킴으로써 다음 노드와 ‘연결’하고, 특별히 헤드 포인터(그림에서 head)가 시작 노드를 가리킨다. 이 헤드 포인터를 참조하면 연결리스트에 접근할 수 있는 것이다.

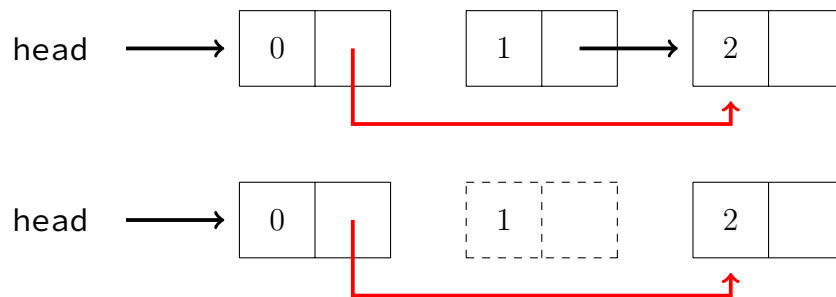
앞에서 연결리스트는 임의 접근을 포기하고 삽입과 삭제를 얻었다고 했다. 실제로 연결리스트에서 임의 접근, 삽입, 삭제가 어떻게 동작하는지 시뮬레이션해보자.

먼저 임의 접근이다. 임의로 k 번째(0-based) 위치에 접근하고 싶다면, 헤드 포인터가 가리키는 노드에서 k 번 더 포인터를 통해 이동한 위치의 값을 가져오면 된다. 연결리스트에서는 발판들을 건너 넘어가는 것처럼, 순서대로 직접 가야 한다. 따라서 k 번의 연산이 필요하고, 이를 점근 표기법으로 나타내면 $O(N)$ 이 되는 것이다.

두 번째로 삽입이다. ‘현재 노드 뒤’(Problem 18.1)에 삽입하는 연산을 생각해보자. 그러면 새로 삽입되는 노드의 포인터를 현재 노드의 포인터로 바꾸고, 현재 노드의 포인터를 새로 삽입되는 노드로 바꾸면 된다. 아래 그림을 참고하자.



세 번째로 삭제이다. ‘현재 노드 뒤’(Problem 18.2)에 있는 노드를 삭제하는 연산을 생각해보자. 그러면 현재 노드의 포인터를 현재 노드에서 바로 뒤에 있는 노드의 포인터로 바꾸고, 현재 노드 뒤에 있는 노드를 제거하면 된다. 아래 그림을 참고하자.



18.3 연결리스트의 구현

연결리스트를 구현하려면 노드 구조체에 데이터 타입 T와 노드를 가리키는 포인터를 담아야 한다. 이렇게 되면 자기참조로써 컴파일러에서 무한루프에 빠지는거 아닐까? 하지만 포인터 자체는 unsigned long long int와 같고, 별(*) 연산자와 붙었을 때 구조체의 정보를 불러오기 때문에 이러한 일은 발생하지 않는다.

C는 클래스와 관련된 것이 하나도 존재하지 않아서 생성자와 소멸자를 사용할 수 없지만, C++에서는 구조체로 선언하였다 하더라도 구조체 자체가 클래스로 구현되어 있어서 생성자와 소

멸자를 사용할 수 있다. 따라서 간단하게 `new`로 노드를 만들고, `del`로 노드를 제거할 수 있다. 메모리 누수 방지를 위해 노드 제거 작업은 꼭 필요하다.

이제 다음 코드 Code 18.1로 연결리스트의 구현을 보자. 포인터를 사용하는 자료구조의 구현은 본질적으로 이와 같고, 각 연산들의 로직만 다르므로 아래 구현 방식에는 익숙해졌으면 한다. 실제 시험장에서 연결리스트를 구현할 일은 없겠지만, 포인터를 사용하는 자료구조 구현은 나중에 때때로 쓰이기 때문이다.

Code 18.1. 연결리스트 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int n;
4
5  struct Node {
6      int a;
7      Node *nxt;
8      Node() {
9          a = 0;
10         nxt = NULL;
11     }
12 };
13 Node *head = new Node();
14
15 void add(Node *node, int val) {
16     Node *added = new Node();
17     added->a = val;
18     added->nxt = node->nxt;
19     node->nxt = added;
20 }
21
22 void rem(Node *node) {
23     Node *tar = node->nxt;
24     if (tar == NULL) return;
25     node->nxt = tar->nxt;
26     delete tar;
27 }

```

구조체 `Node` 안에 선언된 `Node()` 함수는 생성자로, `new`를 사용하여 구조체 변수를 만들 때 자동으로 실행된다. 함수 이름을 구조체 이름과 똑같이 설정하면 된다. 만약, 소멸자가 필

요하다면 `Node()`와 같이 물결을 붙여서 함수 이름을 정하면 된다. 이 소멸자는 `delete`를 사용하여 구조체 변수를 제거할 때 자동으로 실행된다.

18.4 STL 라이브러리의 연결리스트

STL 라이브러리는 연결리스트를 지원한다. <https://en.cppreference.com/w/cpp/container/list>에 가면 어떤 함수를 지원하고 어떻게 사용하는지, 시간복잡도까지 모두 확인할 수 있다. 그 중에서 중요한 함수만을 소개하겠다. 그 전에 몇 가지 용어를 정의하자. STL container에서는 C언어의 포인터와 같은 기능을 하는 것을 **iterator**라고 표현한다. 그리고 iterator로 표현된 구간은 기본적으로 반열린구간 $[s, e)$ 로 처리된다. 또, 저장하는 데이터의 자료형은 T로 표기한다. 즉, 함수 인자에 T가 있다면 데이터를 함수 인자로 넣으라는 뜻이다.

- `front()`를 통해 제일 앞의 노드를 참조할 수 있다.
- `back()`를 통해 제일 뒤의 노드를 참조할 수 있다.
- `begin()`를 통해 제일 앞의 노드를 가리키는 포인터를 참조할 수 있다.
- `end()`를 통해 제일 뒤의 노드의 다음 노드를 가리키는 포인터를 참조할 수 있다.
- `push_front(T)`를 통해 제일 앞에 새로운 데이터를 추가할 수 있다.
- `push_back(T)`를 통해 제일 뒤에 새로운 데이터를 추가할 수 있다.
- `pop_front()`를 통해 제일 앞의 노드를 제거할 수 있다.
- `pop_back()`를 통해 제일 뒤의 노드를 제거할 수 있다.
- `erase(iterator)`를 통해 포인터(iterator)가 가리키는 노드를 제거할 수 있다.
- `insert(iterator, T)`를 통해 포인터(iterator)의 위치에 새로운 데이터를 추가할 수 있다.
- `empty()`를 통해 비어있는지 여부를 확인할 수 있다.
- `size()`를 통해 저장된 데이터의 개수를 확인할 수 있다.
- `clear()`를 통해 모든 데이터를 지울 수 있다.

상술된 함수들은 대부분의 STL container에서 공통적으로 존재하는 함수들이다. 따라서 같은 이름의 함수를 앞으로 계속 만나게 된다. 그리고, 각 함수들이 매번 비슷한 기능을 하므로 다양한 STL container를 만나면서 점차 친숙해지면 된다.

코드에서 사용하는 방법은 다음 예시 Code 18.2를 참고하라.

Code 18.2. `std::list`의 예시 코드

```
1 #include <algorithm>
2 #include <iostream>
3 #include <list>
4
5 int main()
```

```

6 {
7     // Create a list containing integers
8     std::list<int> l = {7, 5, 16, 8};
9
10    // Add an integer to the front of the list
11    l.push_front(25);
12    // Add an integer to the back of the list
13    l.push_back(13);
14
15    // Insert an integer before 16 by searching
16    auto it = std::find(l.begin(), l.end(), 16);
17    if (it != l.end())
18        l.insert(it, 42);
19
20    // Print out the list
21    std::cout << "l = { ";
22    for (int n : l)
23        std::cout << n << ", ";
24    std::cout << "};\n";
25 }

```

18.5 연습문제 A

Problem 18.1. 연결리스트에서 헤드 포인터와 삽입하고 싶은 위치의 인덱스 k (0-based)가 주어질 때 연산의 시간복잡도는 무엇인가? 삽입이 아닌 삭제에서는 어떠한가?

Problem 18.2. 연결리스트에서 **현재 포인터**를 $O(1)$ 에 삭제하려면 어떻게 수정하면 될까? 방법을 고안하고 직접 구현해보라.

18.6 연습문제 B

Problem 18.3. (5680번) <https://www.acmicpc.net/problem/5680>

Problem 18.4. (3294번) <https://www.acmicpc.net/problem/3294>

Problem 18.5. (4621번) <https://www.acmicpc.net/problem/4621>

Problem 18.6. (6542번) <https://www.acmicpc.net/problem/6542>

Problem 18.7. (22645번) <https://www.acmicpc.net/problem/22645>

Problem 18.8. (23309번) <https://www.acmicpc.net/problem/23309>

Problem 18.9. (25252번) <https://www.acmicpc.net/problem/25252>

Problem 18.10. (30885번) <https://www.acmicpc.net/problem/30885>

Problem 18.11. (31857번) <https://www.acmicpc.net/problem/31857>

Problem 18.12. (3217번) <https://www.acmicpc.net/problem/3217>

Problem 18.13. (21201번*) <https://www.acmicpc.net/problem/21201>

Problem 18.14. (17820번*) <https://www.acmicpc.net/problem/17820>

Problem 18.15. (23774번*) <https://www.acmicpc.net/problem/23774>

Problem 18.16. (28974번*) <https://www.acmicpc.net/problem/28974>

18.7 연습문제 C

Problem 18.17. (26032번) <https://www.acmicpc.net/problem/26032>

Problem 18.18. (24971번*) <https://www.acmicpc.net/problem/24971>

Problem 18.19. (17692번**) <https://www.acmicpc.net/problem/17692>

Part IV

동적계획법 (Easy)

Chapter 19

1차원 동적계획법

동적계획법은 큰 문제를 작은 문제의 합으로 표현하여 해결하는 모든 알고리즘을 총칭한다. 작은 문제를 푸는 방식이므로 재귀함수와 유사하지만, 메모이제이션을 통해 시간복잡도를 획기적으로 단축한다는 차이점이 있다. 이번 장에서는 동적계획법 문제의 다양한 예시를 알아본다.

19.1 동적계획법 개론

동적계획법과 가장 비슷한 수학 개념은 점화식이다. 점화식은 수열 $\{a_n\}$ 에서 $a_n = a_{n-1}^2 - a_{n-2}$ 와 같은 식으로 수열을 정의하는 방식이다. 실제로, 동적계획법은 점화관계와 초항을 수학적으로 구하고, 점화식을 따라 수열의 n 번째 값과 같은 원하는 값을 구해낸다.

동적계획법과 점화식의 차이는 각각의 목표가 다르다는 것이다. 수학에서는 점화식을 바탕으로 일반항을 구하는 것에 주목한다. 하지만 동적계획법에서는 원하는 값만 구하면 충분하다. 물론, 일반항을 구하면 원하는 값을 쉽게 구할 수 있겠지만, PS 문제의 점화식에서 일반항이 존재하는 경우는 손에 꼽는다. 또한, 꽤 자주 n, k 의 두 개의 변수가 포함된 수열의 점화관계가 유도되기도 하는데(2차원 동적계획법), 이 경우에는 일반항을 구하기 더욱 어려워진다.

따라서 동적계획법 문제는 크게 ‘점화관계 파악’과, 그리고 ‘점화식의 계산’의 두 부분으로 나눌 수 있다. 이를 조금 더 컴퓨터과학과 어울리게 쓴다면 ‘재귀관계’라고 표현할 수 있다. 구현 과정에서 재귀적인 접근이 사용되기 때문이다. 예를 들어, a_n 을 구하는 함수를 $f(n)$ 이라 하면, $f(n)$ 에서는 $f(n-1)$ 등을 호출하게 되며, $f(n)$ 은 재귀함수가 된다.

따라서, 동적계획법의 과정을 요약하면 다음과 같다:

- (1) 문제를 재귀적으로 정의한다. (‘재귀관계 파악’)

- 풀고자 하는 문제를 수학적으로 명확히 쓴다.
 - 부분문제를 수학적으로 정의하고,
 - 부분문제의 답으로 전체 문제를 해결하는 재귀적 식을 찾는다.
- (2) 부분문제를 재귀적 방식으로 해결하되, 답을 저장하여 중복을 방지한다. ('재귀관계의 계산')
- 작은 문제를 먼저 푼다.
 - 전부 풀릴 때까지 이를 반복한다.

19.2 동적계획법의 구현

이제, 구체적으로 동적계획법의 구현 방법을 알아보자. 구현 방식은 큰 수부터 재귀함수 호출로 처리하는 top-down 방식과, 작은 수부터 배열을 채워나가는 bottom-up 방식이 있다. 피보나치 수를 구하는 문제를 예시로 들어보자.

Example 19.1. $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2} (n \geq 2)$ 로 정의되는 점화식에서, F_1 부터 F_{30} 까지의 값을 출력하는 프로그램을 작성하라.

19.2.1 top-down

먼저 top-down 방식이다. 이 경우 점화식의 변수들을 함수의 인자로 넣는다. 이 문제에서는 `int solve(int n)`으로 쓰면 충분하지만, 때로는 함수의 인자가 두 개 이상일 수도 있음에 유의하라. 이 이후에는 재귀적으로 함수를 쓰면 된다.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int solve(int n) {
5      if (n == 0) return 0;
6      if (n == 1) return 1;
7      return solve(n-1) + solve(n-2);
8  }
9
10 int main() {
11     for (int i = 1; i <= 30; i++) cout << solve(i) << " ";
12     return 0;
13 }
```

이 정도 구현은 스스로 할 수 있을 것이다. 그런데, 위 코드는 동적계획법의 중요한 원리를 누락했다. 동적계획법을 쓰면 '답을 저장하여 중복을 방지'하는 부분이 포함되어야 한다. `solve(5)`

는 분명히 여러 번 호출된다. 그 때마다 다시 `solve(4)`와 `solve(3)`을 호출할 필요는 없다. 한 번 계산하면, 이후부터는 계산된 값을 반환하면 된다.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int dp[31];
5 int solve(int n) {
6     if (dp[n] >= 0) return dp[n];
7     if (n == 0) return dp[n] = 0;
8     if (n == 1) return dp[n] = 1;
9     return dp[n] = solve(n-1) + solve(n-2);
10 }
11
12 int main() {
13     for (int i = 0; i <= 30; i++) dp[i] = -1;
14     for (int i = 1; i <= 30; i++) cout << solve(i) << " ";
15     return 0;
16 }
```

19.2.2 bottom-up

다음은 bottom-up 방식이다. 이것은 사람이 표를 채우는 방법과 유사하다. 손으로 피보나치 수를 구한다고 하면, 우리는 0번 칸과 1번 칸에 0과 1을 적고, 0번 칸과 1번 칸의 값을 더해 2번 칸에 적고, 1번 칸의 값과 2번 칸의 값을 더해 3번 칸에 적고, 이 과정을 계속 반복할 것이다. 이것을 그대로 코드로 구현하면 된다.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int dp[31];
5
6 int main() {
7     dp[0] = 0; dp[1] = 1;
8     for (int i = 2; i <= 30; i++) dp[i] = dp[i-1] + dp[i-2];
9     for (int i = 1; i <= 30; i++) cout << dp[i] << " ";
10     return 0;
11 }
```

bottom-up 방식은 배열을 채우는 순서가 중요하다. 당연히, 8번째 줄에서 i 를 30부터 시작해 점점 줄여나가면 제대로 된 답을 구할 수 없다. 아직 28번째와 29번째 값이 구해지지 않았기 때문이다. 즉, 배열을 채우는 순서를 잘 모르겠다면 top-down 방식을 사용해야 한다. 물론 top-down 방식에도 단점이 있다. 코드가 길고, 함수 호출이 많아서 bottom-up 방식보다 느릴 수 있다.

19.3 일차원 동적계획법 문제의 풀이

다양하고 대표적인 일차원 동적계획법 문제들의 예시를 들고 이를 푸는 방법을 소개할 것이다. 독자들은 알려주는 문제 해결 프로세스를 숙지하고 자연스러운 질 때까지 연습문제를 풀어보며 연습해야 한다.

Example 19.2. 연속한 두 개 이상의 1을 포함하지 않고 0과 1로만 이루어진 N 자리 문자열의 개수를 구하는 프로그램을 작성하라.

Example 19.3. 길이 N 의 수열 $A_1, A_2, \dots, A_N$ 에서 $A_i + A_{i+1} + \dots + A_j$ ($1 \leq i \leq j \leq N$)의 가능한 최댓값을 구하는 프로그램을 작성하라.

Example 19.4. (역추적) 길이 N 의 수열 $A_1, A_2, \dots, A_N$ 에서, $A_{p_1} < A_{p_2} < \dots < A_{p_k}$ 를 만족하는 가장 긴 증가수열 $p_1 < p_2 < \dots < p_k$ 를 LIS(longest increasing subsequences)라고 한다. LIS를 구하는 프로그램을 작성하라.

Example 19.5. (트리 dp로 넘겨야 하는데) 트리 그래프 $G = (V, E)$ 가 주어질 때, $V' \subseteq V$ 이면서 V' 에 속하는 임의의 다른 두 정점이 간선으로 연결되어 있지 않은 V' 들을 생각하자. V' 의 크기로 가능한 최댓값을 구하는 프로그램을 작성하라.

Example 19.6. (이차원 동적계획법) 문자열 S 에 삭제, 삽입, 교환을 최소로 사용하여 T 로 만드려고 할 때, 필요한 횟수의 최솟값을 구하는 프로그램을 작성하라.

Example 19.7. (이차원 동적계획법) 양수로 이루어진 집합 S 가 주어질 때, $T \subseteq S$ 이며 T 에 속하는 모든 원소의 합이 k 가 되는 경우가 있는지 판단하는 프로그램을 작성하라.

Example 19.8. (부분합 최적화) BST를 만들되, 각 key의 탐색 빈도를 바탕으로 연산 횟수가 최소가 되도록 정점들을 배치하라. 단, 연산 횟수는 각 key의 깊이와 같다.

Example 19.9. 배낭 문제

Example 19.10. (이차원 동적계획법) 행렬 곱에 필요한 연산량은 곱셈 순서에 따라 달라진다. 행렬의 크기가 주어질 때, 필요한 연산량의 최솟값을 출력하라.

Example 19.11. (스위핑) N 개의 구간과 각 구간의 가중치가 주어질 때, 겹치지 않게 구

간들을 뽑으면서 뽑힌 구간들의 가중치의 합이 최대가 되도록 하고 싶다. 가능한 최댓값을 출력하라.

Example 19.12. (이차원 동적계획법) 선형 회귀는 오차가 최소가 되도록 하는 하나의 선을 찾는 방식이지만, 때로는 여러 개의 선을 찾는 것이 도움이 될 수 있다. 다중 선분 선형 회귀 최적화는, L 개의 선분으로 구성된 연속함수로 주어진 데이터를 회귀하는 방식이다. 고정된 상수 c 에 대해서, 최소제곱법에 따른 오차의 합 E 와 사용하는 선분의 개수 L 로 계산한 실질적 오차 $E + c \cdot L$ 의 값의 최솟값을 출력하라.

19.4 연습문제 A

Problem 19.1. 만약 가중치가 음수일 수 있다면 최단거리가 항상 존재하는가? 이에 대해 논의하라.

Problem 19.2. Code 30.1와 Code 30.2에서 최단거리를 역추적하는 코드를 작성하라.

Problem 19.3. 두 번째 최단거리를 구하는 $O(M \log M)$ 코드를 작성하라. 일반적으로 K 번째 최단거리를 알아야 한다면 어떻게 구현해야 하는가? 시간복잡도는 어떻게 되는가?

19.5 연습문제 B

Problem 19.4. <https://www.acmicpc.net/problem/2765>

Problem 19.5. (15078번*) $n \times m$ 보드에서 노노그램을 푸는 코드를 작성하라. 칠해야 한다면 X, 칠하지 않아야 한다면 ., 알 수 없다면 ?로 출력하면 된다. n 과 m 은 모두 100 이하의 자연수이다. [이차원 dp]

19.6 연습문제 C

Problem 19.6. <https://www.acmicpc.net/problem/2765>

Chapter 20

배낭 문제

Chapter 21

다차원 동적계획법

Part V

그리디

Chapter 22

그리디

Part VI

그래프 이론

Chapter 23

그래프 기초

그래프(Graph)는 정보 문제에서 큰 비중을 차지하는 분야로 정점과 정점을 연결하는 간선으로 이루어진 집합의 일종이다. 이번 챕터에서는 이 그래프의 정의와 그래프 자료의 저장 방식, 그리고 주로 사용되는 몇 가지 용어를 알아본다.

23.1 그래프의 정의

그래프의 엄밀한 정의는 이와 다르지만, PS 공부에 있어서는 다음과 같이 간략하게 이해하여도 충분하다.

Definition 23.1. 그래프 G 는 정점의 집합 V 와 간선의 집합 E 에 대해 (V, E) 의 순서쌍을 나타낸다. 또한, E 의 임의의 원소 e 는 V 에 속하는 임의의 두 원소 u 와 v 로 이루어진 순서쌍이어야 한다. 이러한 V 와 E 에 대해 그래프 G 는 V 에 속하는 정점으로 이루어져 있고, E 에 속하는 임의의 원소 e 의 성분 두 정점이 연결된 것을 나타낸다.

일반적으로 그래프는 어떠한 두 개체의 연결관계를 표시할 때 유용하다. 많은 그래프 문제에서 직접 그래프를 제공하기도 하지만, 때때로는 주어진 상황으로부터 직접 그래프를 고안해야 함에 유의하라. 대표적인 예시로는 격자 사이에서 인접한 칸만 이동할 수 있는 경우를 그래프로 나타낼 때가 있을 것이다. $N \times M$ 격자를 그래프로 나타내고 그 이동을 시뮬레이션 할 때, V 를 모든 격자의 각 칸을 모은 집합, E 를 각 칸에 대해 인접한 네 칸과 연결한 간선으로 정의하면 그래프로 표현할 수 있게 된다.

23.2 그래프의 표현

우리는 그래프 사진을 보고 직관적으로 그래프의 구조와 모습을 이해할 수 있지만, 컴퓨터는 그렇지 않다. 우리는 직접 그래프의 모양을 적절한 규칙에 따라 입력해주어야 한다. 일반적으로 정점은 숫자 하나로 나타낸다. 정점의 개수가 N 개인 정점의 경우에는 0부터 $N - 1$ 까지 또는 1부터 N 까지를 사용한다. 일반적으로 간선은 총 M 개의 줄에 걸쳐 각 줄마다 써진 u 와 v 를 입력받는데, 다른 말로 하면 E 의 원소를 직접 입력받는다는 의미이다.

하지만 구현의 관점에서는 정점 E 의 집합을 입력받는 방식은 유용하지 않은데, 만약 어떤 정점 v 에서 갈 수 있는 모든 정점을 탐색하려면 $O(M)$ 의 시간 복잡도가 필요하기 때문이다. 따라서 일반적으로 인접행렬 또는 인접리스트를 사용하여 그래프를 표시한다.

- 인접행렬은 i 행 j 열의 성분이 정점 i 에서 정점 j 로 갈 수 있는지 여부를 저장한 행렬이다. $O(N^2)$ 의 공간과 v 에서 갈 수 있는 모든 정점 탐색에 $O(N)$ 의 시간을 요구한다.
- 인접리스트는 각 정점 i 에 대해 i 에서 갈 수 있는 모든 정점을 저장한 리스트이다. 보통 `std::vector`를 사용해 구현하며, $O(M)$ 의 공간과 v 에서 갈 수 있는 모든 정점 탐색에 연결된 정점 개수만큼의 시간을 요구한다. 다른 말로 낭비되는 반복이 없다.

다음은 인접행렬과 인접리스트를 만드는 예시 코드이다.

Code 23.1. 인접행렬 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, m;
5  int adj[1010][1010];
6
7  int main() {
8      ios_base::sync_with_stdio(false);
9      cin.tie(0);
10     cin >> n >> m;
11     for (int i = 1; i <= m; i++) {
12         int u, v;
13         cin >> u >> v;
14         adj[u][v] = 1;
15         adj[v][u] = 1;
16     }
17 }
```

Code 23.2. 인접리스트 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, m;
5  vector<int> adj[101010];
6
7  int main() {
8      ios_base::sync_with_stdio(false);
9      cin.tie(0);
10     cin >> n >> m;
11     for (int i = 1; i <= m; i++) {
12         int u, v;
13         cin >> u >> v;
14         adj[u].push_back(v);
15         adj[v].push_back(u);
16     }
17 }

```

23.3 몇 가지 용어들

그래프 이론에서 사용되는 몇 가지 용어가 있다. 다음 용어들은 이 파트에서 꾸준히 사용될 것이므로 꼭 숙지하도록 하자.

- 연결: 두 정점 사이를 간선을 통해 이동할 수 있음을 나타내는 용어
- 경로: 연결된 두 정점 사이를 간선을 통해 이동할 때 지나가는 간선의 나열
- 최단경로: 어떤 두 정점 사이를 있는 모든 경로에 대해 경로의 길이가 최소인 경로
- 사이클: 한 정점에서 출발해 다시 그 정점으로 돌아오는 경로 중 경로를 이루는 간선이 모두 다른 경로
- 차수: 한 정점과 연결된 다른 정점의 수. 다른 말로 인접 리스트에서 리스트의 길이
- 컴포넌트: 임의의 두 정점 사이를 간선을 통해 이동할 수 있는 V 의 부분집합
- 완전 그래프: 각 정점 사이를 정확히 하나의 간선으로 연결하는 그래프 (K_n)
- 클릭: 임의의 두 정점 사이가 간선을 통해 연결되어 있는 G 의 부분집합, 즉 부분 완전 그래프
-

23.4 연습문제 A

Problem 23.1. Code 23.1와 Code 23.2를 보고 어떤 그래프 하나의 입력 데이터를 직접 만들어보라.

23.5 연습문제 B

23.5.1 기초문제

Problem 23.2. <https://www.acmicpc.net/problem/3132>

Problem 23.3. <https://www.acmicpc.net/problem/5247>

Problem 23.4. <https://www.acmicpc.net/problem/9993>

Problem 23.5. <https://www.acmicpc.net/problem/10328>

Problem 23.6. <https://www.acmicpc.net/problem/10965>

Problem 23.7. <https://www.acmicpc.net/problem/11255>

Problem 23.8. <https://www.acmicpc.net/problem/14676>

Problem 23.9. <https://www.acmicpc.net/problem/17511>

Problem 23.10. <https://www.acmicpc.net/problem/18097>

Problem 23.11. <https://www.acmicpc.net/problem/18788>

Problem 23.12. <https://www.acmicpc.net/problem/22035>

Problem 23.13. <https://www.acmicpc.net/problem/25953>

Problem 23.14. <https://www.acmicpc.net/problem/29955>

23.5.2 응용문제

Problem 23.15. <https://www.acmicpc.net/problem/1050>

Problem 23.16. <https://www.acmicpc.net/problem/1533>

Problem 23.17. <https://www.acmicpc.net/problem/2008>

- Problem 23.18. <https://www.acmicpc.net/problem/2099>
- Problem 23.19. <https://www.acmicpc.net/problem/2549>
- Problem 23.20. <https://www.acmicpc.net/problem/3378>
- Problem 23.21. <https://www.acmicpc.net/problem/4165>
- Problem 23.22. <https://www.acmicpc.net/problem/6691>
- Problem 23.23. <https://www.acmicpc.net/problem/11642>
- Problem 23.24. <https://www.acmicpc.net/problem/11749>
- Problem 23.25. <https://www.acmicpc.net/problem/13143>
- Problem 23.26. <https://www.acmicpc.net/problem/19544>
- Problem 23.27. <https://www.acmicpc.net/problem/19934>

23.5.3 도전문제

- Problem 23.28. <https://www.acmicpc.net/problem/2939>
- Problem 23.29. <https://www.acmicpc.net/problem/11755>
- Problem 23.30. <https://www.acmicpc.net/problem/15261>
- Problem 23.31. <https://www.acmicpc.net/problem/16661>
- Problem 23.32. <https://www.acmicpc.net/problem/18346>
- Problem 23.33. <https://www.acmicpc.net/problem/18971>
- Problem 23.34. <https://www.acmicpc.net/problem/19221>
- Problem 23.35. <https://www.acmicpc.net/problem/20689>
- Problem 23.36. <https://www.acmicpc.net/problem/21117>
- Problem 23.37. <https://www.acmicpc.net/problem/22025>
- Problem 23.38. <https://www.acmicpc.net/problem/26134>
- Problem 23.39. <https://www.acmicpc.net/problem/27167>

Problem 23.40. <https://www.acmicpc.net/problem/29257>

Problem 23.41. <https://www.acmicpc.net/problem/30042>

Problem 23.42. (5472번, IOI08) x 축을 따라 x 좌표가 증가하는 방향으로 계속 이동하는 게임이 있다. 그런데 x 축 위에 N 개의 텔레포터가 있어서 텔레포터를 만난다면 **무조건** 텔레포터를 타고 다른 위치로 이동하게 된다. 엄밀하게 말하면, 텔레포터의 두 끝점이 a 와 b 라면 $x = a$ 에 도달하면 $x = b$ 로, $x = b$ 에 도달하면 $x = a$ 에 이동하게 될 것이다. 물론 텔레포터를 탄 이후에도 항상 x 좌표가 증가하는 방향으로 이동한다. 여기에 더해 우리는 M 개의 텔레포터를 원하는 위치에 추가할 수 있다. 다음은 N 개의 텔레포터가 만족하는 조건과 M 개의 텔레포터가 만족해야 할 조건을 보여준다.

- N 개의 텔레포터의 두 끝점의 좌표는 정수이다. 하지만 추가하는 M 개의 텔레포터의 두 끝점 좌표는 정수일 필요가 없다.
- 모든 텔레포터의 두 끝점은 $(0, 2000001)$ 에 속해야 하고, 모든 텔레포터 중 임의의 두 텔레포터를 골랐을 때 텔레포터의 두 끝점이 겹치면 안된다.

위 조건을 만족하는 N 개의 텔레포터의 두 끝점 정보가 주어질 때 위 조건을 만족하도록 적절히 M 개의 텔레포터를 추가해 $x = 0$ 에서 $x = 2000001$ 에 도달할 때까지 텔레포터를 사용하는 횟수를 최소화하자. 그 최댓값을 구하여라. $N \leq 10^6$, $M \leq 10^6$ 이다.

Chapter 24

그래프의 종류

앞 장에서 그래프의 정의와 저장 방법, 그리고 용어들을 알아보았다. 이번 장에서는 그래프의 큰 분류와 특수한 그래프 형태 몇 가지를 소개하고, 각각이 어떻게 이용되는지 설명한다.

24.1 그래프의 분류

24.1.1 방향성

일반적으로 이전까지 살펴보았던 그래프의 간선은 두 정점의 연결 관계를 나타내는 것으로, 양방향 이동이 가능하다고 가정하였다. 이러한 그래프를 무방향 그래프라 부르고, 무방향 그래프의 간선은 무방향 간선이라고 한다. 하지만 그래프의 간선이 두 정점의 연결 관계가 아닌 이동 관계를 나타낼 수도 있다. 이러한 경우에는 v 와 u 를 연결하는 간선이 존재하더라도 V 에서 u 로의 이동은 허용되지만, u 에서 v 로의 이동은 허용되지 않을 수 있다. 이러한 그래프를 방향 그래프라고 하며, 방향 그래프의 간선을 방향 간선이라고 한다.

많은 문제에서 무방향 그래프를 다루게 될 것이므로 앞으로의 그래프 알고리즘 설명에서 무방향 그래프를 가정한다. 또한, 일반적으로 무방향 그래프에서 구현된 알고리즘을 방향 그래프로 바꾸는 것이 어렵지 않으므로 방향 그래프에 대한 논의는 생략하겠다(위상 정렬에서만 특수히 방향 그래프를 다룰 것이다).

24.1.2 가중치

일반적인 그래프에는 가중치가 존재하지 않다. 하지만 때때로 간선에 가중치가 존재하는 경우가 있으며, 이 가중치는 최단경로에만 영향을 준다. 앞에서 최단경로를 경로 중에 가장 적은 개수의 간선으로 이루어진 것으로 정의했지만, 가중치가 있는 경우에는 경로를 이루는 간선의

가중치의 합이 최소인 것이다. 가중치가 포함될 경우에는 최단 거리 이외의 성질은 달라지지 않으므로 가중치가 포함된 그래프는 최단거리 알고리즘 및 최소 스패닝 트리에서만 언급될 것이다.

24.2 특수한 그래프

이전 장에서 컴포넌트의 개념을 배웠다. 임의의 두 정점이 연결된 V 의 부분집합을 컴포넌트라고 하는데, V 자체가 컴포넌트인 그래프를 연결 그래프라고 한다. 다시 말해 전체 그래프의 임의의 두 정점 사이가 연결되어 있으면, 그 그래프는 연결 그래프라고 부른다.

Definition 24.1 (연결 그래프). $G = (V, E)$ 에서 V 가 컴포넌트이면 G 를 **연결 그래프**라고 한다.

정점 N 개로 이루어진 무방향 연결 그래프 중 가능한 간선 개수 M 의 최솟값은 얼마일까? 바로 $N - 1$ 이다. 적어도 N 개의 정점이 연결되려면 $N - 1$ 개의 간선이 필요하다. 트리가 바로 이러한 그래프이다. 다시 말해 정점 N 개와 간선 $N - 1$ 개로 이루어진 연결 그래프를 트리라고 한다.

Definition 24.2 (트리). $G = (V, E)$ 에서 V 의 크기가 E 의 크기에 1을 더한 값이면 G 를 **트리**라고 한다.

왜 트리라고 부를까? 트리는 하나의 정점에서 시작하여 위로 뻗어나가는 나무 모양의 구조를 이루기 때문이다. 이러한 의미에서 트리라는 이름이 붙었으며, 계층 구조나 상하 구조와 같이 나뭇가지처럼 뻗어나가야 하는 경우 트리를 통해 나타내는 경우가 많다.

트리에는 매우 많은 성질이 있다. 특히, 어떤 그래프 G 가 트리인 것과 다음 명제들은 필요충분 조건이다.

- G 가 무방향 연결 그래프이고, 간선 수에 1을 더하면 정점 수가 된다.
- G 가 무방향 연결 그래프이고, 두 정점 사이를 잇는 경로는 유일하다.
- G 가 무방향 연결 그래프이고, 사이클이 없다.

임의의 간선이 둘 이상의 사이클에 포함되지 않는 그래프를 생각해보자. 이러한 그래프는 트리는 아니지만 뽕족뽕족한 선인장과 같은 모양을 갖는다. 이러한 그래프를 선인장 그래프라고 한다.

Definition 24.3 (선인장 그래프). $G = (V, E)$ 에서 E 의 임의의 원소(간선)이 많아야 하나의 사이클에 속하면 G 를 **선인장 그래프**라고 한다.

선인장 그래프의 판별은 중요한 문제 중 하나이다. 이러한 문제는 이후에 이중 연결 요소와 같이 살펴볼 것이다.

중복 간선이 없어야 한다는 가정이 있다면, 무방향 연결 그래프에 존재할 수 있는 간선의 최대

개수는 몇 개일까? 당연히 모든 정점쌍이 간선으로 연결된 경우에 $M = N(N+1)/2$ 까지 가능할 것이다. 이러한 그래프를 완전 그래프라고 한다.

Definition 24.4 (완전 그래프). $G = (V, E)$ 에서 V 에 속하는 임의의 정점 u, v 에 대해 (u, v) 간선이 E 에 존재하면 G 를 **완전 그래프**라고 한다.

때때로 어떤 그래프는 양쪽 집합 사이를 연결하기도 한다. 이 경우에는 각 집합 내부의 연결 간선이 없을 것이다. 이러한 그래프를 이분 그래프라고 한다.

Definition 24.5 (이분 그래프). 어떤 그래프 $G = (V, E)$ 에서 V 를 정확히 두 집합 A 와 B 로 $A \subset G, B \subset G, A \cup B = G, A \cap B = \emptyset$ 가 되도록 나눌 때, A 의 임의의 두 원소 u, v 에 대해 $(u, v) \notin E$ 가 성립하고 B 의 임의의 두 원소 u, v 에 대해 $(u, v) \notin E$ 가 성립하는 A 와 B 가 존재할 때 G 를 **이분 그래프**라고 한다.

우리는 이분 그래프에서 두 집합의 원소를 간선으로 연결된 경우에 한해 매칭할 수 있다(당연히 같은 원소는 두 번 이상 매칭되면 안될 것이다). 이러한 문제를 이분 매칭 문제라고 하며 이분 매칭 문제는 유량 문제의 시작점이 되는 문제로써 중요하다.

정의역과 공역이 같은 함수 f 를 그래프로 나타낼 수 있다. 정의역과 공역을 정점 집합 V 로 놓고, 각 V 의 원소 v 에 대해 v 에서 $f(v)$ 로 가는 방향 간선을 설계하면 된다. 역으로 말하면 각 정점 v 에 대해 진출 차수가 1인 모든 그래프는 적절한 함수 f 를 찾을 수 있으므로 함수형 그래프라고 말할 수 있다.

Definition 24.6 (함수형 그래프). $G = (V, E)$ 가 방향 그래프이고 임의의 정점 $v \in V$ 에 대해 진출 차수가 1이면 G 를 **함수형 그래프**라고 한다.

함수형 그래프는 f 의 합성함수를 처리하는데 유용하다. 단순히 $f(f(f(v)))$ 를 구하고 싶다면 v 에서 간선을 따라 세 번 이동하면 되기 때문이다. 이러한 함수형 그래프는 희소 배열을 시각적으로 나타낸 것과 같다.

다른 종류의 특수한 방향 그래프도 있다. 만약, 어떤 방향 그래프가 사이클을 갖지 않는다면, 이러한 그래프를 방향 비순환 그래프(DAG)라고 한다.

Definition 24.7 (방향 비순환 그래프). $G = (V, E)$ 가 방향 그래프이고, 이 그래프에 어떠한 사이클도 존재하지 않는다면 G 를 **방향 비순환 그래프**라고 한다.

어떤 그래프가 DAG인 것과 위상 정렬이 존재하는 것은 필요충분조건이다. 이와 관련된 자세한 내용은 위상 정렬 단원에서 살펴보겠다.

어떤 그래프는 간선끼리 교점이 존재하지 않도록 평면에 나타낼 수 있다. 즉, 주어진 그래프를 평면의 분할로 나타낼 수 있는 그래프가 존재한다. 이러한 그래프를 평면 그래프라고 한다.

Definition 24.8 (평면 그래프). $G = (V, E)$ 가 간선끼리 교점이 존재하지 않도록 한 평면에

정점과 그 정점을 잇는 간선을 그릴 수 있을 때 G 를 **평면 그래프**라고 한다. 매우 엄밀하게 말하면 평면에 graph embedding이 가능한 그래프이다.

당연하게도 평면 그래프는 오일러의 등식 $v - e + f = 2$ 을 만족시킨다.

24.3 연습문제 A

Problem 24.1. Code 23.1와 Code 23.2를 참고하여 방향 그래프의 인접행렬과 인접리스트를 구하는 코드를 작성하라.

Problem 24.2. Code 23.1와 Code 23.2를 참고하여 가중치가 있는 무방향 그래프의 인접행렬과 인접리스트를 구하는 코드를 작성하라.

Problem 24.3. 정점 N 개로 이루어진 무방향 연결 그래프의 간선 개수 최솟값이 $N - 1$ 임을 증명하라. (hint. 수학적 귀납법)

Problem 24.4. 트리의 성질로 주어진 세 명제가 동치임을 증명하라.

Problem 24.5. 정점의 개수와 간선의 개수가 같은 연결 그래프(선인장)에 정확히 하나의 사이클이 존재하고, 그 사이클에 연결된 부분 그래프는 트리임을 증명하라.

Problem 24.6. 임의의 연결 그래프에서 간선 몇 개를 적절히 제거하여 트리를 만들 수 있음을 보여라(이러한 트리를 스패닝 트리라고 한다).

Problem 24.7. graph embedding의 정의를 명확히 찾아보고 평면 그래프와 어떠한 관련이 있는지 분석하라.

Problem 24.8. 평면 그래프의 필요충분조건을 말해주는 **쿠라토프스키 정리**가 있다. 이 정리를 이해하고 증명을 읽어보자.

24.4 연습문제 B

24.4.1 기초문제

Problem 24.9. <https://www.acmicpc.net/problem/9644>

Problem 24.10. <https://www.acmicpc.net/problem/15166>

Problem 24.11. <https://www.acmicpc.net/problem/22100>

Problem 24.12. <https://www.acmicpc.net/problem/22392>

Problem 24.13. <https://www.acmicpc.net/problem/23039>

Problem 24.14. <https://www.acmicpc.net/problem/30411>

24.4.2 응용문제

Problem 24.15. <https://www.acmicpc.net/problem/3461>

Problem 24.16. <https://www.acmicpc.net/problem/4634>

Problem 24.17. <https://www.acmicpc.net/problem/9666>

Problem 24.18. <https://www.acmicpc.net/problem/13331>

Problem 24.19. <https://www.acmicpc.net/problem/15010>

Problem 24.20. <https://www.acmicpc.net/problem/21002>

Problem 24.21. <https://www.acmicpc.net/problem/23154>

Problem 24.22. <https://www.acmicpc.net/problem/24689>

Problem 24.23. <https://www.acmicpc.net/problem/28111>

24.4.3 도전문제

Problem 24.24. <https://www.acmicpc.net/problem/10591>

Problem 24.25. <https://www.acmicpc.net/problem/12625>

Problem 24.26. <https://www.acmicpc.net/problem/14519>

Problem 24.27. <https://www.acmicpc.net/problem/16746>

Problem 24.28. <https://www.acmicpc.net/problem/17772>

Problem 24.29. <https://www.acmicpc.net/problem/18447>

Problem 24.30. <https://www.acmicpc.net/problem/19220>

Problem 24.31. <https://www.acmicpc.net/problem/19445>

Problem 24.32. <https://www.acmicpc.net/problem/19552>

Problem 24.33. <https://www.acmicpc.net/problem/20573>

Problem 24.34. <https://www.acmicpc.net/problem/20966>

Problem 24.35. <https://www.acmicpc.net/problem/24133>

Problem 24.36. <https://www.acmicpc.net/problem/25355>

Problem 24.37. <https://www.acmicpc.net/problem/28332>

Chapter 25

그래프 탐색

그래프 탐색은 그래프의 정보를 바탕으로 그래프의 모든 정점을 탐색하는 것을 말한다. 기본적으로는 모든 정점을 한 번씩 빠짐없이 탐색하는 것을 기본으로 하지만, 이 탐색 방법의 특징을 이용해 문제를 해결하는 경우도 있기 때문에 꼭 알아두어야 할 것이다. 이 장에서는 대표적인 그래프 탐색 방식인 깊이 우선 탐색과 너비 우선 탐색을 배운다.

25.1 깊이 우선 탐색

깊이 우선 탐색(DFS)은 이름에서도 알 수 있듯이 시작 정점으로부터 점점 더 깊어지는(멀어지는) 방향으로 탐색하는 방식이다. Example 25.1에서 이 방식을 정확히 설명하고 있다.

Example 25.1. 그래프 $G = (V, E)$ 의 정점 수와 간선이 주어질 때, 1번 정점을 시작 정점으로 하여 다음과 같은 방식으로 탐색한 경로를 출력하는 프로그램을 작성하라.

- 제일 처음 시작(1번) 정점을 방문한다.
- 만약 연결된 간선 중 방문하지 않은 정점이 있다면 아무 것이나 방문하고 탐색 경로에 방문한 정점을 추가한다.
- 만약 연결된 간선 중 방문하지 않은 정점이 없다면 바로 직전에 방문했던 정점에서 다시 이 과정을 수행한다.

위 방식대로 재귀함수로 구현하면 쉽게 구현할 수 있다. Code 25.1은 Example 25.1를 연결리스트 방식으로 구현한 코드이다.

Code 25.1. 연결리스트 기반 DFS 구현 코드

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, m;
5  vector<int> adj[101010];
6
7  int visited[101010];
8  vector<int> path;
9  void DFS(int v) {
10     path.push_back(v);
11     for (int i = 0; i < adj[v].size(); i++) {
12         int u = adj[v][i];
13         if (visited[u] == 0) {
14             visited[u] = 1;
15             DFS(u);
16         }
17     }
18 }
19
20 int main() {
21     ios_base::sync_with_stdio(false);
22     cin.tie(0);
23     cin >> n >> m;
24     for (int i = 1; i <= m; i++) {
25         int u, v;
26         cin >> u >> v;
27         adj[u].push_back(v);
28         adj[v].push_back(u);
29     }
30     DFS(1);
31     //path is answer
32 }
```

그래프 탐색은 모든 점에 방문하는 것 자체로 사용되는 경우가 많기 때문에 구현이 간단한 DFS가 자주 사용된다. 하지만 DFS로 알아낼 수 있는 것이 없는 것은 아니다. DFS를 사용하면 시작 정점을 포함하는 컴포넌트를 파악할 수 있다. 왜냐하면 연결된 모든 정점을 도달해보기 때문이다.

25.2 너비 우선 탐색

너비 우선 탐색(BFS)은 깊이 우선 탐색과 다르게 시작 정점으로부터 멀어지는 방향이 아니라 거리가 최대한 가까운 것부터 탐색하는 방식이다. Example 25.2에서 이 방식을 정확히 설명하고 있다.

Example 25.2. 그래프 $G = (V, E)$ 의 정점 수와 간선이 주어질 때, 1번 정점을 시작 정점으로 하여 다음과 같은 방식으로 탐색한 경로를 출력하는 프로그램을 작성하라.

- 이 과정은 방문 대기열을 따라 이루어진다. 제일 처음 시작(1번) 정점을 방문 대기열에 추가한다. 이후 방문 대기열의 순서대로 정점을 본다.
- 만약 연결된 간선 중 방문하지 않은 정점이 있다면 그 정점들을 모두 방문 대기열에 추가한다.
- 만약 연결된 간선 중 방문하지 않은 정점이 없다면 방문 대기열을 따라 다음 정점으로 이동한다.

방문 대기열의 처리는 동적 배열에 포인터를 쓰거나 정적 배열에 두 개의 포인터를 사용해도 구현할 수 있으나, 큐 자료구조를 사용하면 쉽게 구현할 수 있다. Code 25.2은 Example 25.2를 연결리스트 방식으로 구현한 코드이다.

Code 25.2. 연결리스트 기반 BFS 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, m;
5  vector<int> adj[101010];
6
7  int visited[101010];
8  queue<int> q;
9  vector<int> path;
10
11 int main() {
12     ios_base::sync_with_stdio(false);
13     cin.tie(0);
14     cin >> n >> m;
15     for (int i = 1; i <= m; i++) {
16         int u, v;
17         cin >> u >> v;
18         adj[u].push_back(v);
19         adj[v].push_back(u);

```

```

20     }
21
22     q.push(1);
23     visited[1] = 1;
24     while (!q.empty()) {
25         int v = q.front();
26         q.pop();
27         path.push_back(v);
28         for (int i = 0; i < adj[v].size(); i++) {
29             int u = adj[v][i];
30             if (visited[u] == 0) {
31                 visited[u] = 1;
32                 q.push(u);
33             }
34         }
35     }
36     // path is answer
37 }

```

BFS는 DFS보다는 조금 더 복잡하지만 가장 거리가 가까운 정점부터 방문하기 때문에 최단거리를 구할 수 있다는 장점이 있다(가중치가 없는 경우에). 이 외에는 거의 사용되지 않을 것이다. 구현도 매우 간단한데, 큐에 새로운 정점 u 를 넣기 전에 $dis[u] = dis[v] + 1$ 를 이용해 거리를 구하면 된다. 이미 $dis[v]$ 가 최단거리이기 때문에 수학적 귀납법에 의해 구해진 $dis[u]$ 도 최단거리가 된다.

25.3 0-1 BFS

BFS는 간선의 가중치가 있으면 사용될 수 없는 방법이다. 하지만 특수하게 간선의 가중치가 0이거나 모두 같은 수라면 BFS를 활용하여 최단거리를 구할 수 있다. 기본적인 아이디어는 가중치가 있는 경우 최단거리를 구할 때 다시 설명하겠지만 현재까지 최단거리가 구해진 정점 중 가장 거리가 짧은 정점으로부터 거리를 구하면 최단거리가 유지된다는 것이다. 이를 바탕으로 생각해보면, 거리가 0인 간선을 통해 도달한 정점의 경우에는 여전히 현재까지 있는 정점 중 가장 짧은 거리에 있는 정점이므로 방문 대기열의 제일 앞에 넣고, 거리가 1인 간선을 통해 도달한 정점의 경우에는 현재까지 거리가 구해진 정점 중 가장 먼 거리에 있는 정점이므로 방문 대기열의 제일 끝에 넣겠다는 아이디어가 성립한다. 텍은 이러한 연산을 지원하므로 텍을 사용하면 가중치가 0 또는 1인 가중치가 있는 그래프의 최단거리를 구해낼 수 있다.

25.4 연습문제 A

Problem 25.1. 인접행렬 기반 DFS 구현과 BFS 구현의 시간복잡도를 논의하라.

Problem 25.2. DFS와 BFS에서 `visited` 배열을 사용하지 않으면 실행 결과가 어떻게 될지 생각해보라.

Problem 25.3. Example 25.1와 Example 25.2에서 연결된 간선 중 방문하지 않은 정점이 있다면 가장 번호가 작은 정점을 방문하게 한다면 코드를 어떻게 수정해야 할지 생각해보라.

Problem 25.4. BFS를 통해 최단 경로의 길이 뿐만 아니라 최단 경로도 구하는 코드를 작성하라.

Problem 25.5. BFS에서 k 개의 정점 $v_1, v_2, \dots, v_k$ 와 임의의 정점 u 에 대해 v_i 와 u 사이 거리의 최솟값을 찾는 코드를 작성하라.

Problem 25.6. 가중치 값이 0 또는 1이라고 가정하고, 입력 코드부터 0-1 BFS를 바탕으로 최단거리를 찾는 코드까지 작성하라.

25.5 연습문제 B

25.5.1 기초문제

Problem 25.7. <https://www.acmicpc.net/problem/1012>

Problem 25.8. <https://www.acmicpc.net/problem/1260>

Problem 25.9. <https://www.acmicpc.net/problem/1261>

Problem 25.10. <https://www.acmicpc.net/problem/2668>

Problem 25.11. <https://www.acmicpc.net/problem/3255>

Problem 25.12. <https://www.acmicpc.net/problem/5925>

Problem 25.13. <https://www.acmicpc.net/problem/6166>

Problem 25.14. <https://www.acmicpc.net/problem/7569>

Problem 25.15. <https://www.acmicpc.net/problem/11967>

Problem 25.16. <https://www.acmicpc.net/problem/13078>

Problem 25.17. <https://www.acmicpc.net/problem/13549>

Problem 25.18. <https://www.acmicpc.net/problem/16397>

Problem 25.19. <https://www.acmicpc.net/problem/17244>

Problem 25.20. <https://www.acmicpc.net/problem/20419>

Problem 25.21. <https://www.acmicpc.net/problem/22352>

25.5.2 응용문제

Problem 25.22. <https://www.acmicpc.net/problem/1981>

Problem 25.23. <https://www.acmicpc.net/problem/2424>

Problem 25.24. <https://www.acmicpc.net/problem/5507>

Problem 25.25. <https://www.acmicpc.net/problem/10094>

Problem 25.26. <https://www.acmicpc.net/problem/14599>

Problem 25.27. <https://www.acmicpc.net/problem/15019>

Problem 25.28. <https://www.acmicpc.net/problem/15514>

Problem 25.29. <https://www.acmicpc.net/problem/15730>

Problem 25.30. <https://www.acmicpc.net/problem/16835>

Problem 25.31. <https://www.acmicpc.net/problem/18251>

Problem 25.32. <https://www.acmicpc.net/problem/21935>

Problem 25.33. <https://www.acmicpc.net/problem/25718>

Problem 25.34. <https://www.acmicpc.net/problem/26051>

Problem 25.35. <https://www.acmicpc.net/problem/27602>

Problem 25.36. <https://www.acmicpc.net/problem/28104>

Problem 25.37. <https://www.acmicpc.net/problem/28143>

25.5.3 도전문제

Problem 25.38. <https://www.acmicpc.net/problem/10483>

Problem 25.39. <https://www.acmicpc.net/problem/10743>

Problem 25.40. <https://www.acmicpc.net/problem/10955>

Problem 25.41. <https://www.acmicpc.net/problem/11742>

Problem 25.42. <https://www.acmicpc.net/problem/12343>

Problem 25.43. <https://www.acmicpc.net/problem/13088>

Problem 25.44. <https://www.acmicpc.net/problem/13949>

Problem 25.45. <https://www.acmicpc.net/problem/13963>

Problem 25.46. <https://www.acmicpc.net/problem/16663>

Problem 25.47. <https://www.acmicpc.net/problem/17385>

Problem 25.48. <https://www.acmicpc.net/problem/17667>

Problem 25.49. <https://www.acmicpc.net/problem/18929>

Problem 25.50. <https://www.acmicpc.net/problem/19012>

Problem 25.51. <https://www.acmicpc.net/problem/20349>

Problem 25.52. <https://www.acmicpc.net/problem/20657>

Problem 25.53. <https://www.acmicpc.net/problem/21085>

Problem 25.54. <https://www.acmicpc.net/problem/21346>

Problem 25.55. <https://www.acmicpc.net/problem/21799>

Problem 25.56. <https://www.acmicpc.net/problem/23052>

Problem 25.57. <https://www.acmicpc.net/problem/23164>

Problem 25.58. <https://www.acmicpc.net/problem/24902>

Problem 25.59. <https://www.acmicpc.net/problem/27867>

Problem 25.60. <https://www.acmicpc.net/problem/28686>

Chapter 26

최소 스패닝 트리

트리는 연결 그래프 중 가장 적은 간선으로 이루어진 그래프이기 때문에 연결 그래프 $G = (V, E)$ 에서 $E' \subset E$ 이고 $T = (V, E')$ 가 트리를 이루는 T 가 무한히 많이 존재한다. 이번 장에서는 가중치가 주어졌을 때 E' 을 이루는 간선의 가중치 합이 가장 작은 T 를 찾는 문제인 최소 스패닝 트리 문제를 다룬다.

26.1 최소 스패닝 트리

서두에서도 간단히 언급했지만, 가중치가 있는 연결 그래프에서 일부 간선을 제거하여 만든 트리인 스패닝 트리 중에서 간선 가중치의 합이 가장 작은 트리를 최소 스패닝 트리라고 한다. 이번 절에서 다룰 문제는 like-linear한 시간에 최소 스패닝 트리를 찾는 것이다.

Example 26.1. 그래프 $G = (E, V)$ 가 주어질 때, 이 그래프 G 의 스패닝 트리 중 가장 간선 가중치가 작은 스패닝 트리(최소 스패닝 트리) T 의 간선 가중치 합을 출력하라.

이 문제는 그리디 기반으로 해결할 수 있으며, 두 가지 알고리즘을 소개한다.

26.2 Kruskal 알고리즘

가장 가중치가 작은 간선부터 보면서 추가하거나 추가하지 않는 방식으로 그리디를 사용한다. 추가하는 경우는 연결했을 때 트리가 유지되는 경우(사이클이 생기지 않는 경우)이고, 추가하지 않는 경우는 연결했을 때 트리가 깨지는 경우이다. 증명은 일반적인 그리디의 증명 방법과 유사하게 할 수 있다.

Proof. 위 방법대로 선택되지 않은 최소 스패닝 트리가 존재한다고 가정하자. 그 스패닝 트리에는 u 와 v 를 잇는 가중치 w 인 간선이 골라지지 않고, 트리 상에서 u 와 v 를 잇는 경로 중 가중치가 w 보다 큰 간선이 존재할 것이다. 그 간선을 u 와 v 를 연결하는 간선으로 바꾸어도 여전히 트리가 되기 때문에 처음의 트리가 MST라는 가정에 모순이다. $\square$

그렇다면 추가해도 트리가 유지되는지는 어떻게 판단할까? 연결해도 트리가 유지되려면, 사이클이 생기면 안되므로 간선의 양쪽 정점이 같은 컴포넌트에 속하지 않으면 된다. 이는 분리 집합을 이용해 쉽게 판단할 수 있다. 매번 간선을 연결할 때마다 양 끝점에 union 연산을 하면 쉽게 판단할 수 있을 것이다. 간선의 가중치에 따라 저장해야 하기 때문에 인접행렬 또는 인접리스트 저장 방식보다 간선을 배열로 저장하는 것이 더 유용하다. 구현 코드는 Code 26.1을 참고하라. Union 함수와 Find 함수는 코드에서 생략하였다.

Code 26.1. Kruskal 알고리즘으로 MST를 찾는 코드 (분리집합 구현 코드는 생략)

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, m, ans = 0;
5
6  struct edge{
7      int u, v, w;
8  };
9  vector<edge> edges;
10 bool compare(edge a, edge b) {
11     return a.w < b.w;
12 }
13
14 // Find and Union function
15
16 int main() {
17     ios_base::sync_with_stdio(false);
18     cin.tie(0);
19     cin >> n >> m;
20     for (int i = 1; i <= m; i++) {
21         int u, v, w;
22         cin >> u >> v >> w;
23         edges.push_back({u, v, w});
24     }
25     sort(edges.begin(), edges.end(), compare);
26

```

```

27     int cnt = 0;
28     for (int i = 0; i < m; i++) {
29         if (Find(edges[i].u) == Find(edges[i].v)) continue;
30         else {
31             Union(edges[i].u, edges[i].v);
32             cnt++;
33             ans += edges[i].w;
34         }
35     }
36     //ans is answer
37 }

```

시간복잡도는 정렬이 지배하므로 $O(E \log E)$ 이다.

26.3 Prim 알고리즘

Kruskal 알고리즘이 간선을 기준으로 했다면 Prim 알고리즘은 정점을 기준으로 한다. 새로운 정점을 적절한 간선으로 추가하는 방식으로 그리드를 이어나간다. 기본적으로 Kruskal 알고리즘과 동치가 되어야 하므로 이미 선택된 정점끼리는 연결하지 않고, 이미 선택된 정점과 아직 선택되지 않은 정점 사이에서 가장 가중치가 작은 것을 고른다. 결과적으로 만들어지는 트리는 연결 그래프이기 때문에 이렇게 고르더라도 Kruskal 알고리즘과 고르는 순서만 다를 뿐 골라지는 간선은 같음을 쉽게 증명할 수 있다. (증명과 구현 추가)

26.4 연습문제 A

Problem 26.1. 최소 스패닝 트리로 가능한 것 하나를 출력하는 프로그램을 작성하라.

Problem 26.2. 최소 스패닝 트리로 가능한 것이 하나인지, 여러 개인지 판별하는 프로그램을 작성하라.

Problem 26.3. N 개의 수로 이루어진 수열 A_i 가 주어진다. 여러분은 i 번 정점과 j 번 정점을 비용 $\gcd(A_{i+1}, A_{i+2}, \dots, A_j)$ 로 연결할 수 있다. 1번부터 N 번까지의 정점이 포함된 그래프 G 를 연결 그래프로 만들기 위해 필요한 최소 비용을 $O(N^2)$ 의 시간복잡도로 구하는 프로그램을 작성하라. 단, $\gcd$ 는 최대공약수 함수이다.

26.5 연습문제 B

26.5.1 기초문제

Problem 26.4. <https://www.acmicpc.net/problem/1197>

Problem 26.5. <https://www.acmicpc.net/problem/4343>

Problem 26.6. <https://www.acmicpc.net/problem/4386>

Problem 26.7. <https://www.acmicpc.net/problem/9134>

Problem 26.8. <https://www.acmicpc.net/problem/18769>

Problem 26.9. <https://www.acmicpc.net/problem/20010>

Problem 26.10. <https://www.acmicpc.net/problem/24010>

Problem 26.11. <https://www.acmicpc.net/problem/25545>

Problem 26.12. <https://www.acmicpc.net/problem/27009>

Problem 26.13. <https://www.acmicpc.net/problem/28019>

26.5.2 응용문제

Problem 26.14. <https://www.acmicpc.net/problem/1396>

Problem 26.15. <https://www.acmicpc.net/problem/3900>

Problem 26.16. <https://www.acmicpc.net/problem/7788>

Problem 26.17. <https://www.acmicpc.net/problem/10335>

Problem 26.18. <https://www.acmicpc.net/problem/12667>

Problem 26.19. <https://www.acmicpc.net/problem/14574>

Problem 26.20. <https://www.acmicpc.net/problem/18919>

Problem 26.21. <https://www.acmicpc.net/problem/20263>

Problem 26.22. <https://www.acmicpc.net/problem/23108>

Problem 26.23. <https://www.acmicpc.net/problem/23736>

26.5.3 도전문제

Problem 26.24. <https://www.acmicpc.net/problem/4203>

Problem 26.25. <https://www.acmicpc.net/problem/5257>

Problem 26.26. <https://www.acmicpc.net/problem/8527>

Problem 26.27. <https://www.acmicpc.net/problem/11592>

Problem 26.28. <https://www.acmicpc.net/problem/12668>

Problem 26.29. <https://www.acmicpc.net/problem/14828>

Problem 26.30. <https://www.acmicpc.net/problem/16074>

Problem 26.31. <https://www.acmicpc.net/problem/17582>

Problem 26.32. <https://www.acmicpc.net/problem/18482>

Problem 26.33. <https://www.acmicpc.net/problem/18963>

Problem 26.34. <https://www.acmicpc.net/problem/19251>

Problem 26.35. <https://www.acmicpc.net/problem/19852>

Problem 26.36. <https://www.acmicpc.net/problem/20086>

Problem 26.37. <https://www.acmicpc.net/problem/20503>

Problem 26.38. <https://www.acmicpc.net/problem/24953>

Problem 26.39. <https://www.acmicpc.net/problem/24961>

Problem 26.40. <https://www.acmicpc.net/problem/25121>

Problem 26.41. <https://www.acmicpc.net/problem/27071>

Problem 26.42. <https://www.acmicpc.net/problem/27877>

Problem 26.43. <https://www.acmicpc.net/problem/28002>

Chapter 27

이중 연결 요소

이중 연결 요소(정점-서로소 이중 연결 요소)는 그래프를 트리로 압축하는 중요한 도구 중 하나이다. 임의의 무향 연결 그래프는 이중 연결 요소라 불리는 것으로 묶을 수 있고, 각 이중 연결 요소를 정점으로 한 트리로 압축할 수 있다. 이 절에서는 단절점과 단절선, 그리고 이중 연결 요소를 다룬다.

27.1 단절점

단절점은 무향 그래프에서 한 정점과 연결된 간선들을 제거했을 때 그래프가 둘 이상으로 나뉘는 정점을 말한다. 이러한 정점들을 구하는 문제를 생각해보자.

Example 27.1. 연결 그래프 $G = (V, E)$ 가 주어질 때, V 의 각 원소 v 에 대해 다음 조건을 만족하는지 판단하라.

조건: E 의 원소들 중 v 와 다른 한 정점을 연결하는 간선들을 모두 제거한 집합 E' 에 대해 $G' = (V - \{v\}, E')$ 이 연결 그래프가 아니다.

다음과 같은 관찰을 기반으로 하면 단절점은 DFS를 이용하여 선형 시간에 구할 수 있다.

Observation. 시작 정점이 아닌 정점 v 에 대해 v 를 방문하기 이전에 방문한 정점 a 와 v 를 방문한 이후 방문한 정점 b 가 연결된 순서쌍 (a, b) 가 존재하면 v 는 단절점이 아니다. 즉, v 가 단절점이라는 것은 DFS order에서 v 앞에 있는 임의의 정점과 v 뒤에 있는 임의의 정점을 골랐을 때 연결되지 않았다는 것과 필요충분관계이다.

즉, DFS order에서 자신의 자식 노드들이 방문한 정점(자신을 방문한 이후에 확인한 정점)들 중

자신보다 일찍 방문한 정점이 있다면 그 정점은 단절점이 아니고, 나머지 정점은 단절점이다.

단, 이러한 논의는 시작 정점에 대해서는 적용할 수 없는데, 시작 정점은 자신보다 일찍 방문한 정점이 없기 때문이다. 하지만 시작 정점도 단절점이 될 수 있는데, 시작 정점에서 출발한 DFS가 종료되고 다시 시작 정점으로 돌아왔을 때 아직 방문하지 않은 정점이 남아있다면 둘 이상의 닫힌 부분이 있다는 뜻이므로 시작 정점이 단절점이 된다. 즉, 시작 정점의 경우 시작 정점에서 둘 이상의 재귀 함수 호출이 있으면 단절점이고, 그렇지 않으면 단절점이 아니다.

다음 Code 27.1는 이 과정을 구현한 DFS 코드이다.

Code 27.1. Example 27.1의 DFS 부분 코드

```

1  int r = 0;
2  int solve(int v) {
3      ord[v] = ++r;
4      int fast = 1e9, child = 0;
5
6      for (int i = 0; i < adj[v].size(); i++) {
7          int u = adj[v][i];
8          if (ch[u] == 1) fast = min(fast, ord[u]);
9          else {
10             child++;
11             ch[u] = 1;
12             int prev = solve(u);
13             if (v != 1 && prev >= ord[v]) iscut[v] = 1;
14             fast = min(fast, prev);
15         }
16     }
17
18     if (v == 1 && child >= 2) iscut[v] = 1;
19
20     return fast;
21 }
```

27.2 단절선

단절선은 단절점과 비슷한 개념이지만 정점이 아닌 간선을 제거하는 것이다. 즉, 무향 그래프에서 한 간선을 제거했을 때 그래프가 둘 이상으로 나뉘는 간선을 말한다. 이러한 간선들을 구하는 문제를 생각해보자.

Example 27.2. 연결 그래프 $G = (V, E)$ 가 주어질 때, E 의 각 원소 e 에 대해 다음 조건을 만족하는지 판단하라.

조건: $G' = (V, E - \{e\})$ 이 연결 그래프가 아니다.

다음과 같은 관찰을 기반으로 단절선도 DFS를 이용하여 선형 시간에 구할 수 있다.

Observation. 어떤 간선 e 에서 시작 정점에 가까운 정점을 v 라고 하자. v 로 도달하는 경로 중 DFS order에 따라 온 경로가 아닌 다른 경로에서 v 보다 방문 순서가 이른 정점이 없으면 그 간선 e 는 단절선이다.

구현 자체는 단절점과 크게 다르지 않다. 간선의 시작 정점과 가까운 정점 v 가 v 를 방문한 이후에 확인한 정점들 중 v 의 부모가 아니고 자신보다 일찍 방문한 정점이 있다면 그 간선은 단절선이 아니고, 나머지 간선은 단절선이다.

구현은 다음 Code 27.2를 참고하라.

Code 27.2. Example 27.2의 DFS 부분 코드

```

1  int r = 0;
2  int solve(int v, int p) {
3      ord[v] = ++r;
4      int fast = 1e9;
5
6      for (int i = 0; i < adj[v].size(); i++) {
7          int u = adj[v][i].first, j = adj[v][i].second;
8          if (u == p) continue;
9          if (ch[u] == 1) fast = min(fast, ord[u]);
10         else {
11             ch[u] = 1;
12             int prev = solve(u, v);
13             if (prev > ord[v]) iscut[j] = 1;
14             fast = min(fast, prev);
15         }
16     }
17
18     return fast;
19 }
```

27.3 이중 연결 요소

이중 연결 요소는 단절점과 관련된 개념으로 한 정점과 연결된 간선을 제거했을 때 서로 연결되어 있는 가장 큰 부분 그래프를 말한다. 여기서 단절점을 기준으로 구분하면 각각의 부분 그래프가 이중 연결 요소가 되며 이러한 방식으로 그래프를 나눌 수 있다.

Definition 27.1 (이중 연결 요소). 연결 그래프 $G = (V, E)$ 의 이중 연결 요소는 G 의 부분 연결 그래프이며 단절점이 없는 그래프 중 자신을 부분 그래프로 갖는 G 의 이중 연결 요소가 없는 그래프를 말한다.

한 그래프의 이중 연결 요소는 정점은 겹칠 수 있으나 간선은 겹치지 않는다. 따라서 구현에서 이중 연결 요소는 간선으로 저장한다.

구체적인 구현은 단절점의 구현에 간선을 묶어주는 부분만 추가하면 된다. 현재 보고 있는 점에 대해 연결된 간선이 아직 추가하지 않은 간선이면 스택에 간선을 넣는다. 지금 보고 있는 점이 단절점이라면 이 시점까지 확인한 부분이 하나의 이중 연결 요소이므로 스택에 넣은 간선 중 현재 보고 있는 간선까지 하나의 이중 연결 요소로 묶으면 된다. 구현 코드는 Code 27.3를 참고하라.

Code 27.3. 이중 연결 요소의 DFS 부분 코드

```

1  stack< pair<int, int> > stk;
2  vector< vector< pair<int, int> > > bcc;
3
4  int r = 0;
5  int solve(int v) {
6      ord[v] = ++r;
7      int fast = 1e9;
8
9      for (int i = 0; i < adj[v].size(); i++) {
10         int u = adj[v][i];
11         if (ord[u] > ord[v]) stk.push({ord[v], ord[u]});
12         if (ch[u] == 1) fast = min(fast, ord[u]);
13         else {
14             ch[u] = 1;
15             int prev = solve(u);
16             if (prev >= ord[v]) {
17                 vector< pair<int, int> > _bcc;
18                 while (stk.top() != {ord[v], ord[u]}) {
19                     _bcc.push_back(stk.top());
20                     stk.pop();

```

```

21         }
22         _bcc.push_back({ord[v], ord[u]});
23         stk.pop();
24         bcc.push_back(_bcc);
25     }
26     fast = min(fast, prev);
27 }
28 }
29
30 return fast;
31 }

```

27.4 블록-컷 트리

이중 연결 요소를 바탕으로 그래프를 트리로 압축할 수 있다. 각 이중 연결 요소를 블록으로, 단절점을 컷으로 표현하면 블록과 컷을 연결하는 간선만으로 이루어진 블록-컷 트리를 만들 수 있다. 정확히는 각 단절점을 컷 정점으로, 각 이중 연결 요소를 블록 정점에 대응시키고 단절점이 이중 연결 요소의 정점 집합에 포함되는 컷과 블록을 연결하여 트리를 만든다.

블록-컷 트리를 사용하는 대표적인 문제를 하나 보자.

Example 27.3. 어떤 무향 연결 그래프 $G = (V, E)$ 가 주어질 때, 다음 두 쿼리를 처리하는 프로그램을 작성하라.

- u 와 v 를 잇는 간선이 E 에 포함되어 있을 때 u 와 v 를 잇는 간선을 제거한 후 u 와 v 가 연결되어 있는지 판단하라.
- w 를 포함하는 모든 간선을 E 에서 제거한 후 u 와 v 가 연결되어 있는지 판단하라.

(<https://www.acmicpc.net/problem/1734>)

두 쿼리 모두 같은 방식으로 해결할 수 있다. 일단 쿼리 1에서는 u 와 v 를 잇는 간선이 단절선이 아니면 무조건 연결되어 있고 쿼리 2에서는 w 가 단절점이 아니면 무조건 연결되어 있다. 이제 블록-컷 트리에서 u 와 v 를 잇는 유일한 경로를 생각하자. 이 경로 위에 u 와 v 를 잇는 간선을 포함하는 이중 연결 요소의 블록이 존재하면, 단절점 w 에 해당하는 컷이 존재하면 u 에서 v 로 가는 경로가 절단되므로 더 이상 연결되지 않게 된다. 이는 트리에서 최소 공통 조상을 사용하면 구현할 수 있으나, 아직 배우지 않았으므로 그 전까지를 구하는 것을 목표로 해보자. 이를 실제로 구현하는 문제는 Problem 27.2에 있다.

27.5 연습문제 A

Problem 27.1. 이중 연결 요소를 구하면 단절선도 구해낼 수 있다. 이중 연결 요소로부터 단절선을 구하는 방법을 생각해보고, 코드로 구현하라.

Problem 27.2. Example 27.3을 해결하는 코드를 작성하라. 배우지 않은 부분은 생략하여도 좋다.

Problem 27.3. Code 27.3에서 각 이중 연결 요소에 포함되는 정점도 저장하도록 코드를 수정하여라.

27.6 연습문제 B

27.7 연습문제 C

Chapter 28

위상 정렬

위상 정렬(Topology Sort)은 방향 비순환 그래프(Directed Acyclic Graph, DAG)의 모든 정점을 방향성에 맞게 나열하는 것을 말한다. 즉, 그래프에 존재하는 모든 간선이 앞에서 뒤로 향하도록 정점들을 나열하는 것이다. 위상 정렬은 작업의 선후 관계를 나타내는 그래프에서 작업을 수행하는 순서를 결정하는 데 유용하다. 예를 들어, 어떤 작업이 다른 작업보다 먼저 완료되어야 한다면, 그 작업을 먼저 수행하도록 작업 순서를 정하는 것이다. 이번 장에서는 방향 비순환 그래프에서 위상 정렬을 수행하는 알고리즘을 다룬다.

28.1 위상 정렬

방향 비순환 그래프는 방향 그래프 중에서 사이클이 없는 방향 그래프를 말한다. 만약 사이클이 있다면 위상 정렬은 수행될 수 없다. 사이클에 속하는 그 어떤 작업도 동일 사이클에 있는 다른 작업에 의존하기 때문에, 어떤 작업도 먼저 수행될 수 없기 때문이다. 따라서 위상 정렬은 방향 비순환 그래프에서만 정의된다.

Definition 28.1 (위상 정렬). 유향 그래프 $G = (V, E)$ 의 위상 정렬은 G 의 모든 정점을 나열하는 것으로, 이때 모든 간선이 앞에서 뒤로 향하도록 정점들을 나열하는 것을 말한다.

위상 정렬을 구하는 방법은 큐를 활용한 방법과 DFS를 활용한 방법이 있다. 두 알고리즘 모두 $O(V + E)$ 의 시간복잡도로 위상 정렬을 구할 수 있다.

28.2 큐 활용 위상정렬 알고리즘

방향 비순환 그래프의 재미있는 성질 중 하나는 진입 차수가 0인 정점이 무조건 존재한다는 것이다. 만약 그렇지 않다면 사이클이 존재하게 된다. 더 나아가, 진입 차수가 0인 정점과, 그 정점과 연결된 간선들을 제거하면 여전히 방향 비순환 그래프가 된다. 따라서 진입 차수가 0인 정점을 큐에 넣고, 큐에서 하나씩 꺼내면서 그 정점과 인접한 정점들의 간선을 제거시키는 과정을 반복하면 위상 정렬이 가능하다. 큐에서 꺼낸 순서대로 놓으면 되기 때문이다.

여기서 문제는 간선을 제거하는 것이다. 인접행렬 방식의 구현이라면 간선 제거가 매우 쉽겠지만, 인접 리스트 방식에서는 어렵다. 이러한 문제의 해결을 위해서는 그래프의 인접 리스트를 업데이트하지 않고, 단순히 진입 차수만 관리하는 배열을 만든 후, 자신으로 들어오는 간선 하나가 사라질 때마다 진입 차수를 관리하는 배열의 값을 1씩 감소시키면 된다. 그러다가 진입차수가 0이 되면 큐에 삽입하는 것이다. 구현 코드는 Code 28.1를 참고하라.

Code 28.1. 큐를 활용한 위상 정렬 구현

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int n, m, in[202020];
4  vector<int> adj[202020];
5  queue<int> q;
6  vector<int> ans;
7
8  int main() {
9      // make adj
10     for (int i = 1; i <= n; i++) {
11         for (int v : adj[i]) in[v]++;
12     }
13     for (int i = 1; i <= n; i++) {
14         if (in[i] == 0) q.push(i);
15     }
16     while (!q.empty()) {
17         int v = q.front();
18         q.pop();
19         ans.push_back(v);
20         for (int u : adj[v]) {
21             in[u]--;
22             if (in[u] == 0) q.push(u);
23         }
24     }
25     // ans contains the topological order

```

```

26     // if ans.size() < n, there is a cycle in the graph
27 }

```

시간복잡도는 $O(V + E)$ 이다.

28.3 DFS 활용 위상정렬 알고리즘

DFS를 활용한 위상 정렬 알고리즘은 DFS의 종료 시간을 활용한다. 핵심 아이디어는

Observation. 방향 비순환 그래프의 각 v 에서 u 로 가는 간선에 대해, v 의 종료 시간이 u 의 종료 시간보다 이른다.

이다. 위상 정렬은 모든 간선이 앞에서 뒤로 향하도록 정점을 나열하는 것이므로, 종료 시간 순으로 정점을 나열하면 위상 정렬이 된다. 구현 코드는 Code 28.2를 참고하라.

Code 28.2. DFS를 활용한 위상 정렬 구현

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int n, m;
4  vector<int> adj[202020];
5  int ch[202020], val[101010];
6  stack<int> stk;
7  void dfs(int v) {
8      ch[v] = 1;
9      for (int i = 0; i < adj[v].size(); i++) {
10         int u = adj[v][i];
11         if (ch[u]) continue;
12         dfs(u);
13     }
14     stk.push(v);
15 }
16 int main() {
17     // make adj
18     for (int i = 1; i <= n; i++) ch[i] = 0;
19     for (int i = 1; i <= n; i++) {
20         if (ch[i]) continue;
21         dfs(i);
22     }
23     while (stk.size() > 0) {

```

```
24         int v = stk.top();
25         stk.pop();
26         // v is in topological order
27     }
28 }
```

이 방법은 주어진 그래프가 DAG일 때는 위상정렬을 구해주지만, 사이클이 있는지 여부는 탐지하지 못한다는 단점이 있다.

28.4 연습문제 A

Problem 28.1. Code 28.2를 약간만 변형하여 시간복잡도를 유지하면서 사이클이 있는지 여부를 탐지할 수 있다. 주어진 그래프가 DAG인지 판별하는 코드를 추가하라.

28.5 연습문제 B

28.5.1 기초문제

28.5.2 응용문제

28.5.3 도전문제

Chapter 29

강한 연결 요소

강한 연결 요소(strongly connected component, SCC)는 방향 그래프에서 정점 u 에서 정점 v 로 가는 경로와 정점 v 에서 정점 u 로 가는 경로가 모두 존재하는 정점들의 집합이다. 즉, 강한 연결 요소는 방향 그래프에서 서로 도달할 수 있는 정점들의 집합이다. 강한 연결 요소를 이용하면 임의의 방향 그래프를 방향 비순환 그래프로 압축할 수 있다. 이번 장에서는 방향 그래프에서 강한 연결 요소를 찾는 알고리즘을 다룬다.

29.1 강한 연결 요소

그래프에 사이클이 있다면 유향 그래프여도 사이클에 포함된 모든 정점 사이를 도달하는 경로가 존재한다. 따라서 이러한 정점들의 집합을 하나로 묶어보자는 아이디어를 낼 수 있다. 수학적으로 엄밀히 서술하면 다음과 같다:

Definition 29.1 (강한 연결 요소). 유향 그래프 $G = (V, E)$ 의 강한 연결 요소는 G 의 부분 그래프이며, 이 부분 그래프의 모든 정점 쌍 (u, v) 에 대해 u 에서 v 로 가는 경로와 v 에서 u 로 가는 경로가 모두 존재하는 그래프를 말한다.

강한 연결 요소를 구하는 가장 쉬운 방법은 $O(V^2)$ 의 시간복잡도를 가지는 알고리즘을 사용하는 것이다. 모든 정점에 대해 DFS 또는 BFS를 수행하여 도달할 수 있는 정점을 찾고, 그 정점에서 다시 DFS 또는 BFS를 수행하여 원래의 정점에 도달할 수 있는지 확인하자. 도달할 수 있다면 SCC에 포함시키면 되고, 아니라면 새로운 SCC를 만들면 된다. 이 방법은 모든 정점에 대해 DFS 또는 BFS를 수행하므로 시간복잡도는 $O(V(V + E))$ 가 된다.

하지만 더 효율적인 알고리즘이 존재한다. 대표적으로 Kosaraju 알고리즘과 Tarjan 알고리즘이 있다. 두 알고리즘 모두 DFS를 기반으로 하여 $O(V + E)$ 의 시간복잡도로 강한 연결 요소를

찾을 수 있다.

29.2 Kosaraju 알고리즘

코사라주 알고리즘은 두 번의 DFS를 이용하여 강한 연결 요소를 찾는 알고리즘이다. 첫 번째 DFS에서는 각 정점의 종료 시간을 기록하고, 두 번째 DFS에서는 그래프의 모든 간선의 방향을 뒤집은 후 종료 시간이 가장 늦은 정점부터 DFS를 수행하여 강한 연결 요소를 찾는다. 현재 까지 방문하지 않은 정점들 중에서, 뒤집어진 그래프에서 방문 가능한 정점을 모으면 하나의 강한 연결 요소가 된다.

이 방법으로 SCC를 구할 수 있는 이유를 간단하게 생각해보자. 먼저,

- (1) 어떤 정점 v 에서 DFS를 수행한다고 하자. 그러면, v 와 같은 SCC에 속한 정점과, v 에서 도달할 수 있는 다른 SCC에 속한 정점들을 모두 방문하게 된다.
- (2) 이제 거꾸로 뒤집은 그래프의 v 에서 DFS를 수행한다고 하자.
 - 원래 그래프의 v 에서 도달할 수 없었던 점들은 더 늦은 종료 시간을 갖는다.
 - 뒤집어진 그래프의 v 에서 도달할 수 있는 점들은 원래 그래프의 v 에서 도달할 수 없었던 점들이거나 같은 SCC에 속한 점들이다.
 - 따라서 뒤집어진 그래프의 v 에서 DFS를 수행하면, v 와 같은 SCC에 속한 정점들만 모두 방문하게 된다. 왜냐하면 원래 그래프에서 v 에서 도달할 수 없었던 점들은 더 늦은 종료 시간을 가져 이미 두 번째 과정에서 DFS를 수행하면서 방문했기 때문이다.

이 내용을 다음과 같이 엄밀히 수학적 귀납으로 증명할 수 있다.

Proof. (보충)

□

구현 시에 종료 시간을 기록하여도 되지만, 먼저 종료된 정점부터 스택에 넣어도 충분하다. 구현 코드는 Code 29.1을 참고하라.

Code 29.1. Kosaraju 알고리즘으로 SCC를 찾는 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int n, m;
4  vector<int> adj1[202020], adj2[202020];
5
6  int ch[202020], val[101010];
7  stack<int> stk;
8  vector< vector<int> > scc;
9  void dfs1(int v) {

```

```

10     ch[v] = 1;
11     for (int i = 0; i < adj1[v].size(); i++) {
12         int u = adj1[v][i];
13         if (ch[u]) continue;
14         dfs1(u);
15     }
16     stk.push(v);
17 }
18
19 void dfs2(int v) {
20     ch[v] = 1;
21     scc.back().push_back(v);
22     for (int i = 0; i < adj2[v].size(); i++) {
23         int u = adj2[v][i];
24         if (ch[u]) continue;
25         dfs2(u);
26     }
27 }
28
29 int main() {
30     // make adj1 and adj2
31     for (int i = 1; i <= n; i++) ch[i] = 0;
32     for (int i = 1; i <= n; i++) {
33         if (ch[i]) continue;
34         dfs1(i);
35     }
36     for (int i = 1; i <= n; i++) ch[i] = 0;
37     while (stk.size() > 0) {
38         int v = stk.top();
39         stk.pop();
40         if (ch[v]) continue;
41         scc.push_back(vector<int>());
42         dfs2(v);
43     }
44     // scc contains all strongly connected components
45 }

```

시간복잡도는 DFS가 지배하므로 $O(V + E)$ 이다.

29.3 Tarjan 알고리즘

(설명, 증명, 구현 추가)

29.4 2-SAT 문제

2-SAT 문제는 강한 연결 요소를 이용하여 해결할 수 있는 문제이다. 2-SAT 문제는 다음과 같이 정의된다:

29.5 연습문제 A

29.6 연습문제 B

29.6.1 기초문제

29.6.2 응용문제

29.6.3 도전문제

Chapter 30

다익스트라 알고리즘

이번 장부터는 그래프 알고리즘의 꽃인 최단거리 알고리즘을 다룰 것이다. 특히, BFS 알고리즘으로 해결할 수 없었던 일반적인 가중치가 있는 그래프에서 최단거리 알고리즘에 대해 논의할 것이다. 첫 번째 알고리즘은 첫 번째는 양의 가중치에서 단일-전체 최단경로 문제를 해결할 수 있는 다익스트라 알고리즘이다.

30.1 문제 정의

다음과 같은 최단경로 문제를 생각하자.

Example 30.1. 모든 가중치가 양수인 그래프에서 1번 정점에서 시작할 때 각 정점까지의 최단거리를 모두 구하는 프로그램을 작성하라.

이러한 문제를 단일-전체 최단경로 문제라고 부른다. 시작 정점이 정해지고, 그 정점으로부터 모든 정점까지의 거리를 구해야 하기 때문이다. 이런 문제는 가중치가 양수라는 조건 하에서 다익스트라 알고리즘으로 해결할 수 있다. 왜 가중치가 양수여야 하는지는 알고리즘 설명 이후 다시 설명하겠다.

30.2 다익스트라 알고리즘

다익스트라 알고리즘의 기본 아이디어는 BFS에서 사용했던 방법을 확장하여 점화관계를 바탕으로 동적계획법을 사용하는 것이다. 그 전에 다음과 같은 관찰이 필요하다.

Observation. u 에서 v 로 가는 어떤 경로가 최단거리라면, 그 경로상의 한 점 x 에 대해 u 에서

x 까지 가는 부분 나열과 x 에서 v 로 가는 부분 나열 모두 최단거리이다.

따라서 최단거리가 보장된 정점부터 점화식을 이용해 거리를 계산해나가면 위 관찰에 의해 계산된 모든 거리가 최단거리이다. 그렇다면 최단거리가 보장된 정점은 어떤 정점일까? 모든 간선의 가중치가 양수이기 때문에 현재까지 방문하지 않은 점들 중 가장 출발점과의 거리가 짧은 점은 최단거리가 보장된 정점이다.

이제 이 알고리즘을 $O(N^2)$ 으로 naive하게 구현한 후 $O(N \log M)$ 으로 최적화할 것이다.

30.2.1 $O(N^2)$ 구현

$O(N^2)$ 구현은 매우 쉽게 할 수 있다. 매번 출발점과의 거리를 확인하여 $O(N)$ 의 시간에 봐야 할 정점을 구하는 과정을 $O(N)$ 번 반복하면 되기 때문이다. 구현 코드는 Code 30.1를 참고하라.

Code 30.1. $O(N^2)$ 다익스트라 알고리즘 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, m;
5  vector<pii> adj[101010];
6
7  int visited[101010], dis[101010];
8
9  int main() {
10     ios_base::sync_with_stdio(false);
11     cin.tie(0);
12     cin >> n >> m;
13     for (int i = 1; i <= m; i++) {
14         int u, v, w;
15         cin >> u >> v >> w;
16         adj[u].push_back({v, w});
17         adj[v].push_back({u, w});
18     }
19
20     for (int i = 1; i <= n; i++) dis[i] = 1e9; //inf
21     dis[1] = 0;
22
23     while (1) {
24         int v = -1, _min = 1e9;
25         for (int i = 1; i <= n; i++) {
26             if (visited[i] == 0 && _min > dis[i]) {

```

```

27         _min = dis[i];
28         v = i;
29     }
30 }
31 if (v == -1) break;
32 visited[v] = 1;
33 for (int i = 0; i < adj[v].size(); i++) {
34     int u = adj[v][i].first, w = adj[v][i].second;
35     dis[u] = min(dis[u], adj[v] + w);
36 }
37 }
38 }

```

30.2.2 $O(M \log M)$ 구현

Code 30.1의 시간복잡도를 엄밀히 쓰면 $O(N^2 + M)$ 일 것이다. $O(N^2)$ 은 거리가 가장 짧은 정점을 찾는 시간복잡도이고, $O(M)$ 은 점화식을 통해 거리를 구하는 과정의 시간복잡도이다. 이 알고리즘을 $O(M \log M)$ 으로 줄이려면 거리가 가장 짧은 정점을 like-constant한 시간에 찾아야 한다. 이를 위한 대표적인 자료구조가 힙(heap)이다.

힙 구조를 사용하기 위해 힙에 현재 계산된 출발점으로부터의 거리와 정점 번호를 쌍으로 묶어 넣고, 거리를 기준으로 정렬하자. 이제 힙에 가장 꼭대기에 있는 거리와 정점 번호 (d, v) 를 받자. 이제 다음 과정에 따라 (d, v) 를 처리하면 된다.

- v 를 이미 처리한 적 있는 경우: 넘어간다.
- d 가 현재 저장된 출발점과 v 사이의 거리가 아닌 경우: 업데이트 이전에 heap에 들어간 정보이므로 넘어간다.
- 나머지 경우: v 와 연결된 정점에 점화관계를 통해 거리를 갱신하고, 거리가 갱신된 경우에 한해 heap에 $(d + w, u)$ 쌍을 넣는다.

Code 30.2은 이 과정을 구현한 코드이다.

Code 30.2. $O(M \log M)$ 다익스트라 구현 코드

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef pair<int, int> pii;
4
5 int n, m;
6 vector<pii> adj[101010];

```

```

7
8  int visited[101010], dis[101010];
9  priority_queue<pii> pq;
10
11  int main() {
12      ios_base::sync_with_stdio(false);
13      cin.tie(0);
14      cin >> n >> m;
15      for (int i = 1; i <= m; i++) {
16          int u, v, w;
17          cin >> u >> v >> w;
18          adj[u].push_back({v, w});
19          adj[v].push_back({u, w});
20      }
21
22      for (int i = 1; i <= n; i++) dis[i] = 1e9; //inf
23      dis[1] = 0;
24
25      pq.push({-dis[1], 1});
26
27      while (!pq.empty()) {
28          int v = pq.top().second, d = -pq.top().first;
29          if (visited[v] == 1) continue;
30          if (dis[v] < d) continue;
31          visited[v] = 1;
32          for (int i = 0; i < adj[v].size(); i++) {
33              int u = adj[v][i].first, w = adj[v][i].second;
34              if (d + w < dis[u]) {
35                  dis[u] = d + w;
36                  pq.push({-dis[u], u});
37              }
38          }
39      }
40  }

```

시간복잡도는 어떻게 될까? 점화식을 통해 거리를 업데이트하는 횟수는 $O(M)$ 으로 같고, while 문은 많아야 $2M$ 번 실행되기 때문에 시간복잡도는 $O(M \log M)$ 이 된다.

30.3 연습문제 A

Problem 30.1. 만약 가중치가 음수일 수 있다면 최단거리가 항상 존재하는가? 이에 대해 논의하라.

Problem 30.2. Code 30.1와 Code 30.2에서 최단거리를 역추적하는 코드를 작성하라.

Problem 30.3. 두 번째 최단거리를 구하는 $O(M \log M)$ 코드를 작성하라. 일반적으로 K 번째 최단거리를 알아야 한다면 어떻게 구현해야 하는가? 시간복잡도는 어떻게 되는가?

30.4 연습문제 B

30.4.1 기초문제

Problem 30.4. <https://www.acmicpc.net/problem/1162>

Problem 30.5. <https://www.acmicpc.net/problem/1238>

Problem 30.6. <https://www.acmicpc.net/problem/1504>

Problem 30.7. <https://www.acmicpc.net/problem/1753>

Problem 30.8. <https://www.acmicpc.net/problem/1800>

Problem 30.9. <https://www.acmicpc.net/problem/1916>

Problem 30.10. <https://www.acmicpc.net/problem/2211>

Problem 30.11. <https://www.acmicpc.net/problem/5972>

30.4.2 응용문제

Problem 30.12. <https://www.acmicpc.net/problem/1848>

Problem 30.13. <https://www.acmicpc.net/problem/1854>

Problem 30.14. <https://www.acmicpc.net/problem/1857>

Problem 30.15. <https://www.acmicpc.net/problem/2325>

Problem 30.16. <https://www.acmicpc.net/problem/5529>

Problem 30.17. <https://www.acmicpc.net/problem/5542>

- Problem 30.18. <https://www.acmicpc.net/problem/5719>
- Problem 30.19. <https://www.acmicpc.net/problem/5822>
- Problem 30.20. <https://www.acmicpc.net/problem/9446>
- Problem 30.21. <https://www.acmicpc.net/problem/10217>
- Problem 30.22. <https://www.acmicpc.net/problem/12930>
- Problem 30.23. <https://www.acmicpc.net/problem/13308>
- Problem 30.24. <https://www.acmicpc.net/problem/13907>
- Problem 30.25. <https://www.acmicpc.net/problem/14611>
- Problem 30.26. <https://www.acmicpc.net/problem/15264>
- Problem 30.27. <https://www.acmicpc.net/problem/15556>
- Problem 30.28. <https://www.acmicpc.net/problem/16763>
- Problem 30.29. <https://www.acmicpc.net/problem/18262>

30.4.3 도전문제

- Problem 30.30. <https://www.acmicpc.net/problem/2765>
- Problem 30.31. <https://www.acmicpc.net/problem/7138>
- Problem 30.32. <https://www.acmicpc.net/problem/8236>
- Problem 30.33. <https://www.acmicpc.net/problem/10120>
- Problem 30.34. <https://www.acmicpc.net/problem/10847>
- Problem 30.35. <https://www.acmicpc.net/problem/12752>
- Problem 30.36. <https://www.acmicpc.net/problem/13145>
- Problem 30.37. <https://www.acmicpc.net/problem/15004>
- Problem 30.38. <https://www.acmicpc.net/problem/15208>
- Problem 30.39. <https://www.acmicpc.net/problem/15292>

- Problem 30.40. <https://www.acmicpc.net/problem/16312>
- Problem 30.41. <https://www.acmicpc.net/problem/16606>
- Problem 30.42. <https://www.acmicpc.net/problem/17719>
- Problem 30.43. <https://www.acmicpc.net/problem/18253>
- Problem 30.44. <https://www.acmicpc.net/problem/19335>
- Problem 30.45. <https://www.acmicpc.net/problem/19378>
- Problem 30.46. <https://www.acmicpc.net/problem/19616>
- Problem 30.47. <https://www.acmicpc.net/problem/19928>
- Problem 30.48. <https://www.acmicpc.net/problem/20536>
- Problem 30.49. <https://www.acmicpc.net/problem/21731>
- Problem 30.50. <https://www.acmicpc.net/problem/21787>
- Problem 30.51. <https://www.acmicpc.net/problem/21847>
- Problem 30.52. <https://www.acmicpc.net/problem/23461>

Chapter 31

벨만-포드 알고리즘

다익스트라 알고리즘은 가중치가 음수인 경우의 문제를 해결할 수 없다는 문제가 있다. 이번 장에서는 음의 가중치가 있어도 단일-전체 최단경로 문제를 해결할 수 있는 벨만-포드 알고리즘을 배울 것이다.

31.1 문제 정의

다음과 같은 최단경로 문제를 생각하자.

Example 31.1. 가중치가 음수일 수 있는 그래프에서 가중치 합이 음수인 사이클이 있는지 판단하고, 없다면 1번 정점에서 시작할 때 각 정점까지의 최단거리를 모두 구하는 프로그램을 작성하라.

가중치가 음수이기 때문에 가중치 합이 음수인 사이클이 존재할 수 있다. 만약 그런 사이클에 도달할 수 있다면 그 사이클을 계속 순환하면 거리가 작아지기 때문에 최단거리가 존재하지 않게 된다. 따라서 음의 가중치가 있는 경우의 단일-전체 최단경로 문제는 가중치 합이 음수인 사이클 존재 여부 판별이 우선되어야 한다. 벨만-포드 알고리즘을 이용하면 좀 더 일반적인 단일-전체 최단경로 문제를 해결할 수 있다.

31.2 벨만-포드 알고리즘

다음과 같은 관찰을 기반으로 한다.

Observation. 가중치 합이 음수인 사이클이 없다면 각 간선이 N 번 이상으로 사용되는 최단

경로는 존재할 수 없다. 만약 그렇다면 경로 상에 같은 정점이 여러 번 등장할 수 밖에 없고, 이는 가중치 합이 음수인 사이클의 존재를 나타내기 때문이다.

이제 우리는 모든 정점에 대해 인접한 정점까지 거리를 점화식으로 계산하면 된다. 이 과정의 횟수가 N 번으로 제한되므로 $O(NM)$ 에 문제를 해결할 수 있다. Code 31.1는 구현 코드이다.

Code 31.1. 벨만-포드 알고리즘 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef pair<int, int> pii;
4
5  int n, m;
6  vector<pii> adj[101010];
7
8  int dis[101010];
9
10 int main() {
11     ios_base::sync_with_stdio(false);
12     cin.tie(0);
13     cin >> n >> m;
14     for (int i = 1; i <= m; i++) {
15         int u, v, w;
16         cin >> u >> v >> w;
17         adj[u].push_back({v, w});
18         adj[v].push_back({u, w});
19     }
20
21     for (int i = 1; i <= n; i++) dis[i] = 1e9;
22     dis[1] = 0;
23     for (int i = 1; i <= n; i++) {
24         for (int v = 1; v <= n; v++) {
25             for (int j = 0; j < adj[v].size(); j++) {
26                 int u = adj[v][j].first, w = adj[v][j].second;
27                 if (i == n && dis[u] > dis[v] + w) {
28                     // there is a negative cycle
29                     return 1;
30                 }
31                 dis[u] = min(dis[u], dis[v] + w);
32             }
33         }

```

```
34     }  
35     // there is not any negative cycle  
36     // dis is answer  
37 }
```

31.3 연습문제 A

Problem 31.1. Code 31.1에서 경로를 역추적하는 코드를 추가하라.

Problem 31.2. 가중치 값이 제약이 없는 그래프에서 1번 정점에서 시작할 때 최장 경로를 존재하는지 판별하고 구하는 프로그램을 작성하라.

31.4 연습문제 B

31.4.1 기초문제

Problem 31.3. <https://www.acmicpc.net/problem/1738>

Problem 31.4. <https://www.acmicpc.net/problem/1865>

Problem 31.5. <https://www.acmicpc.net/problem/6002>

Problem 31.6. <https://www.acmicpc.net/problem/6817>

Problem 31.7. <https://www.acmicpc.net/problem/10360>

Problem 31.8. <https://www.acmicpc.net/problem/11463>

Problem 31.9. <https://www.acmicpc.net/problem/11657>

Problem 31.10. <https://www.acmicpc.net/problem/13317>

Problem 31.11. <https://www.acmicpc.net/problem/26019>

Problem 31.12. <https://www.acmicpc.net/problem/30221>

31.4.2 응용문제

Problem 31.13. <https://www.acmicpc.net/problem/1219>

Problem 31.14. <https://www.acmicpc.net/problem/2081>

Problem 31.15. <https://www.acmicpc.net/problem/3680>

Problem 31.16. <https://www.acmicpc.net/problem/3860>

Problem 31.17. <https://www.acmicpc.net/problem/4324>

Problem 31.18. <https://www.acmicpc.net/problem/5942>

Problem 31.19. <https://www.acmicpc.net/problem/6141>

Problem 31.20. <https://www.acmicpc.net/problem/7040>

Problem 31.21. <https://www.acmicpc.net/problem/7141>

Problem 31.22. <https://www.acmicpc.net/problem/7144>

Problem 31.23. <https://www.acmicpc.net/problem/7145>

Problem 31.24. <https://www.acmicpc.net/problem/7634>

Problem 31.25. <https://www.acmicpc.net/problem/13271>

Problem 31.26. <https://www.acmicpc.net/problem/22522>

31.4.3 도전문제

Problem 31.27. <https://www.acmicpc.net/problem/4109>

Problem 31.28. <https://www.acmicpc.net/problem/6297>

Problem 31.29. <https://www.acmicpc.net/problem/7332>

Problem 31.30. <https://www.acmicpc.net/problem/19153>

Problem 31.31. <https://www.acmicpc.net/problem/19330>

Problem 31.32. <https://www.acmicpc.net/problem/27400>

Chapter 32

플로이드-워셜 알고리즘

마지막 최단경로 알고리즘은 임의의 두 정점 u 와 v 사이의 거리를 모두 구하는 문제인 전체-전체 최단경로 문제를 해결하는 플로이드-워셜 알고리즘이다. 물론 단일-전체 최단경로를 N 번 사용할 수 있지만, 플로이드-워셜 알고리즘은 그것보다 빠르다. 이번 장에서는 플로이드-워셜 알고리즘을 배울 것이다.

32.1 문제 정의

다음과 같은 최단경로 문제를 생각하자.

Example 32.1. 모든 가중치가 양수인 한 그래프에서 임의의 두 정점 u 와 v 사이의 거리를 모두 저장한 거리 행렬을 구하는 프로그램을 작성하라.

이러한 문제를 전체-전체 최단경로 문제라고 부른다. 시작 정점도 모든 정점이고, 도착 정점도 모든 정점이기 때문이다. 이런 문제는 가중치가 양수라는 조건 하에서 플로이드-워셜 알고리즘으로 해결할 수 있다. 꽤 특이한 아이디어를 사용하고, 이러한 아이디어는 때때로 사용되기도 하니 꼭 숙지하도록 하자.

32.2 플로이드-워셜 알고리즘

거리 행렬의 i 행 j 열 성분, 즉 i 번 정점에서 j 번 정점까지의 최단거리를 d_{ij} 라고 놓자. 초깃값은 직접 연결하는 간선이 있으면 그 간선의 가중치이고, 없으면 무한대이며, $i = j$ 이면 $d_{ij} = 0$ 이다. 핵심 아이디어는 i 번 정점에서 j 번 정점으로 가는 경로는 모든 k 에 대해 k 번 정점을 거치는 경로의 최단거리의 최솟값이라는 것이다. 다른 말로 하면 $d_{ij} \leftarrow d_{ik} + d_{kj}$ 의 상태전이를

사용한다.

문제는 d_{ik} 와 d_{kj} 가 최솟값임이 보장되어야 d_{ij} 가 제대로 계산될 수 있다는 것이다. 이를 보장하기 위해서는 i, j, k 에 대한 삼중 반복문을 쓸 때 제일 상위 반복문을 k 에 대해 반복하면 된다. 이러한 접근이 가능한 이유는 최초의 d_{ij} 경로에서 1번 정점부터 N 번 정점까지 순서대로 경로에 포함시켰을 때 최적이 되는지 파악하기 때문이다. 동적계획법과 비슷하게 수학적 귀납법 기반으로 진행된다고 이해하면 쉽다.

시간복잡도는 $O(N^3)$ 으로, 단일-전체 최단경로를 N 번 시행했을 때 시간복잡도인 $O(N^3 \log N)$ 보다 빠르다. 구현 코드는 Code 32.1을 참고하라.

Code 32.1. 플로이드-워셜 알고리즘 구현 코드

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, m;
5  int dis[1010][1010]
6
7  int main() {
8      ios_base::sync_with_stdio(false);
9      cin.tie(0);
10     cin >> n >> m;
11     for (int i = 1; i <= n; i++) for (int j = 1; j <= n; j++) dis[i][j]
        = 1e9;
12     for (int i = 1; i <= n; i++) dis[i][i] = 0;
13     for (int i = 1; i <= m; i++) {
14         int u, v, w;
15         cin >> u >> v >> w;
16         dis[u][v] = w;
17         dis[v][u] = w;
18     }
19
20     for (int k = 1; k <= n; k++) {
21         for (int i = 1; i <= n; i++) {
22             for (int j = 1; j <= n; j++) {
23                 dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
24             }
25         }
26     }
27     //dis is answer

```

28 | }

32.3 연습문제 A

Problem 32.1. 플로이드-워셜 알고리즘에서, 임의의 i, j, k 반복문 순서를 사용하더라도 똑같은 코드를 세 번 반복하면 답이 구해짐이 보장된다. 이를 증명하라.

Problem 32.2. Code 32.1에서 경로를 역추적하는 코드를 추가하라.

32.4 연습문제 B

32.4.1 기초문제

Problem 32.3. <https://www.acmicpc.net/problem/11403>

Problem 32.4. <https://www.acmicpc.net/problem/11404>

Problem 32.5. <https://www.acmicpc.net/problem/1389>

Problem 32.6. <https://www.acmicpc.net/problem/2458>

Problem 32.7. <https://www.acmicpc.net/problem/1956>

Problem 32.8. <https://www.acmicpc.net/problem/14938>

Problem 32.9. <https://www.acmicpc.net/problem/2660>

Problem 32.10. <https://www.acmicpc.net/problem/10159>

Problem 32.11. <https://www.acmicpc.net/problem/1058>

Problem 32.12. <https://www.acmicpc.net/problem/1613>

Problem 32.13. <https://www.acmicpc.net/problem/11780>

Problem 32.14. <https://www.acmicpc.net/problem/1507>

Problem 32.15. <https://www.acmicpc.net/problem/1719>

Problem 32.16. <https://www.acmicpc.net/problem/2617>

Problem 32.17. <https://www.acmicpc.net/problem/2610>

Problem 32.18. <https://www.acmicpc.net/problem/11562>

Problem 32.19. <https://www.acmicpc.net/problem/17182>

Problem 32.20. <https://www.acmicpc.net/problem/13424>

Problem 32.21. <https://www.acmicpc.net/problem/11265>

32.4.2 응용문제

Problem 32.22. <https://www.acmicpc.net/problem/1602>

Problem 32.23. <https://www.acmicpc.net/problem/3349>

Problem 32.24. <https://www.acmicpc.net/problem/4142>

Problem 32.25. <https://www.acmicpc.net/problem/4603>

Problem 32.26. <https://www.acmicpc.net/problem/6430>

Problem 32.27. <https://www.acmicpc.net/problem/7141>

Problem 32.28. <https://www.acmicpc.net/problem/7143>

Problem 32.29. <https://www.acmicpc.net/problem/7768>

Problem 32.30. <https://www.acmicpc.net/problem/8877>

Problem 32.31. <https://www.acmicpc.net/problem/9502>

Problem 32.32. <https://www.acmicpc.net/problem/13141>

Problem 32.33. <https://www.acmicpc.net/problem/15526>

Problem 32.34. <https://www.acmicpc.net/problem/15843>

Problem 32.35. <https://www.acmicpc.net/problem/16617>

Problem 32.36. <https://www.acmicpc.net/problem/18264>

Problem 32.37. <https://www.acmicpc.net/problem/19648>

Problem 32.38. <https://www.acmicpc.net/problem/20445>

Problem 32.39. <https://www.acmicpc.net/problem/23258>

Problem 32.40. <https://www.acmicpc.net/problem/24617>

Problem 32.41. <https://www.acmicpc.net/problem/27116>

32.4.3 도전문제

Problem 32.42. <https://www.acmicpc.net/problem/2125>

Problem 32.43. <https://www.acmicpc.net/problem/7577>

Problem 32.44. <https://www.acmicpc.net/problem/9669>

Problem 32.45. <https://www.acmicpc.net/problem/10518>

Problem 32.46. <https://www.acmicpc.net/problem/13009>

Problem 32.47. <https://www.acmicpc.net/problem/15308>

Problem 32.48. <https://www.acmicpc.net/problem/15826>

Problem 32.49. <https://www.acmicpc.net/problem/16662>

Problem 32.50. <https://www.acmicpc.net/problem/17516>

Problem 32.51. <https://www.acmicpc.net/problem/17681>

Problem 32.52. <https://www.acmicpc.net/problem/21006>

Problem 32.53. <https://www.acmicpc.net/problem/23171>

Problem 32.54. <https://www.acmicpc.net/problem/23493>

Problem 32.55. <https://www.acmicpc.net/problem/27425>

Problem 32.56. <https://www.acmicpc.net/problem/29765>

Part VII

트리 알고리즘

Chapter 33

트리 기초

Chapter 34

트리 탐색

Chapter 35

최소 공통 조상

Chapter 36

오일러 투어 테크닉

Chapter 37

센트로이드

Chapter 38

Heavy-Light 분할

Chapter 39

트리에서의 동적계획법

Chapter 40

작은 집합에서 큰 집합으로 합치는 테크닉

Part VIII

자료구조 (Hard)

Chapter 41

제공근 분할

Chapter 42

mo's

Chapter 43

세그먼트 트리

Chapter 44

느리게 갱신되는 세그먼트 트리

Chapter 45

바이너리 인덱스 트리

Chapter 46

머지 소트 트리

Chapter 47

다차원 세그먼트 트리

Chapter 48

퍼시스턴트 세그먼트 트리

Chapter 49

스플레이 트리

Chapter 50

링크/컷 트리

Chapter 51

오프라인 동적 연결성

Part IX

동적계획법 (Hard)

Chapter 52

단조 큐를 이용한 동적계획법 최적화

Chapter 53

덱을 이용한 구간 최댓값 트릭

Chapter 54

비트필드를 이용한 동적계획법

Chapter 55

블록 꺾질을 이용한 동적계획법 최적화

Chapter 56

분할 정복을 이용한 동적계획법 최적화

Chapter 57

커넥션 프로파일을 이용한 동적계획법

Chapter 58

Knuth 최적화

Chapter 59

Aliens 트릭

Chapter 60

Slope 트릭

Chapter 61

Hirschburg 알고리즘

Chapter 62

별리캠프-매시

Chapter 63

키타마사법

Part X

문자열

Chapter 64

KMP

Chapter 65

아호-코라식

Chapter 66

트라이

Chapter 67

점미사 배열과 LCP 배열

Part XI

기하

Chapter 68

선분 교차 판정

Chapter 69

볼록 껍질

Chapter 70

회전하는 켈리퍼스

Part XII

네트워크 플로우

Chapter 71

이분 매칭

Chapter 72

최대 유량

Chapter 73

최소 비용 최대 유량

Chapter 74

최대 유량 최소 컷 정리

Part XIII

수학

Chapter 75

게임 이론

Chapter 76

소수 판정 및 소인수분해

Chapter 77

모듈러 곱셈 역원

Chapter 78

중국인의 나머지 정리

Chapter 79

확장 유클리드 호제법

Chapter 80

고속 푸리에 변환

Appendix A

Hint and Solution

Problem A.1. Code 11.1의 시간 복잡도는 무엇인가?

Solution A.1. 사실 백트래킹 문제의 시간복잡도를 논의하기는 어렵다. Code 11.1를 보면 `isvalid` 함수를 통과하는지 여부에 따라 다음 재귀 호출이 실행될 수도 있고 실행되지 않을 수도 있기 때문이다. 다만 확실한 것은 재귀 함수의 호출 횟수가 $T(N)$ 이면 시간복잡도는 $O(N \cdot T(n))$ 이라는 사실이다.

고로 $T(n)$ 의 상한을 잡아 시간복잡도의 상한을 잡자. 대각선을 고려하지 않았을 때, 매 번 선택되는 열이 다르므로 $T(N) \leq N!$ 을 쉽게 보일 수 있다. 따라서 시간복잡도의 상한은 $O(N \times N!)$ 이다.

Cf. 실제로 상한과 실제 실행 시간이 다름을 3번 문제에서 확인할 수 있을 것이다.